

Java 热门面试题 200 道速记通关版 _ 面试刷题

mianshiya.com

本资源来自面试鸭: <https://www.mianshiya.com>

推荐更多免费学编程资源:

1. 编程导航学习网站: [学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库: [开始刷题, 面试遇原题!](#)
3. 精选简历模板大全: [1 分钟搞定简历!](#)
4. AI 资源导航网站: [获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试: [随时随地提升面试能力](#)

Java 中有哪些集合类? 请简单介绍

Java 的集合类主要分两大派, 一个是 Collection 接口下的各种实现, 另一个是 Map 系的键值对结构。面试里聊到这块, 重点其实是搞清楚每种集合适用的场景和底层机制。

Collection 里最常用的是 List、Set 和 Queue。

List 是有序可重复的, ArrayList 内部用数组实现, 随机访问快, 但中间插入删除得搬数据, 性能损耗大; LinkedList 是双向链表, 适合频繁增删的场景, 尤其是头尾操作。

Set 不允许重复元素, HashSet 基于 HashMap 实现, 查找添加都是 $O(1)$, 但不保序; LinkedHashSet 能维持插入顺序; TreeSet 则能自然排序, 底层是红黑树。

Queue 一般用在任务调度, 比如 ArrayDeque 是数组实现的双端队列, ConcurrentLinkedQueue 是无锁线程安全队列, DelayQueue 适合定时任务。

Map 才是脏活累活的主力。HashMap 最常见, 数组 + 链表/红黑树结构, 非线程安全, 但性能好; Hashtable 就是老古董了, 方法全 synchronized, 基本被 ConcurrentHashMap 取代, 后者用了分段锁 + CAS, 在高并发下能扛住压力。

LinkedHashMap 能记住插入顺序, 适合做 LRU 缓存; TreeMap 支持按键排序。

- 1) ArrayList 查找快, 增删慢
- 2) LinkedList 增删快, 查找慢
- 3) HashMap 性能高, 并发用 ConcurrentHashMap
- 4) 需要排序考虑 TreeSet / TreeMap
- 5) 讲究插入顺序用 LinkedHashMap

MySQL 的索引类型有哪些?

MySQL 的索引类型主要看存储引擎支持, 最常用的是 InnoDB, 它的索引结构基于 B+ 树。

1) 主键索引 (聚簇索引)

行数据和主键一起存储在 B+ 树的叶子节点, 物理顺序和主键顺序一致。每张表只能有一个, 查询效率最高, 范围查询也快。

2) 二级索引 (普通索引)

非主键列上的索引, B+ 树叶子节点存的是主键值。查非主键字段会先走二级索引找到主键, 再回表查聚簇索引, 这个过程叫回表。

3) 唯一索引

和二级索引类似，但要求列值唯一，允许一个 NULL。创建唯一约束时自动建这种索引。

4) 联合索引

多个列组合的索引，遵循最左前缀原则。比如 (a, b, c) 能命中 a、(a,b)、(a,b,c)，但不能用 b 单独查。

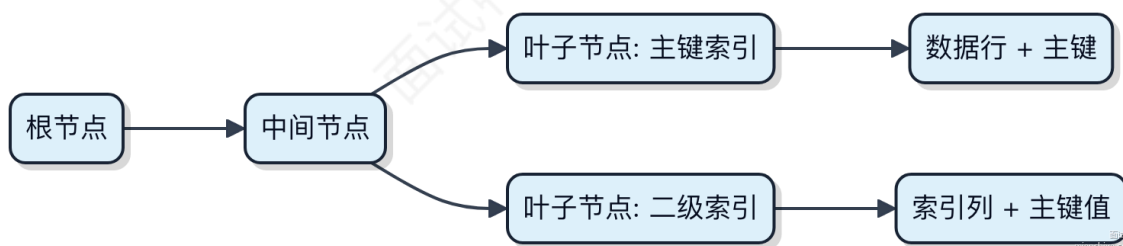
5) 全文索引

用于文本内容的关键词搜索，支持 MATCH AGAINST 语法。MyISAM 和 InnoDB 都支持，但使用场景不多，一般用 Elasticsearch 或 OpenSearch 替代。

6) 前缀索引

对字符串前 N 个字符建索引，节省空间。比如 index(email(10))，但会影响区分度，可能增加扫描行数。

覆盖索引是个重要优化点。如果查询的字段都在索引里，压根不经过回表，直接从二级索引返回数据，速度很快。比如给 (name, age) 建联合索引，select name, age from t where name=? 就能覆盖。



MySQL 索引的最左前缀匹配原则是什么？

当你的 SQL 查询条件用到了联合索引，但不是从第一个字段开始，也不是连续使用时，能不能命中索引，就得看最左前缀匹配原则。

这条规则的意思是，MySQL 会一直向右匹配直到遇到范围查询（比如 >、<、BETWEEN、LIKE）或者没有使用的列为止，中间不能断。

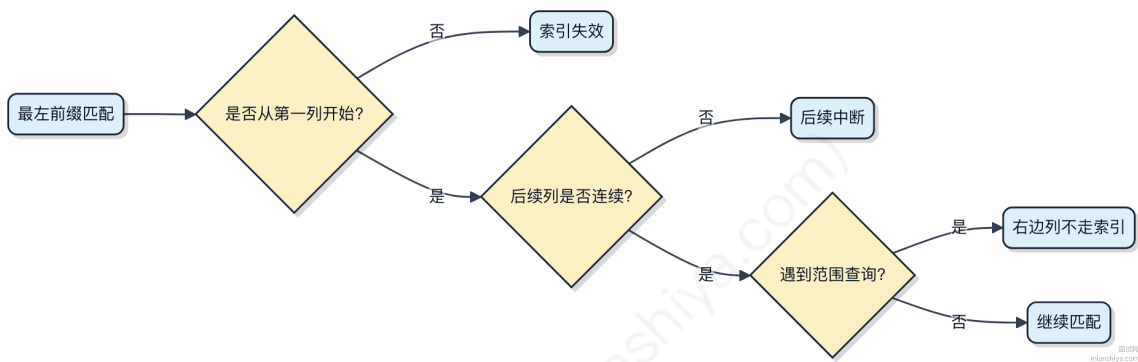
举个例子，你有个联合索引 (a, b, c)，那么下面这些查询能走索引：1) 只查 a 2) 查 a 和 b 3) 查 a、b、c

但如果查的是 b 单独，或者 b 和 c，压根不经过 a，那这个联合索引就用不上。因为不符合“最左”这个前提。

特别注意范围查询的影响。比如 WHERE a = 1 AND b > 2 AND c = 3，虽然三个字段都用了，但因为 b 是范围，c 就没法再利用索引了。也就是说，**范围查询后面的列不会走索引**。

实际开发中，像 user_id 和 create_time 这种高频组合，如果你把 create_time 放前面，而查询经常是等值查 user_id + 范围查时间，那索引顺序错了就搞不定查询性能。

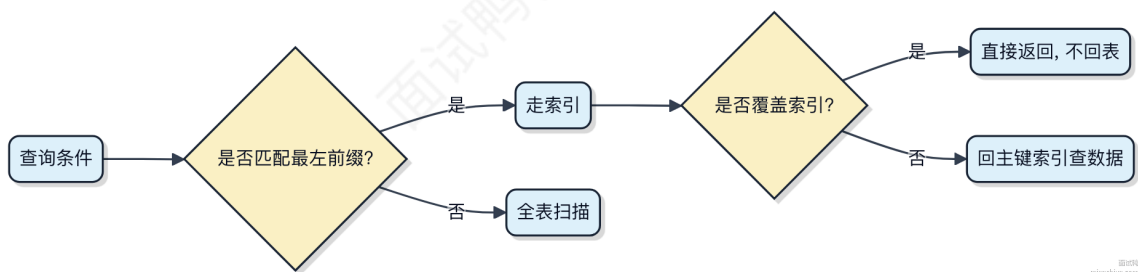
所以建联合索引的时候，顺序很关键：等值查询放前面，范围查询放后面，并且尽量让高频字段靠左。



在 MySQL 中建索引时需要注意哪些事项?

索引不是建得越多越好，搞不好反而拖慢写入性能。每条 INSERT、UPDATE 都要维护索引结构，尤其是 B+ 树的分裂合并，成本不低。

- 1) 选择区分度高的列作为索引键。比如用户表用 `user_id` 比用 `gender` 强多了，后者只有两个值，查起来几乎全表扫。
- 2) 联合索引注意最左前缀原则。比如你建了 `(a, b, c)` 的组合索引，查询条件必须从 `a` 开始，只查 `b` 或 `c` 压根不经过这个索引。
- 3) 避免在索引列上做计算或函数操作。像 `WHERE YEAR(create_time) = 2023` 这种写法，即使 `create_time` 有索引也用不上，得改成范围查询。
- 4) 覆盖索引能显著提升性能。如果查询字段都在索引里，MySQL 直接从 B+ 树的叶子节点取数据，不用回表查主键索引，减少 IO。
- 5) 字符串字段别全建索引，太占空间。比如 `VARCHAR(255)` 可以只对前 10 个字符建前缀索引，但得保证前缀区分度够高。
- 6) 大表加索引要小心锁表。可以用 `ALTER TABLE ... ALGORITHM=INPLACE, LOCK=NONE`，或者借助 `pt-online-schema-change` 工具在线操作。
- 7) 定期检查冗余和未使用索引。`information_schema.statistics` 能看索引使用情况，长时间没被用到的可以考虑删掉。



MySQL 中的事务隔离级别有哪些?

事务隔离级别是为了解决并发场景下多个事务同时操作数据时产生的问题。MySQL 支持四种标准隔离级别，每种解决的问题和带来的副作用都不同。

读未提交 (Read Uncommitted)

一个事务能读到另一个事务还没提交的数据。这种情况最容易出问题，比如读到了“脏数据”，也就是对方事务回滚了，你读到的就是废的。一般没人用这个级别。

读已提交 (Read Committed)

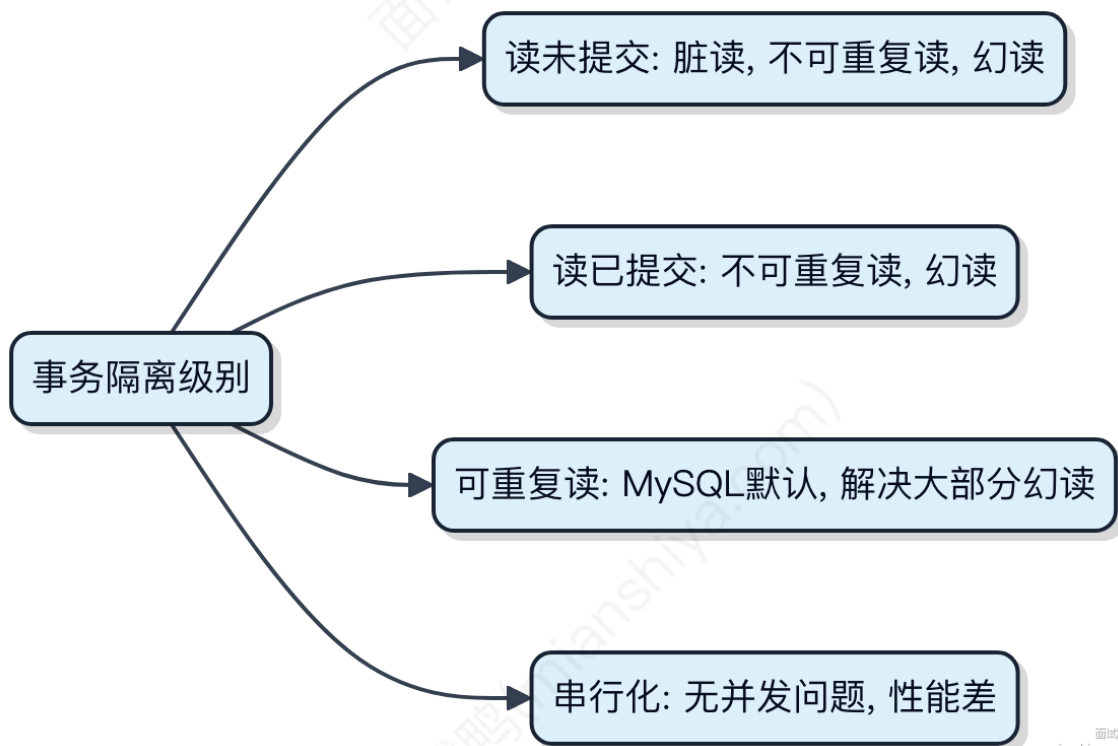
只能读到已经提交的数据，解决了脏读。但会出现不可重复读，就是同一个事务里两次读同一行数据，结果不一样，因为中间有别的事务提交了修改。PostgreSQL 默认就是这个级别。

可重复读 (Repeatable Read)

MySQL 的默认隔离级别。保证在同一个事务中多次读取同一数据结果一致，解决了不可重复读。通过 **MVCC**（多版本并发控制）实现快照读，普通查询不加锁。但理论上还可能存在幻读问题，不过 InnoDB 在这个级别通过间隙锁解决了大部分幻读场景。

串行化 (Serializable)

最高的隔离级别，所有事务排着队执行。读会加共享锁，写加排他锁，完全避免并发问题。但性能最差，吞吐降得厉害，只有对数据一致性要求极端严格的场景才会考虑。



实际业务中，基本都在用读已提交或可重复读。选择哪个看场景，比如金融系统可能倾向可重复读，而高并发读多写的系统可能选读已提交更合适。

MySQL 默认的事务隔离级别是什么？为什么选择这个级别？

MySQL 默认的事务隔离级别是**可重复读** (Repeatable Read)。这个级别能解决脏读和不可重复读问题，大多数业务场景下够用又不会太影响并发性能。

- 1) 脏读是指一个事务读到了另一个未提交事务的数据，这在任何生产环境都是不能接受的。
- 2) 不可重复读是指同一个事务内两次查询结果不一致，因为期间其他事务修改了数据。**可重复读**通过 MVCC（多版本并发控制）机制让事务看到一致的快照，避免这个问题。

但它不解决幻读问题——不过 MySQL 的 InnoDB 在这个级别用了间隙锁（Gap Lock）+ 行锁的组合，实际上把幻读也基本遏制住了。比起串行化那种一刀切的锁表操作，它在数据一致性与并发之间取得了不错的平衡。

代码上你不需要显式设置，除非要改：

```
SELECT @@transaction_isolation;  
-- 默认输出 REPEATABLE-READ
```

如果系统对一致性要求极高，比如金融核心账务，可以升到串行化。但大部分 Web 应用，像电商库存、用户积分这类操作，**可重复读**完全扛得住，还不会因为过度加锁导致线程堵死。

要不要升级隔离级别，得看业务能不能容忍一点点幻读风险。其实很多项目连这个级别的坑都没遇到过，就是因为底层存储引擎做了额外保护。

MySQL 中 count(*)、count(1) 和 count(字段名) 有什么区别？

count(*)、count(1) 和 count(字段名) 看起来都是统计行数，但底层处理方式其实有差别，用错了可能影响性能甚至结果。

count(*) 是标准的行计数方式，它不关心具体列，只统计**所有行**，包括 null 值。MySQL 优化器对它做了专门优化，尤其是 InnoDB 引擎下会选择成本最低的索引树来遍历，不一定真的全表扫描。

count(1) 其实和 count(*) 非常接近，因为 1 是常量，每一行都能取到值，所以也是统计所有行。在大多数情况下，它的执行计划和 count(*) 一样，性能几乎无差别。别被“1”误导，它不是按第一列算。

真正有区别的是一些人写的 count(字段名)。这个函数只统计该字段**非空 (not null) 的行**。如果这个字段允许为 null，那结果就可能比 count(*) 少。比如 count(email) 会漏掉 email 为空的用户，这在统计总数时容易出错。

- 1) count(*) 统计所有行，推荐用于总行数统计
- 2) count(1) 效果基本等同于 count(*)，可互换
- 3) count(字段名) 只统计该字段非空行，要小心语义差异

一般情况下，想查总行数就老老实实用 count(*)，别搞花活。有些 ORM 框架默认生成 count(1)，问题不大，但看到 count(id) 还是得确认下 id 是不是主键且非空。

什么是分库分表？分库分表有哪些类型（或策略）？

分库分表本质是为了解决单库扛不住数据量和并发压力的问题。当一张表数据达到千万甚至上亿级别，查询变慢、锁竞争激烈，主从复制延迟也会加剧。这时候就得把数据拆开，让多个数据库一起干活。

1) 垂直分库

按业务模块拆分，比如用户服务用一个库，订单服务用另一个库。这种拆法最常见于微服务架构中，像电商系统里把商品、库存、交易拆到不同库，彼此独立部署，互不影响。

2) 垂直分表

把一张宽表按字段拆开，比如用户表的登录信息和扩展资料分开存储。一般用在表字段特别多、冷热分明的场景，减少 I/O 开销。

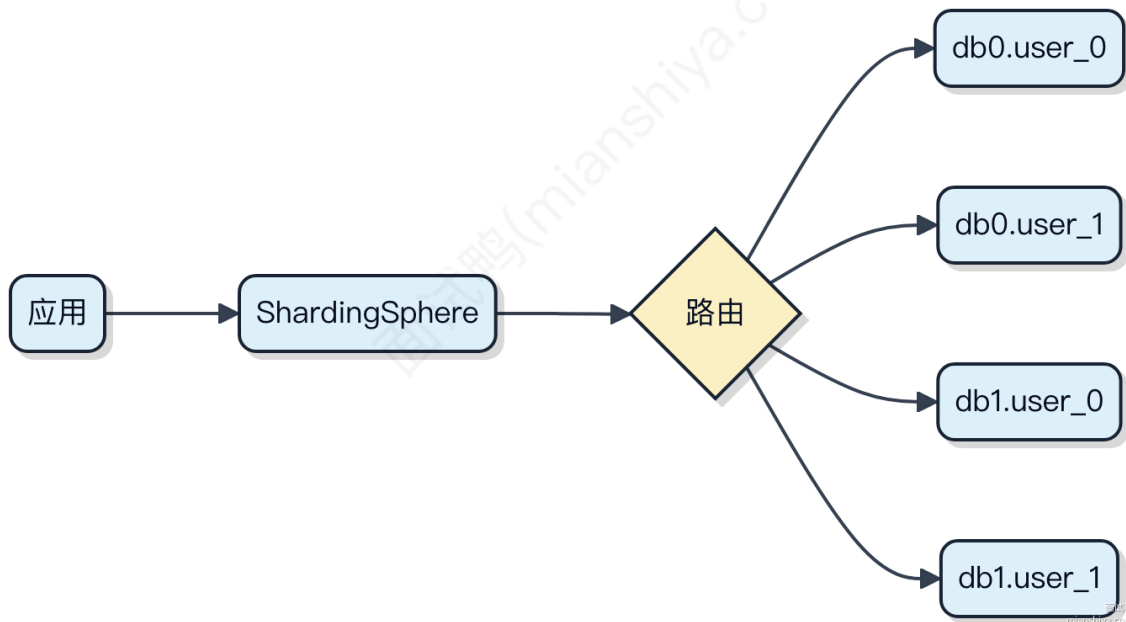
3) 水平分表

同一张表的数据按某种规则打散到多个结构相同的表里，比如用户表按 user_id 取模分成 16 张表。适合写多读少、数据增长快的场景，能有效降低单表容量。

4) 水平分库

在水平分表基础上再跨数据库实例存储，比如把订单数据按 order_id 分到 8 个 MySQL 实例上。这是真正意义上的分布式拆分，能大幅提升存储和吞吐能力。

分片键 (Sharding Key) 的选择特别关键，它决定了数据是否均匀、查询能否下推。常见的有取模、范围、一致性哈希等策略。像 ShardingSphere、MyCat 这类中间件就是靠解析 SQL + 路由规则自动完成数据寻址的。

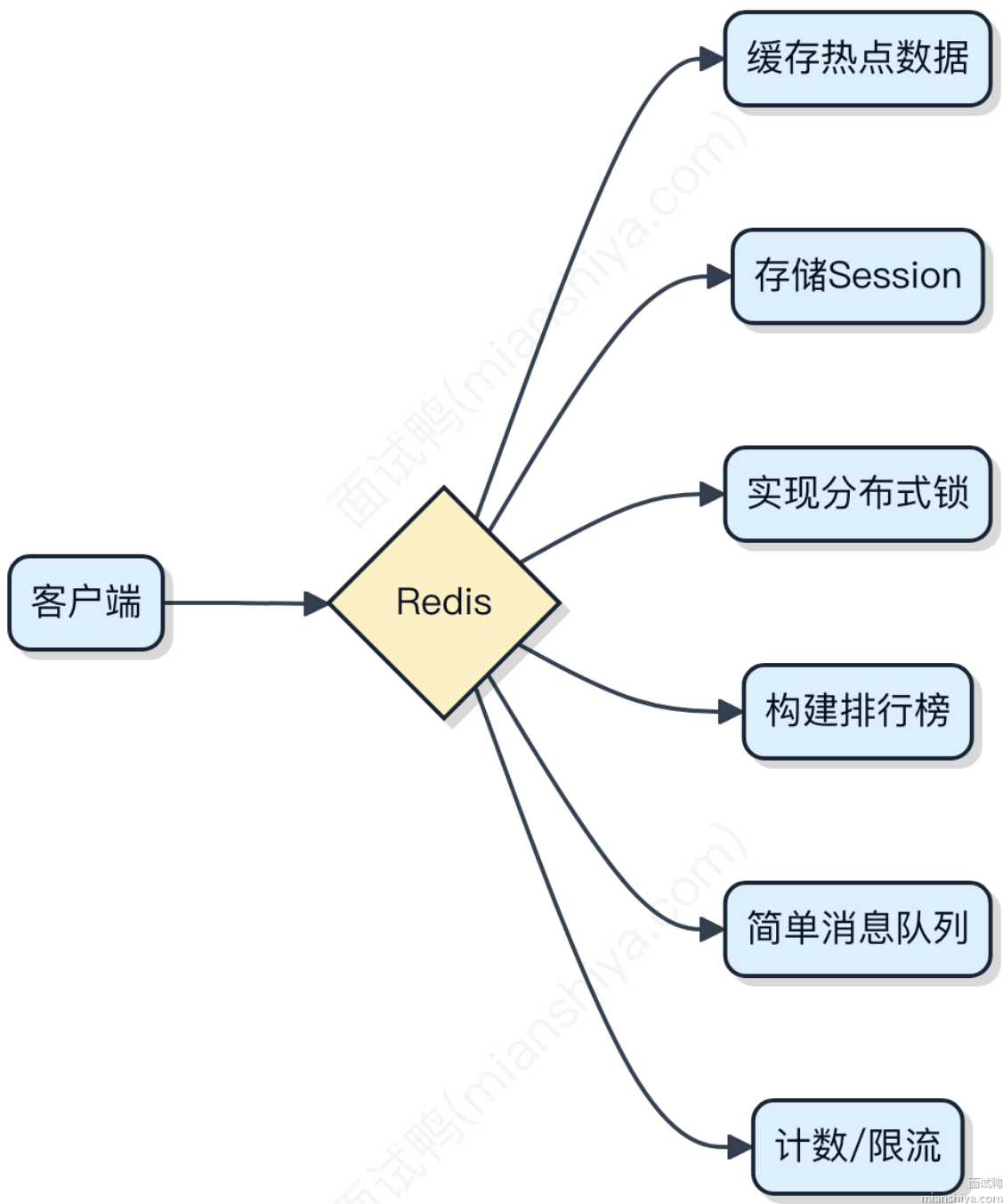


需要注意的是，一旦分库分表，跨库事务、全局排序、聚合查询就变得复杂，得靠柔性事务或引入 ES 做汇总查询来兜底。

Redis 通常应用于哪些场景？

缓存是最常见的用途，把热点数据放在 Redis 里，查询能从几十毫秒降到亚毫秒级。比如电商商品详情页，用 Redis 能扛住每秒几十万的并发请求。

- 1) Session 存储也常用它，像 Spring Session + Redis 可以实现分布式会话，在集群环境下用户登录状态不丢失。
- 2) 分布式锁基于 SETNX 或 RedLock 实现，多个服务节点协调执行，避免资源冲突。注意锁过期时间设置要合理，不然可能误删别人的锁。
- 3) 排行榜或计数场景特别适合，ZSet 支持按分数排序，微博热搜、游戏积分榜基本都这么干。
- 4) 消息队列也能凑合用，通过 List 结构做简单的生产消费，但功能比不上 RocketMQ、Kafka 这种专业中间件，复杂场景还是得换。
- 5) 限流经常结合 Lua 脚本做原子操作，比如统计 1 分钟内接口调用次数，超过阈值就拒绝请求，保护后端服务不被搞崩。



持久化场景要谨慎，RDB 和 AOF 都有性能损耗，一般不拿它当主数据库用。临时数据、高频读写、需要快速响应的场景才是它的主战场。

Redis 中常见的数据类型有哪些？

Redis 支持几种核心数据结构，每种都有明确的使用场景。

字符串（String）是最基础的类型，能存文本、序列化对象，也常用来做计数器。比如用 `INCR` 实现接口调用次数统计，简单高效。

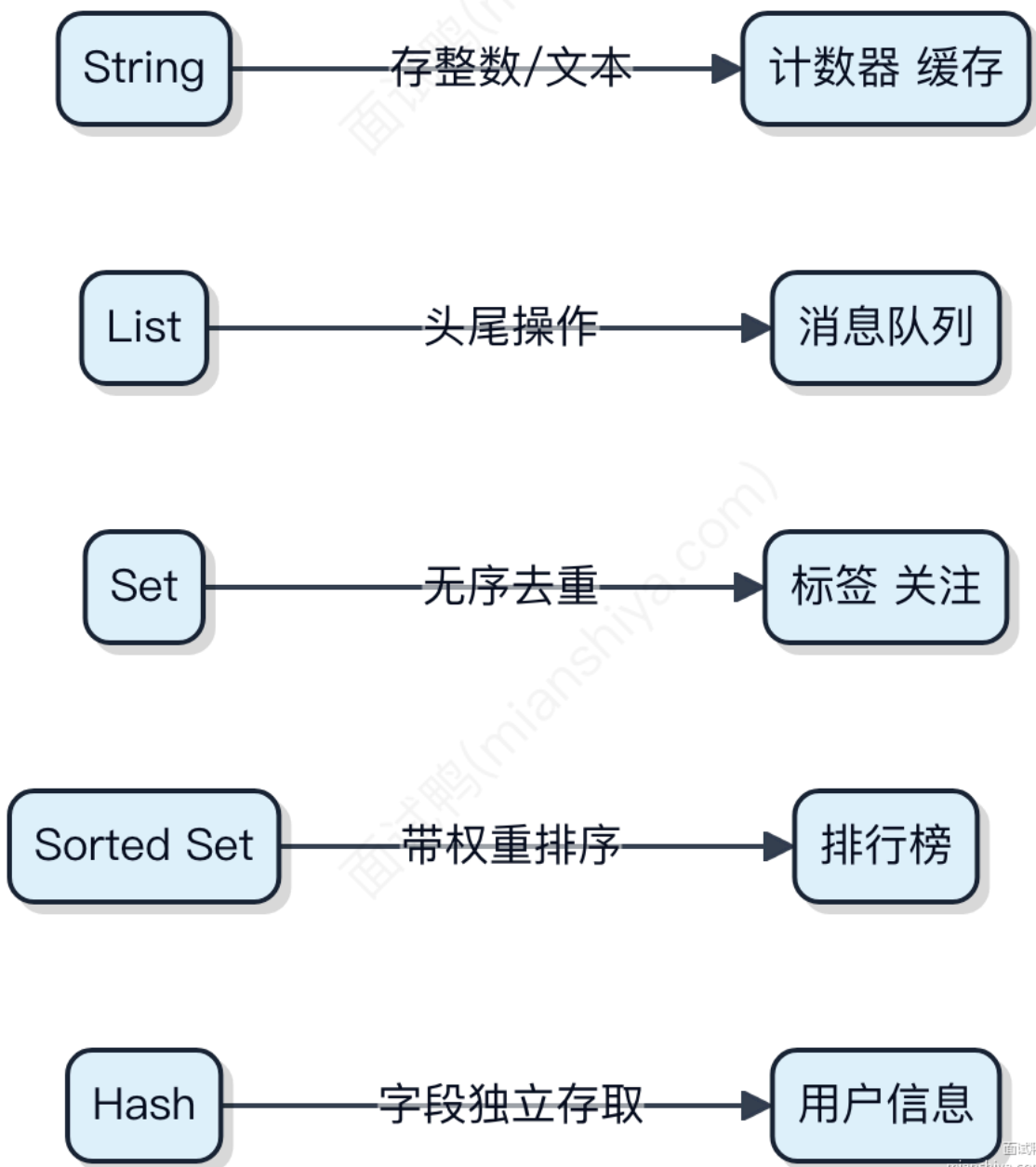
列表（List）底层是双向链表，支持从头尾插入删除，适合做消息队列。但随机访问慢，一般不推荐当数组用。
LPOP/RPOP 配合 LPUSH/RPUSH 能实现简单的生产消费模型。

集合（Set）是无序不重复的字符串集合，底层用哈希表实现，所以增删查都是 $O(1)$ 。可以用在用户标签管理、社交关系去重这些需要快速判断成员存在的场景。

有序集合（Sorted Set）在 Set 基础上加了 score 排序，比如排行榜类需求，ZADD 加数据，ZRANGE 按排名取，性能很好。score 可重复，member 才是唯一键。

哈希（Hash）适合存储对象字段，比如用户信息，field 对应属性名，value 存值。相比直接序列化整个对象，可以单独更新某个 field，省带宽也省内存。

这五种是日常最常用的，像 Stream、Geo、Bitmap 这些属于特定场景扩展类型，用得相对少一些。



你在项目中使用的 Redis 客户端是什么？

项目里用 Redis 客户端，一般首选 **Lettuce**，尤其是 Spring Boot 2.0 之后的项目。它基于 Netty 实现，支持响应式编程，连接是长连接，多个线程可以共享一个连接，资源利用率高。

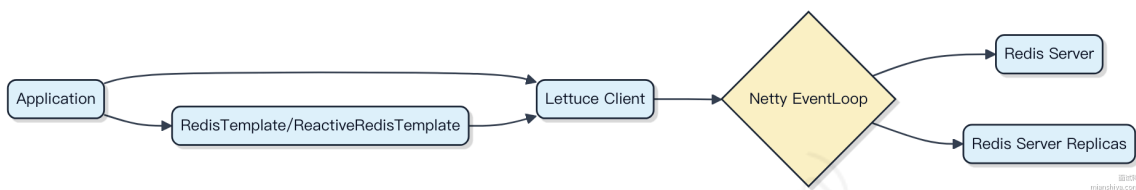
Jedis 也用过，早期项目常见，但它默认是同步阻塞的，每个操作都独占一个连接，高并发下连接数容易打满。除非是特别轻量的场景，不然现在基本不选它了。

Lettuce 和 Spring Data Redis 配合起来特别顺手，配置简单，API 也清晰。比如用 `ReactiveRedisTemplate` 做响应式缓存，或者普通 `RedisTemplate` 操作字符串、哈希都没问题。

```
@Bean
public RedisConnectionFactory redisConnectionFactory() {
    return new LettuceConnectionFactory(new
        RedisStandaloneConfiguration("localhost", 6379));
}
```

遇到过一次连接泄漏，是因为用了 Lettuce 的异步 API 但没处理好 Future 回调，导致 Netty 线程堆积。后来统一用 `RedisTemplate` 封装，避免直接操作底层连接。

1) 生产环境一定要配连接池，Lettuce 虽然原生无连接池，但可以通过 `LettucePoolingClientConfiguration` 启用。2) 大 key 或频繁批量操作要控制粒度，避免阻塞主线程。3) 开启 SSL 和设置超时时间是上线前的必备动作。

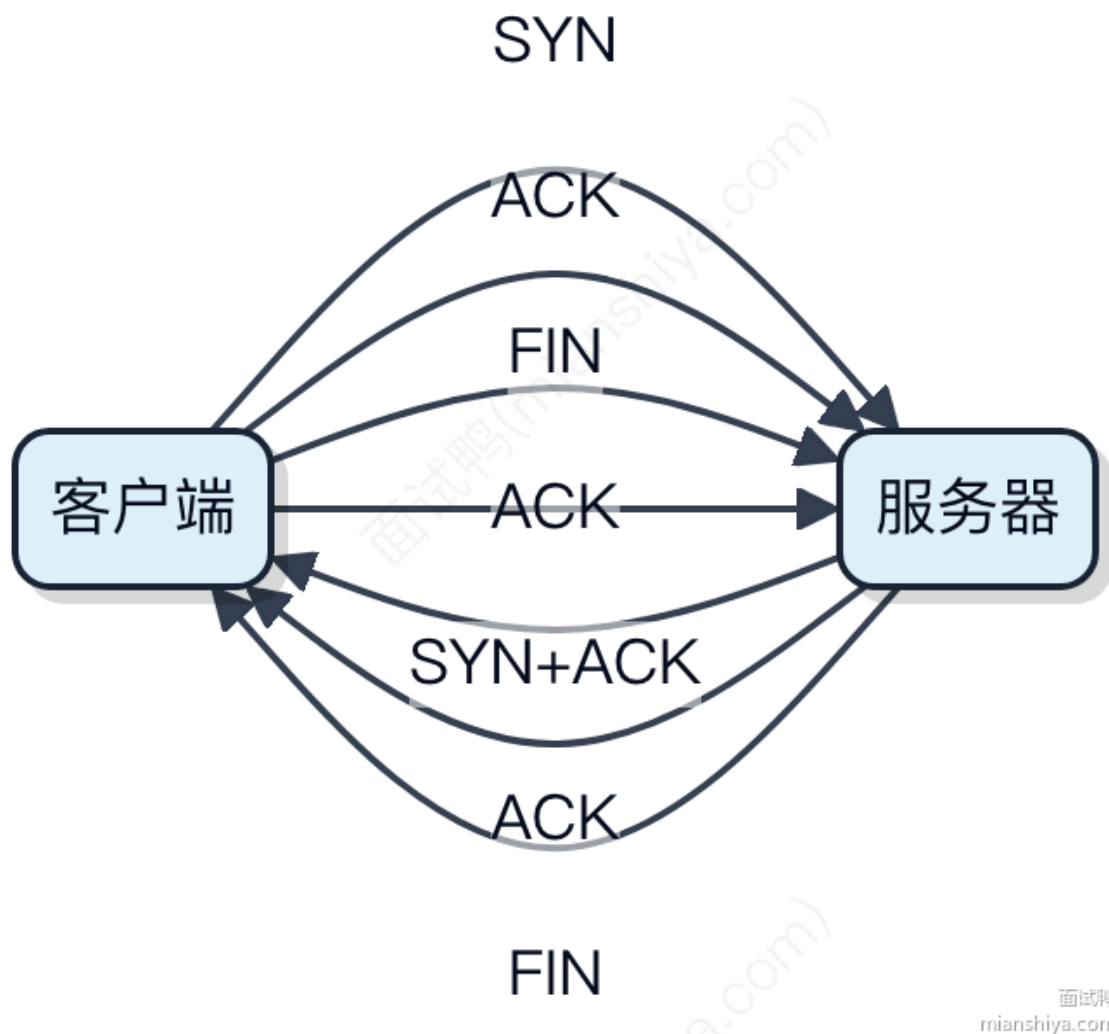


到底什么是 TCP 连接？

TCP 连接本质上是通信双方维护的一对状态，不是物理线路。建立连接靠三次握手，客户端和服务端交换各自的初始序列号（ISN），确认彼此的收发能力。

- 1) 第一次：客户端发 SYN=1, seq=x 给服务器
- 2) 第二次：服务器回 SYN=1, ack=x+1, seq=y
- 3) 第三次：客户端再发 ACK=1, ack=y+1

握手完成后，双方内核里的 TCP 状态机都进入 ESTABLISHED，开始传输数据。断开连接则是四次挥手，因为 TCP 是双工的，读写通道要分别关闭。



面试鸭
mianshiya.com

很多人以为连接是“通路”，其实它只是操作系统里两个 socket 对象在内核中保存的状态信息，比如缓冲区、窗口大小、序列号等。网络中断或超时，状态不一致，连接就失效了。

像 Nginx、Redis 这类高并发服务，最怕 TIME_WAIT 太多或者文件描述符耗尽，本质都是连接资源没释放。所以得调优内核参数，比如开启 `SO_REUSEADDR` 或调整 `tcp_tw_reuse`。

状态同步 和 **可靠传输** 才是 TCP 连接的核心目标，它通过序列号、确认应答、超时重传来保证数据不丢、不乱序。连接本身，不过是实现这个目标的上下文容器。

TCP 是用来解决什么问题？

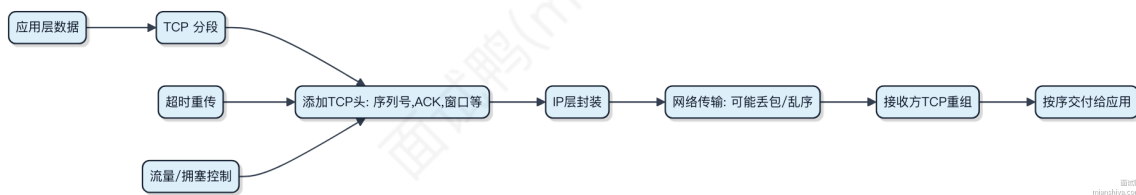
TCP 的本质是解决不可靠网络下的可靠传输问题。现实中的网络会丢包、乱序、重复，甚至中断，而 TCP 就是在这种环境下，让数据像“流水”一样完整、有序地从一端送到另一端。

它不关心你传的是网页、文件还是视频，只保证：你说要发 10000 字节，对方就得收到完整的 10000 字节，少一个都不行。

实现靠的是几个关键机制：

- 1) 序列号和确认应答。每个字节都编了号，接收方回个 ACK 告诉发方“我收到到第多少了”。没收到确认？定时器一到就重发，确保不丢。
- 2) 滑动窗口。不是发一个等一个 ACK，那样太慢。TCP 允许连续发一批，只要在窗口范围内，对方处理得过来就行，效率高。
- 3) 流量控制和拥塞控制。接收方通过窗口字段告诉发送方“我现在只能收这么多”，避免被压垮；网络堵了就主动降速，比如用 Reno 或 Cubic 算法调整发送节奏。
- 4) 连接管理。三次握手建立连接，双方同步初始序列号，避免历史连接干扰；四次挥手确保双方都确认关闭。

典型场景比如 HTTP 协议跑在 TCP 上，浏览器加载页面不能丢包，否则 HTML 解析就错了。而像音视频通话这种实时性要求高、能容忍少量丢失的，则更倾向用 UDP。



所以，TCP 的价值就是为上层屏蔽底层网络的复杂性，提供一条“可靠的字节流管道”。

TCP 和 UDP 有什么区别？

TCP 和 UDP 的本质区别在于，一个追求可靠交付，一个追求极致效率。

TCP 会建立连接，通过三次握手确认双方收发能力没问题，数据包丢了会重传，顺序乱了会重新排序，保证你收到的数据和发送的一致。像 HTTP、数据库连接这些不能出错的场景都用它，背后是 **可靠性** 在撑腰。

UDP 完全不握手，发出去就完事，不管对方听没听到。没有重传、没有流量控制，但正因为没这么多约束，性能高、延迟低。音视频通话用 UDP 多，比如 WebRTC，偶尔丢个包顶多画面卡一下，总比延迟高强。

- 1) TCP 有连接状态，维护成本高，并发百万连接对服务器资源消耗大
- 2) UDP 无状态，每个包独立处理，天生适合做广播、组播，像 DNS 查询、DHCP 分配 IP 都靠它
- 3) TCP 头部至少 20 字节，UDP 只有 8 字节，头部开销小一半以上

代码上看也很直观：

```
// UDP 发送片段
DatagramSocket socket = new DatagramSocket();
byte[] data = "hello".getBytes();
InetAddress addr = InetAddress.getByName("127.0.0.1");
DatagramPacket packet = new DatagramPacket(data, data.length, addr, 8080);
socket.send(packet); // 扔出去就不管了
```

```
// TCP 发送片段
Socket socket = new Socket("127.0.0.1", 8080);
OutputStream out = socket.getOutputStream();
out.write("hello".getBytes()); // 写完也不一定立刻发，内核会缓冲
```

简单说，要准确就选 TCP，要快就选 UDP。没有谁更好，只有合不合适。

说说 TCP 的三次握手？

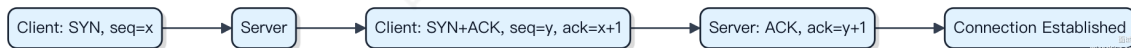
TCP 三次握手是建立连接的必经之路，主要目的是同步双方的初始序列号，确保通信能正常开始。

客户端先发一个 SYN 包，带上自己的初始序列号 $seq=x$ 。这个包就像敲门说“我想连你”。服务器收到后，得回应一个 $ACK=x+1$ 表示确认收到，同时再发一个 $SYN=y$ 告诉客户端“我也想连你”，这就是 SYN+ACK 包。最后客户端再回一个 $ACK=y+1$ ，表示你也收到了。这时候连接才算真正建立起来。

整个过程涉及的状态变化要清楚。客户端从 SYN_SENT 到 ESTABLISHED，服务器从 LISTEN 进入 SYN_RCVD，最后双方都到 ESTABLISHED。如果中途丢包，比如第一个 SYN 没到，客户端会超时重传，一般重试 5 次，间隔从 1 秒开始指数增长。

为啥不能两次？因为两次无法保证双向通道都通。客户端发了 SYN，服务器回 ACK，但这个 ACK 可能丢失，客户端不知道服务器是否准备好了，直接发数据就可能白发。三次能确保两边都能收也能发。

现实里像 Nginx、Redis 这些服务启动后就在 listen，等着客户端来握手。高并发场景下可能出现半连接队列（SYN 队列）满的情况，这时候新的连接请求会被丢弃，得调大 `somaxconn` 和 `tcp_max_syn_backlog`。



最后一次 ACK 如果丢了，服务器会重发 SYN+ACK，客户端收到后会重新回复 ACK，避免连接卡住。

说说 TCP 的四次挥手？

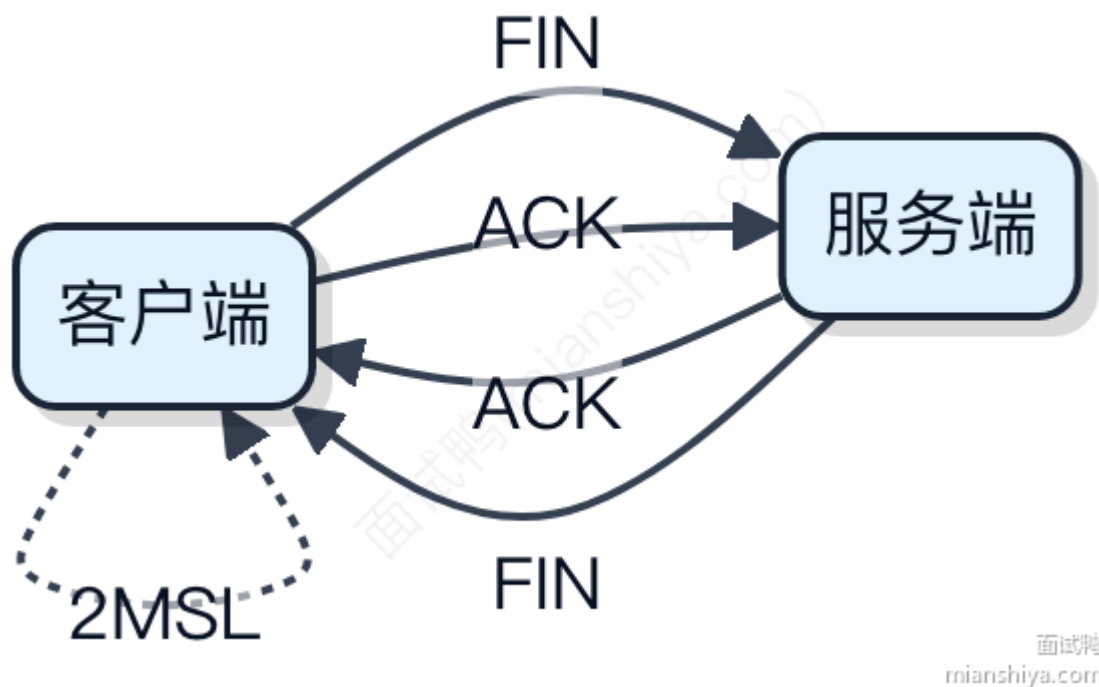
TCP 四次挥手是连接释放的标准流程，确保双方都能可靠地关闭数据传输。

客户端发送 FIN 报文，表示自己不再发数据了。服务端收到后回复 ACK，这个阶段叫半关闭，服务端还能继续发数据。

等服务端也准备好关闭时，会发自己的 FIN 报文。客户端收到后回复最后一个 ACK，等待 2MSL 时间后彻底关闭连接。

整个过程需要四次交互，主要是因为 TCP 是全双工通信，两端必须独立关闭。如果任意一端不确认对方的 FIN，连接就不能安全释放。

- 1) 客户端发 FIN
- 2) 服务端回 ACK（此时客户端进入 FIN_WAIT_2，服务端进入 CLOSE_WAIT）
- 3) 服务端发 FIN
- 4) 客户端回 ACK（进入 TIME_WAIT，服务端收到后连接关闭）



TIME_WAIT 状态很关键，它防止最后一个 ACK 丢包导致服务端重传 FIN 失败。客户端要等够 2 倍的最大报文生存时间，确保网络中残留的该连接报文都消失。

常见问题是在高并发短连接场景下，客户端大量处于 TIME_WAIT 状态，占用端口资源。可以通过开启 `SO_REUSEADDR` 或调整内核参数来缓解。

TCP 超时重传机制是为了解决什么问题？

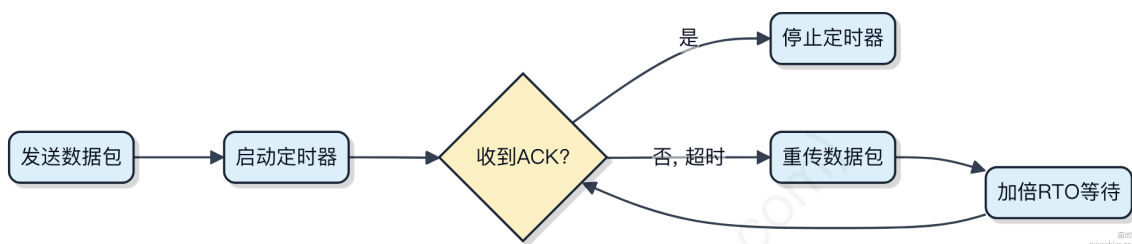
网络传输不是理想环境，数据包可能在路上丢了，或者回执丢了。TCP 作为可靠的传输协议，必须能发现这种情况并补救，不然应用层就收不到完整数据。

超时重传解决的就是丢包问题。发送方发出去一个报文段后，会启动一个定时器。如果在规定时间内没收到对方的 ACK 确认，就认为这个包丢了，于是重新发一次。整个过程对上层应用透明。

这个“规定时间”叫 RTO (Retransmission Timeout)，它不是固定值。TCP 会根据当前网络的实际往返时间 RTT 动态计算 RTO。比如刚开始估计是 200ms，后来发现网络变慢了，RTT 升到 500ms，RTO 也会跟着调整，避免误判重传。

- 1) 每次发数据都会启动定时器
- 2) ACK 到了就清除对应定时器
- 3) 超时未到 ACK 就重发并加倍等待时间（指数退避）

这样既能应对暂时的网络波动，又不会在网络彻底卡住时疯狂重试。



如果连续重传几次（一般是 15 次）还是没响应，TCP 连接就会被断开。这种机制让 TCP 能在不可靠的 IP 层之上，建立起可靠的数据通道。

TCP/IP 四层模型是什么？

TCP/IP 不是死记硬背的四层，而是为了解决网络通信问题演化出来的分层设计。每一层各司其职，把复杂的联网拆解成可管理的部分。

1) 链路层

负责物理传输，比如以太网、Wi-Fi 这一层处理 MAC 地址和帧的收发。交换机工作在这里，数据包压根不经过 IP，直接靠 MAC 转发。

2) 网络层

核心是 **IP 协议**，负责主机到主机的寻址和路由。路由器根据 IP 地址转发数据包，中间可能经过十几个节点。ICMP、ARP 也归这一层管。

3) 传输层

提供端到端的通信能力。TCP 保证可靠、有序、重传，适合 HTTP、数据库连接；UDP 不保证送达，但快，音视频通话、DNS 查询就用它。

4) 应用层

所有协议都为具体应用服务。HTTP、HTTPS、FTP、SMTP 都属于这层。你的程序代码最常接触的就是这一层接口。



实际抓包时你会发现，一个 HTTP 请求背后是四层协议层层封装：应用数据加上 TCP 头，再加 IP 头，最后套上以太网帧头，发出去。

什么是服务雪崩？

服务雪崩指的是某个服务出现故障或响应变慢，导致上游服务的请求堆积，线程资源耗尽，最终引发整个系统连锁性崩溃的现象。这种情况在微服务架构中特别常见，尤其是当服务调用链很长的时候。

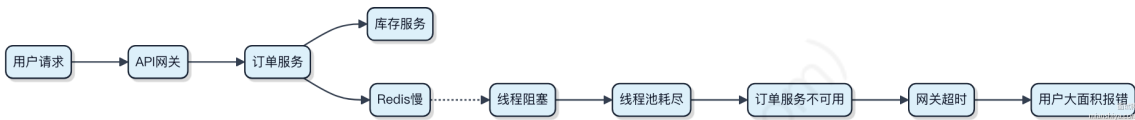
比如你用 Spring Cloud 调用一个订单服务，它又去调 Redis 查数据，结果 Redis 响应慢了。订单服务的线程一直被占着，新请求进来只能等，线程池撑爆，自己也挂了。上层的用户服务调不到订单服务，同样卡住，一层层往上倒，最后整个系统瘫痪。

1) 核心是级联故障，一个点的问题扩散成面的问题

2) 根本原因是同步阻塞调用 + 无熔断机制，请求卡住不释放资源

3) 典型场景就是高并发下数据库慢查询拖垮服务，进而影响整个调用链

解决思路主要是三点：超时控制、限流、降级和熔断。像 Hystrix、Sentinel 这类工具就是干这个的。给每个依赖服务设置最大等待时间和最大并发数，一旦超了就快速失败，不让它把整个线程池拖死。



MyBatis 中 #{} 和 \${} 的区别是什么？

#{ } 和 \${ } 看似都是占位，但处理时机完全不同。

#{ } 是预编译参数占位符，MyBatis 会把 SQL 中的 #{ } 替换成 ?，然后通过 PreparedStatement 的 set 方法赋值。这个过程能有效防止 SQL 注入，因为参数内容不会参与 SQL 解析。比如：

```
<select id="getUser" resultType="User">
    SELECT * FROM user WHERE id = #{id}
</select>
```

最终执行的是 SELECT * FROM user WHERE id = ?，安全又高效。

\${ } 是字符串拼接，直接把变量值原样插入到 SQL 中，不经过预编译。这意味着如果传入恶意内容，就可能被当作 SQL 执行。比如：

```
ORDER BY ${columnName}
```

这里 \${columnName} 如果是 id; DROP TABLE user，那后果就严重了。所以 \${ } 只能在无法使用预编译的场景下用，比如动态表名、动态排序字段，而且必须确保输入可控。

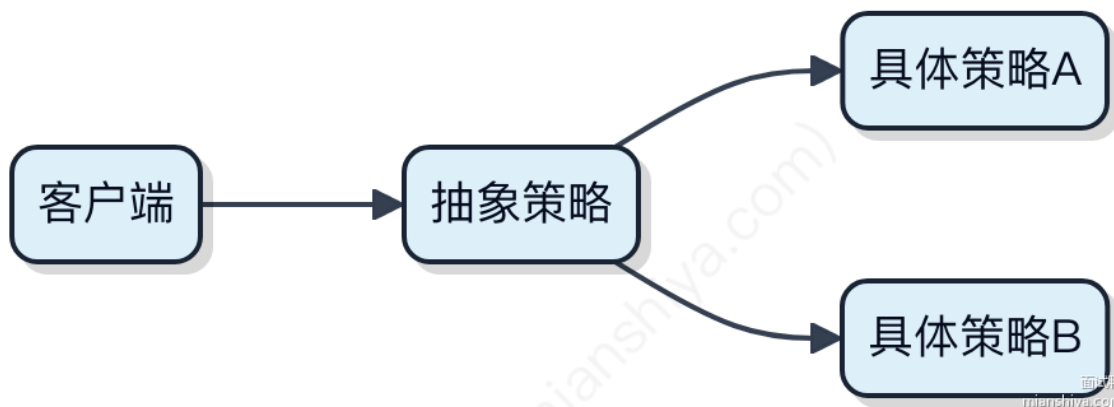
一般情况下，能用 #{ } 就别碰 \${ }。非要用 \${ }，得自己做白名单校验，不能让用户随便输。

- 1) #{ } 走预编译，安全，性能好
- 2) \${ } 是拼接，有注入风险，慎用
- 3) \${ } 适用场景：动态表名、动态排序、动态 in 条件（需转义）

什么是设计模式？请简述其作用。

设计模式不是语法也不是框架，它是前人总结出的、针对特定问题的可复用解决方案模板。本质上是代码结构和组织方式的经验结晶。

- 1) 解决重复出现的设计问题
比如对象创建太散乱，就用工厂模式统一出口；系统耦合太紧，就引入观察者模式解耦。Spring 里的 BeanFactory 就是简单工厂的体现，MyBatis 的插件机制用了责任链。
- 2) 提升代码可维护性和扩展性
以策略模式为例，不同算法独立封装，新增逻辑不用改原有 if-else。电商项目里计算运费、优惠规则基本都这么干。
- 3) 提供通用语言
团队沟通时说“这里用模板方法”，比解释“父类定流程子类重步骤”更高效。就像说“MVC”大家立刻明白分层结构。



注意别过度设计。小项目硬套 23 种模式反而搞复杂了。GOF 提出的分类（创建型、结构型、行为型）可以作为学习路线图，但实战中按需选用才是正道。

请描述简单工厂模式的工作原理。

简单工厂模式的核心是把对象的创建过程集中到一个“工厂”类里，客户端不用关心具体怎么创建实例，只要告诉工厂想要什么类型就行。

- 1) 工厂类内部通常有一个静态方法，接收一个参数比如类型标识，通过 if-else 或 switch 判断该返回哪个具体实现。比如传入 "mysql" 就返回 `MySQLDriver`，传入 "redis" 就返回 `RedisClient`。
- 2) 客户端代码只需要依赖抽象接口或父类，不直接调用 new。这样新增产品时，理论上只需修改工厂逻辑，而不必改动所有使用方。
- 3) 这种模式其实违背了开闭原则，每加一种新产品就得改工厂方法。所以适合产品种类固定、变化少的场景，比如配置解析器，支持 JSON 和 XML 两种格式就够了。

代码示例如下：

```
public class ParserFactory {
    public static Parser create(String type) {
        if ("json".equals(type)) return new JsonParser();
        if ("xml".equals(type)) return new XmlParser();
        throw new IllegalArgumentException("Unknown type");
    }
}
```

说白了，它就是个封装了 new 的工具类，让调用方更干净，但扩展性有限。真要搞灵活，还是得上 **工厂方法模式** 或 **抽象工厂**。

什么是观察者模式？一般用在什么场景？

观察者模式解决的是一对多依赖关系，当一个对象状态改变，所有依赖它的对象都能自动收到通知。这种模式在事件驱动系统里特别常见。

- 1) 被观察的对象叫主题（Subject），它维护一个观察者列表。状态变更时，遍历列表调用每个观察者的更新方法。
- 2) 观察者实现统一接口，只需定义如何响应变化，不需要主动去拉数据。

这种设计把通知逻辑收拢到主题内部，避免散落在各处。像 Spring 的 `ApplicationEvent` 机制就是典型应用，发布一个事件，多个监听器自动触发。Swing 的按钮点击、前端框架的事件绑定也都是这个思路。

代码上其实很简单：

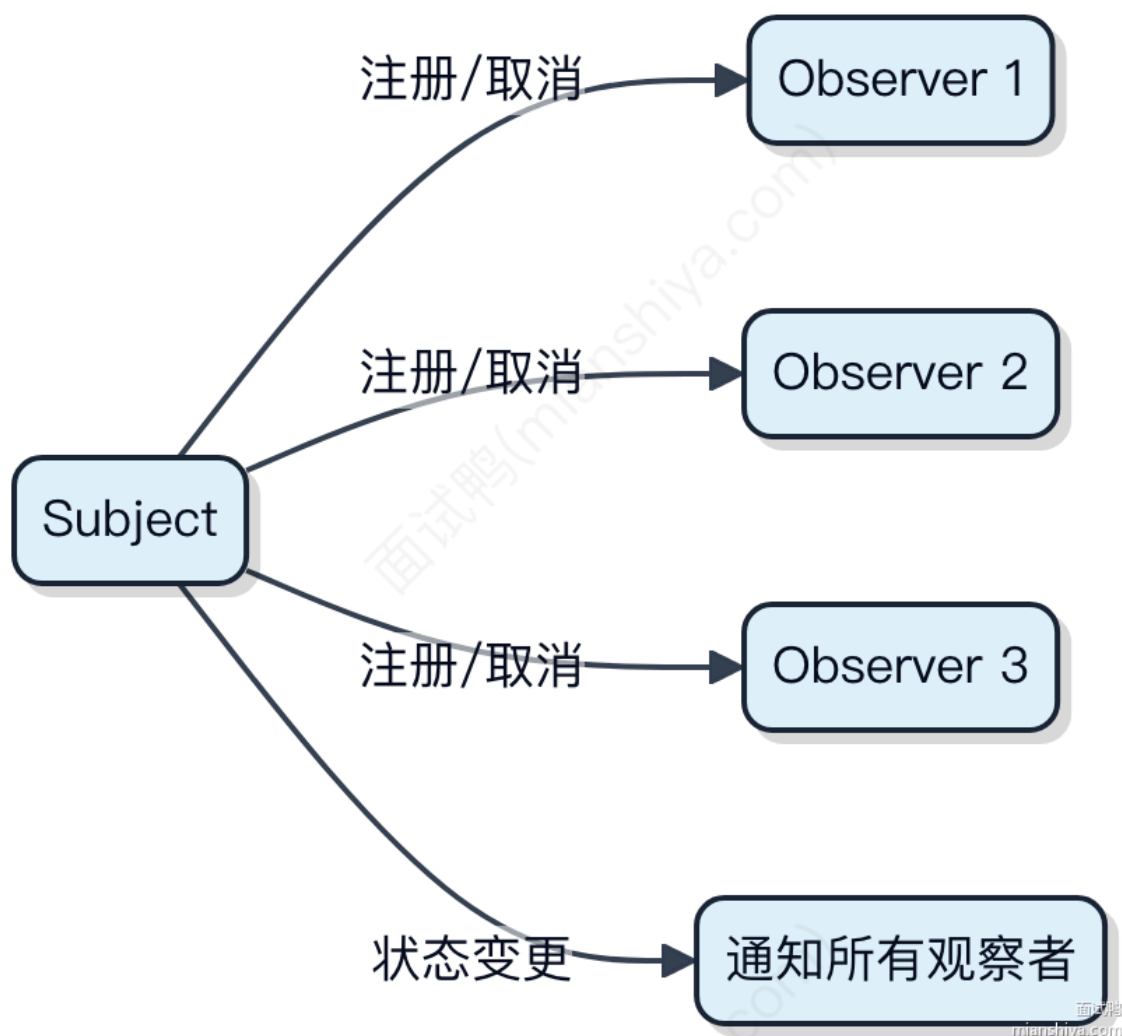
```
interface Observer {
    void update(String message);
}

class Subject {
    private List<Observer> observers = new ArrayList<>();
    private String state;

    public void setState(String state) {
        this.state = state;
        notifyObservers();
    }

    private void notifyObservers() {
        observers.forEach(o -> o.update(state));
    }
}
```

适合用在业务解耦的场景，比如订单创建后要发短信、更新库存、记录日志。把这些操作做成观察者，主流程只管发事件，不用硬编码后续动作。但要注意别滥用，观察者太多会搞不清执行链路，调试起来头疼。



什么是模板方法模式？一般用在什么场景？

模板方法模式说白了就是把一个算法的流程固定下来，把不变的部分在抽象类里写死，变的部分交给子类去实现。父类控制整体步骤，子类只负责具体细节。

- 1) 定义了一个算法骨架，比如处理请求的流程是：校验 -> 处理 -> 记录日志。这个流程不能改。
- 2) 把校验、处理这些步骤声明成抽象方法，子类根据业务实现自己的逻辑。比如支付流程里，微信支付和支付宝支付的处理方式不同，但整体流程一致。
- 3) 钩子方法可以用来做条件分支，父类通过 `isDebug()` 这样的默认返回值控制是否打印日志，子类可选择性覆盖。

这种模式在框架设计里特别常见。Spring 的 `JdbcTemplate` 就是个典型例子，你写 SQL，它帮你搞定连接管理、异常转换、资源释放这些脏活累活。还有像 `AbstractList` 里的 `addAll` 方法，底层调用 `add`，而 `add` 是抽象的，留给具体集合实现。

代码长这样：

```

abstract class ProcessTemplate {
    public final void execute() {
        validate();
        process();
        log();
    }

    abstract void validate();
    abstract void process();

    void log() { System.out.println("default log"); }
}

class MyProcess extends ProcessTemplate {
    void validate() { /* 自定义校验 */ }
    void process() { /* 业务处理 */ }
}

```

适合用在多个子类有相同工作流程，只是细节不同的场景。要是每个子类流程都不一样，硬套模板反而更麻烦。

什么是策略模式？一般用在什么场景？

策略模式解决的是“一堆 if-else 或 switch 分支里塞不同算法”的问题。你想啊，比如做支付功能，微信支付、支付宝、银联，每种逻辑完全不同，要是全写在一个方法里，代码立马爆炸，维护起来根本搞不定。

它的核心是把每个算法封装成独立的类，都实现同一个接口。运行时根据条件选一个具体实现来用。这样新增支付方式不用改老代码，符合开闭原则。

一般在这些场景特别好使：1) 有明显的行为分支，比如计算运费，按城市、重量、会员等级有不同算价规则 2) 需要动态切换算法，像导出报表支持 CSV、Excel、PDF 三种格式 3) 避免重复的条件判断，像促销活动满减、打折、返券

举个例子，Spring 的 `Resource` 接口就是典型，`ClassPathResource`、`FileSystemResource`、`UrlResource` 各自处理不同路径类型的资源加载，调用方压根不经过具体实现。

```

public interface Strategy {
    void execute();
}

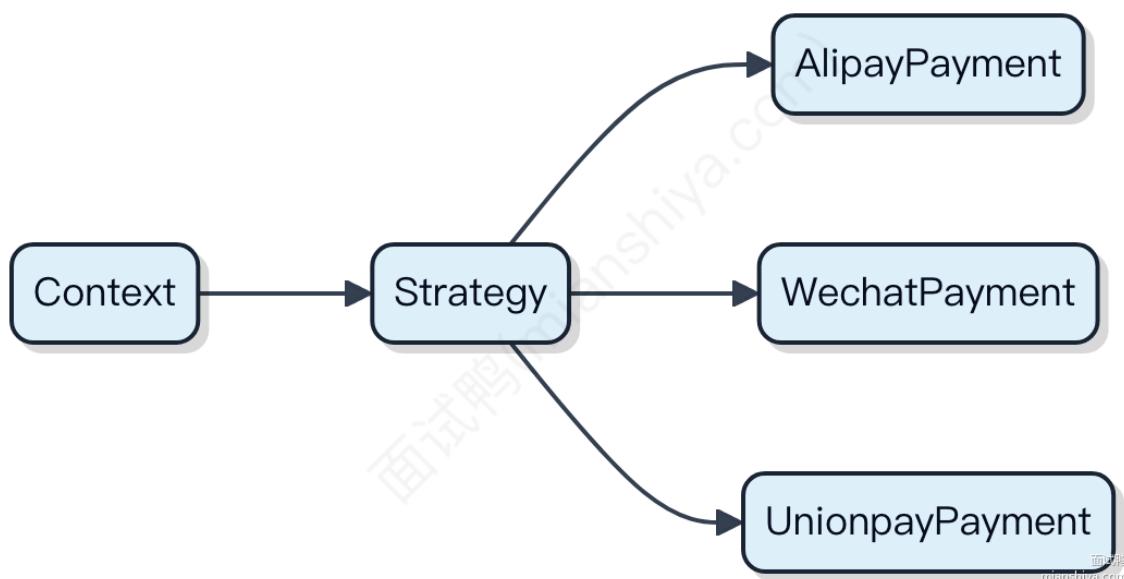
public class Context {
    private Strategy strategy;

    public void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }

    public void run() {
        strategy.execute();
    }
}

```

实际项目里，配合 Spring 的 IoC，直接把所有策略实现注入进来，用 map 存好，key 是策略名，取的时候 `strategies.get("alipay").execute()` 就行，干净利落。



什么是责任链模式？一般用在什么场景？

责任链模式就是把多个处理器串成一条链，请求沿着链往下传，直到某个处理器处理它为止。每个节点只关心自己能不能处理，能就办了，不能就丢给下一个，调用者不用管背后是谁干的。

这种模式的关键在于解耦。你往链上加新处理器，完全不用动原来的代码，符合开闭原则。比如在 Web 开发里，过滤器（Filter）就是典型的责任链，像 Spring Security 的安全过滤器链，一堆 Filter 依次执行，有的做认证，有的做授权，漏掉一个都不行。

再比如电商场景下的审批流，报销申请先由直属领导审，然后财务审，金额超过 1 万还得总监过一遍。每层只判断自己该不该拦，不用知道前后是谁。

写法上一般让处理器实现同一个接口，里面有个 next 引用指向下一个处理器，处理方法里先判断自己是否处理，否则调用 `next.handle()`。

```
public abstract class Handler {
    protected Handler next;
    public void setNext(Handler next) { this.next = next; }
    public abstract void handle(Request request);
}

public class Level1Handler extends Handler {
    public void handle(Request request) {
        if (request.getType() == "type1") {
            System.out.println("Level1 处理");
        } else if (next != null) {
            next.handle(request);
        }
    }
}
```



```
}  
}
```

注意别搞成死循环，还要考虑是否要支持多路径或并行处理，原生责任链是线性的。有些框架像 Netty 的 ChannelPipeline，也是责任链，但更复杂，支持入站出站双向传播。

Java 中如何创建多线程？

创建多线程在 Java 里最常用的就两种方式：继承 Thread 类和实现 Runnable 接口。其实本质上都是靠 Thread 对象来启动线程，只是任务定义的方式不同。

1) 直接 new Thread，重写 run 方法

这种方式简单直观，但一般不推荐，因为 Java 不支持多继承，你继承了 Thread 就没法再继承别的类了。

```
new Thread() {  
    public void run() {  
        System.out.println("我在子线程运行");  
    }  
}.start();
```

2) 实现 Runnable 接口，把任务传给 Thread

更灵活，解耦了任务和线程本身。你现在写的任务类可以交给 Thread，也可以交给线程池，比如提交给 **ThreadPoolExecutor**。

```
Runnable task = () -> System.out.println("通过 Runnable 执行");  
new Thread(task).start();
```

3) 用 Callable + FutureTask 拿返回值

如果需要线程执行完后返回结果，就得用 Callable。它能抛异常、有返回值，配合 FutureTask 使用。

```
Callable<Integer> call = () -> {  
    return 42;  
};  
FutureTask<Integer> future = new FutureTask<>(call);  
new Thread(future).start();  
Integer result = future.get(); // 阻塞等待结果
```

现在实际开发中，几乎不会手动 new Thread。高并发场景都用线程池，比如用 Executors 工厂类创建，背后其实是封装了 **ThreadPoolExecutor**，能复用线程、控制资源，避免频繁创建线程把系统搞崩。

什么是 MyBatis-Plus？它有什么作用？它和 MyBatis 有哪些区别？

MyBatis-Plus 是在 MyBatis 基础上封装的增强工具，目标是简化开发、减少模板代码。你用过 MyBatis 就知道，增删改查写 XML 太重复，MyBatis-Plus 直接帮你把单表操作全包了，基本不用写 SQL。

它最核心的功能是 **自动 CRUD** 和 **条件构造器**。比如实体类配上注解，Mapper 继承 BaseMapper，立刻拥有 insert、deleteById、selectList 等方法，连分页都能自动处理。

1) 不需要再为每个单表写 XML 映射，90% 的简单操作直接调用接口就行

2) 内置 QueryWrapper 构建查询条件，拼接 WHERE、LIKE、IN 都是链式调用，避免手写字符串 SQL

3) 支持逻辑删除、自动填充创建时间/更新时间等常见需求，配置一下就能用

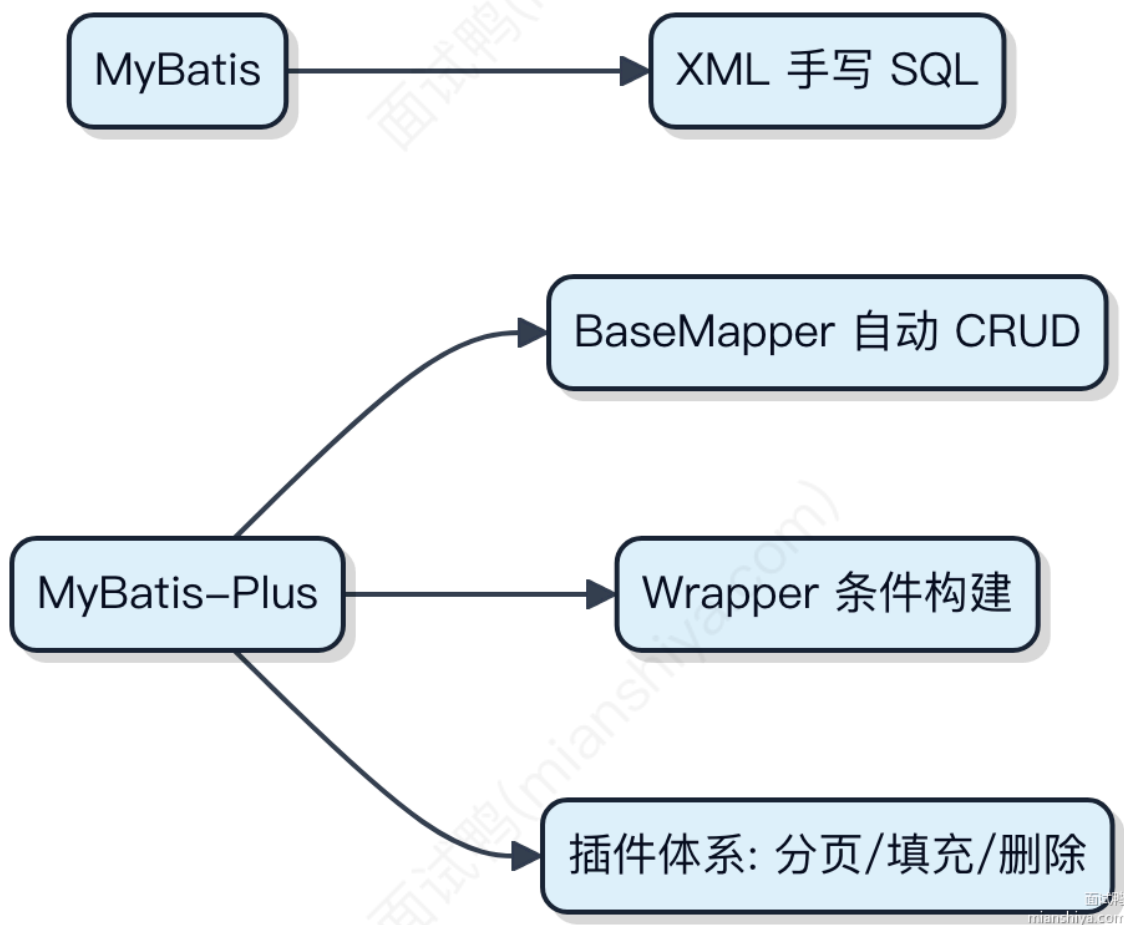
```
QueryWrapper<User> qw = new QueryWrapper<>();  
qw.eq("status", 1).like("name", "张");  
userMapper.selectList(qw);
```

和 MyBatis 的区别其实就两点：

一是开发效率，MyBatis-Plus 把通用操作抽成公共实现，你只专注业务复杂 SQL；

二是功能扩展，像分页插件 PageHelper 要配合使用，而 MP 内置分页、SQL 性能分析、租户插件等。

但复杂多表关联、存储过程还是得回退到 XML 或原生方式，毕竟它只是增强，不是替代。如果你项目里大量单表操作，用 MP 能省下不少脏活累活。



如何设计一个点赞系统？

设计点赞系统，关键是在数据一致性、性能和扩展性之间找平衡。用户点赞这种操作并发高、写多读少，直接怼数据库肯定扛不住。

一般用分层架构来消化压力。前端写请求先打到消息队列，比如 Kafka 或 RocketMQ，把瞬时流量削峰填谷。这样后端服务不会被突发的点赞洪流冲垮，系统更稳。

数据存储通常分两层。热数据用 Redis 存，比如用 Set 或 Sorted Set 记录用户对某内容的点赞状态，查得快、写得也快。Redis 里可以按内容 ID 聚合点赞用户，也可以加个计数器用 INCR 维护总数。定时任务再把 Redis 的变更异步落库，MySQL 或 TiDB 都行，保证最终一致。

1) 用户点赞先写消息队列，响应立刻返回 2) 消费者从队列拉数据，更新 Redis 中的点赞集合和计数 3) 后台任务周期性将 Redis 数据合并到数据库



有个细节是去重。同一个用户不能重复点，Redis 的 Set 天然支持唯一性，SADD 操作直接搞定。取消点赞用 SREM 就行，干净利落。

如果内容热度差异大，热门内容的计数器可能成为热点 key。可以分片计数，比如用 `user_id % 16` 分 16 个 key，查的时候再汇总。这种设计在微博、小红书这类产品里都验证过，能撑住千万级互动。

数组和链表在 Java 中的区别是什么？

数组和链表是两种基础的数据结构，在 Java 里的表现和适用场景差异挺大，关键得看你怎么用。

- 1) 内存布局上，数组在堆里申请一块连续的内存空间，大小固定，初始化时就定下来。你声明一个 `int[1000]`，JVM 就得一口气找 1000 个连续的 int 位置，中间不能断。链表不一样，每个节点 Node 分散在堆里，靠 next 指针连起来，增删节点就是改指针，不用找连续空间。
- 2) 访问效率，数组支持随机访问，`get(i)` 是 $O(1)$ ，直接通过 `base address + offset` 算出位置。链表必须从头开始遍历，`get(i)` 最坏要 $O(n)$ 。
- 3) 插入删除，数组中间插一个数，后面所有元素都得往后挪，代价是 $O(n)$ 。链表只要改前后节点的指针， $O(1)$ 搞定，前提是已经定位到位置。所以像 LinkedList 在频繁增删的场景比 ArrayList 强。

```
// ArrayList 底层是数组
List<Integer> list = new ArrayList<>();
list.add(0, 1); // 后面元素全后移

// LinkedList 底层是双向链表
List<Integer> linked = new LinkedList<>();
linked.add(0, 1); // 只改指针
```

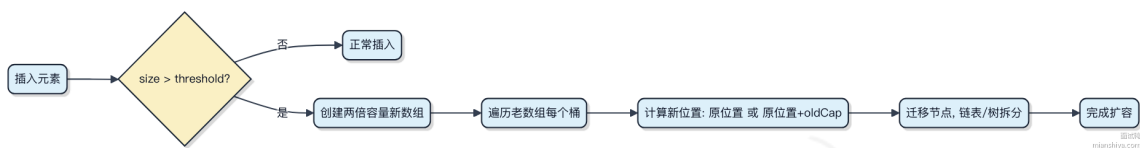
- 4) 空间开销，数组只存数据，内存紧凑。链表每个节点得多存 1-2 个引用 (next、prev)，空间利用率低一些。

Java 中 HashMap 的扩容机制是怎样的？

HashMap 扩容发生在元素数量超过阈值 (threshold) 时，这个阈值是容量 (capacity) 乘以加载因子 (loadFactor)，默认是 0.75。比如初始容量 16，阈值就是 $16 * 0.75 = 12$ 。一旦元素数超过 12，就会触发扩容。

扩容不是简单地把数组扩大一倍，而是创建一个新数组，长度翻倍，然后把老数据全部 rehash 搬到新桶里。这里有个优化点：因为容量总是 2 的幂，所以元素在新数组中的位置要么是原位置，要么是原位置加旧容量。JDK 8 利用这一点，通过位运算直接判断，避免重复计算 hash。

链表节点在迁移时会根据 `hash & oldCap` 的结果决定去向，等于 0 的留在原索引，不等于 0 的移到原索引 + 旧容量的位置。红黑树节点也类似，但会先拆分树，再判断是否需要转回链表。



注意扩容是**懒触发**的，只在 put 后检查时发生。多线程下并发 put 可能导致数据覆盖或链表成环，所以 HashMap 不是线程安全的。如果担心性能，可以预设足够大的容量，减少扩容次数。ConcurrentHashMap 在这方面做了优化，扩容时支持并发读写，用的是**渐进式 rehash**。

为什么 HashMap 在 Java 中扩容时采用 2 的 n 次方倍？

HashMap 的容量设计成 2 的 n 次方，主要是为了**快速计算元素的存储位置**。数组下标是通过 `hash & (capacity - 1)` 算出来的，这个位运算只有在容量为 2^n 时才等价于取模 `%`。

如果容量是普通数字，就得用取模运算 `hash % capacity`，这比位运算慢得多。而当 capacity 是 16、32、64 这种数时，`capacity - 1` 就是 15 (0b1111)、31 (0b11111) 这种低位全 1 的二进制数，& 运算天然就能截断高位，实现均匀分布。

扩容翻倍也保证了这个特性一直成立。比如从 16 扩到 32，还是 2 的幂，下标计算方式不变，迁移数据时也能用 `hash & oldCapacity` 判断是否需要移动。

还有一个隐藏好处：JDK 8 的树化阈值判断和红黑树拆分逻辑，都依赖这个均匀分布特性来保证性能稳定。要是随便设个 100 当初始容量，不仅取模慢，还容易导致链表过长，查询退化成 $O(n)$ 。

1) 必须是 2 的幂才能用 & 替代 % 2) 扩容翻倍维持了该性质连续性 3) 配合扰动函数减少哈希冲突

为什么 Java 中 HashMap 的默认负载因子是 0.75？

负载因子 (load factor) 决定了 HashMap 什么时候扩容。默认值 0.75 是空间和时间成本之间的一个权衡结果。

- 1) 如果负载因子太小，比如 0.5，那数组还没装满就触发扩容，内存利用率低，但冲突少，查找快。
- 2) 如果负载因子太大，比如 0.9，内存用得更多，但哈希冲突概率显著上升，链表变长，查找性能下降。

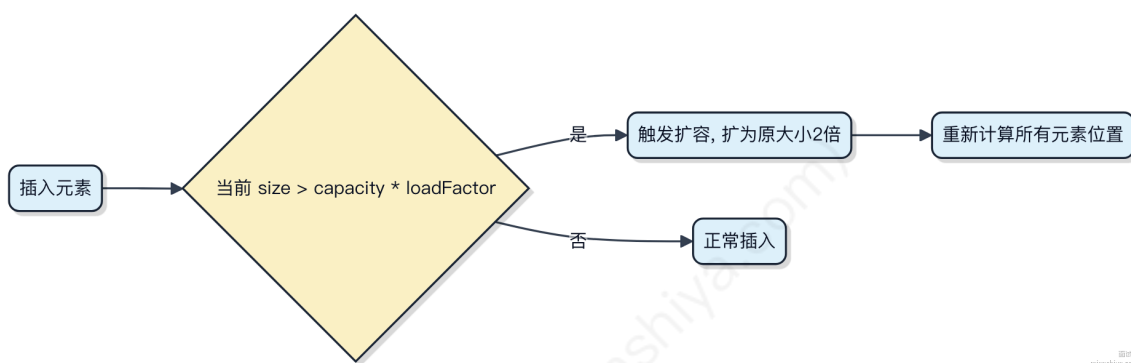
HashMap 的设计目标是在**平均情况下**实现 $O(1)$ 的存取效率。0.75 这个值能让哈希桶的填充程度适中，在不浪费太多空间的前提下，把冲突控制在可接受范围。

JDK 源码注释里也提过，基于泊松分布统计，当负载因子为 0.75 时，一个桶里链长度超过 8 的概率极低，这 and 红黑树转换阈值的设计也能对上。

代码上看，初始化时如果不指定，就是 0.75：

```
public HashMap() {
    this.loadFactor = 0.75f;
}
```

实际开发中，如果你知道数据量会很大且稳定，可以提前算好容量，避免频繁 rehash。比如要存 1000 个元素，按 0.75 算，初始容量至少设成 $1000 / 0.75 \approx 1333$ ，再向上取最接近的 2 的幂，也就是 2048。



为什么 JDK 1.8 对 HashMap 进行了红黑树的改动？

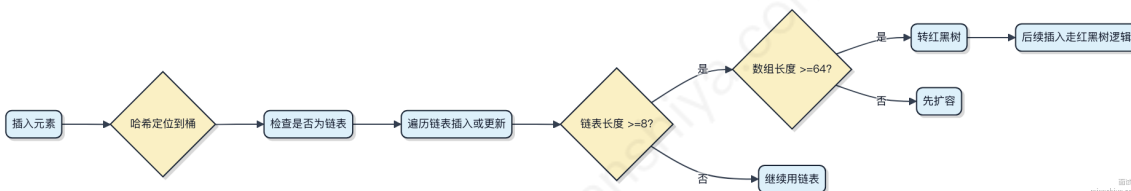
JDK 1.8 对 HashMap 的优化，主要解决哈希冲突严重时的性能退化问题。当大量元素落在同一个桶时，链表会变得很长，查找从 $O(1)$ 退化到 $O(n)$ 。为控制最坏情况的时间复杂度，引入了红黑树机制。

具体触发条件是：某个桶内的节点数达到 8，并且当前数组长度大于等于 64，链表就会转成红黑树。这样最坏查找性能能稳定在 $O(\log n)$ 。反过来，如果红黑树节点减少到 6，又会退化回链表，避免维护红黑树的额外开销。

这个设计其实是空间和时间的权衡。链表占用小，但查询慢；红黑树查询快，结构更复杂。阈值选 8 和 6 是为了防止频繁来回转换，用了泊松分布做依据，碰撞达到 8 的概率已经极低。

常见误区是认为只要 put 就可能转红黑树，其实前提是**扩容也解决不了散列问题**。比如所有 key 都只落在少数几个桶里，扩容后还是冲突，这时候链表长度才会持续增长。

代码上你不需要感知这个过程，HashMap 内部自动切换。但如果你的 key 没有合理实现 hashCode()，比如总返回 1，那所有元素都挤在一个桶，性能立马崩成 $O(n)$ 。



JDK 1.8 对 HashMap 除了红黑树还进行了哪些改动？

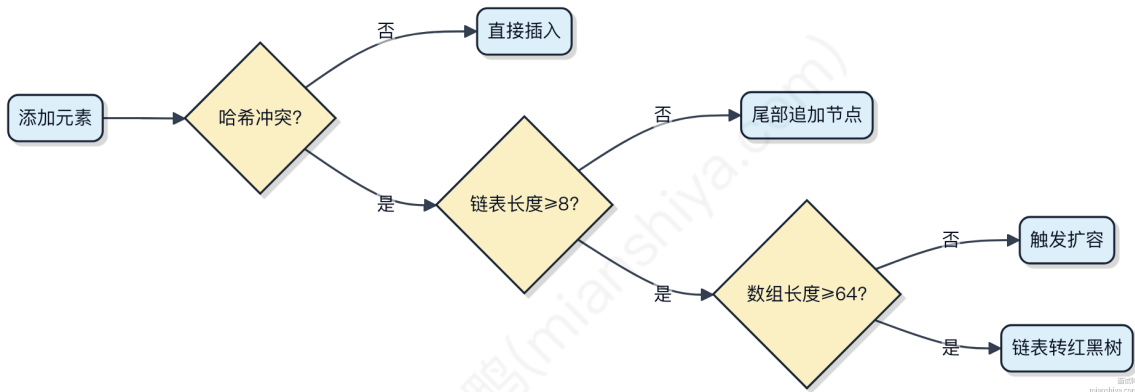
JDK 1.8 对 HashMap 的优化不止是引入红黑树这么简单，还有几个关键调整让它的性能更稳。

链表转红黑树的条件是 **bin 中节点数 ≥ 8 且 数组长度 ≥ 64** 。如果只是链表长但数组太小，会优先扩容，而不是直接建树。这样避免在哈希冲突还没真正严重时就引入复杂的树结构，毕竟树的开销比链表大。

另一个容易被忽略的点是 hash 计算方式。JDK 1.8 仍然用 $(n - 1) \& \text{hash}$ 定位桶，但对 key 的 hashCode 做了**二次扰动**：先异或低 16 位和高 16 位，减少低位碰撞概率。这招在 key 的 hashCode 分布不均时特别有用，比如只递增的 Integer。

扩容机制也有变化。老版本是头插法，多线程下可能形成环，导致死循环。1.8 改成尾插法，迁移时节点顺序不变，彻底避开这个问题。而且扩容后的新位置要么在原索引，要么在原索引 + oldCap 位置，计算也快。

代码上看插入逻辑，核心还是 `putVal` 方法，但处理树化和拆分树的分支明显变多了。尤其是 `treeifyBin` 里会先看数组大小，不够就不树化。



这些改动合起来，让 `HashMap` 在高冲突场景下从 $O(n)$ 降到 $O(\log n)$ ，同时日常使用也没增加额外负担。

Java 中 `ConcurrentHashMap` 1.7 和 1.8 之间有哪些区别?

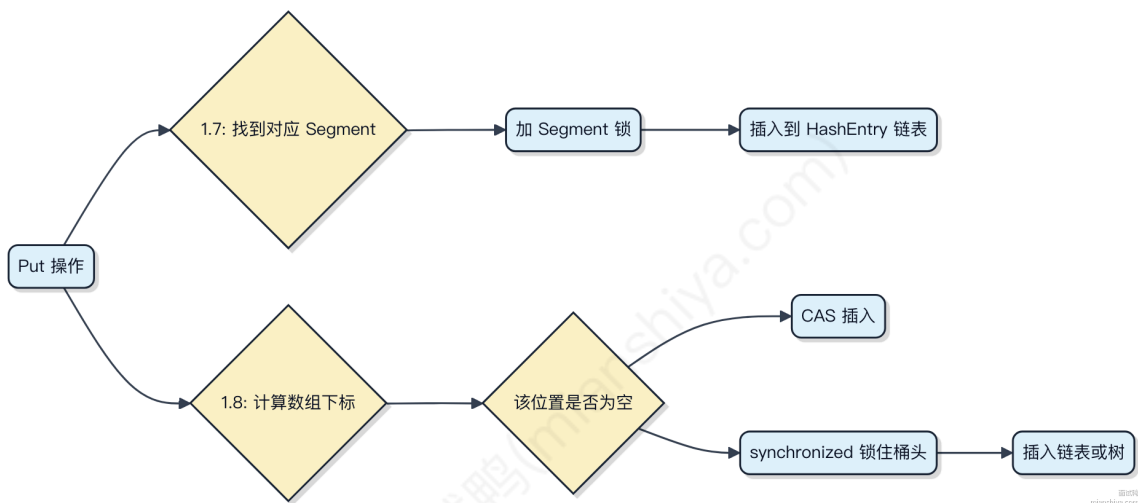
`ConcurrentHashMap` 的演进其实是个典型的“从锁竞争到 `CAS` + `synchronized` 优化”的过程，1.7 到 1.8 的改动几乎是重写级别。

1.7 用的是分段锁 `Segment`，每个 `Segment` 是一个 `ReentrantLock` 保护的小哈希表。默认 16 个 `Segment`，意味着最多 16 个线程能并发写。一旦线程数超过这个数，后面的就只能阻塞等待锁。`Segment` 继承自 `ReentrantLock`，结构复杂，内存占用也高。

到了 1.8，直接抛弃了 `Segment`，改用 **`CAS` + `synchronized`** 控制并发。`Node` 数组加链表/红黑树，写操作对数组桶位加 `synchronized` 锁，粒度更细。读操作完全无锁，靠 `volatile` 保证可见性。扩容时还能支持并发迁移，由多个线程一起搬数据。

另一个关键变化是哈希冲突的处理。1.7 是纯链表，容易在极端情况下退化成 $O(n)$ 。1.8 引入了链表转红黑树机制，当链表长度超过 8 且数组长度大于 64，就转换成红黑树，查找性能从 $O(n)$ 提升到 $O(\log n)$ ，抗攻击能力更强。

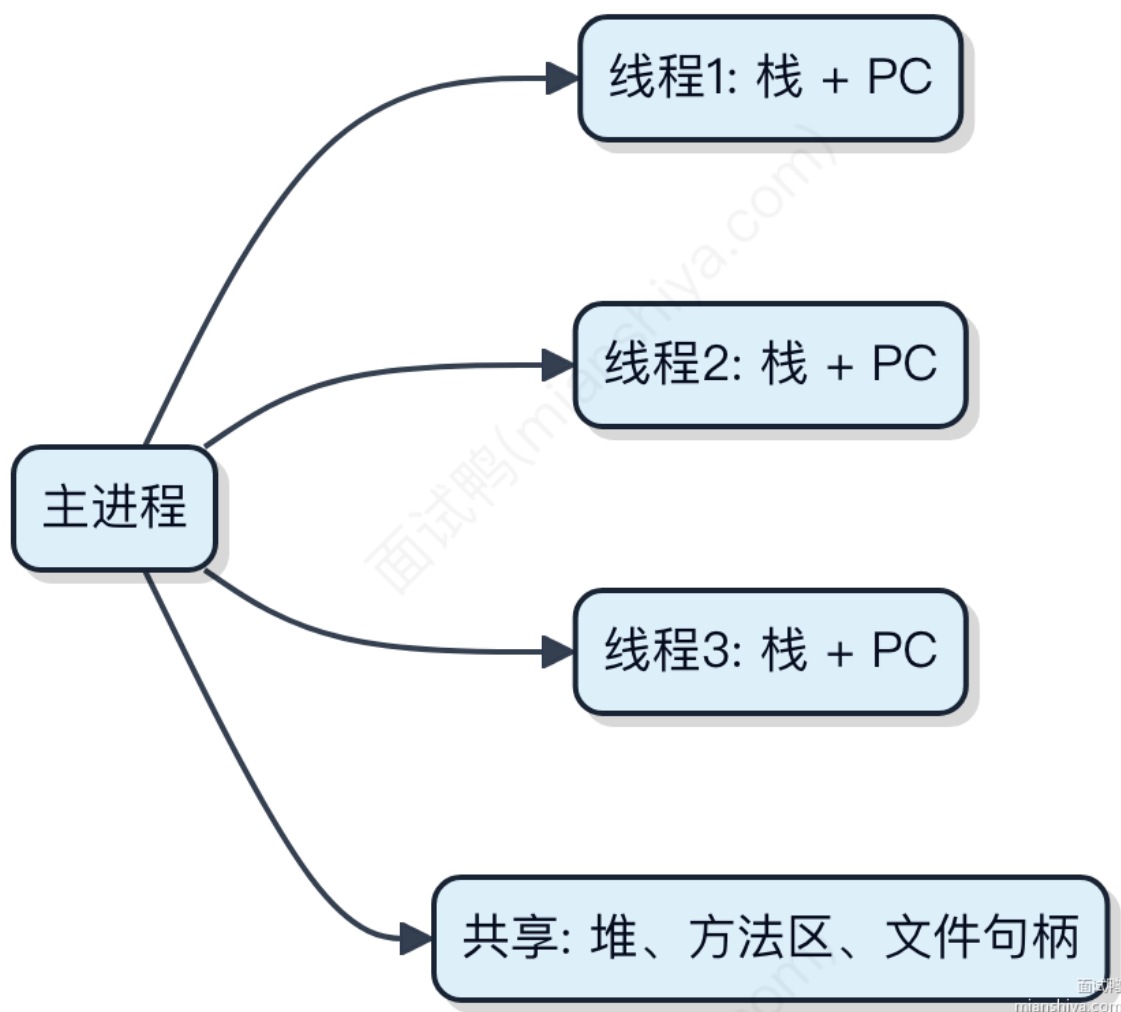
扩容效率也提升了。1.8 用了“多线程协助扩容”机制，一个线程发现需要扩容，可以只负责搬一部分桶，其他线程写数据时也会顺手帮忙搬，真正做到并发扩容。



线程和进程有什么区别？

进程是资源分配的基本单位，线程是CPU调度的基本单位。一个进程可以包含多个线程，这些线程共享进程的内存空间和文件描述符等资源，但每个线程有自己独立的程序计数器、栈和寄存器状态。

- 1) 进程之间相互隔离，一个进程崩溃通常不会直接影响另一个进程。比如你运行两个Java应用，它们分别在不同的JVM进程中，一个OOM了，另一个照常运行。
- 2) 线程共享所属进程的堆内存，所以多线程编程要注意数据竞争问题。像HashMap这种非线程安全的结构，多个线程同时写就会出问题，得用ConcurrentHashMap或者加锁来保护。
- 3) 创建进程开销大，因为要分配独立的地址空间，还要做页表映射。Linux下fork()虽然用了写时复制优化，但依旧比创建线程贵得多。Java里new Thread()底层对应pthread_create，在同一个进程内启动新执行流，代价小很多。
- 4) 线程上下文切换成本低于进程。进程切换时CPU要把整个地址空间的映射都换掉，而线程切换只刷新少量寄存器，TLB也可能不用全清。



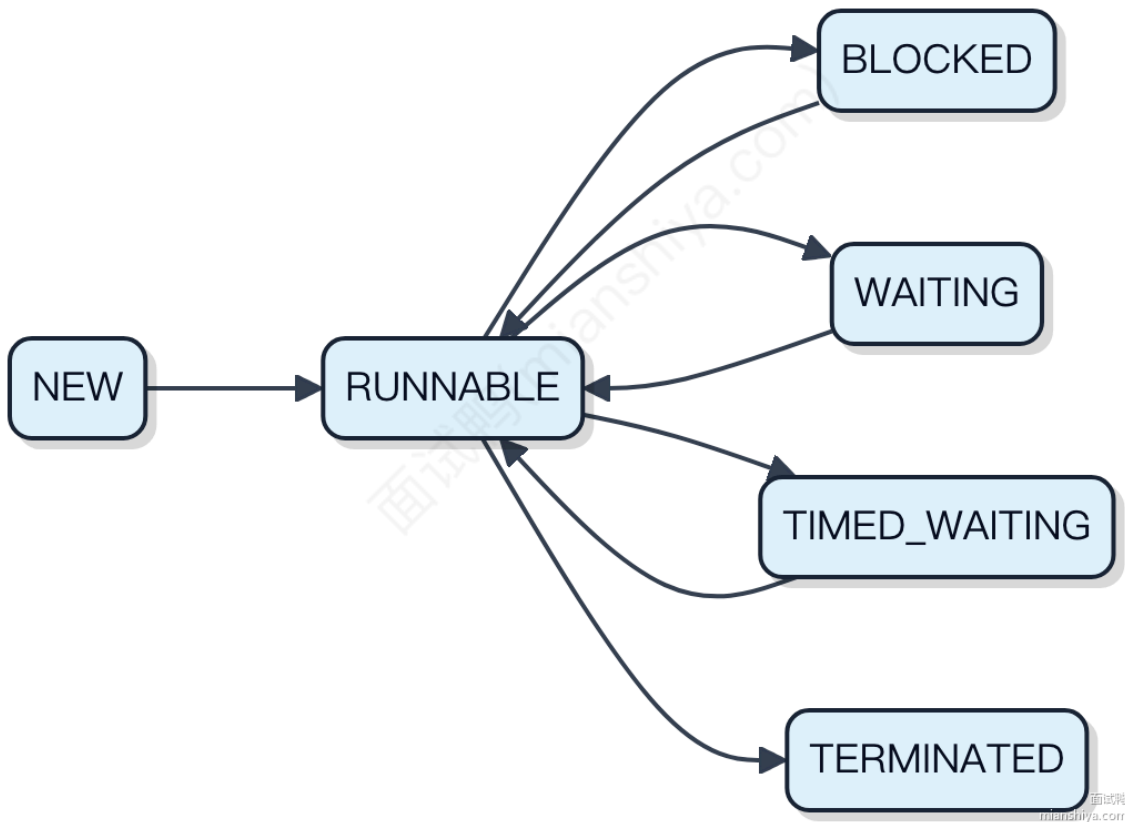
多线程适合高并发场景，比如Tomcat用线程池处理HTTP请求，每个请求一个线程；而多进程更适合隔离性要求高的场合，比如浏览器为每个标签页启动独立进程防崩溃。

线程的生命周期在 Java 中是如何定义的？

线程在 Java 中的生命周期由 `Thread.State` 枚举定义，总共包含 6 种状态，它们反映了线程从创建到终止的完整过程。

- 1) **新建 (NEW)**: 线程对象已创建，但还没调用 `start()` 方法，此时线程还没进入运行状态。
- 2) **可运行 (RUNNABLE)**: 调用了 `start()` 后，线程进入这个状态。它可能正在运行，也可能在等待 CPU 调度，JVM 不区分就绪和运行中。
- 3) **阻塞 (BLOCKED)**: 线程试图获取一个被其他线程持有的监视器锁 (synchronized 块/方法)，就会进入阻塞状态，直到锁释放。
- 4) **等待 (WAITING)**: 线程调用了 `wait()`、`join()` 或 `LockSupport.park()`，会无限期待，直到被其他线程显式唤醒。
- 5) **限期等待 (TIMED_WAITING)**: 调用带超时参数的方法，比如 `sleep(long)`、`wait(long)`、`join(long)` 或 `parkNanos`，会在指定时间后自动恢复。

6) **终止 (TERMINATED)**：线程任务执行完毕，或者抛出未捕获异常，状态变为终止，此时线程不可复用。



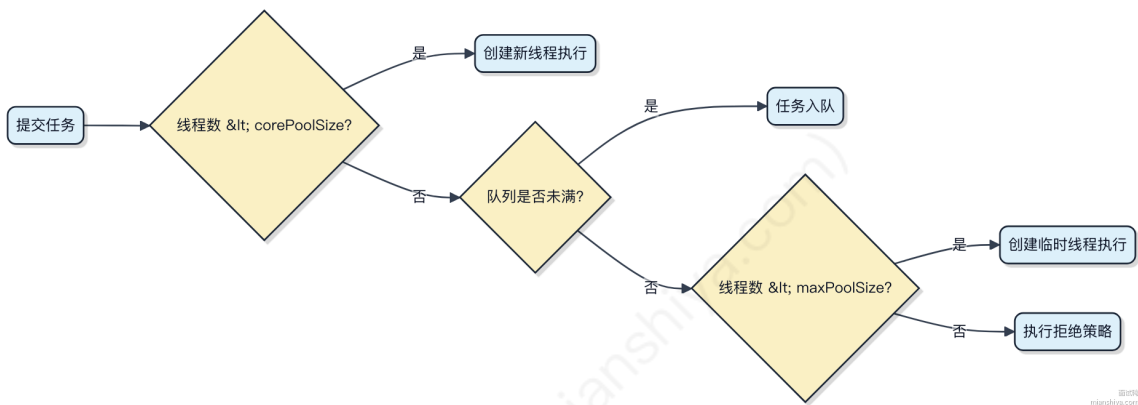
注意，操作系统层面的线程状态和 Java 层不完全一致，JVM 做了一层抽象映射。比如 **WAITING** 状态可能对应内核的休眠，但开发者只需关注 Java 定义的状态流转即可。

你了解 Java 线程池的原理吗？

线程池的核心是用少量线程处理大量任务，避免频繁创建销毁线程带来的系统开销。它通过一个阻塞队列暂存待执行任务，核心线程数内的线程会一直存活，超过核心线程数的线程在空闲时会被回收。

ThreadPoolExecutor 的工作流程是这样的：

- 1) 提交任务时，如果当前线程数小于核心线程数，直接创建新线程执行任务，哪怕有空闲线程也一样。
- 2) 如果线程数已达核心线程数，任务会被放入阻塞队列等待。
- 3) 如果队列满了，且线程数小于最大线程数，就会创建临时线程执行任务。
- 4) 如果队列满且线程数达到最大值，就会触发拒绝策略，默认是抛出 `RejectedExecutionException`。



常见的线程池类型中，**FixedThreadPool** 使用固定大小线程和无界队列，容易导致内存溢出；**CachedThreadPool** 允许创建无限线程，适合短任务但可能压垮系统；**ScheduledThreadPool** 支持定时和周期任务，类似 Timer 但更强大。

实际项目里，Dubbo 和 Tomcat 都自定义了线程池策略。比如 Dubbo 会根据业务场景调整队列长度和拒绝策略，防止雪崩。自己用的时候别直接用 Executors 工厂方法，一定要显式构造 ThreadPoolExecutor，把参数设清楚，不然线上容易搞不定。

如何合理地设置 Java 线程池的线程数？

设置线程数不是拍脑袋决定的，得看任务类型。CPU 密集型和 IO 密集型的策略完全不一样。

CPU 密集型任务，比如大量计算、数据处理，线程太多反而会因为上下文切换拖慢整体性能。这种场景下，线程数一般设为 CPU 核心数 + 1 就够了。JVM 可以通过 `Runtime.getRuntime().availableProcessors()` 拿到核心数。

IO 密集型就不同了，像读写文件、网络请求、数据库操作，线程经常在等 IO 返回，这时候 CPU 是空闲的。为了**压榨 CPU 利用率**，就得开更多线程去“占坑”。常见公式是 $\text{线程数} = \text{CPU核心数} * (1 + \text{平均等待时间} / \text{平均工作时间})$ 。如果没法精确测算，一个经验做法是设成 $\text{CPU核心数} * 2$ 或更高。

```
// 示例：基于 CPU 核心数动态设置
int corePoolSize = Runtime.getRuntime().availableProcessors() * 2;
ExecutorService executor = Executors.newFixedThreadPool(corePoolSize);
```

别直接用 Executors 工厂类创建线程池，容易搞出大问题。比如 `newFixedThreadPool` 用的是无界队列，任务堆积可能 OOM。应该用 `ThreadPoolExecutor` 手动传参，把核心参数都控制住。

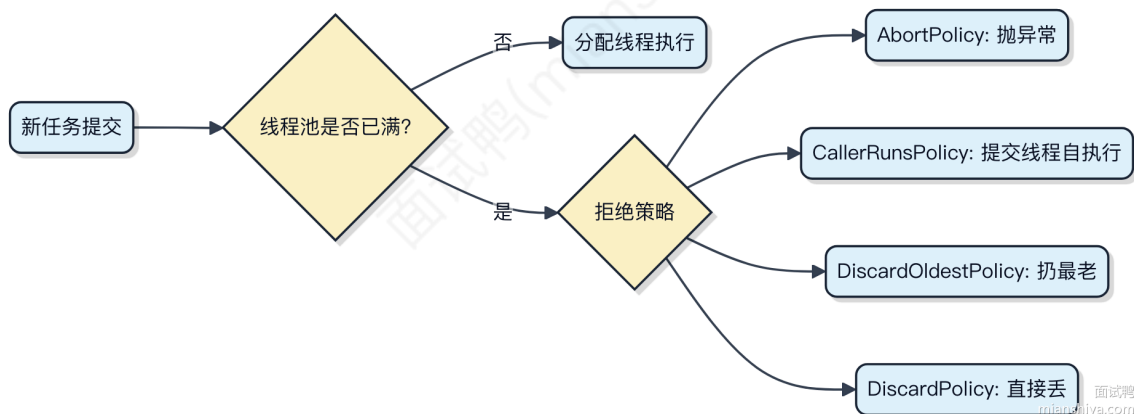
还有种情况是混合型任务，建议拆分，用不同的线程池隔离。比如用 `ForkJoinPool` 处理并行计算，用自定义线程池跑 HTTP 调用，避免互相影响。

Java 线程池有哪些拒绝策略？

线程池的拒绝策略，说白了就是任务太多，队列也满了，新来的任务怎么处理。JDK 里内置了四种标准方式，每种应对不同的业务需求。

1) 直接抛异常。用 `AbortPolicy` 的时候，一旦线程池饱和，新任务进来立刻抛出 `RejectedExecutionException`。适合那种不能丢任务、也不能阻塞的场景，比如核心交易下单，扛不住就让调用方立刻知道。

- 2) 谁提交谁处理。CallerRunsPolicy 比较特别，它不会另起线程，而是让提交任务的线程自己去执行这个任务。相当于“你送过来我忙不过来，那你亲自干吧”。这种能减缓任务提交速度，常用于负载较高的服务端，配合 Netty 或 Tomcat 使用效果不错。
- 3) 丢弃最老的任务。DiscardOldestPolicy 会把队列里待得最久的那个任务扔掉，腾位置给新任务。前提是队列得支持优先级或有序性，比如用 LinkedBlockingQueue 就行。适用于实时性要求高、老任务不如新任务重要的场景，像监控数据采集。
- 4) 默默丢弃。DiscardPolicy 最干脆，新任务来了啥也不做，直接丢掉。适合那些允许丢失任务的场景，比如日志上报、埋点数据，丢了几次影响不大。



实际项目中，如果这四种不够用，也可以自定义，实现 `RejectedExecutionHandler` 接口就行。比如记录日志、写入死信队列，防止任务彻底丢失。

你使用过哪些 Java 并发工具类？

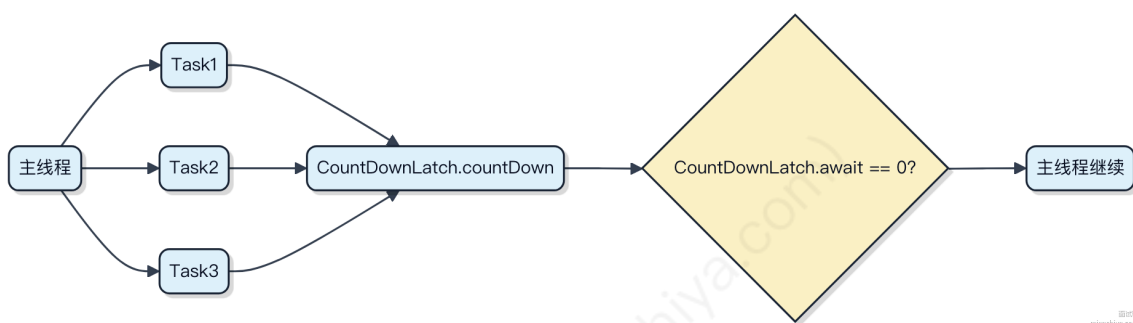
Java 并发包里的工具类，用得最多的是 **JUC** 这一块。日常开发里，几乎离不开这几个场景：线程池管理、并发控制、线程协作。

比如处理批量订单导入，一般会用 `ThreadPoolExecutor` 自定义线程池，核心线程数根据 CPU 核心来定，IO 密集型任务通常设为 $2 * \text{CPU 数}$ 。拒绝策略选 `RejectedExecutionHandler` 的时候，用 `CallerRunsPolicy` 能让主线程自己执行，避免雪崩。

要控制并发读写，`ReentrantReadWriteLock` 就很实用。像缓存系统这种读多写少的场景，多个读线程可以同时进入，写的时候才独占，性能比直接用 `synchronized` 高不少。

信号量 `Semaphore` 适合做限流。假设调用第三方支付接口，QPS 上限是 100，那就初始化一个 permit 数为 100 的 `Semaphore`，每次请求前 `acquire`，结束后 `release`，简单又有效。

还有 `CountDownLatch` 和 `CyclicBarrier`，虽然都用来协调线程，但用途不一样。前者是一等多，比如主线程等 3 个加载任务都完成再启动服务；后者是多等一，所有线程互相等待，到齐了才继续，像并发压测时统一发压。



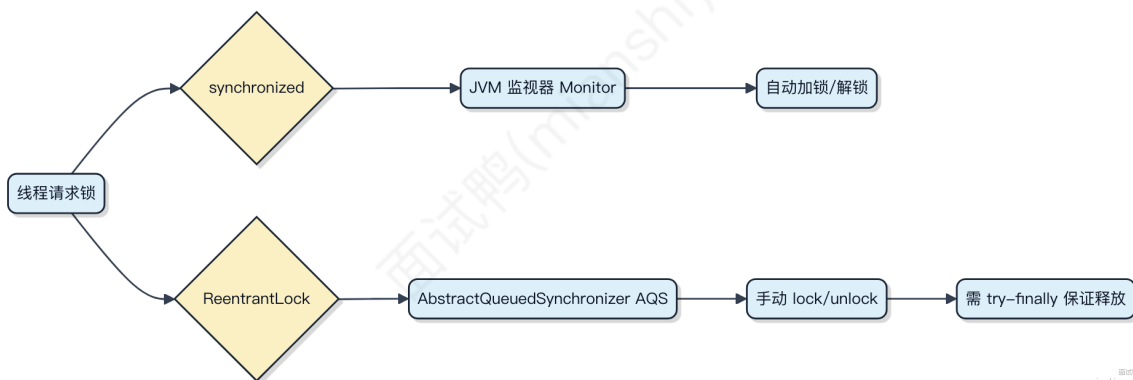
`ConcurrentHashMap` 更是高频，缓存计数、共享状态存储都靠它。和 `Collections.synchronizedMap` 不同，它分段锁，1.8 后直接用 CAS + synchronized 控制桶，冲突少，吞吐高。

这些工具类组合起来，基本能搞定大多数并发场景的脏活累活。

Synchronized 和 ReentrantLock 有什么区别？

`synchronized` 是 JVM 层面实现的互斥锁，代码进入同步块时自动加锁，退出时自动释放，哪怕抛异常也不会死锁。`ReentrantLock` 是 JDK 提供的 API 级别可重入锁，必须手动调用 `lock()` 和 `unlock()`，通常要配合 `try-finally` 使用。

- 1) 等待可中断：`ReentrantLock` 能响应中断，比如线程 A 拿着锁不放，B 等太久可以主动中断放弃，避免无限等下去。`synchronized` 只能傻等。
- 2) 公平锁支持：`ReentrantLock` 可以通过构造函数指定是否使用公平模式，按排队顺序获取锁。`synchronized` 始终是非公平的，谁抢到算谁的。
- 3) 条件等待：`ReentrantLock` 配合 `Condition` 接口能实现更灵活的线程通信，比如生产者消费者模型里可以分别控制等待和唤醒。`synchronized` 只能靠 `wait/notify`，灵活性差一些。
- 4) 性能上早期差距明显，但自从 `synchronized` 加了偏向锁、轻量级锁优化后，大多数场景下性能已经和 `ReentrantLock` 差不多，甚至更好。

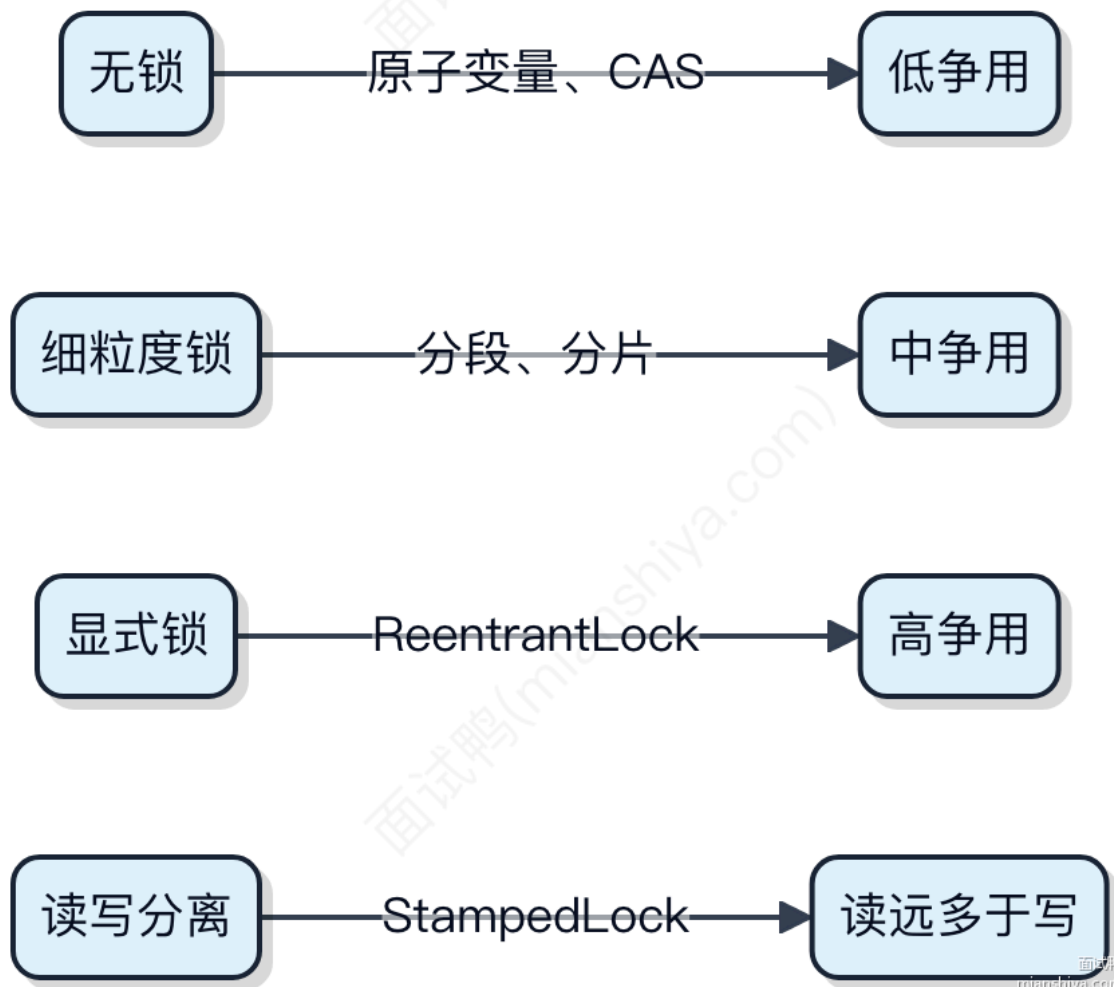


一般业务开发优先用 `synchronized`，足够简单安全。只有在需要超时尝试、可中断、公平锁这些高级特性时，才选 `ReentrantLock`。

如何优化 Java 中的锁的使用？

锁这玩意儿用不好就是性能瓶颈，尤其在高并发场景下。优化的关键是减少争用、降低开销，而不是一上来就 synchronized 啥都包住。

- 1) 能不用锁就不用，优先考虑无锁结构。比如计数用 `AtomicInteger`，状态机切换用 `AtomicReference`，这些底层靠 CAS 实现，比 `synchronized` 轻太多。像 Kafka、Netty 里大量用原子类扛流量，压根不经过内核态。
- 2) 缩小锁粒度，别一把锁管全场。`ConcurrentHashMap` 就是个例子，它把数据分段，每个 segment 独立加锁，读写不同桶根本互不干扰。你也可以拆成多个锁对象，按业务维度隔离，比如按用户 ID 取模分片加锁。
- 3) 替换重锁。`synchronized` 在 JDK 1.6 后其实做了很多优化，偏向锁、轻量级锁能自旋抢，但竞争激烈时还是会膨胀成重量级锁。这时候用 `ReentrantLock` 更灵活，支持公平锁、可中断、超时获取，还能配合 `Condition` 实现精准通知。不过要注意 `finally` 里 `unlock`，漏了会死锁。
- 4) 读多写少用读写锁。`ReentrantReadWriteLock` 允许多个读线程并发，写线程独占。但要注意“写饥饿”，大量读请求可能让写线程一直抢不到锁。JDK 8 引入的 `StampedLock` 支持乐观读，性能更好，但使用更复杂，得小心版本校验。



说说 AQS 吧？

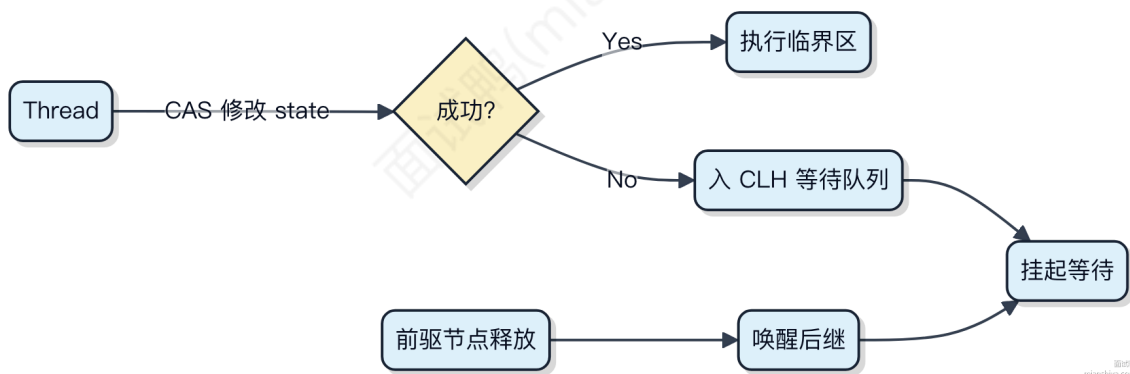
AQS，也就是 `AbstractQueuedSynchronizer`，是 Java 并发包里构建锁和同步器的底层基础。像 `ReentrantLock`、`Semaphore`、`CountDownLatch` 这些都是基于它实现的。

它的核心是用一个 `volatile` 修饰的 `int` 类型 `state` 来表示同步状态，比如加锁次数、信号量许可数。线程通过 CAS 去修改 `state`，改成功就拿到资源，改失败就进等待队列。

这个等待队列是个双向链表，也叫 CLH 队列，每个节点代表一个线程。没抢到锁的线程会被包装成 Node 入队，然后挂起。前面的线程释放资源后，会唤醒后继节点，实现公平调度。

AQS 把具体的获取和释放逻辑留给子类实现，比如 `tryAcquire` 和 `tryRelease`，自己只管排队和阻塞这些脏活累活。这种模板模式让开发者能方便地定制自己的同步组件。

- 1) 独占模式下，只有一个线程能拿到 `state`，比如 `ReentrantLock`
- 2) 共享模式下，多个线程可以同时拿到，比如 `Semaphore` 控制并发数
- 3) 支持公平和非公平策略，是否让新来的线程插队

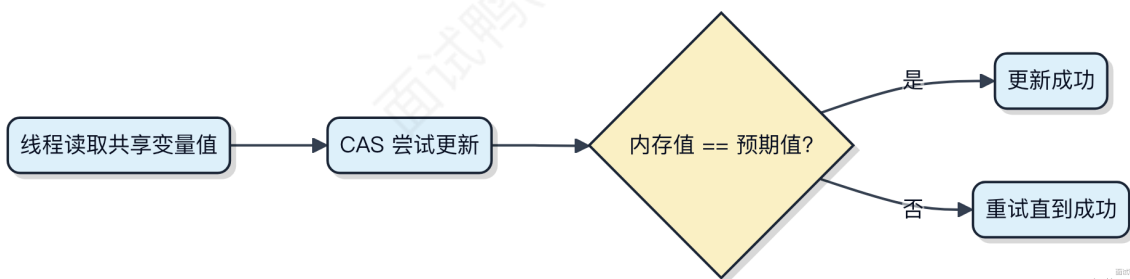


注意 AQS 不是万能的，如果场景只是简单计数或一次性事件，直接用 JDK 提供的现成工具更稳妥，没必要重复造轮子。

什么是 Java 的 CAS (Compare-And-Swap) 操作?

CAS (Compare-And-Swap) 是 Java 并发编程里实现原子操作的一种底层机制，它直接由 CPU 提供指令支持，比如 x86 的 `cmpxchg` 指令。这个操作包含三个参数：内存位置 `V`、预期旧值 `A` 和要更新的新值 `B`。只有当 `V` 位置的当前值等于 `A` 时，才会把 `V` 更新为 `B`，否则什么都不做。整个过程是原子的，不会被线程调度打断。

Java 里的 `Unsafe` 类封装了这些底层操作，而我们日常用的 `AtomicInteger`、`AtomicLong` 等原子类就是基于它实现的。比如调用 `incrementAndGet()`，底层其实就是不断用 CAS 尝试加 1，直到成功为止。



这种机制避免了传统锁的阻塞问题，适合**多读少写、竞争不激烈**的场景。但也有几个坑得注意。1) ABA 问题，就是值从 A 变成 B 又变回 A，CAS 会误判没变过，可以用 `AtomicStampedReference` 带版本号解决。2) 自旋开销，如果冲突频繁，线程会一直重试，CPU 占用可能飙高。3) 只能保证单个变量的原子性，多个变量就得上锁或者用 `synchronized`。

代码上很简单：

```
private AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet(); // 底层就是 CAS 自旋
```

所以你看，像 `ConcurrentHashMap`、`AQS` 这些高并发组件，底层都大量依赖 CAS 来扛住并发压力，干的都是无锁同步的脏活累活。

Java 中的 final 关键字是否能保证变量的可见性？

final 关键字在 Java 中不仅能防止变量被重新赋值，还能在多线程环境下**保证可见性**，但这依赖于正确的使用场景。

- 1) 对于引用类型，final 能确保对象初始化完成后，其他线程看到的都是完全构造好的实例。比如单例模式中用 `private final` 字段，配合构造完成后的发布，不需要额外同步就能安全共享。
- 2) Java 内存模型规定，final 字段的写操作不会被重排序到构造方法之外。这意味着一旦对象引用对其他线程可见，其所有 final 字段的值也一定可见。
- 3) 非 final 字段则没有这个保障。即使主线程构造完对象并发布引用，其他线程仍可能读到未初始化或部分初始化的字段值。

代码示例：

```
public class FinalFieldExample {
    final int x;
    int y;

    public FinalFieldExample(int x, int y) {
        this.x = x; // final 写
        this.y = y; // 普通写
    }
}
```

只要对象是正常构造（没有 this 逸出），x 的值对所有线程都可见且一致。

但注意，final 不适用于后续修改。如果想让非 final 字段也有可见性，得靠 `volatile` 或 `synchronized`。



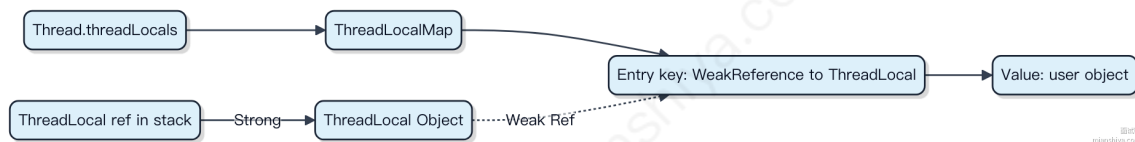
为什么 Java 中的 ThreadLocal 对 key 的引用为弱引用？

ThreadLocal 的 key 是弱引用，主要是为了解决内存泄漏的隐患。每个线程都有自己的 ThreadLocalMap，这个 map 的 key 就是 ThreadLocal 实例，value 是线程本地的值。如果这里用强引用，那么即使 ThreadLocal 实例在外部已经被置为 null，只要线程还没结束，map 里的 key 依然能被访问到，GC 就没法回收它，造成内存泄漏。

使用弱引用后，当外部没有强引用指向 ThreadLocal 时，下一次 GC 就能把 key 回收掉，这时候 key 变成 null，但 value 还在 map 里，这就是所谓的“脏 entry”。如果不处理，value 依然会累积。所以真正靠得住的做法是在调用 `remove()` 主动清理，像 Tomcat 复用线程池时就特别注意这点，不然可能几十个请求就搞不定内存。

虽然 key 是弱引用，但不能完全依赖它来避免内存泄漏。最佳实践是每次用完 ThreadLocal 都显式调用 remove()，尤其是在高并发、长生命周期的线程场景下，比如使用线程池时。

- 1) 弱引用让 key 可被回收，防止 ThreadLocal 实例本身泄漏
- 2) 但 value 不会被自动清理，必须手动 remove()
- 3) 不及时清理会导致 Entry 对象堆积，最终 OOM



不要以为用了弱引用就万事大吉，**弱引用**只是减轻了泄漏风险，真正的安全还得靠主动清理。

Java 中什么情况会导致死锁？如何避免？

死锁最经典的场景就是两个线程各自持有对方需要的锁，然后互相等。比如线程 A 拿了锁 1，想去拿锁 2；线程 B 拿了锁 2，却在等锁 1。结果谁也别想过去，卡死了。

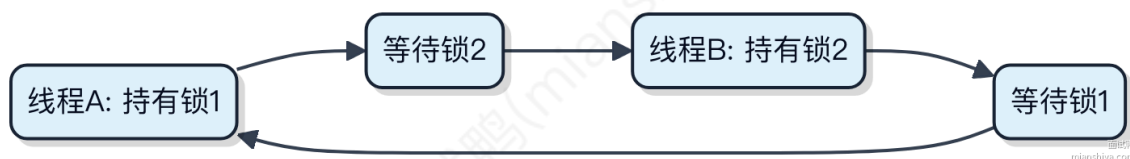
这种情况要发生，得同时满足四个条件：

- 1) 互斥：锁这东西本来就是排他的，同一时间只能一个线程用
- 2) 占有并等待：我已经拿着一个锁了，还等着另一个
- 3) 不可抢占：别人不能强行把我的锁拿走
- 4) 循环等待：A 等 B，B 等 C，C 又等 A，形成闭环

只要打破其中一个，死锁就搞不定你。

最常见的避免方式是**统一线程加锁顺序**。比如大家都约定先申请 ID 小的锁，再申请 ID 大的。这样就不会出现 A→B 而 B→A 的反向依赖。

还有一个办法是使用 tryLock 带超时，不傻等。比如用 ReentrantLock，最多等 5 秒，拿不到就释放已有资源，退回来重试或者报错，而不是死磕。



实际开发里，像数据库的行锁、Redis 分布式锁处理不当都容易出这问题。Spring 里事务方法嵌套调用，如果加锁顺序乱来，也可能中招。最稳妥的还是设计阶段就想清楚资源依赖关系，别让锁的获取路径打结。

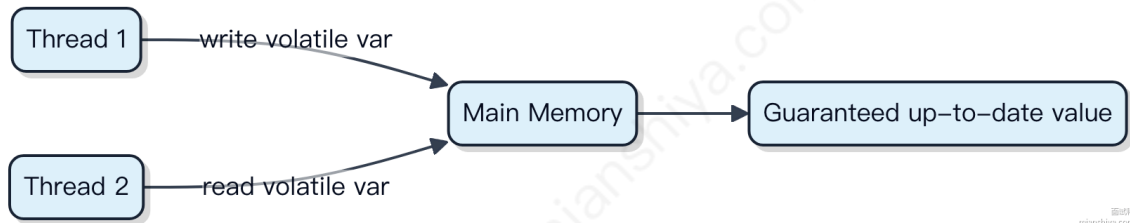
Java 中 volatile 关键字的作用是什么？

volatile 关键字主要解决多线程环境下的可见性和指令重排序问题。当一个变量被声明为 volatile，JVM 会保证这个变量的修改对所有线程立即可见，不会因为线程本地缓存导致读取到过期值。

- 1) 每次读取 volatile 变量都会直接从主内存加载，每次写入也会立刻刷新回主内存，绕开了线程的工作内存。这使得一个线程的修改能被其他线程及时感知。

2) 编译器和处理器不会对 volatile 写操作与其前面的读写操作进行重排序, 同样也不会对 volatile 读操作与其后面的读写操作重排。这种内存屏障机制保证了代码执行顺序与程序逻辑一致。

但要注意, volatile 不保证原子性。比如 `i++` 这种复合操作, 即使变量是 volatile 的, 依然可能出错。需要原子性时得用 synchronized 或 **AtomicInteger** 这类工具。



典型使用场景是标志位控制, 比如用 `volatile boolean running = true;` 来控制循环是否继续。像 Double-Checked Locking 模式中, 单例对象引用也必须用 volatile 修饰, 防止对象还没初始化完就被其他线程读到。

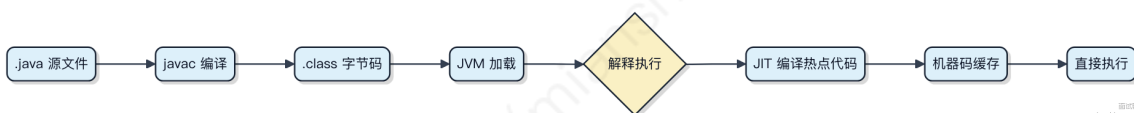
别拿它当 synchronized 用, 搞不定复杂同步逻辑。简单状态标记、一次性安全发布, 这些才是它的主战场。

编译执行与解释执行的区别是什么? JVM 使用哪种方式?

Java 代码的执行混合了编译和解释两种方式, 得从整个流程来看。

- 1) 源代码先由 `javac` 编译成字节码 (.class 文件), 这个阶段是**编译执行**的一部分。但这里的“编译”不是直接生成机器码, 而是生成一种中间形态的字节码, 平台无关。
- 2) 运行时 JVM 读取字节码, 再通过解释器逐行解释成机器指令执行。这就是**解释执行**, 启动快, 但执行慢, 因为每次都要翻译。
- 3) 为了提升性能, JVM 还引入了 **JIT (Just-In-Time) 编译器**。它会在运行时把热点代码 (频繁执行的方法) 直接编译成机器码缓存起来, 下次调用就不用解释了, 直接执行本地代码。

所以 JVM 实际上是“解释 + 编译”的混合模式。早期 HotSpot VM 就是靠这种策略兼顾了启动速度和运行效率。



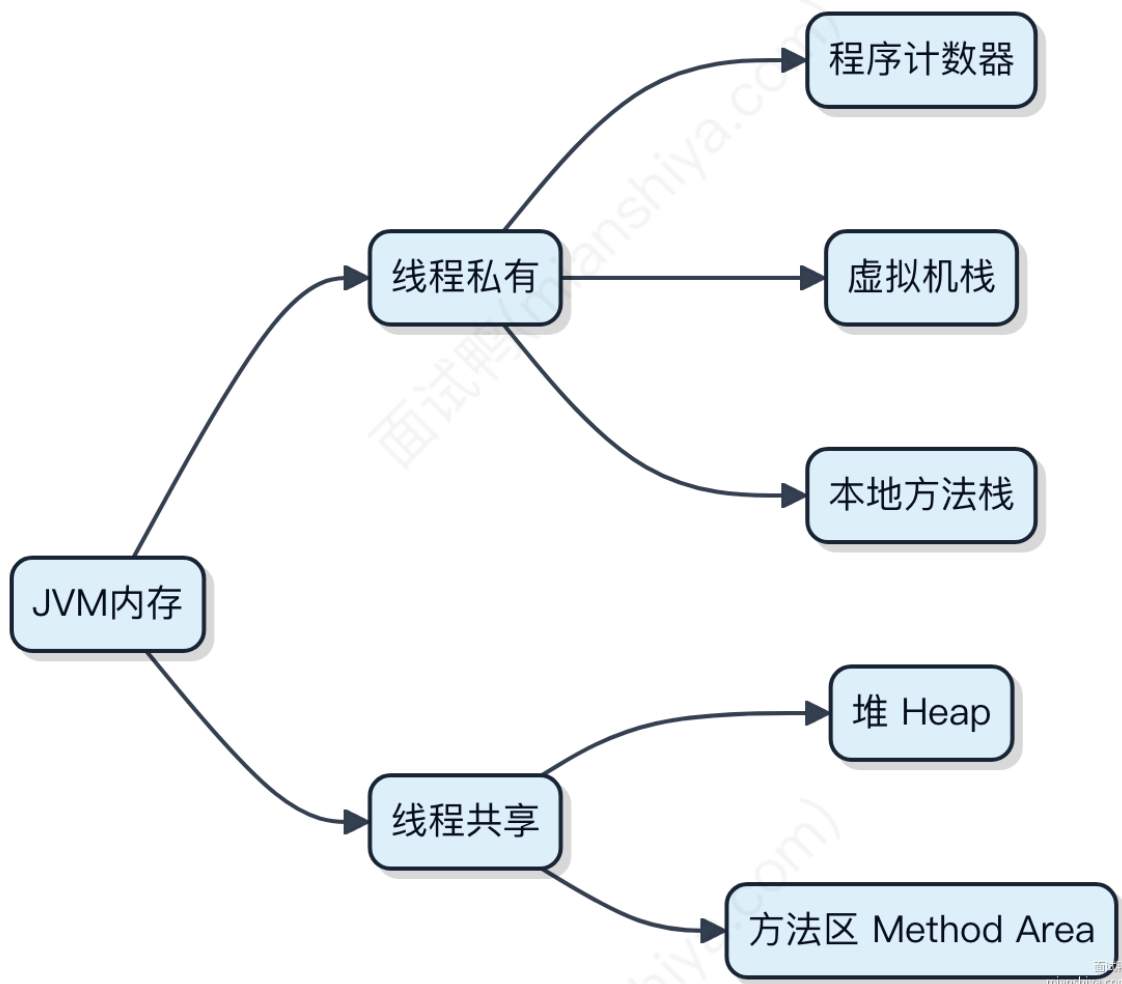
像 Spring Boot 应用刚启动时接口响应慢, 过一会儿变快, 就和 JIT 的预热过程有关。GraalVM 提供的 AOT (提前编译) 可以进一步优化这个问题, 把部分工作提前到构建阶段。

JVM 的内存区域是如何划分的?

JVM 内存区域划分, 其实是为了高效管理程序运行时的数据。不同数据生命周期不一样, 比如方法执行的局部变量和整个类的结构信息, 肯定不能混着放。

线程私有的部分包括 **程序计数器**、虚拟机栈和本地方法栈。程序计数器记录当前线程执行到哪条字节码指令了, 线程切换回来能接着执行。每个方法调用时都会创建一个栈帧压入虚拟机栈, 里面存了局部变量表、操作数栈、动态链接这些, 方法执行完就弹出。本地方法栈则是为 native 方法服务的。

线程共享的区域主要是 **堆** 和 **方法区**。堆是最大的一块，几乎所有对象实例都在这里分配内存。GC 主要就是管这块区域，像 G1、ZGC 这些垃圾回收器优化的也是它。方法区存放类信息、常量、静态变量等，JDK 8 以后用元空间实现，放在本地内存里，避免永久代时代因加载类过多导致 OOM。



直接内存虽然不在 JVM 规范里定义，但 NIO 的 ByteBuffer 分配会用到，得小心 OutOfMemoryError，毕竟不受堆大小限制。

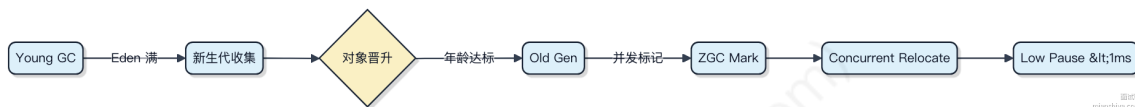
Java 中常见的垃圾收集器有哪些？

Java 的垃圾收集器得按年代来看，不同年代用的策略和实现不一样。早期 CMS 主要解决老年代回收停顿时间长的问題，用标记-清除算法，并发收集，但容易产生碎片，搞不定的时候还得靠 Serial Old 收尾。

G1 收集器是 JDK 9 默认的，主打“低延迟”，把堆拆成多个 Region，优先回收垃圾最多的区域，用写屏障维护 Remembered Set 来追踪跨区引用，避免全局扫描。它在大部分场景下能控制 STW 时间在目标范围内，适合大堆、多核机器。

到了 JDK 11+，ZGC 和 Shenandoah 是新主力。它们都追求亚毫秒级停顿，支持几十 GB 甚至 TB 级堆。ZGC 用着色指针 + 读屏障，在标记和转移阶段几乎不暂停应用线程，停顿基本压在 1ms 以内。Shenandoah 类似，但它不依赖着色指针，而是额外加了个转发指针。

- 1) 新生代这边，Parallel Scavenge 注重吞吐量，适合后台批处理服务
- 2) CMS 已经被废弃，ZGC 在生产环境越来越常见，尤其对延迟敏感的系统比如交易、实时计算



选型上，一般服务用 G1 足够了，要是堆特别大、要求极低延迟，直接上 ZGC。不过 ZGC 对操作系统有要求，比如 Linux 上需要透明大页关闭。

Java 中有哪些垃圾回收算法？

Java 的垃圾回收算法主要围绕怎么找垃圾、怎么清理垃圾展开。不同算法适合不同场景，一般 CMS、G1 这些收集器会组合使用多种算法。

1) 标记-清除 (Mark-Sweep)

先标记所有存活对象，然后统一回收未被标记的垃圾。最大的问题是会产生内存碎片，比如老年代用 Serial 收集器时就用这个算法，长时间运行后可能因为碎片触发 Full GC。

2) 复制 (Copying)

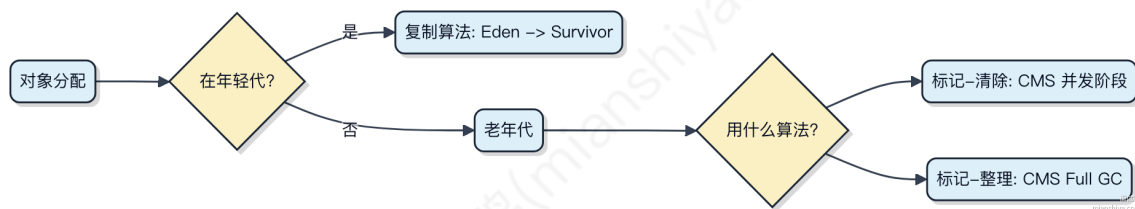
把内存分成两块，每次只用其中一块。当这一块满了，就把存活对象复制到另一块，然后清空原来的那块。效率高，不会产生碎片，但可用内存直接砍半。年轻代的 Minor GC 基本都用这个思路，比如 ParNew 和 G1 在年轻代阶段。

3) 标记-整理 (Mark-Compact)

标记过程和“标记-清除”一样，但后续不是直接清理，而是把存活对象往一端滑动，再清理边界外的内存。适合老年代这种对象多、回收少的场景，避免碎片问题。CMS 在并发失败降级后就会用这个算法做 Full GC。

4) 分代收集 (Generational Collection)

这不是具体算法，而是策略。基于对象生命周期分布：90% 对象朝生夕死。年轻代用复制算法，回收快；老年代用标记-清除或标记-整理。G1 和 ZGC 都是基于分代思想演进出来的。



像 G1 会把堆拆成多个 Region，逻辑上保留分代，回收时优先选垃圾多的区域，实现“增量整理”。ZGC 和 Shenandoah 更激进，主打并发整理，停顿时间控制在 10ms 以内。

JVM 垃圾回收调优的主要目标是什么？

JVM 垃圾回收调优不是为了把 GC 时间压到零，那不现实。目标很实际：在应用的吞吐量、延迟和内存占用之间找到平衡点。

1) 吞吐量优先的场景，比如离线计算任务，像 Spark 批处理这种，可以选 G1 或 Parallel GC，目标是让用户代码跑得更久，GC 停顿少打断。

2) 低延迟是关键的应用，比如交易平台下单链路，每次 GC 停顿必须控制在 100ms 内，这时候 ZGC 或 Shenandoah 就更合适，它们能把 STW (Stop-The-World) 拆成小段，压根不经过长时间卡顿。

3) 内存 footprint 小也很重要，特别是在容器化环境，K8s 里跑着几百个 Java 实例，用 ZGC 虽然停顿短，但内存开销大，可能反而不如 G1 稳定。

调的时候常见翻车点是堆设太大，结果老年代回收拖不动，一次 Full GC 几秒直接超时。一般来说，新生代别超 4G，避免 Minor GC 时间过长。



重点盯住 GC 日志里的 Pause Time 和 Frequency，配合监控看 P99 延迟有没有毛刺。调优到最后，其实是业务容忍度和硬件成本的权衡。

如何对 Java 的垃圾回收进行调优？

调优垃圾回收不是上来就调参数，而是先搞清楚应用的 GC 行为。大部分问题其实出在对象创建太猛，而不是 GC 本身不行。

1) 先用 `jstat -gc` 或 `jcmd <pid> GC.run_finalization` 观察频率和耗时。如果 Young GC 超过 50ms 或频繁到几十毫秒一次，说明对象分配速率太高，得查代码里有没有短生命周期大对象。

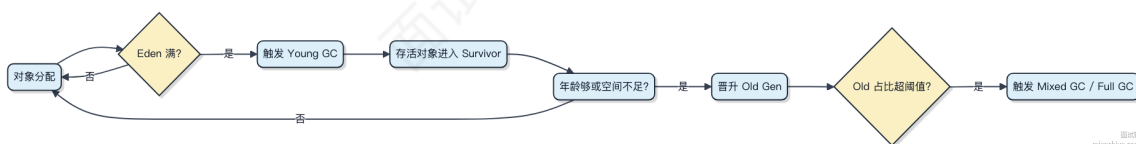
2) 老年代增长快？可能是内存泄漏，也可能是晋升太快。打开 `-XX:+PrintGCDetails` 看对象从 Eden 到 Old 的晋升日志。可以加 `-XX:MaxTenuringThreshold=1` 强制提前晋升测试是否扛得住。

3) 停顿时间敏感的应用，比如接口响应要求 200ms 内，CMS 已经过时了，直接上 **ZGC** 或 **Shenandoah**。它们的暂停时间基本稳定在 10ms 以内，哪怕堆到 32GB 也压不垮。

```
// 启动参数示例：用 ZGC，堆设 8G
-XX:+UseZGC -Xmx8g -XX:+UnlockExperimentalVMOptions
```

4) 别迷信“最大吞吐量”。吞吐型服务可以用 G1，但记得设置 `-XX:MaxGCPauseMillis=200` 目标停顿，JVM 会自动调整分区回收数量。

真正要动手调的，一般就三个方向：控制对象分配速率、减少晋升压力、选对收集器。工具链推荐配合 Arthas 的 `vm.gc` 命令做线上诊断，比 `jstat` 更直观。



常用的 JVM 配置参数有哪些？

JVM 参数选对了，服务能少掉一半链路。先分三类来看，最常用的是内存相关。

堆内存一般用 `-Xms` 和 `-Xmx` 设成一样，避免动态扩容开销，比如 `-Xms2g -Xmx2g`。年轻代大小用 `-Xmn`，通常设为堆的 1/3 到 1/2。如果对象多，可以调大一点。

垃圾回收器得看场景。响应优先用 **G1**，加 `-XX:+UseG1GC` 就行，再配 `-XX:MaxGCPauseMillis=200` 控制停顿时间。吞吐优先就上 `ParallelGC`，`-XX:+UseParallelGC`，适合后台批处理。`ZGC` 现在用得更多，`-XX:+UseZGC` 能把 STW 压到 10ms 内，但得 JDK 15+。

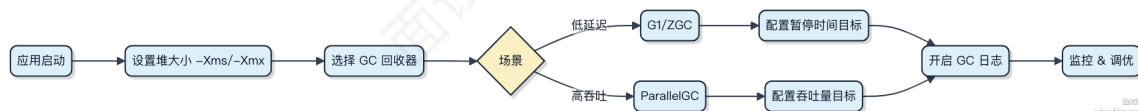
日志参数不能少。`-Xlog:gc*` 开启 GC 日志，最好加上 `time,uptime,level,tags` 打全信息。线上出问题靠这个定位。

还有几个关键开关。`-XX:+HeapDumpOnOutOfMemoryError` 自动导出堆 dump，配合 `-XX:HeapDumpPath=/path` 存文件。`-XX:+PrintCommandLineFlags` 启动时打印实际生效的参数，排查配置有没有被覆盖。

别乱调 `-XX:MaxMetaspaceSize`，除非你真遇到元空间爆了。多数时候让它自动扩就行。

代码示例：

```
java -Xms4g -Xmx4g -Xmn2g -XX:+UseG1GC -XX:MaxGCPauseMillis=200 \
-Xlog:gc*:gc.log:time,uptime,filecount=5,filesize=100m \
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dump MyApp
```



看过源码吗？说下 Spring 由哪些重要的模块组成？

Spring 框架不是单一的一个库，它是一套组合拳，模块之间各司其职。面试问这个，其实是想看你有没有从整体上理解过它的设计结构。

先说最核心的几个部分。**IoC 容器**是整个 Spring 的基石，对应 `spring-core`、`spring-beans` 和 `spring-context` 这三个模块。没有它们，就没有依赖注入这回事。`spring-core` 提供底层工具类，比如类型转换、资源抽象；`spring-beans` 实现了 Bean 的定义、配置和生命周期管理；而 `spring-context` 在前者基础上扩展出 `ApplicationContext`，支持国际化、事件传播等高级特性。

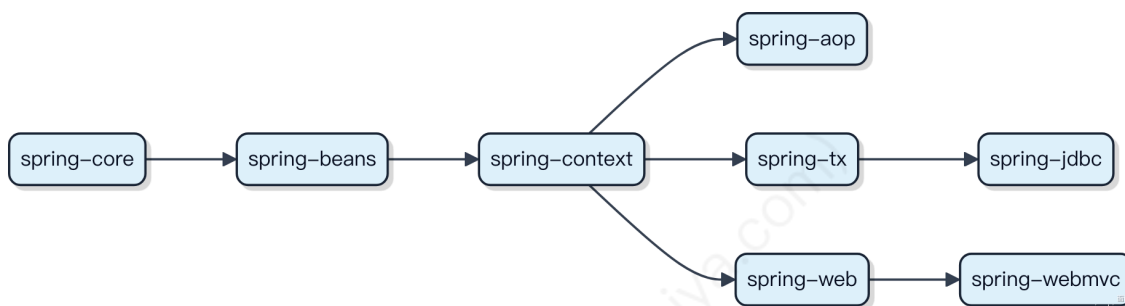
AOP 能力由 `spring-aop` 模块提供，它基于动态代理实现方法级别的拦截。如果你用了 `@Transactional` 或自定义切面，背后就是它在工作。这个模块依赖 `spring-core` 和 `spring-beans`，但可以独立使用。

数据访问方面，`spring-jdbc` 封装了 JDBC 操作，避免你写重复的 try-catch 和资源释放代码。`spring-tx` 统一管理事务，不管是编程式还是声明式事务都靠它支撑。这两个模块让 DAO 层变得干净很多。

Web 开发主要靠 `spring-web` 和 `spring-webmvc`。后者就是我们常说的 Spring MVC，处理 HTTP 请求分发、控制器映射、视图解析这一套流程。现在用 Boot 多了，但底层还是这些模块在驱动。

还有个 `spring-expression`，也就是 SpEL 表达式语言，在注解里写 `#[systemProperties['os.name']]` 这种语法就靠它解析。

这些模块之间有明确的依赖关系，但也能按需引入。比如做批处理可以用 `spring-batch`，做消息可以用 `spring-jms`，都不是必须的。



什么是 Spring IOC?

Spring IOC 解决的是对象之间的依赖关系管理问题。我们不再用 `new` 去创建对象，而是把对象的控制权交给容器去管理，这就是“控制反转”——原来由程序自己控制的东西，现在反转给了外部容器。

容器启动时会读取配置（比如 XML 或注解），把一个个对象实例化好，放到 **IOC 容器**里。需要用的时候，直接从容器拿，不需要自己 `new`。对象之间的依赖关系，比如 Service 需要 Dao，容器会自动注入进去，这个过程叫依赖注入（DI），是 IOC 的实现方式。

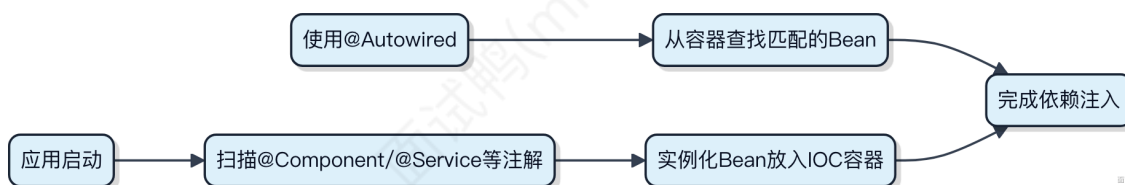
举个例子，你写了个 `UserService` 上面加了 `@Service`，`UserDao` 加了 `@Repository`，Spring 启动时就把它们都实例化，当你在 `UserController` 里用 `@Autowired` 注入 `UserService`，容器会直接把准备好的实例塞进去。

```

@Service
public class UserService {
    @Autowired
    private UserDao userDao;
}
  
```

常见场景像 Bean 的作用域控制，单例默认就是容器里只有一份。如果是 Web 应用，结合 `@Controller`、`@Component` 这些注解，整套对象体系都能被自动装配起来，省得手动维护对象生命周期。

IOC 让代码更松耦合，测试也方便，Mock 对象可以直接塞进容器替换真实实现。但过度依赖注解和隐式注入的话，排查问题会有点麻烦，尤其是 Bean 冲突或者循环依赖的时候。



Spring 中的 DI 是什么?

依赖注入（DI）是 Spring 框架最核心的能力之一，它让对象之间的依赖关系由容器来管理，而不是在代码里硬编码。

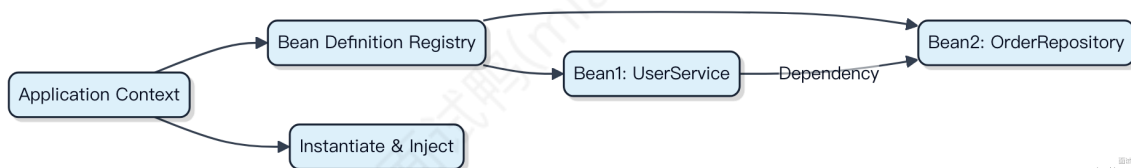
比如一个 Service 需要用到 Repository，传统做法是在 Service 里 `new` 一个 Repository 实例，耦合度高，测试困难。DI 的做法是，Spring 容器在启动时根据配置或注解把 Repository 实例“塞”进 Service 里，运行时直接用就行。

最常见的注入方式有三种：

- 1) 通过构造函数注入，推荐这种方式，不可变且便于单元测试
- 2) 通过 setter 方法注入，灵活性高但对对象可能处于不完整状态
- 3) 通过字段直接注入（@Autowired 注解字段），写起来最简单，但降低了可测试性和灵活性，不推荐在复杂项目中使用

Spring 在创建 Bean 时会解析依赖关系图，按需实例化并完成注入。整个过程基于反射和 JavaBean 规范实现，支持循环依赖的处理（借助三级缓存）。

举个例子，你用 @Component 标记组件，@Autowired 标记要注入的字段或构造函数，Spring 启动时自动完成装配。像 MyBatis 的 SqlSessionFactory、Redis 的 RedisTemplate，都是通过 DI 管理的典型场景。



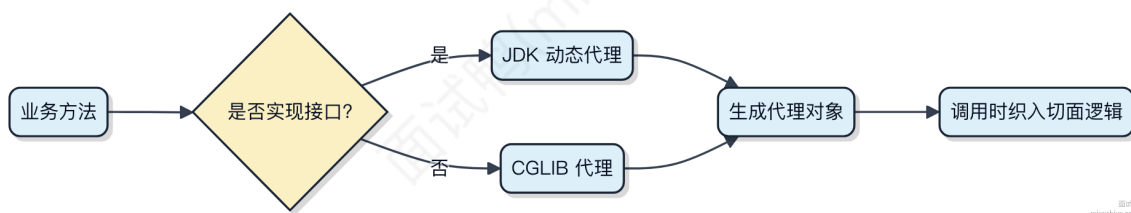
本质上，DI 是控制反转（IoC）思想的具体实现，把对象的创建和组装责任从应用代码转移到容器，专注业务逻辑即可。

什么是 AOP?

AOP，也就是面向切面编程，本质是为了解决横切关注点的代码复用和集中管理。像日志记录、权限校验、事务管理这些功能，散落在各个业务方法里，改起来费劲还容易漏。AOP 就是把这些横切逻辑抽出来，统一织入到目标方法的特定位置。

Spring AOP 的实现依赖动态代理。如果目标类实现了接口，默认用 JDK 动态代理；没有接口就用 CGLIB。整个过程不需要修改原有代码，通过配置或注解就能把切面加进去。比如你用 @Around 写个环绕通知，就能在方法执行前后插入逻辑。

- 1) 切点（Pointcut）决定哪些方法要被拦截，通常用表达式匹配方法签名
- 2) 通知（Advice）定义在切点的哪个时机执行，比如前置、后置、环绕
- 3) 切面（Aspect）是切点和通知的组合，描述“在哪执行”和“执行什么”



典型场景就是 Controller 层统一打日志，或者 Service 方法上加 @Transactional。但别滥用，像业务逻辑本身就不该塞进切面里，否则别人看不懂流程。简单说，AOP 是工具，不是魔法，用好了事半功倍，用歪了就是坑。

Spring AOP默认用的是什么动态代理，两者的区别？

Spring AOP 在生成代理对象时，会根据目标类是否实现接口来决定使用哪种动态代理。如果目标类实现了至少一个接口，Spring 就用 **JDK 动态代理**，所有接口方法调用都会被拦截。要是没实现接口，那就上 **CGLIB**。

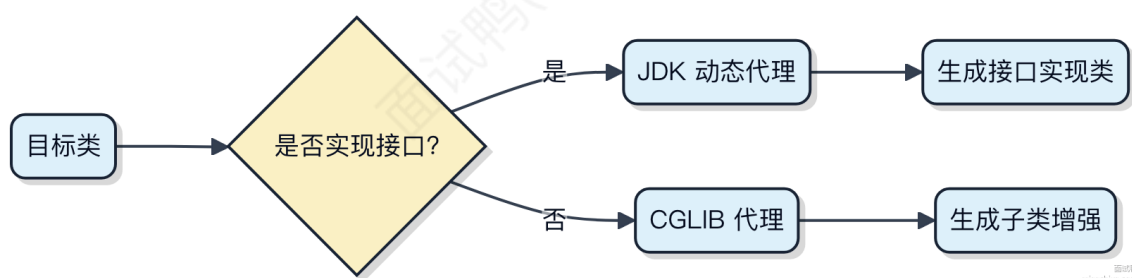
JDK 动态代理是 Java 原生支持的，核心是 `InvocationHandler` 和 `Proxy`，它要求被代理类必须有接口，生成的代理对象是接口的实现类实例。这种方式性能不错，而且不依赖第三方库。

CGLIB 是基于 ASM 字节码框架实现的，它通过继承的方式创建子类来增强功能，所以目标类不能是 `final` 的，方法也不能是 `final` 或 `private` 的。它没有接口也能代理，适用性更广，但会多一层继承关系，启动时要生成 class 文件，稍微耗点内存。

举个例子，你写了个 `UserService` 实现了 `UserOperations` 接口，Spring 默认走 JDK 代理。但如果你的类是 `package-private` 或者压根没定义接口，那只能靠 CGLIB 来搞定了。

实际项目里像 Spring Boot 自动配置、事务管理这些场景，底层都可能混用这两种方式，具体看你的类结构。

要不要切换？可以强制用 CGLIB，只要在配置类上加 `@EnableAspectJAutoProxy(proxyTargetClass = true)` 就行。不过一般没必要，Spring 自己能选对。



什么是循环依赖（常问）？

Spring 里的循环依赖，指的是两个或多个 Bean 互相依赖，形成闭环。比如 A 依赖 B，B 又依赖 A，初始化时就可能卡住。

Spring 能解决部分循环依赖，靠的是三级缓存。创建中的 Bean 会提前暴露到缓存里，让对方面引用到一个“半成品”实例。

1) 一级缓存是 `singletonObjects`，存完全初始化好的单例。2) 二级缓存是 `earlySingletonObjects`，存提前暴露的原始对象（还没填充属性完）。3) 三级缓存是 `singletonFactories`，存能生成这个早期对象的工厂函数。

只有单例 Bean 且通过 `setter` 或 `字段注入`，Spring 才能自动解环。构造器注入的循环依赖，压根不经过缓存，直接抛异常。



原型（prototype）Bean 不走缓存，每次都要新创建，遇到循环依赖直接报错。所以设计时要警惕这种隐式耦合，必要时用 `@Lazy` 推迟加载。

Spring 如何解决循环依赖？

Spring 解决循环依赖主要靠三级缓存和提前暴露对象引用的机制，核心是把实例化和初始化分开。

1) 当 A 依赖 B，B 又依赖 A 时，Spring 在创建 A 的过程中，new 出 A 实例后就马上把它放进早期暴露缓存（`singletonFactories`），这个工厂能返回尚未完全初始化的 A 对象。

2) 接着去创建 B，发现需要 A，就会从 singletonFactories 拿到那个“半成品”A 实例。此时 B 能完成注入，然后 B 初始化完毕放入一级缓存（单例池）。

3) 再回到 A 的初始化，从一级缓存拿到完整的 B，继续完成 A 的属性填充和初始化，最后 A 也进一级缓存，整个流程闭环。

这里的关键是三个缓存层级：

- 一级：singletonObjects，放完全初始化好的 Bean
- 二级：earlySingletonObjects，放提前暴露的原始对象（用于解决普通循环依赖）
- 三级：singletonFactories，存放 ObjectFactory，能生成早期引用

但要注意，这种机制只能解决单例作用域下的 setter 注入循环依赖。构造器注入的循环依赖 Spring 压根不经过缓存，直接抛异常。还有原型（prototype）作用域也不支持，每次都是新对象，没法缓存。

代码层面不需要额外操作，只要用 @Autowired 注入就行，框架自动处理。



为什么 Spring 循环依赖需要三级缓存，二级不够吗？

Spring 解决循环依赖靠的是提前暴露对象的机制，而三级缓存的设计是为了在不破坏 Bean 生命周期的前提下，拿到正确的代理对象。

- 1) 一级缓存是 singletonObjects，存完全初始化好的单例 Bean。
- 2) 二级缓存是 earlySingletonObjects，存提前曝光的原始对象（还未完成属性填充和初始化）。
- 3) 三级缓存是 singletonFactories，存能创建早期引用的 ObjectFactory，用来生成可能需要被代理的对象。

关键点在于：如果两个 Bean 都涉及 AOP 代理，比如都加了 @Transactional，那么它们相互注入时必须是代理对象，而不是原始实例。

假设只有二级缓存：

- A 创建过程中发现自己被 B 引用，就把原始实例放二级缓存。
- B 去拿 A 的时候，只能拿到原始对象，后续再对 A 进行代理，那 B 拿到的就是错的，压根不经过代理逻辑。

有了三级缓存就不一样了：

- A 创建时，把自己包装成一个 ObjectFactory 放进三级缓存。
- B 需要 A 时，通过这个工厂触发 getEarlyBeanReference，拿到的是经过所有 BeanPostProcessor 处理后的早期引用——也就是正确的代理对象。
- 等 A 最终创建完，还会把最终版本升级到一级缓存，保证一致性。



所以，三级缓存的核心价值就是：延迟代理对象的生成时机，同时确保中间环节拿到的是正确增强过的早期引用。

说下 Spring Bean 的生命周期？

Spring Bean 的生命周期从创建到销毁，整个过程由容器全权管理，我们可以在关键节点插入自定义逻辑。

- 1) 实例化 Bean：容器通过反射调用构造方法创建对象，这时候还没进行属性注入。
- 2) 属性填充：依赖的字段通过 setter 或自动注入方式赋值，比如 @Autowired 标注的成员会被处理。

- 3) 初始化前处理：如果类实现了 BeanPostProcessor，它的 postProcessBeforeInitialization 方法会执行，这是个扩展点，像 AOP 代理就可能在这里生成。
- 4) 初始化操作：先看有没有实现 InitializingBean 接口，有的话调用 afterPropertiesSet；然后检查是否配置了 init-method，比如 XML 中指定或 @Bean(initMethod="xxx")，都会在这个阶段触发。
- 5) 初始化后处理：BeanPostProcessor 的 postProcessAfterInitialization 再次介入，很多代理逻辑都在这步完成，比如事务代理。
- 6) 放入单例池：如果是单例，默认会把最终对象放进一级缓存 singletonObjects，后续 getBean 直接返回。
- 7) 使用阶段：Bean 被应用程序正常使用，比如作为 Controller 处理请求，或被其他 Bean 引用。
- 8) 销毁阶段：容器关闭时，先调用 DisposableBean 的 destroy 方法，再执行 destroy-method 指定的方法，比如释放数据库连接、关闭线程池。



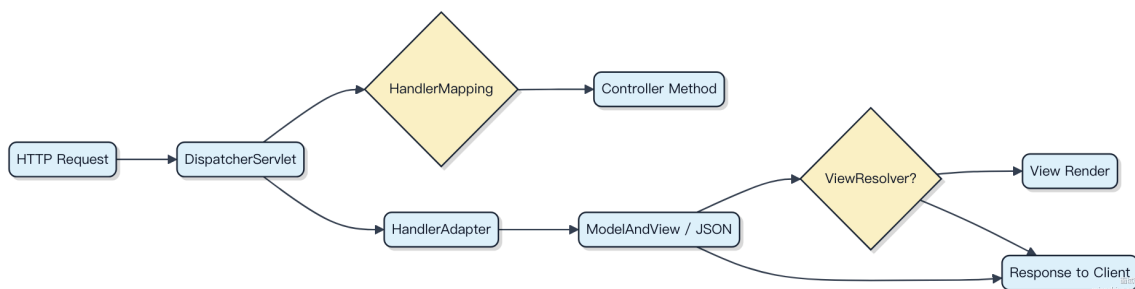
prototype 类型的 Bean 容器不负责销毁，不会调用销毁方法。这个流程里最常被利用的是两个 **BeanPostProcessor** 扩展点，Spring 的各种功能都靠它织入。

Spring MVC 具体的工作原理？

Spring MVC 的工作核心是围绕 DispatcherServlet 展开的，它本质上是一个前端控制器（Front Controller），所有请求都先打到它这里，再由它分发给具体的处理器。

请求进来后，首先是 DispatcherServlet 接手，它会委托一系列组件完成映射、适配、处理和渲染。整个流程可以拆成几步来看：

- 1) HandlerMapping 负责根据 URL 找到对应的 Controller 方法。比如你访问 /user/1，框架得知道该调 UserController 里的 getById 方法。常见的实现有 RequestMappingHandlerMapping，它基于注解扫描路由。
- 2) 找到方法后，HandlerAdapter 出场，它负责真正去调用那个被 @RequestMapping 标记的方法。不同的处理器类型需要不同的适配器，比如处理简单字符串返回的和处理 @ResponseBody 的不一样。
- 3) 方法执行完得到 ModelAndView 或响应数据，接着 ViewResolver 对视图名进行解析。比如返回 "userView"，它可能被解析成一个 Thymeleaf 模板或 JSP 页面。不过现在前后端分离多，很多接口直接写 @ResponseBody，这一步就跳过了，数据直接走 Jackson 序列化 JSON。
- 4) 最后是渲染和响应输出。如果是 REST API，压根不经过视图渲染，数据通过 HttpMessageConverter 写回客户端。

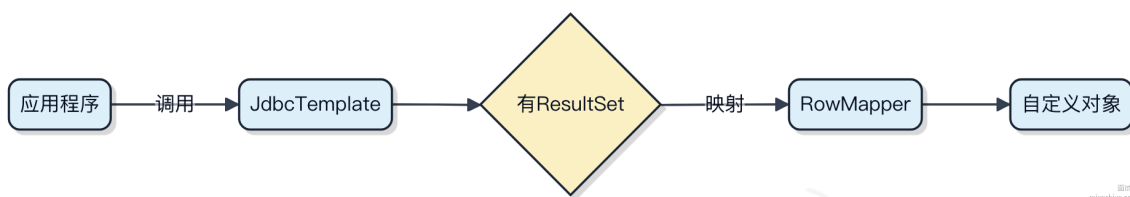


整个过程里，Interceptor 的拦截逻辑穿插在前后，适合做日志、权限校验这些脏活累活。像 Spring Boot 自动配置把这些组件都默认搭好了，我们写个 @RestController 就能跑起来，背后其实是这套机制在扛住。

你了解的 Spring 都用到哪些设计模式？

Spring 框架里设计模式用得非常自然，不是为了套模式而套，而是为了解决实际问题。

- 1) **单例模式**是应用最广的，默认的 Bean 作用域就是单例。容器启动时创建一次实例，后续获取都返回同一个对象，减少频繁创建销毁的开销。但要注意线程安全，无状态 Bean 没问题，有状态的就得自己管理。
- 2) **工厂模式**体现在 BeanFactory 和 ApplicationContext 上。它们负责实例化、配置、组装 Bean，用户不需要手动 new。比如 XML 或注解定义的 Bean，由工厂统一管理生命周期，实现解耦。
- 3) **代理模式**是 AOP 的基础。Spring 默认用 JDK 动态代理（接口-based），有接口走 Proxy + InvocationHandler；没有接口就切到 CGLIB，生成子类增强方法。事务 @Transactional 就是典型场景，方法前后织入开启/提交事务逻辑。
- 4) 模板方法模式在 JdbcTemplate 里体现明显。它把获取连接、执行 SQL、处理结果集这些流程定好，把变化的部分——比如结果映射——留给开发者通过 RowMapper 实现。
- 5) 观察者模式对应 Spring 的事件机制。发布 ApplicationEvent，监听器 @EventListener 接收处理，像上下文刷新、异常通知都可以基于这个链路做扩展。

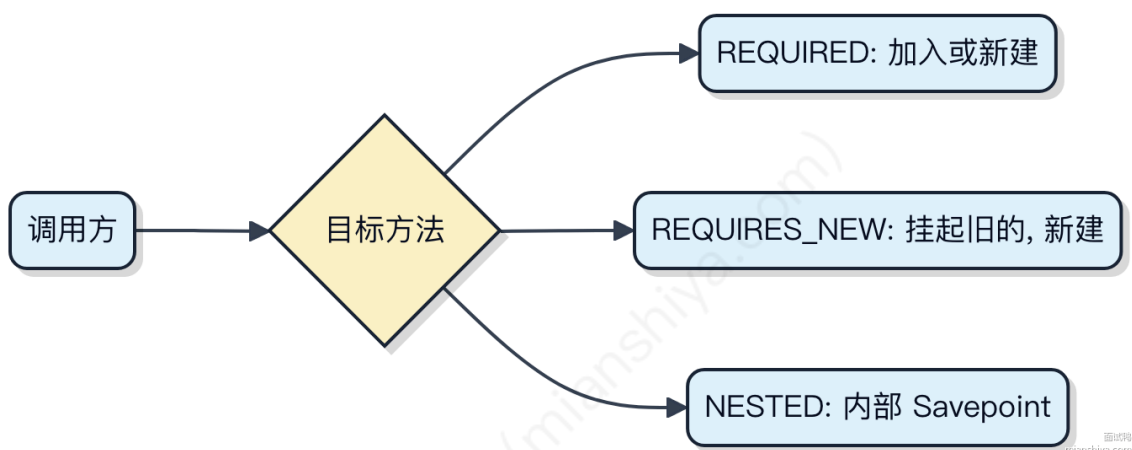


这些模式不是孤立的，经常组合使用，比如 AOP + 事务 + 代理，形成一套完整的声明式事务解决方案。

Spring 有几种事务传播行为？

Spring 的事务传播行为决定了多个事务方法相互调用时，事务如何“传播”下去。一共七种，最常用的其实是 **PROPAGATION_REQUIRED** 和 **PROPAGATION_REQUIRES_NEW**。

- 1) **REQUIRED** 当前没事务就新建，有就用已有的。这是默认行为，大部分增删改都走这个。
- 2) **REQUIRES_NEW** 搞个全新的事务，挂起当前的。适合独立提交的场景，比如记日志、发通知，不能因为主业务回滚就跟着回滚。
- 3) **SUPPORTS** 有事务就加入，没有也正常运行。适合查询类操作，不强制事务。
- 4) **NOT_SUPPORTED** 不支持事务，总以非事务方式执行。比如调用第三方接口，不想让它卷进当前事务。
- 5) **MANDATORY** 必须在已有事务里运行，没有就抛异常。用于强制约束，比如某个方法只允许在事务中被调用。
- 6) **NEVER** 绝对不要事务，有事务就抛异常。安全控制场景会用到。
- 7) **NESTED** 嵌套事务，外层事务回滚，它也回滚；但它自己回滚不影响外层。基于 Savepoint 实现，注意不是所有数据库都支持。



别死记数字和名字，关键理解每个行为在什么业务场景下适用。比如订单创建中扣库存和发券，发券失败要独立回滚就得用 REQUIRES_NEW。

详细描述一条 SQL 语句在 MySQL 中的执行过程。

一条 SQL 语句从发起到执行完成，MySQL 要走完一整套流程，涉及多个组件协作。

首先客户端发起请求，连接器先验证身份和权限，同时维持连接状态。长连接用得小心，内存占用会累积，8 小时空闲也会自动断开。

接着解析 SQL。**查询缓存**在这阶段起作用，但默认是关闭的，因为命中率低，8.0 直接移除了。然后词法和语法分析生成解析树，再通过预处理器校验字段、表名是否存在，权限是否够。

接下来进入优化阶段。优化器决定执行路径，比如多表 JOIN 的顺序、走哪个索引。它会基于成本模型选一个它认为“最快”的方案，生成**执行计划**。可以用 EXPLAIN 看这个计划。

执行时，优化器调用存储引擎接口，比如 InnoDB。如果是 SELECT，就通过索引找到数据行。InnoDB 的聚簇索引结构决定了主键查询最快，二级索引需要回表。更新操作还会写 redo log 和 undo log，保证崩溃恢复和事务回滚。

整个过程还可能涉及锁机制，比如行锁、间隙锁，在 RR 隔离级别下防止幻读。如果是事务，还得遵循两阶段提交，尤其是开启 binlog 的情况下，确保 crash-safe。



实际排查慢查询，一般从慢日志入手，结合 EXPLAIN 分析执行计划，看有没有全表扫描、索引失效等问题。

MySQL 的存储引擎有哪些？它们之间有什么区别？

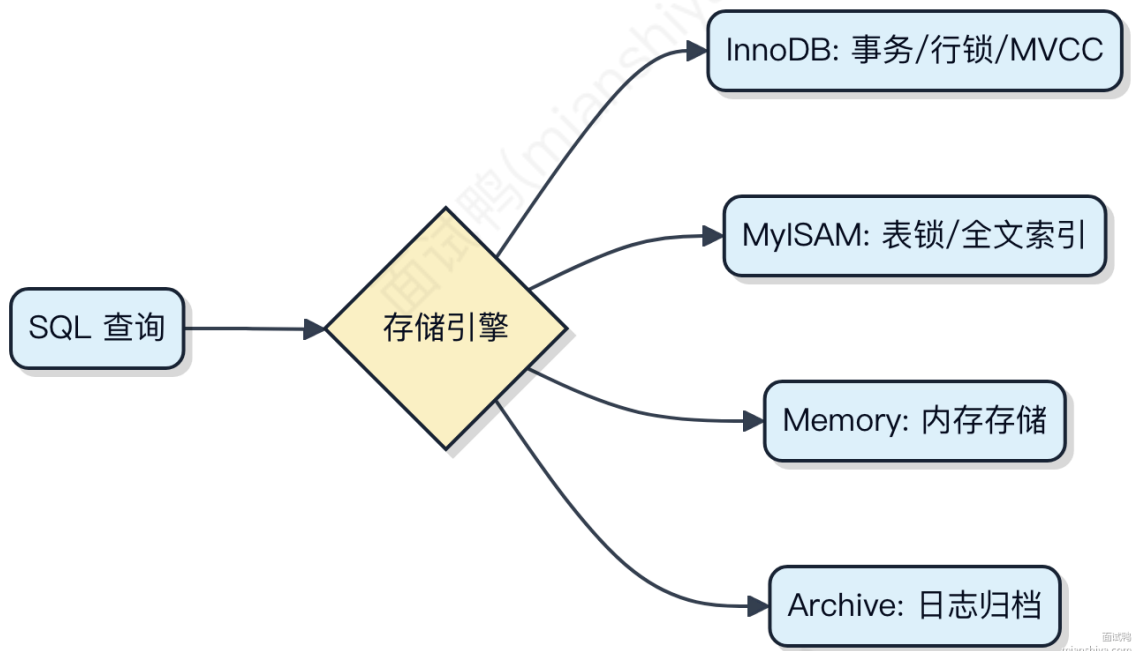
MySQL 支持多种存储引擎，但最常用的其实是 InnoDB 和 MyISAM。面试里问这个，核心是看你对 InnoDB 的掌握程度。

1) InnoDB 是 MySQL 默认引擎，从 5.5 版本开始就扛住主要业务场景。它支持事务、行级锁、外键约束，还实现了 MVCC 来提升并发性能。大多数高并发、要求数据一致性的系统都会选它，比如电商订单、支付这类模块。

2) MyISAM 虽然查询快，尤其是 `count(*)` 这种全表扫描，但它压根不支持事务和行锁，写入时会锁整张表。现在基本被淘汰了，只在一些日志类、读多写少的场景还能见到，比如老系统的数据分析表。

3) 其他引擎像 Memory 是把数据放内存的，断电就丢，适合做临时缓存表；Archive 用来存归档数据，压缩比高，但查询能力弱。

InnoDB 的数据是按聚簇索引组织的，主键索引直接带行数据，二级索引指向主键。而 MyISAM 不管什么索引，都指向数据文件地址，结构上就简单很多。



选引擎的关键是看业务需求。要事务和并发写，必须用 InnoDB；纯读且统计频繁，可以考虑 MyISAM，但现在一般通过缓存或数仓解决。别在新项目里用 MyISAM，搞不定故障恢复。

MySQL InnoDB 引擎中的聚簇索引和非聚簇索引有什么区别？

聚簇索引的叶子节点直接存储完整的行数据，而非聚簇索引存的是索引列值和主键ID。InnoDB 表必须有**聚簇索引**，一般是主键，如果没有主键会选唯一非空索引，再没有就隐式生成一个。

查询时，走聚簇索引可以直接拿到数据，一次查找就行。走非聚簇索引需要先查到主键ID，再拿主键去聚簇索引回表查数据，多了一次IO。

比如你用 `SELECT * FROM users WHERE id = 1`，id 是主键，直接走聚簇索引搞定。但用 `SELECT * FROM users WHERE name = 'Tom'`，name 上有普通索引，就得先查 name 索引得到 id，再去主键索引查整行，这叫回表。

非聚簇索引的结构就像一棵B+树，内部节点只存 name 和指针，叶子存 name + id。聚簇索引的B+树叶子才真正放整行记录。

更新主键成本很高，因为聚簇索引里数据是按主键物理排序的，改主键等于挪动数据页位置。所以主键建议用自增int或bigint，避免页分裂。

- 1) 聚簇索引决定数据存储顺序
- 2) 每个表只有一个聚簇索引

3) 非聚簇索引依赖主键做“跳转”



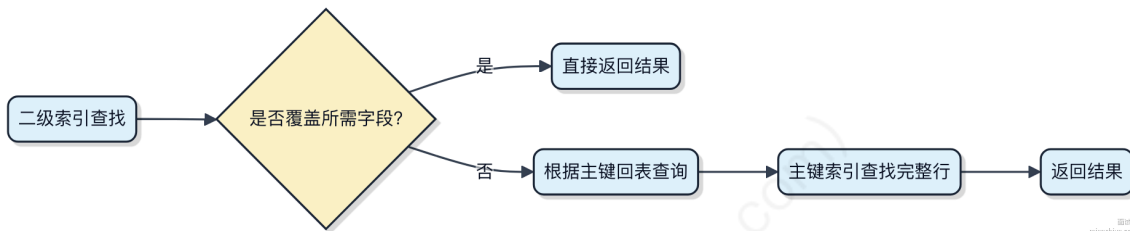
别在频繁更新的列上建主键，也别让主键太长，像用UUID那种，一页存不下几条数据，B+树层数变深，查询就慢了。

MySQL 中的回表是什么？

回表指的是当查询不能完全通过索引树获取所需数据，必须回到主键索引（聚簇索引）中去查找剩余字段的过程。InnoDB 的二级索引叶子节点存的是主键值，不是行数据地址，所以一旦需要非索引列，就得拿主键再去查一遍主键索引。

比如你有个联合索引 (name)，但你要查 name 和 email，而 email 不在索引里，这时就会先在 name 索引树上找到主键 ID，再拿着 ID 去主键索引上找完整的行记录，这就是一次回表。

- 1) 回表的代价其实不低，一次查询可能变成两次 B+ 树遍历，尤其是结果集大时，随机 IO 会明显增加。
- 2) 避免回表的关键是让查询能走**覆盖索引**，也就是所有要查的字段都在索引里。像 `SELECT id, name FROM users WHERE name = 'Tom'` 就不需要回表，因为 id 和 name 都在索引中。
- 3) 使用联合索引或调整索引结构可以有效减少回表。例如把 (name, email) 建成联合索引，查这两个字段就直接命中，压根不经过主键索引。



简单说，回表就是“查了索引还得再查数据行”，能覆盖就不回，不能覆盖就得来回跑。

MySQL 的覆盖索引是什么？

覆盖索引指的是，查询的字段都包含在某个索引中，MySQL 直接通过索引就能返回数据，压根不经过主键索引的回表操作。

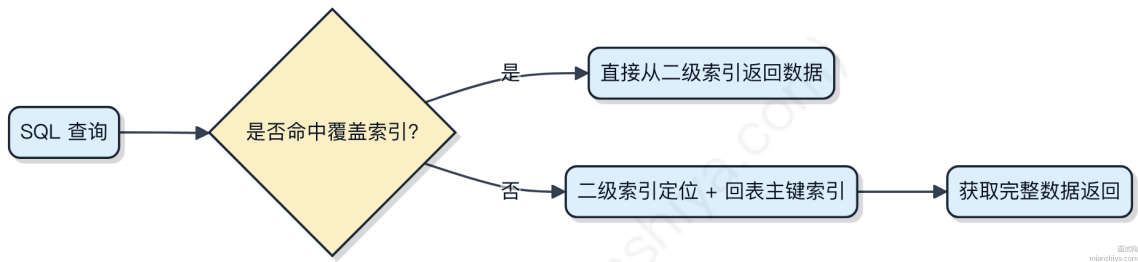
比如有张订单表 orders，建了联合索引 (status, create_time, user_id)。如果执行这样的查询：

```
SELECT user_id FROM orders WHERE status = 1 AND create_time > '2023-01-01';
```

它只需要走索引树查找，找到对应索引项后直接取出 user_id 返回，不需要再去主键索引捞整行数据。这种情况下，这个索引就起到了**覆盖索引**的作用。

有几个关键点要注意：1) 如果查询里有个字段没在索引里，比如 `SELECT amount`，那肯定得回表。2) 覆盖索引是提升性能的利器，尤其在大表上能减少大量随机 IO。3) 但别为了覆盖所有查询建宽索引，索引太宽写入代价高，B+ 树层数可能变多，反而影响整体性能。

实际场景中，像用 Elasticsearch 同步 MySQL 数据做搜索时，常会为高频查询字段建覆盖索引，让 MySQL 只负责写和增量同步，查都交给 ES，这时候覆盖索引能让同步查询轻量化。



MySQL 的索引下推是什么？

索引下推（ICP）是 MySQL 5.6 引入的优化机制，主要解决二级索引查询时回表次数过多的问题。没有 ICP 时，存储引擎找到一条符合索引条件的记录，就会立刻回表查聚簇索引，再由 Server 层判断其他 WHERE 条件是否满足。如果筛选条件多，很多回表其实是白做的。

有了 ICP，存储引擎层会把能用索引字段的过滤条件也带上，在索引遍历过程中就提前过滤掉不满足条件的数据，**只把真正符合条件的才回表**。这样大大减少了回表次数，尤其在联合索引包含多个筛选字段时效果明显。

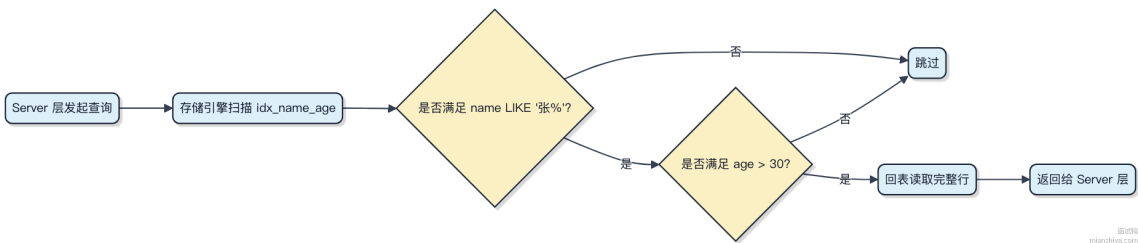
比如这张表：

```
CREATE TABLE users (
  id int primary key,
  name varchar(50),
  age int,
  city varchar(20)
);
-- 建立联合索引
ALTER TABLE users ADD INDEX idx_name_age (name, age);
```

执行这条查询：

```
SELECT * FROM users
WHERE name LIKE '张%'
AND age > 30;
```

name 是范围查询，传统方式只能用到 name 的索引部分，age 的条件得回表后判断。但启用 ICP 后，age > 30 这个条件会被“下推”到存储引擎层，在遍历 idx_name_age 索引时直接跳过 age <= 30 的记录，减少一半以上的回表量。



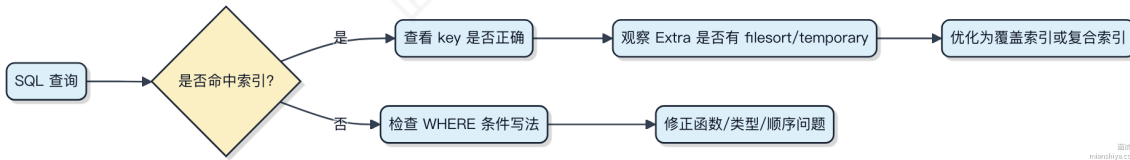
注意 ICP 不适用于主键索引，也不支持覆盖索引的情况——因为压根不经过回表。一般像 InnoDB、MyISAM 都支持，但具体还得看执行计划里 Extra 字段有没有 Using index condition。

MySQL中使用索引一定有效吗？如何排查索引效果？

索引不是银弹，用不对压根不生效。常见原因比如写法导致无法命中，像在索引字段上做函数操作 WHERE YEAR(create_time) = 2023，这直接让B+树失效。或者隐式类型转换，比如索引字段是 varchar，但用数字查询没加引号，MySQL会全表扫描。

联合索引要遵守最左前缀原则，中间跳字段或者顺序颠倒都可能让索引失效。比如索引 (a,b,c)，查 b=1 and c=2 就用不上。还有 like 以 % 开头，LIKE '%abc' 这种也无法利用索引查找。

想确认索引有没有起作用，核心是看执行计划。用 EXPLAIN 分析 SQL，重点关注 type、key 和 Extra 字段。type 至少到 range 级别才算有效使用，最好能达到 ref 或 const。key 显示实际走的索引，如果是 NULL 就说明没用上。Extra 里出现 Using index 表示覆盖索引命中，性能好；要是 Using filesort 或 Using temporary，那多半有优化空间。



线上排查可以用 slow query log 配合 pt-query-digest 分析慢查询，定位哪些 SQL 没走索引。有时候加了索引但数据量小，优化器觉得全表扫更快，也会放弃索引，这种情况一般不用管。

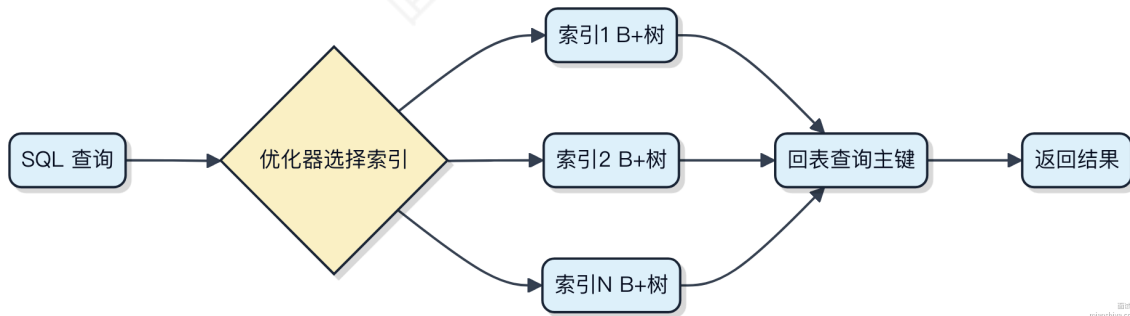
MySQL中的索引数量是否越多越好？为什么？

索引不是越多越好，它是一把典型的双刃剑。你加一个索引，查询可能快了 10 毫秒，但写入就得付出额外代价。

每次执行 INSERT、UPDATE 或 DELETE，MySQL 都得去维护对应的 B+ 树索引。每多一个索引，就意味着多一次磁盘 I/O 和内存操作。比如一张用户表，如果你给 name、email、phone、age、city 各建一个单列索引，那每次插入一条数据，引擎最多得更新 5 棵 B+ 树。这开销在高并发写入场景下会直接拖垮性能。

另外，索引占用存储空间。每个索引都对应物理上的数据结构，尤其是组合索引列多的时候，可能比数据本身还大。更麻烦的是优化器选错执行计划——索引太多会让优化器计算成本时耗时增加，甚至选中不合适的索引，导致慢查询。

一般建议控制单表索引数量在 5 个以内，优先使用覆盖索引和组合索引。比如你经常查 city 和 age，就建 (city, age) 而不是两个单列索引。这样既能命中联合查询，又能减少维护成本。



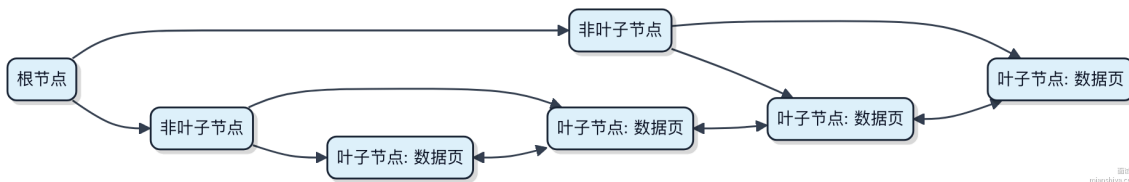
重点是权衡读写比例。读多写少的分析表可以适当多建索引；频繁写入的交易流水表，就得精打细算。

为什么 MySQL 选择使用 B+ 树作为索引结构？

MySQL 的索引选型，关键是要在磁盘 I/O 效率、数据有序性和范围查询能力之间找到平衡。B+ 树在这几个方面表现非常均衡。

- 1) B+ 树的非叶子节点不存数据，只存索引，这意味着同样大小的页可以容纳更多键值，树的高度更低。一般 3 层的 B+ 树就能支撑上百万行的数据查询，查一次数据最多 2~3 次磁盘 IO，效率很高。
- 2) 所有数据都集中在叶子节点，并且叶子节点之间用双向链表连接，这使得范围查询特别高效。比如 `WHERE id BETWEEN 100 AND 200`，只需要定位到第一个 key，然后顺序往后扫就行，不需要回溯父节点。
- 3) B+ 树的结构稳定，插入删除时通过分裂和合并维持平衡，不会频繁重构整棵树。像 MyISAM 和 InnoDB 都基于 B+ 树实现主键和二级索引，InnoDB 的聚簇索引甚至把主键和行数据直接绑在一起，减少一次回表。

对比一下，哈希索引适合等值查询，但压根不支持范围查询；二叉搜索树虽然快但深度太深，IO 次数多；B 树每个节点都存数据，导致单层能放的 key 变少，树更高。



这种设计让 MySQL 能扛住大量并发查询和写入，尤其是业务系统里常见的分页、排序、区间统计场景。

MySQL 中长事务可能会导致哪些问题？

长时间运行的事务在 MySQL 里是个隐患，尤其在高并发场景下容易引发连锁反应。

1) 锁竞争加剧

长事务持有锁的时间更久，别的事务等锁的时间变长，响应变慢。比如一个事务一直没提交，它锁住的行别人就改不了，严重时直接把线程池拖垮，大量连接堆积。

2) 回滚段压力大

InnoDB 的 MVCC 依赖 undo log 维护多版本数据。长事务不结束，undo log 就不能清理，这些历史版本一直占着空间。极端情况下，可能让 ibdata1 文件疯狂增长，磁盘撑爆。

3) 主从延迟

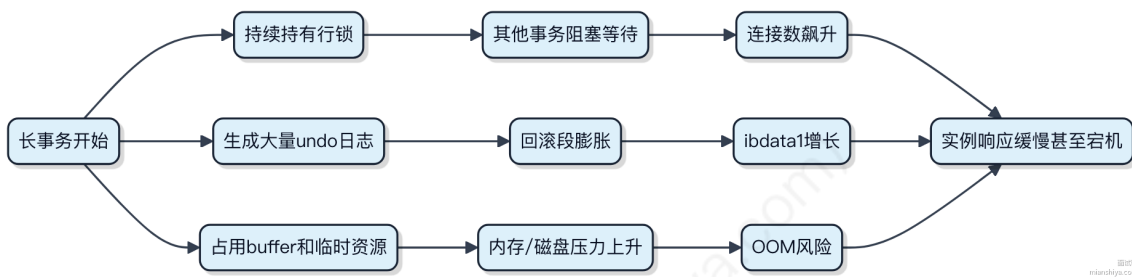
主库上执行了 5 分钟的事务，从库得按同样的节奏重放。如果这个事务还涉及大量行变更，从库 SQL Thread 跟不上节奏，延迟就来了。像阿里云 RDS 这类环境，延迟超过 30 秒可能触发告警。

4) 临时文件和内存泄漏风险

事务过程中产生的临时结果集、排序缓冲都得维持状态。时间越长，内存占用越高，可能触发 OOM。特别是 `ORDER BY`、`GROUP BY` 操作，容易用到磁盘临时表。

5) 逻辑备份失败

mysqldump 默认用 `REPEATABLE READ` 隔离级别加全局读锁，长事务会让 `FLUSH TABLES WITH READ LOCK` 卡住，导致备份超时。换成 `--single-transaction` 也一样，如果备份期间有长事务，快照一致性就保不住。

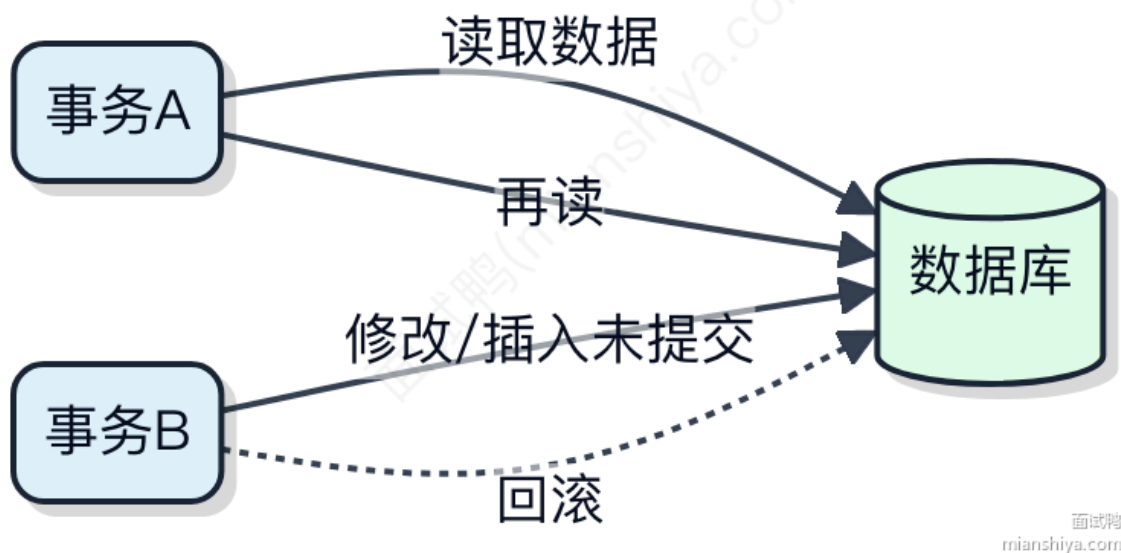


一般建议控制在几秒内完成，超过 30 秒就算危险信号。可以查 `information_schema.innodb_trx` 表定位异常事务。

数据库的脏读、不可重复读和幻读分别是什么？

脏读、不可重复读、幻读，都是在并发事务下读取数据时可能遇到的一致性问题。它们的根源在于事务之间没有做好隔离。

- 1) 脏读是指一个事务读到了另一个事务**未提交**的数据。比如事务 A 更新了一行记录但还没提交，事务 B 此时读到了这行“临时”数据。如果事务 A 最终回滚，那事务 B 读到的就是无效数据。这种情况在金融转账场景里绝对不能接受。
- 2) 不可重复读指的是在同一事务中，两次读取同一行数据结果不一样。原因是另一个事务在这期间修改并提交了这行数据。例如事务 A 先读某条记录 `age=20`，事务 B 修改为 25 并提交，事务 A 再读一次发现 `age` 变了。这会影响需要多次校验一致性的逻辑。
- 3) 幻读和不可重复读类似，但针对的是**数量变化**。比如事务 A 查询 `id > 10` 的记录有 5 条，事务 B 插入一条新的满足条件的记录并提交，事务 A 再查发现变成 6 条，像“幻觉”一样多出一条。典型的场景是分页查询时前后翻页数据错乱。



这三个问题可以通过数据库的事务隔离级别来控制。读已提交能解决脏读，可重复读能解决前两个，串行化才能彻底避免幻读。MySQL InnoDB 在可重复读级别下通过 **间隙锁** 和 **MVCC** 机制也能防止大部分幻读场景。

MySQL 中有哪些锁类型？

MySQL 的锁机制主要是为了解决并发访问带来的数据不一致问题。不同场景下，引擎会选择不同的锁来平衡性能和一致性。

先说 InnoDB 支持的锁类型。最常见的是**行级锁**，它能最大程度减少锁冲突，提升并发能力。行级锁分两种：共享锁（S锁）和排他锁（X锁）。事务对某行加了 S 锁，其他事务还能加 S 锁读，但不能加 X 锁写；加了 X 锁，其他事务啥都不能干。

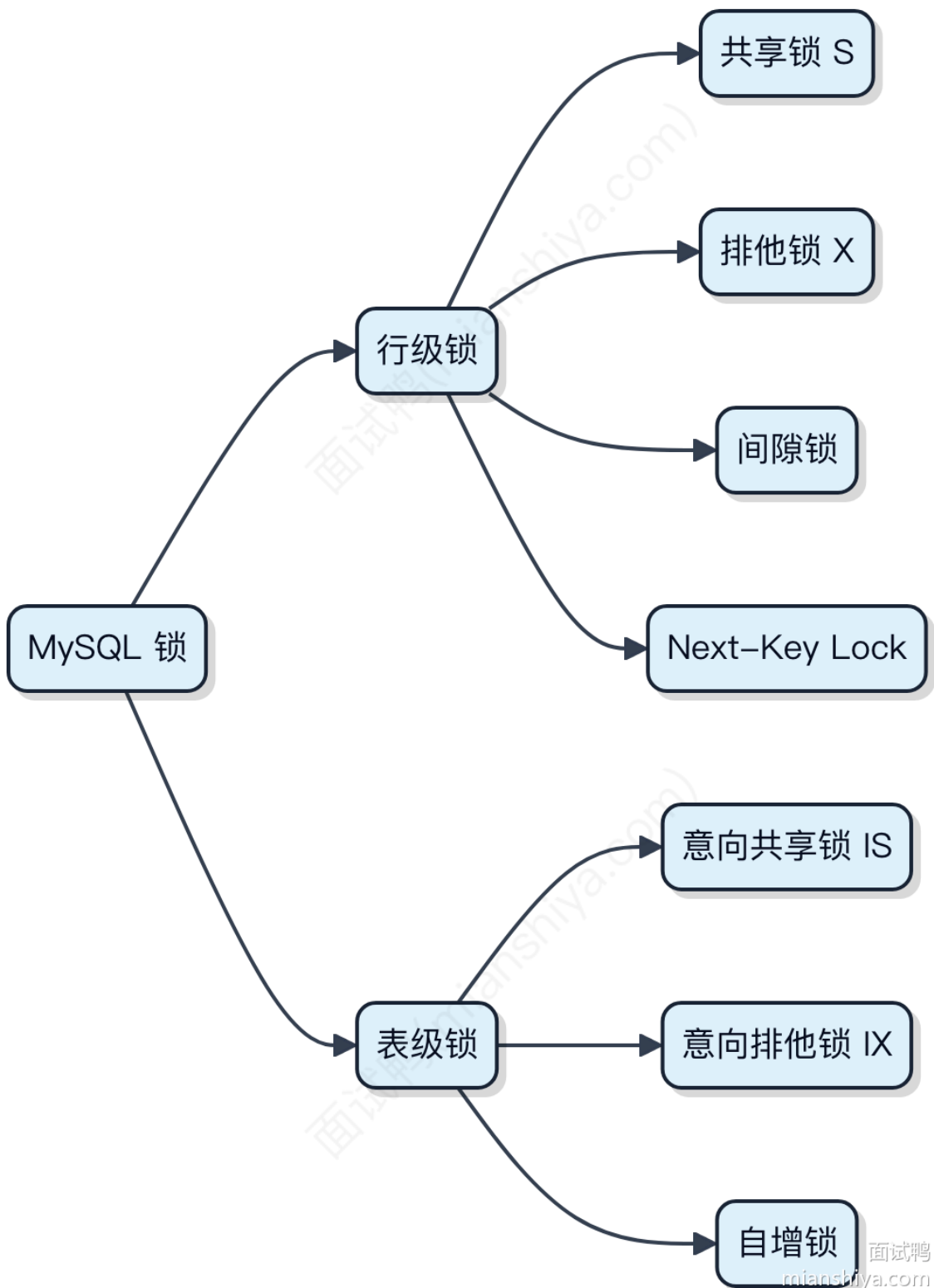
除了行锁，还有**间隙锁**（Gap Lock），它不锁记录本身，而是锁索引之间的“间隙”，防止幻读。比如你查 `WHERE id > 10 AND id < 20`，InnoDB 会锁住 (10,20) 这个区间，不让别人插入新记录。

Next-Key Lock 是行锁和间隙锁的组合，既锁记录又锁间隙，是可重复读隔离级别下防幻读的关键。RR 级别默认用的就是这种锁。

表级锁也有，比如意向共享锁（IS）和意向排他锁（IX）。它们是“声明式”锁，表示事务打算在某个表的某些行上加 S 或 X 锁。有了意向锁，就不需要逐行检查就能快速判断表是否被锁定。

自增锁是特殊的表级锁，用在有 AUTO_INCREMENT 字段的表上，保证自增值分配不冲突，INSERT 完就释放。

- 1) 行级锁：精确控制，并发高
- 2) 间隙锁：锁范围，防幻读
- 3) Next-Key Lock：行锁 + 间隙锁，RR 默认
- 4) 表级锁：意向锁、自增锁等，协调行锁与表操作



[MySQL 的乐观锁和悲观锁是什么？](#)

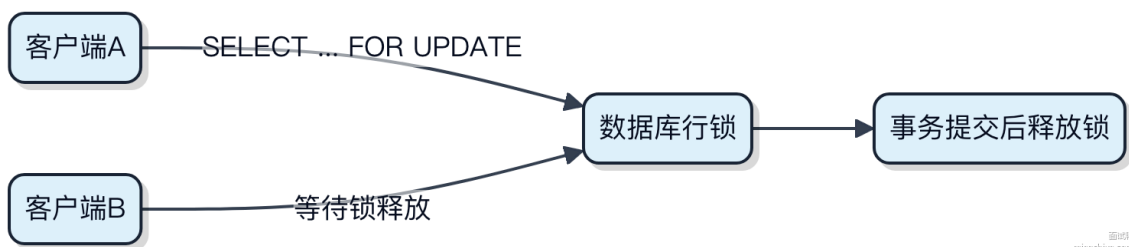
数据库里的锁机制，其实是为了处理并发场景下数据一致性问题的两种思路。乐观锁和悲观锁不是 MySQL 数据库层直接提供的锁类型，而是应用层面的一种设计思想，不过可以用数据库的能力去实现。

- 1) 悲观锁认为冲突大概率会发生，所以访问数据时就直接加锁。在 MySQL 里通常用 `SELECT ... FOR UPDATE` 或 `SELECT ... LOCK IN SHARE MODE` 实现。比如扣减库存这种敏感操作，你查一行记录马上加排他锁，别的事务就只能等着，直到当前事务提交释放锁。这种方式能保证强一致性，但吞吐量会受影响，特别是高并发下容易阻塞。
- 2) 乐观锁相反，它假设冲突很少发生，不会提前加锁。一般通过版本号或时间戳字段来控制。比如读取数据时带一个 `version` 字段，更新时判断这个 `version` 是否被别人改过：

```
UPDATE goods SET stock = stock - 1, version = version + 1
WHERE id = 1001 AND version = 1;
```

如果影响行数为 0，说明 `version` 不匹配，更新失败，需要重试。这种方式没有长时间持有锁，适合读多写少的场景，像秒杀系统里常用这种机制配合 Redis 预减库存来扛住流量。

- 3) 对比来看，悲观锁简单直接，但容易搞成串行执行；乐观锁并发性能好，但会有失败重试的成本。选哪种得看业务场景。比如金融转账偏向悲观锁保安全，内容点赞这类弱一致性操作更适合乐观锁。



MySQL 中如果发生死锁应该如何解决？

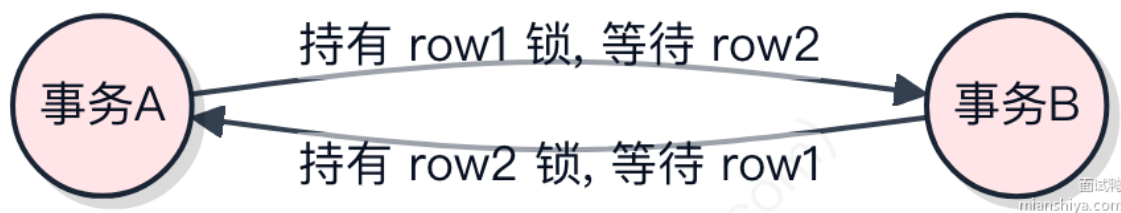
死锁是多个事务循环等待对方持有的锁，谁都继续不了。MySQL 会自动检测到这种情况，在几秒内主动回滚其中一个事务，让另一个先走，避免一直卡着。

被回滚的事务会收到一个错误提示，比如 `Deadlock found when trying to get lock`。这时候应用层得捕获这个异常，一般做法是做几次重试，大概率就能成功了。别忘了加个退避策略，别上来就疯狂重试。

想看到底怎么死的，可以查 `information_schema.innodb_lock_waits` 和 `innodb_trx` 这两表，结合 `SHOW ENGINE INNODB STATUS` 输出的最近一次死锁详情。这里面会告诉你哪个事务持了什么锁，等谁，谁又在等它，链条一清二楚。

避免死锁的关键是访问多张表或同一张表多行时，**顺序一致**。比如都按主键从小到大更新，或者固定先操作用户表再订单表。乱序操作很容易撞上“你等我、我等你”的局面。

代码里注意缩短事务长度，别把网络请求、文件读写这些慢操作包在事务里。长事务持有锁的时间越长，撞车概率直线上升。



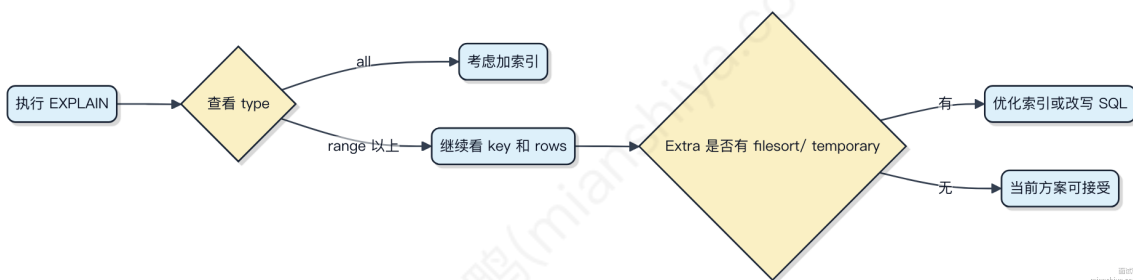
其实大多数死锁不是 MySQL 的问题，而是业务逻辑没设计好。只要保证加锁顺序一致，事务够短，重试机制到位，系统基本能扛住。

如何使用 MySQL 的 EXPLAIN 语句进行查询分析？

EXPLAIN 是分析 SQL 执行计划的最直接工具，执行后返回的每行结果对应查询中的一张表，能看出 MySQL 是怎么执行这条语句的。

- 1) type 列看访问类型，从 system、const、eq_ref、ref 到 range、index、all，性能依次下降。一般优化至少要到 range 级别，all 意味着全表扫描，通常不可接受。
- 2) key 列显示实际走的索引，如果为 NULL 就得警惕了，大概率该建索引了。而 possible_keys 是可能用上的索引，不一定真用。
- 3) rows 列是预估扫描行数，越大越慢。比如查 10 条数据却要扫 10 万行，那肯定有问题。
- 4) Extra 最关键，出现 Using filesort 或 Using temporary 基本意味着需要优化。前者表示无法利用索引排序，后者是用了临时表，都是性能杀手。

举个例子，你用 ORDER BY 创建时间排序但没加索引，Extra 很可能就报 Using filesort。给创建时间字段加个 B+ 树索引，就能消除。



实际排查时结合 WHERE 条件、JOIN 顺序和索引设计一起看，像 **覆盖索引** 这种能让 Extra 显示 Using index 的情况，就是理想状态。

MySQL 中如何进行 SQL 调优？

SQL 调优不是上来就改语句，先知道慢在哪。第一步永远是开慢查询日志，把执行时间超过 1 秒的 SQL 捞出来，这个阈值可以根据业务调整，比如线上核心接口可能得设成 100ms。

找到慢 SQL 后，用 **explain** 看执行计划。重点关注 type 字段，如果是 ALL 就是全表扫描，index 是全索引扫描，最好能做到 range 或 ref。Extra 里出现 Using filesort 或 Using temporary 基本就有性能问题，说明排序或分组没走好索引。

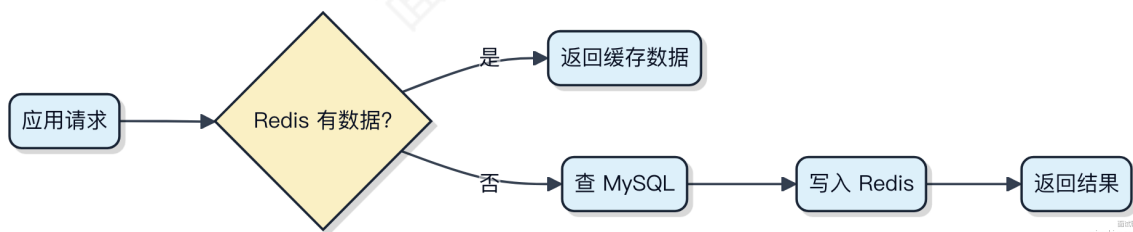
索引设计要结合查询条件。比如经常按 user_id 和 create_time 查订单，那 (user_id, create_time) 的联合索引就很合适。记住最左前缀原则，查询时如果跳过 user_id 直接查 create_time，这个联合索引就用不上。

```
-- 比如这张订单表
SELECT * FROM orders WHERE user_id = 123 AND create_time > '2023-01-01';
-- 建立复合索引
CREATE INDEX idx_user_time ON orders(user_id, create_time);
```

别忘了索引也有成本，写多读少的表加太多索引会拖慢插入更新。大字段比如 text 类型别放索引里，可以用前缀索引或者单独拎出去。

还有些场景需要改架构。单表数据上亿就得考虑分库分表，ShardingSphere 或者 MyCat 可以帮忙。但分了之后跨库 join 和事务就是难题，得从业务设计上规避。

最后提一嘴缓存。热点数据像商品信息，可以放 Redis，避免频繁打到数据库。但要注意缓存穿透、击穿这些坑，合理设置过期时间和空值。



什么是 MySQL 的主从同步机制？它是如何实现的？

MySQL 的主从同步本质是数据变更事件的传递与回放。主库把每一笔写操作记录下来，从库拿到这些记录，在自己身上重新执行一遍，最终达到数据一致。

实现上靠三个关键步骤。主库开启 binlog，所有更改都会追加到这个日志文件里。从库启动一个 I/O 线程，连接主库，请求获取 binlog 里的事件，拉过来存到自己的 relay log（中继日志）里。接着，从库的 SQL 线程读取 relay log，按顺序重放事件，完成数据同步。



整个过程是异步的，从库不强制实时跟进。比如你用 MySQL 搭读写分离，像 ShardingSphere 或 MyCat 这类中间件就能把查询路由到从库，减轻主库压力。

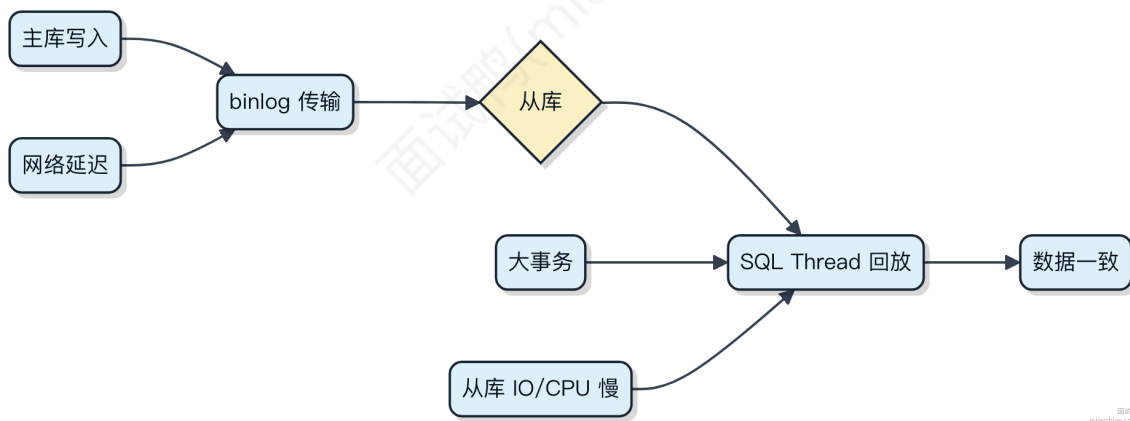
有个坑得注意，主库崩了可能丢数据，因为事务提交后才写 binlog，还没来得及传出去就挂了。半同步复制能缓解这个问题，至少等一个从库确认收到才返回成功。

binlog 是核心载体，格式推荐用 row 模式，能精确记录每一行的变化，避免 statement 模式下因函数或触发器导致的不一致。

如何处理 MySQL 的主从同步延迟？

主从同步延迟本质是备库追不上主库的写入速度。常见于大事务、网络抖动或备库负载过高，得先定位瓶颈在哪一环。

- 1) 先看延迟来源。用 `SHOW SLAVE STATUS` 里的 `Seconds_Behind_Master` 只是参考，更要看 `relay_log` 回放进度。如果 SQL 线程卡住，可能是单线程回放大事务，比如一个 `delete` 删了几百万行，这时候主库瞬间完成，备库得一条条执行，压根不经过优化。
- 2) 解决方案分几类。如果是大事务导致，应用层尽量拆成小批量操作，别搞 10w 行的单 SQL。架构上可以考虑 **并行复制**，MySQL 5.7 之后支持按库并行，8.0 支持更细粒度的逻辑时钟并行，能明显提升回放速度。
- 3) 读写分离场景下，关键请求可以走主库。像用户注册完立刻查信息，这种强一致性需求，代码里就得标记走主库，别指望从库实时同步。TDDL、ShardingSphere 这类中间件都支持 hint 强制主库路由。
- 4) 监控也不能少。一旦延迟超过 30 秒就告警，结合日志判断是网络、IO 还是 SQL 回放问题。极端情况可以临时切主，但要考虑 GTID 断裂风险。



MySQL 中如何解决深度分页的问题？

深度分页在 MySQL 里是个经典性能问题，尤其是用 `LIMIT offset, size` 这种方式，当 `offset` 很大时，比如 `LIMIT 1000000, 20`，数据库得先扫描前 100 万条数据，再取 20 条，查询速度直接崩掉。

根本原因在于这种分页方式依赖**物理偏移**，跳过的行越多，执行越慢。解决思路是避免使用大 `offset`，转而在**基于游标的分页**，也就是利用索引字段（通常是主键或有序列）做条件过滤。

比如你查订单列表，按创建时间排序，就可以记录上一页最后一条的时间戳和 ID，在下一页查询时加 `where` 条件：

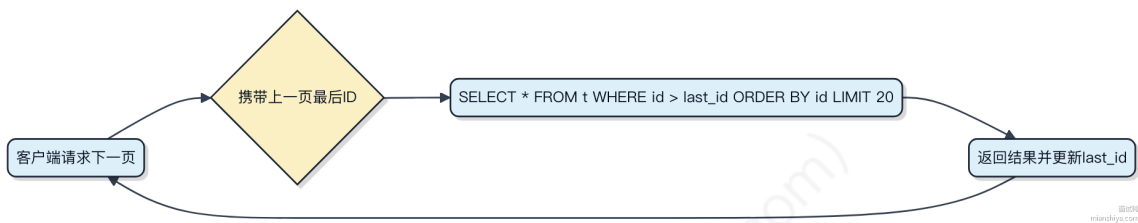
```
SELECT * FROM orders
WHERE id > 1000000
ORDER BY id
LIMIT 20;
```

这种方式能走主键索引，查询效率稳定在 $O(\log n)$ ，完全压根不经过前面那些无效扫描。

如果业务必须支持“跳转到第 N 页”，那就搞不定高性能了，只能提前算好每个分页的边界值，比如用延迟关联或者维护一个分页索引表，但成本高，一般推荐前端改成“加载更多”模式。

还有一个办法是结合业务场景用缓存，比如把热点数据预加载到 Redis，用 ZSET 按 `score` 排序后分页取，像 Feed 流系统里常见。

- 1) 核心是避免大 `offset` 的全范围扫描
- 2) 优先用 `where` + 索引字段替代 `limit offset`
- 3) 配合缓存或物化视图提升极端场景体验



Redis 为什么这么快？

Redis 快不是单一原因造就的，是多个设计选择叠加的结果。很多人第一反应是“因为内存”，但这只是基础，真正关键的是它怎么利用这个基础。

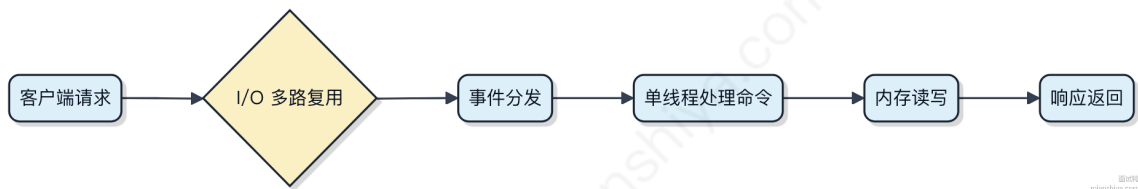
首先，数据存在内存里确实让访问速度上了个台阶，读写都在纳秒到微秒级。但光这样还不够，比如你用 Java 写个 Map 存数据，也快，但扛不住并发请求。Redis 的**多路复用 I/O 模型**才是让它能同时处理几万连接的核心。

它用 epoll (Linux) 这类机制，一个线程就能监听大量 socket 连接，有数据才处理，压根不经过阻塞等待。像 Tomcat 那种每个请求起个线程的方式，在高并发下线程切换开销直接搞垮性能，而 Redis 单线程处理命令，避免了上下文切换和锁竞争，脏活累活都省了。

当然，单线程也有前提：命令执行必须足够快。所以你得注意别用 `KEYS *` 这种慢操作，一执行全库卡住。官方推荐用 `SCAN` 代替。

数据结构上，Redis 直接基于 C 语言实现紧凑结构，比如 SDS 字符串比普通字符串少内存拷贝，哈希表查找接近 $O(1)$ ，这些细节累积起来也让响应更快。

再往后看，像 Pipeline 批量发命令、主从复制异步同步、RDB 快照 fork 子进程由操作系统做写时复制，都是围绕“减少阻塞”做的权衡。



总结一句：**内存 + I/O 多路复用 + 单线程 + 精简数据结构**，这套组合拳打下来，才让 Redis 能轻松扛住 10w+ QPS。

为什么 Redis 设计为单线程？6.0 版本为何引入多线程？

Redis 的单线程模型其实指的是它的核心网络 I/O 和命令执行流程由一个线程串行处理。这么设计最直接的原因是，**避免锁竞争**带来的复杂性和性能损耗。内存操作本身极快，如果命令处理再用多线程加锁，上下文切换和同步开销反而可能压倒收益。

尤其是在 10Gbps 网络下，Redis 处理简单命令的瓶颈通常不在 CPU，而在于网络读写。所以早期版本选择用 **I/O 多路复用 + 单线程事件循环**，像 Redis 这种以微秒级响应为目标的系统，逻辑越简单越可控。

但问题出在高并发场景下，网络 I/O 成了新瓶颈。比如同时有上万连接发来请求，单线程读取 socket 缓冲区就成了串行步骤，CPU 压根没被充分利用。

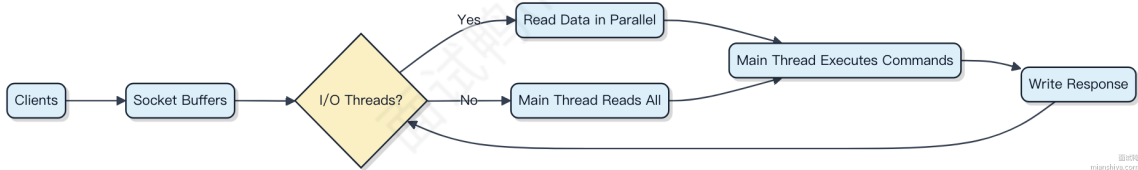
从 Redis 6.0 开始，它引入了多线程来分担这部分脏活累活。具体做法是：主线程继续处理命令调度和执行，但把网络数据的读写拆给 I/O 线程池。你可以通过 `io-threads` 配置开启，比如设为 4，那么读请求就由 4 个线程并行从

socket 读数据，解析完再交回主线程统一执行。

```
// redis.conf 关键配置
io-threads 4
io-threads-do-reads yes
```

注意，命令执行依然在主线程，保证了原子性。多线程只解决“数据搬入搬出”的问题。实测在多核机器上，开启 4 线程能提升吞吐近一倍。

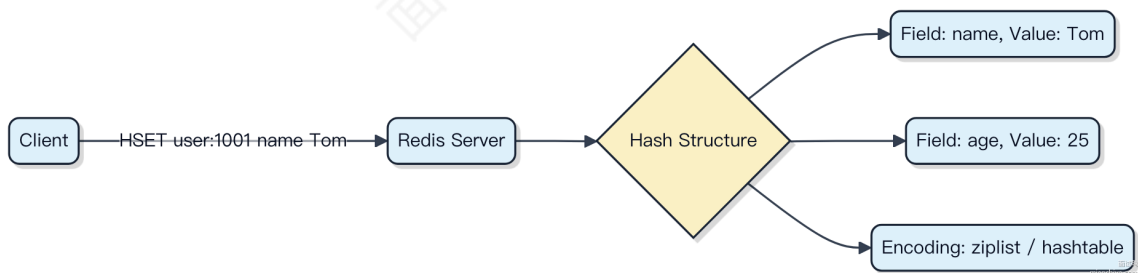
- 1) 单线程为规避锁开销，依赖内存速度和 I/O 多路复用
- 2) 6.0 多线程仅用于网络 I/O，不碰命令执行
- 3) 适合高并发、多核环境，小并发多线程反而增加切换成本



Redis 的 hash 是什么？

Redis 的 hash 本质上就是一个字段和值的映射表，类似 Java 里的 `Map<String, String>` 或 `Map<String, Object>`，但它是存放在一个 key 下的结构。你可以用它来存储对象，比如用户信息，不用把整个对象序列化成一个字符串。

- 1) 你往一个 hash 里塞数据，用的是 `HSET user:1001 name "Tom" age "25"` 这种命令，读的时候用 `HGET user:1001 name`。这样改某个字段特别方便，不用动整个对象。
- 2) 底层实现上，Redis 会根据数据量自动切换两种编码方式。小的 hash 用 **ziplist**（压缩列表），内存紧凑，所有字段和值挨着存；当字段多了或单个值太大，比如超过 64 字节，或者字段数超过 512 个，就会转成 **hashtable**，也就是真正的哈希表，读写都是 $O(1)$ 。
- 3) 适用场景很明显，比如缓存一个商品信息，字段多但更新独立，用 hash 就很合适。像在电商系统里用 Redis 存商品详情，字段如 price、stock、title 各自更新互不影响。但如果你每次都读整个对象，直接用 string 存 JSON 反而更简单。
- 4) 注意别滥用嵌套，Redis 不支持 hash 套 hash。你要存复杂结构，得自己拼 key，比如用 `user:1001:profile` 和 `user:1001:settings` 分开存。



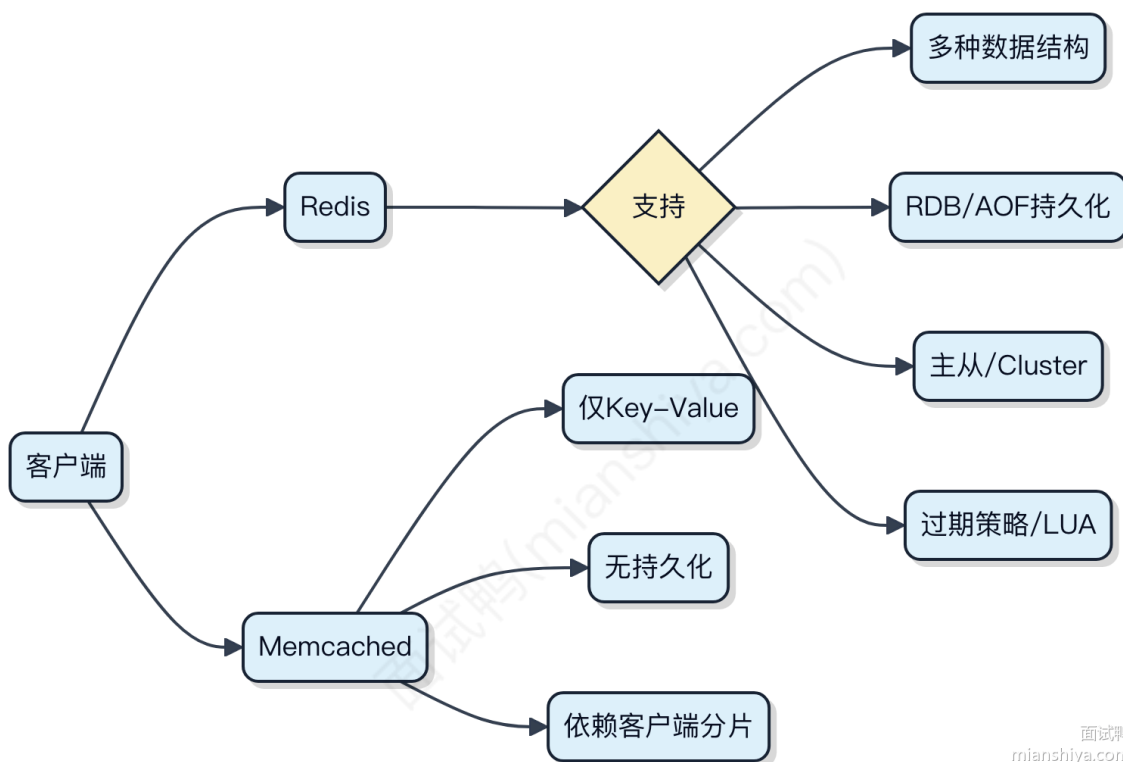
这种设计让 hash 在内存和性能之间做了平衡，小对象省空间，大对象保速度。

Redis 和 Memcached 有哪些区别？

Redis 和 Memcached 都是内存数据存储系统，但设计目标和能力差异明显。

- 1) 数据类型支持。Memcached 只支持简单的 key-value 字符串，存什么取什么。Redis 支持 **String**、Hash、List、Set、Sorted Set、Stream 等丰富类型，能直接操作数据结构，比如用 `HINCRBY` 原子更新哈希字段。
- 2) 持久化能力。Memcached 数据全在内存，重启就丢。Redis 提供 RDB 快照和 AOF 日志两种机制，可以落盘，故障后能恢复，适合对数据可靠性有要求的场景，比如用户会话存储。
- 3) 高可用与集群。Memcached 本身不带主从复制，靠客户端分片。Redis 支持主从同步、哨兵模式自动 failover，以及 Redis Cluster 实现分布式部署，支撑更大规模读写。
- 4) 线程模型。Memcached 用多线程处理并发，多核利用率高。Redis 6.0 之前是单线程事件循环，避免锁竞争，6.0+ 引入了多线程 I/O，但命令执行仍是单线程，保证原子性。
- 5) 内存管理。两者都用预分配内存池减少碎片，但 Redis 对内存更精细控制，支持 LRU 近似淘汰策略，还能设置 key 的 TTL 自动过期。

一般选型时，如果只是做纯缓存，追求极致并发和简单，Memcached 在某些场景仍有优势。但大多数现代应用倾向于 Redis，因为它功能更全面，能当缓存也能当轻量数据库用，像 Session 存储、排行榜、消息队列都能搞定。



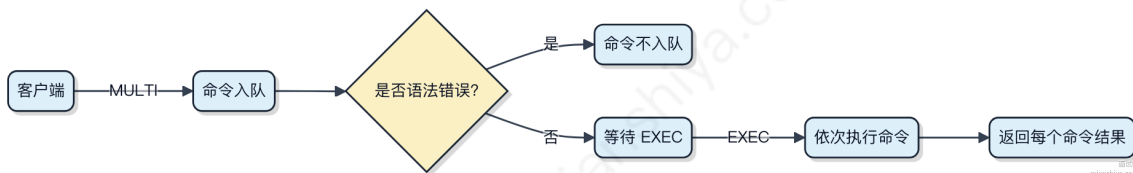
面试鸭
mianshiya.com

Redis 支持事务吗？如何实现？

Redis 确实提供了事务支持，但和传统数据库的事务不一样，它不保证原子性，也不支持回滚。你把一组命令用 `MULTI` 包起来，Redis 会把这些命令放到队列里，然后通过 `EXEC` 一次性发出去，按顺序执行。

1) 开启事务用 `MULTI`，之后的命令不会立即执行，而是进入等待状态。2) 命令被缓存到客户端对应的队列中，直到调用 `EXEC` 或 `DISCARD`。3) 执行 `EXEC` 后，Redis 按顺序串行执行队列里的命令，期间不会被其他客户端中断。

有个关键点，如果某个命令执行出错，比如对字符串类型执行 `HSET`，Redis 不会回滚已经执行的命令，错误命令后面的命令照样继续执行。这也是和 MySQL 这类数据库最大的区别。



可以用 Lua 脚本来弥补这个缺陷，因为 Redis 执行 Lua 脚本是原子的，要么全执行，要么全不执行，能实现真正的原子性操作。像 `redisson` 这类 Java 客户端就是靠 Lua 脚本实现分布式锁的。

另外，Redis 事务也不隔离，中间不会加锁，多个事务可能交错读写同一个 key，需要你自己控制并发访问。

Redis 数据过期后的删除策略是什么？

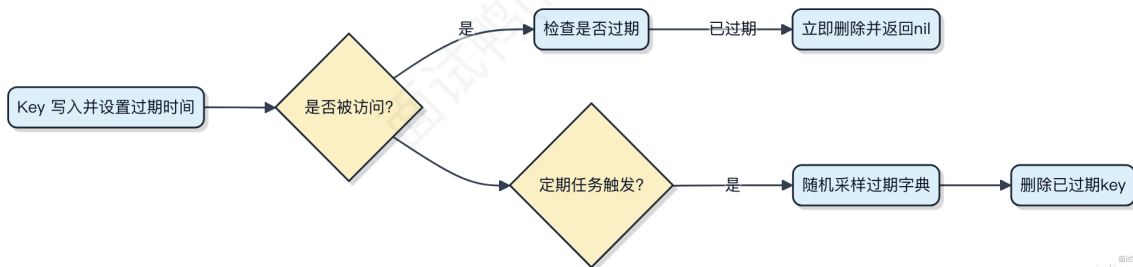
Redis 的过期键删除采用的是 **惰性删除** 和 **定期删除** 两种策略组合，既保证内存能被及时释放，又避免消耗太多 CPU 资源。

1) 惰性删除是在访问 key 的时候才检查它是否已过期，过期就删掉。比如执行 `GET` 操作时，Redis 会先查一下这个 key 是否存在且已过期，如果过期就删除并返回 `nil`。这种方式简单直接，但有个问题：如果一个 key 过期了却没人访问，那它就会一直占着内存。

2) 为了解决这个问题，Redis 还启用了定期删除。它会周期性地随机抽查一部分带过期时间的 key，删掉其中已经过期的。默认每秒运行 10 次，每次最多处理 25 个 key（具体数量可配），并且只会扫描设置了 TTL 的字典。这样可以在不影响性能的前提下，主动清理掉一批“僵尸 key”。

这两种机制互补：定期删除负责兜底，把大部分过期 key 清理掉；惰性删除则确保任何过期 key 都不会在被访问时还活着。

实际场景中，像用 Redis 做缓存、会话存储（比如 `Spring Session + Redis`）都依赖这套机制。如果你发现内存增长异常，可能就是因为过期 key 没被及时删，得看看是不是写得多读得少，导致惰性删除没触发。



Redis 中有哪些内存淘汰策略？

Redis 内存淘汰发生在物理内存不够时，会根据配置的策略清理一些 key 来腾出空间。你得知道，这直接关系到缓存命中率和数据可靠性。

1) noeviction

默认策略，内存满了就不再接受写操作，读请求还能正常处理。适合不能容忍数据丢失的场景，比如用作存储而不是缓存。

2) volatile-lru

只对设置了过期时间的 key 执行 LRU（最近最少使用）淘汰。像 Session 缓存这种有明确生命周期的数据就很合适，避免影响永久数据。

3) allkeys-lru

对所有 key 做 LRU 淘汰，不管有没有设置 TTL。这是最常用的策略之一，适合纯缓存场景，比如用 Redis 做热点数据缓存，像电商商品信息这类。

4) volatile-lfu 和 allkeys-lfu

LFU 是基于访问频率淘汰，比 LRU 更能应对突发流量。allkeys-lfu 现在是推荐的默认策略，尤其适合访问分布不均的场景，比如社交 feed 流。

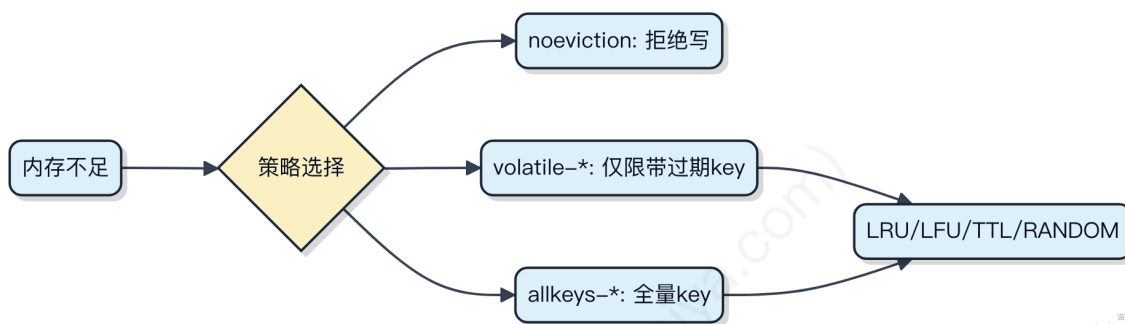
5) volatile-ttl

优先淘汰剩余生存时间短的 key，适用于大量短期 key 的堆积场景。

6) volatile-random 和 allkeys-random

随机删除，简单粗暴。性能开销小，但命中率可能差，一般不作为首选。

实际项目中，如果你用 Redis 做热点缓存，**allkeys-lfu** 或 **allkeys-lru** 能更好地扛住流量。如果混着存了关键数据，就得小心别误删，可以考虑 noeviction 配合监控告警。



选哪个，得看你数据的访问模式和能不能容忍丢失。

Redis 的 Lua 脚本功能是什么？如何使用？

Redis 的 Lua 脚本让你能把一组操作打包成一个原子执行的脚本，整个过程不会被其他命令打断。这在需要保证多个 key 操作一致性时特别有用，比如分布式锁、库存扣减这种场景。

Lua 脚本通过 EVAL 或 EVALSHA 命令执行。EVAL 直接传脚本内容，EVALSHA 用脚本的 SHA1 值调用，避免重复传输代码，提升性能。

脚本里可以用 KEYS 和 ARGV 传参，Redis 会把这两个表映射成客户端传入的键和值。一般建议把会被频繁调用的脚本先加载到 Redis 中，用 SCRIPT LOAD 预加载，再用 EVALSHA 触发。

1) 写个简单的库存扣减脚本：

```
local num = redis.call('GET', KEYS[1])
if not num then return 0 end
```

```
if tonumber(num) >= tonumber(ARGV[1]) then
    return redis.call('DECRBY', KEYS[1], ARGV[1])
else
    return -1
end
```

2) 用 EVAL 执行:

```
EVAL "上述脚本" 1 stock_key 10
```

注意点: 脚本得尽量快, 别做复杂循环或耗时操作, 否则会阻塞主线程。单个脚本执行时间最好控制在几毫秒内, 不然会影响其他请求响应。

像 **原子性**、**减少网络开销**、**逻辑复用** 是它的主要优势。但搞不定长时间任务, 真要处理复杂业务逻辑, 还是得靠服务端程序。



Redis 的 Pipeline 功能是什么?

Redis 的 Pipeline 是一种批量执行命令的机制, 用来减少客户端与服务端之间的网络往返次数。每次 Redis 命令都要经历 “发送命令 → 等待响应” 的过程, 如果连续发 100 条命令, 就会有 100 次 RTT (往返时延)。Pipeline 能把多条命令一次性打包发送, 服务端依次处理后批量返回结果, 整个过程 **压根不经过逐条等待**。

比如你要 set 1000 个 key, 不用 Pipeline 至少要 1000 次网络交互; 用了之后, 可能只要 1 次或几次就能完成, 性能提升非常明显, 在千兆网络下能从几百 QPS 提升到几万 QPS。

它和事务不一样, Pipeline 不保证原子性, 也不支持回滚, 纯粹是优化网络开销的手段。MSET、HSET 这类原生批量命令当然更好, 但如果你要混合执行不同类型的命令, 比如 set、del、incr 各几十个, Pipeline 就是最佳选择。

代码上也很简单, 以 Jedis 为例:

```
try (Jedis jedis = new Jedis("localhost")) {
    Pipeline pipeline = jedis.pipelined();
    for (int i = 0; i < 1000; i++) {
        pipeline.set("key:" + i, "value" + i);
        pipeline.incr("counter");
    }
    List<Object> results = pipeline.syncAndReturnAll();
}
```

- 1) 命令通过 pipelined() 获取管道对象后, 调用不会立即执行
- 2) 实际是把命令缓存在本地队列里
- 3) syncAndReturnAll() 才真正发送并等待所有回复

Redis 中的 Big Key 问题是什么? 如何解决?

Big Key 指的是 Redis 中单个 key 对应的 value 过大, 比如一个 String 超过 10MB, 或者一个 Hash、ZSet 成员数量超过几万个。这种 key 会带来一系列连锁问题。

- 1) 网络传输上，一次读取或写入 Big Key 可能占满带宽，导致其他请求被阻塞。
- 2) 在主从复制时，RDB 或 AOF 重放阶段，Big Key 的加载会拖慢整个恢复过程。
- 3) 内存分配和释放时，Redis 是单线程处理命令，Big Key 的删除操作可能引发长时间阻塞，甚至触发 DEL 命令延迟告警。

常见的解决方案有几个方向。第一是**拆分**，把一个大结构拆成多个小 key。比如原来用一个 Hash 存百万用户积分，现在按用户 ID 分片，用 `user:score:001`、`user:score:002` 这样分散存储。读写时通过计算定位具体 key。

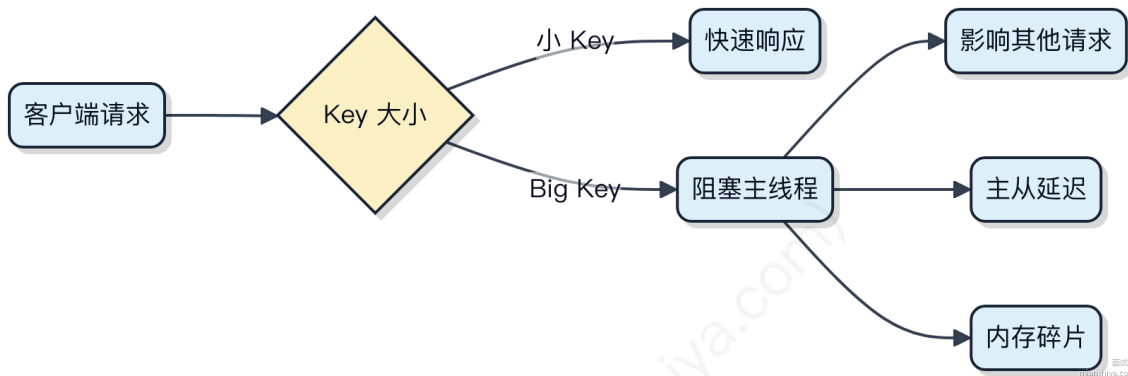
第二是**压缩数据结构**。如果 value 是文本或 JSON，可以考虑用 gzip 压缩后再存入，取出来再解压。虽然增加 CPU 开销，但能显著降低内存和网络开销。

第三是**使用合适的数据类型**。比如本来用 String 存序列化的大对象，不如拆成多个 field 用 Hash，这样可以部分更新，避免全量读写。

代码层面注意批量操作的粒度：

```
// 避免一次性 hgetall 一个超大 hash
List<String> fields = getFields(); // 控制一次只取少量 field
Map<String, String> part = jedis.hgetAll("user:profile");
```

监控上可以用 `redis-cli --bigkeys` 定期扫描，结合 Redis 自身的 slow log 和内存分析工具发现隐患。



如何解决 Redis 中的热点 key 问题？

热点 key 问题说白了就是某个 key 被疯狂访问，QPS 可能上十万，直接把 Redis 实例的 CPU 或网卡打满，甚至拖垮整个服务。

1) 先定位。一般通过 `redis-cli --hotkeys` 或监控系统（比如用 Prometheus + Grafana）看 key 的访问频率。生产环境建议在 proxy 层（比如 TendisProxy 或 Codis）埋点统计 top N 热 key。

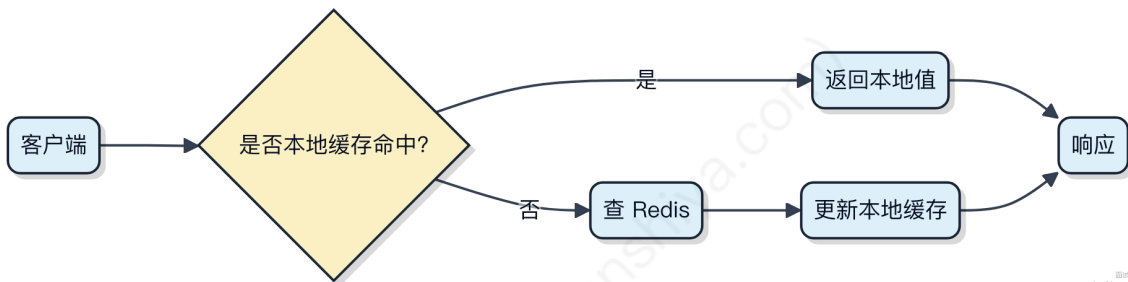
2) 解决思路不是单一的，得组合拳。

本地缓存是第一道防线。应用节点用 Caffeine 或 Guava Cache 把热 key 缓一份，设置短过期时间（比如 1~2 秒），这样大部分请求就压根不经过网络到 Redis。注意缓存穿透问题，可以用布隆过滤器兜底。

3) 拆分 key 也是常用手段。比如原来一个 key 存全量数据，改成 key_1、key_2 分片存储，访问时随机或轮询拉取，把压力分散。适用于静态资源类场景，比如首页 banner。

4) Redis 侧可以做多副本。比如给这个 key 单独起一个只读副本集群，或者利用 Redis Cluster 的 replica 只读节点分流读请求。

5) 极端情况，写扩散。比如把热 key 的值推送到所有客户端内存里，靠消息中间件（比如 Kafka）通知更新。这招重运维，搞不定一致性别轻易上。



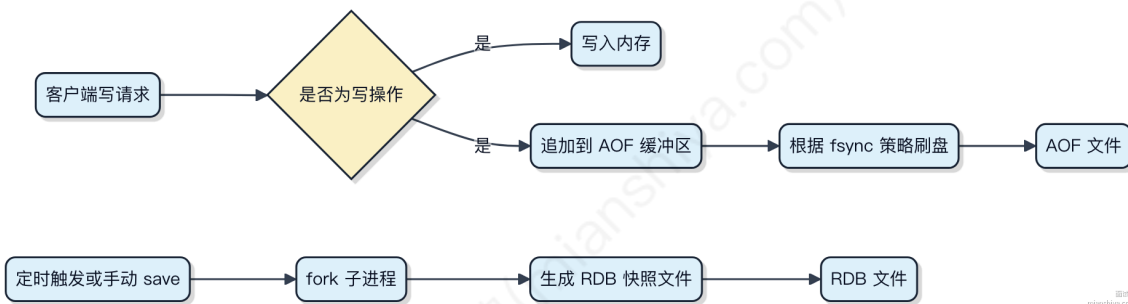
Redis 的持久化机制有哪些?

Redis 的持久化主要是为了在实例重启后能恢复数据，它提供了两种核心机制，实际应用中经常组合使用。

1) RDB 是某个时间点的数据快照。它会 fork 一个子进程，把当前内存数据全量写入 RDB 文件。文件紧凑，恢复速度快，适合做备份。比如用 Redis 做缓存集群时，每天凌晨生成一次 RDB，相当于一次“全量归档”。但两次快照之间如果宕机，中间的数据就丢了，所以无法做到完全不丢数据。

2) AOF 记录的是每一条写操作命令。它像数据库的 binlog，以追加方式写入 aof 文件。通过配置 fsync 策略控制刷盘频率。appendfsync everysec 最常用，性能和安全性比较均衡，最多丢 1 秒数据。不过 AOF 文件通常比 RDB 大，恢复也慢一些，因为要重放命令。

生产环境一般同时开启 RDB 和 AOF。Redis 重启时优先使用 AOF 恢复数据，更完整。AOF 重写机制可以定期压缩日志，去掉无效操作，避免文件无限膨胀。



Redis 在生成 RDB 文件时如何处理请求?

Redis 在生成 RDB 文件时用的是 **fork** 子进程，这个设计很关键。父进程继续处理客户端请求，子进程把内存数据快照写到磁盘，两个进程共享同一份内存数据，互不干扰。

1) fork 之后子进程拿到的是那一刻的内存副本，所以后续的写操作不会影响正在生成的 RDB 内容，保证了快照的一致性。

2) 所有读请求正常由父进程处理，写请求也照样能接，Redis 不会因为持久化就停服。

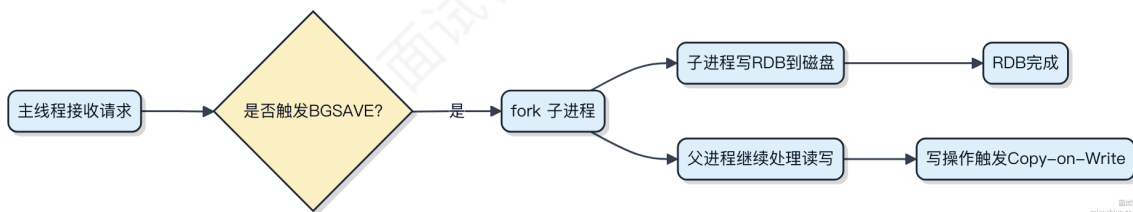
3) 但写请求会触发 **Copy-on-Write** 机制。当有 key 被修改时，操作系统才会把对应内存页复制一份给子进程，避免浪费资源。小改少复制，大改多复制，系统自动控制。

不过这里有个实际风险：如果恰好在做快照时大量 key 被修改，COW 会导致物理内存使用翻倍，极端情况可能把机器搞崩。所以一般建议避开业务高峰期做 BGSAVE，或者直接关掉手动触发，靠 AOF + RDB 混合模式来平衡。

代码上你看到的就是一个系统调用：

```
pid = fork();
if (pid == 0) {
    // 子进程执行 rdbSave()
} else {
    // 父进程继续跑 event loop
}
```

整个过程父进程基本不阻塞，除非 fork 那一瞬间要拷贝页表，数据量特别大时可能卡几十毫秒。其他时候请求压根不经过 RDB 流程，该读读该写写。



Redis 的哨兵机制是什么？

Redis 哨兵机制是为了解决主从架构下主节点故障无法自动恢复的问题。你搭了主从复制，数据能同步，但主节点挂了，整个集群就写不了了，得靠人去切主，线上系统哪得起这种操作。

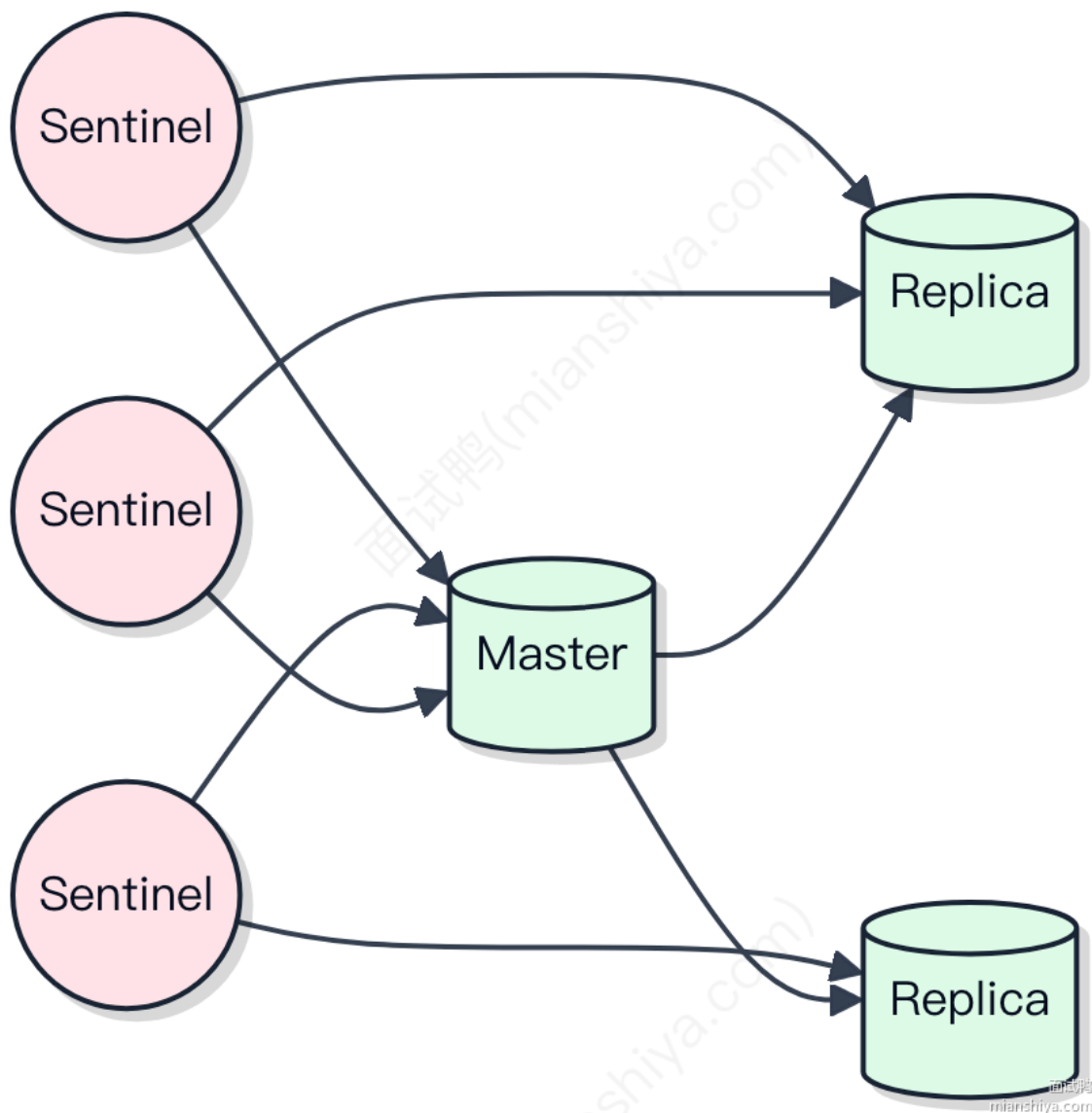
哨兵其实是独立运行的进程，一般部署 3 到 5 个，组成一个哨兵集群。它们会持续监控 Redis 主从节点的健康状态，通过心跳机制判断节点是否在线。一旦发现主节点失联，多个哨兵达成共识（比如 2/3 投票），就会触发故障转移流程。

这个过程分几步走：先选一个从节点提升为新主，再通知其他从节点切换主从关系，最后把旧主恢复后也变成从节点。客户端后续请求会被重定向到新主节点，服务继续可用。

代码层面，Spring Boot 项目通常配置的是 Redis Sentinel 地址列表，而不是直接写主节点。

```
@Bean
public RedisConnectionFactory connectionFactory() {
    RedisSentinelConfiguration config = new RedisSentinelConfiguration();
    config.setMaster("mymaster");
    config.setSentinels(sentinels); // 指向哨兵集群
    return new LettuceConnectionFactory(config);
}
```

整个过程不需要人工介入，高可用性就靠这套机制撑起来。不过要注意，哨兵模式还是单点写入，扩容得靠分片，比如用 Redis Cluster 才能横向扩展。



Redis 主从复制的实现原理是什么？

Redis 主从复制是靠主节点把写操作同步给从节点来实现数据一致的。整个过程分全量同步和增量同步两个阶段，大部分情况下两者会配合使用。

第一次建立复制关系时走全量同步。从节点发起 `psync` 命令带上自己的 replication ID 和 offset，主节点发现不匹配就会触发全量同步。主节点 fork 一个子进程生成 RDB 快照，同时用缓冲区记录这期间的新写命令。RDB 传完后，再把缓冲区的写命令发给从节点，确保数据完整。

之后进入增量同步阶段。主节点每执行一个写命令，都会通过网络把命令传播给所有从节点。从节点按顺序执行这些命令，保持和主节点状态一致。如果网络短暂中断，从节点可以通过 `repl_backlog_buffer` 中的积压日志快速追平，不需要重新全量复制。

心跳检测也很关键。从节点每隔 1 秒发 `REPLCONF ACK` 给主节点，主节点也能感知从节点是否存活。这种机制让复制具备一定容错能力，比如在 Redis Sentinel 或 Cluster 模式下支撑故障转移。



要注意的是，复制是异步的，从节点可能有秒级延迟。另外，默认配置下即使主节点持久化关闭，复制照样进行，这时候一旦宕机重启，从节点跟着同步空数据，会造成雪崩式数据丢失。生产环境一定要开启主节点 AOF，并设置 `min-slaves-to-write` 防止脑裂。

Redis 集群的实现原理是什么？

Redis 集群的核心是数据分片和去中心化协作。它把整个数据空间切成 16384 个槽，每个键通过 CRC16 算出哈希值后对 16384 取模，落到对应的槽上。集群里的每个主节点负责一部分槽，比如节点 A 管 05000，B 管 5001~10000，C 管 10001~16383。

客户端可以直接连任意节点查数据，如果请求的 key 不归这个节点管，它会返回一个 `MOVED` 指令，告诉客户端该去哪个节点找。聪明的客户端会缓存 slot 和节点的映射关系，下次直接访问目标节点，**压根不经过转发**。

节点之间通过 Gossip 协议互相通信，每秒交换几次信息，同步自己负责的槽、健康状态、新节点加入等。这种设计没有中心节点，靠多数主节点投票完成故障转移。比如某个主挂了，它的从节点发现后会发起选举，拿到半数以上同意就能顶上。

集群也支持动态扩缩容。加新节点时先分配槽，再把旧节点上的槽迁过去，迁移过程不影响服务。可以用 `redis-cli --cluster` 工具自动完成。

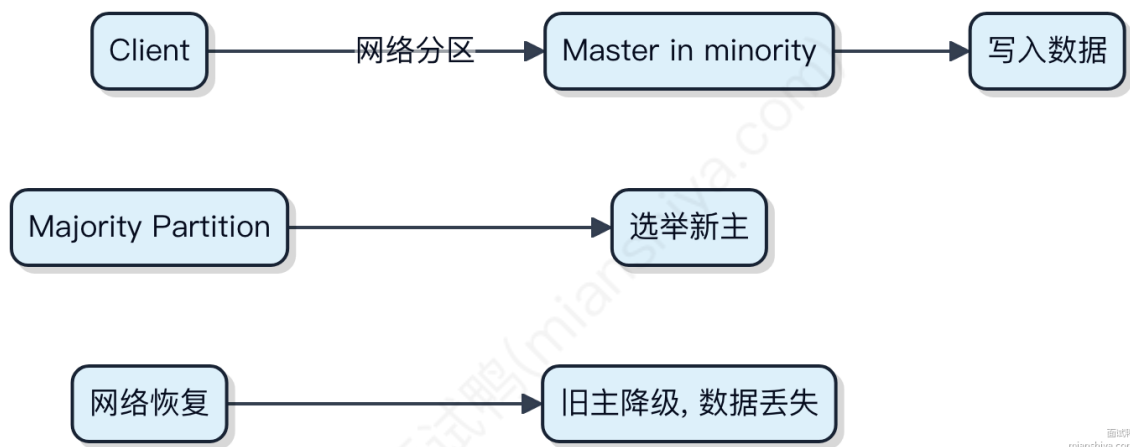
需要注意的是，集群不保证强一致性，异步复制可能导致少量数据丢失。另外多 key 操作必须都在同一个 slot，否则会报错。要用的话得靠 hash tag 强制让多个 key 落到同一槽，比如 `{user1000}.name` 和 `{user1000}.age`。

Redis 集群会出现脑裂问题吗？

Redis 集群在特定网络分区场景下，确实可能出现类似脑裂的现象，但它的设计机制能在一定程度上自动收敛和恢复。

- 1) 集群依赖多数派选举：Redis 集群采用 Gossip 协议传播节点状态，主节点故障时，从节点需要获得集群中**多数主节点的投票**才能晋升为主。如果网络分区导致某边节点数不足半数，就无法选出新主，这一侧会拒绝写入。
- 2) 小分区可能短暂失服：假设 6 节点集群（3 主 3 从），网络分成 4 节点和 2 节点两部分。4 节点这边能凑够多数派，可完成故障转移；而 2 节点那边既不能选主，也不会对外提供服务，相当于主动“自裁”，避免数据不一致。
- 3) 大分区若未隔离主：最危险的情况是，网络分区把多数节点和原主都划在同一侧，原主仍可接收写请求。而另一侧因无法达成多数派，不会升主。这时其实**不会形成双主**，只有原主在工作，不构成传统意义的脑裂。

真正风险在于客户端路由错乱。比如客户端与多数派网络断开，但仍连着一个“孤立”的主节点并持续写入。等网络恢复后，这个主的数据会被新主覆盖，造成**数据丢失**，看起来像脑裂后果。



所以严格说，Redis 集群通过多数派机制防住了双主，但无法完全避免因客户端连接滞留引发的数据不一致问题。生产环境要配合 `min-replicas-to-write` 和 `min-replicas-max-lag` 控制写安全。

Redis 中如何实现分布式锁？

实现分布式锁，最朴素的想法是用一个能共享的状态来标记“锁被谁占了”，Redis 天然适合这个场景。关键不是 set 本身，而是怎么保证 set 的原子性和后续的可靠性。

1) 先说最基础的一招： `SET resource_name random_value NX PX 30000`

NX 是“不存在才设置”，PX 是毫秒级过期时间，random_value 要保证唯一，一般用 UUID 或线程 ID 加时间戳。这一步就能防住并发重复获取。

但光设上锁不行，释放的时候得小心。不能直接 del，不然可能删了别人的锁。正确做法是 Lua 脚本：

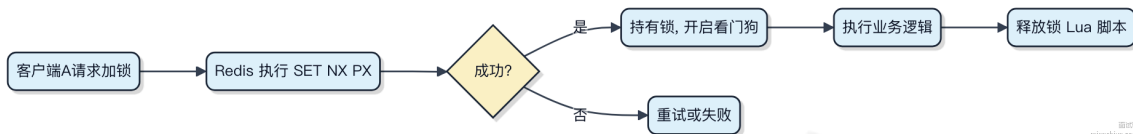
```
if redis.call("get", KEYS[1]) == ARGV[1] then
    return redis.call("del", KEYS[1])
else
    return 0
end
```

这个脚本保证“判断 + 删除”是原子的，值对才删，避免误删。

2) 上面这套在单 Redis 实例下够用，但主从切换时可能丢锁——主节点还没同步到从节点就挂了，新主没这把锁，别人就能抢到。这就是**锁失效**问题。

想更稳就得用 Redlock 算法，它要求跨多个独立 Redis 节点加锁，必须大多数节点成功才算拿到锁。虽然性能差些，但提高了容错性。不过实际中很多人还是用单实例 + 高可用部署，配合超时和重试扛住大部分场景。

3) 生产里常见的是 Redisson，它封装了可重入锁、看门狗自动续期。比如锁快过期了，后台线程会自动延长过期时间，只要线程还活着。这解决了业务执行时间不确定的问题。



Redis 实现分布式锁时可能遇到的问题有哪些？

Redis 实现分布式锁看着简单，其实坑不少，搞不定的话线上容易出大事。

- 1) **锁过期时间不好定**。业务执行时间不确定，设短了锁自动释放了任务还没完，别的实例就能抢锁进来，数据就乱了。设长了，万一节点宕机，锁没法释放，整个系统就卡死了。
- 2) **不是原子操作就危险**。比如先 set key 再 expire，中间 Redis 宕机，expire 没执行，锁就成“永久”了。一定要用 `SET key value NX EX seconds` 一把搞定。
- 3) **主从切换导致锁失效**。Redis 主节点写入了锁，还没同步到从节点，主节点挂了，从节点升主，新来的请求在新主上又能加同名锁——同一个资源被两个客户端同时持有锁，这叫锁的“脑裂”。
- 4) **误删别人的锁**。自己加的锁得自己删，但很多人直接 del key，如果业务超时，锁已经过期被别人获取，你这一删，就把别人的锁干掉了。正确做法是 value 用唯一标识（比如 UUID），删除前先 get 判断是不是自己的锁。
- 5) **Redlock 算法争议大**。虽然它想解决单点问题，但 Martin Kleppmann 和 Antirez 都吵过一轮，实际场景下网络延迟、GC 停顿都会让它的安全性打折扣。一般中小公司用 Redis Cluster + 带唯一值的 SET + Lua 脚本删除就够了。

真要高可靠，建议直接上 ZooKeeper 或 etcd，它们基于一致性协议，天生能保证锁的安全性，就是性能低点。Redis 锁追求的是高性能，牺牲了一定安全性，得清楚这个权衡。



Redis 中的缓存击穿、缓存穿透和缓存雪崩是什么？

缓存击穿是指某个热点 key 在失效的瞬间，大量请求直接打到数据库。因为缓存没命中，数据库扛不住瞬时流量，比如商品秒杀场景下库存 key 过期，成千上万请求同时查库。

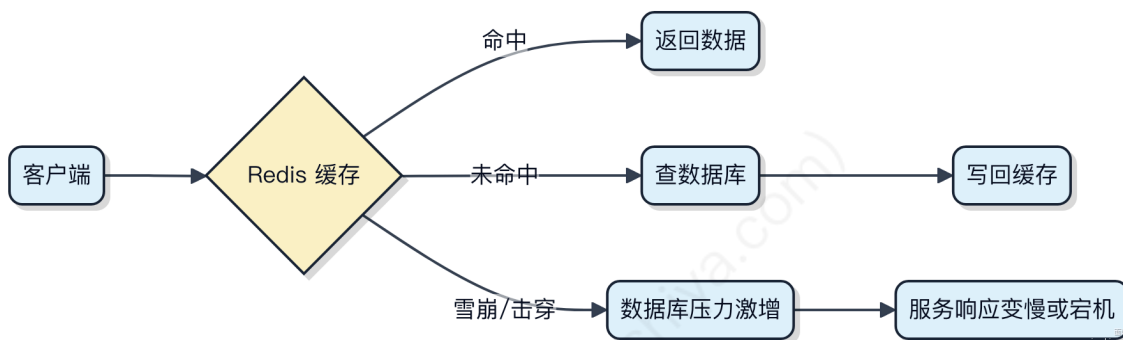
- 1) 可以通过**逻辑过期**或永不过期策略避免物理失效。比如把过期时间存在 value 里，应用层判断是否逻辑过期，再异步更新。
- 2) 也可以用互斥锁，比如 Redis 的 SETNX 控制只有一个线程去加载数据，其他线程等待。但要注意锁超时和单点故障问题，结合 Redlock 谨慎使用。

缓存穿透是查询一个根本不存在的数据库，缓存和数据库都没有，导致每次请求都落库。恶意攻击或非法 id 都可能触发。

布隆过滤器是常见解法，提前拦截无效请求。请求先过布隆过滤器，存在才查缓存和数据库。Guava 或 Redisson 都有实现，注意误判率和内存占用平衡。

缓存雪崩是大量缓存同时失效，大量请求击穿到数据库。比如缓存集群宕机，或者一批 key 设置了相同的过期时间。

解决方案一是过期时间加随机值，比如基础时间 + 300 秒内随机偏移，避免集体退役。二是做二级缓存，比如本地 Caffeine 缓存兜底，虽然可能脏读但能抗住压力。



这三个问题本质都是缓存不可用导致数据库被压垮，解决思路无非是“挡”、“防”、“限”。挡是用布隆过滤器挡住非法请求，防是防止热点失效，限流是最后一道防线，像 Sentinel 可以配合使用。

Redis 中如何保证缓存与数据库的数据一致性？

缓存与数据库的一致性问题，本质上是在写操作发生时，如何让缓存和 DB 的状态保持同步。现实中没有完美方案，只有权衡。

1) 先更新 DB，再删除缓存（推荐）

这是 **Cache Aside Pattern** 的标准做法。写请求来临时，先更新数据库，然后把缓存删掉。下次读请求会触发一次缓存重建。这种策略下，缓存不一致的时间窗口很短，且在高并发场景下相对安全。比如用户信息更新，先改 MySQL，再删 Redis 里的 user:1001。

2) 先删缓存，再更新 DB（有风险）

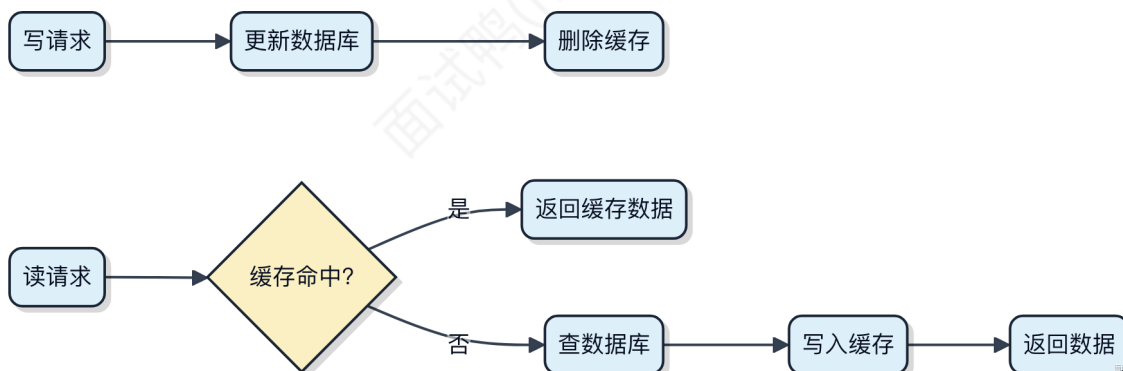
如果删完缓存后，还没来得及更新 DB，这时一个读请求进来，就会把旧数据从 DB 加载到缓存，导致脏数据。这种情况在高并发下可能持续几毫秒，但确实存在。

3) 延迟双删（缓解方案）

针对上面的问题，可以在更新 DB 前先删一次缓存，更新完 DB 后再 sleep 几百毫秒，再次删除缓存。虽然能降低脏数据概率，但 sleep 时间难控制，一般只在强一致性要求高且数据变更频繁的场景用，比如库存系统。

4) 通过 Binlog 异步处理（解耦）

像阿里开源的 **Canal** 可以监听 MySQL 的 Binlog，把数据变更事件发到 MQ，由消费者去清理或更新缓存。这种方式完全解耦，适合复杂系统，比如订单状态变更同步到缓存。



最终建议：优先用“先更新 DB，再删缓存”，配合缓存过期时间兜底。真要强一致，就得上分布式锁或者版本号机制，但性能代价大，多数业务没必要。

如何使用 Redis 快速实现排行榜?

用 Redis 做排行榜，核心是利用 **Sorted Set** 这个数据结构。它天然支持按分数排序，增删改查都是 $O(\log N)$ ，性能完全够用。

- 1) 每个玩家的得分用 `ZADD leaderboard {score} {player}` 写入。后续更新直接覆盖同名 member，Redis 会自动按新分排序。
- 2) 查 Top 10 直接 `ZRANGE leaderboard 0 9 WITHSCORES`，从高到低取前 10。如果要倒序（比如末尾排名），用 `ZREVRANGE`。
- 3) 想查某个玩家的排名，`ZSCORE leaderboard player` 拿分数，再用 `ZRANK leaderboard player` 拿正序名次，或 `ZREVRANK` 拿倒序。

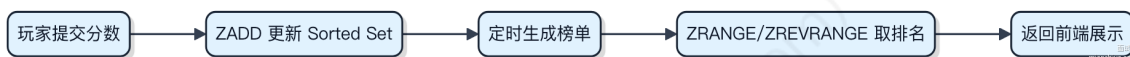
实际场景里，像游戏积分榜、直播热度榜、电商销量排行，基本都这么干。Redis 的 Sorted Set 在内存里维护跳跃表，查询快得飞起，写入也高效。

代码示例：

```
// 添加或更新分数
jedis.zadd("leaderboard", 95.5, "player1");

// 获取 Top 10
Set<Tuple> top10 = jedis.zrevrangeWithScores("leaderboard", 0, 9);
```

注意点：数据量特别大时，比如千万级用户，单个 zset 可能撑不住。可以按维度拆，比如分服排行榜，或者用 zset + 分页缓存组合方案。另外别忘了设过期时间，避免数据堆积。



如何使用 Redis 快速实现布隆过滤器?

布隆过滤器的核心是用多个哈希函数和位数组判断元素是否存在，Redis 本身不直接支持位操作的复合结构，但可以用 **HyperLogLog** 或 **Bitmap** 搭配外部逻辑来实现。实际项目里更常见的是用 Redis Module，比如 **RedisBloom**。

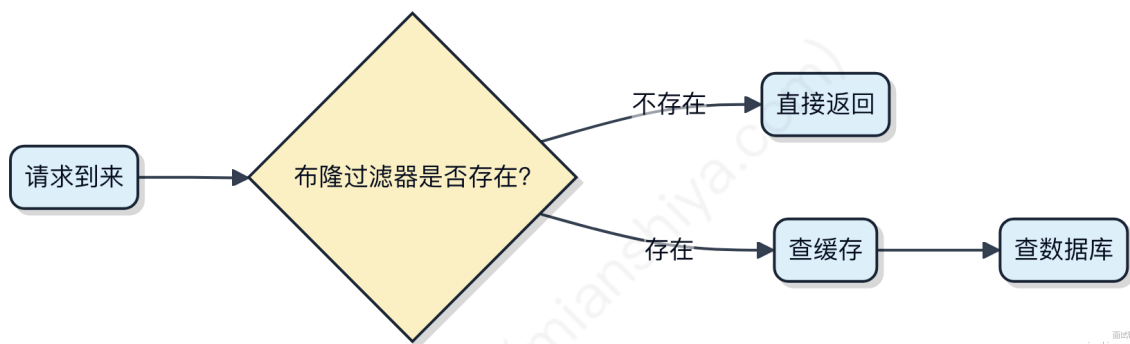
- 1) 先说原理。布隆过滤器往里加数据时，会算出多个哈希值，对应到位数组的不同位置，把这些位设为 1。查的时候看这些位是不是都是 1，有一个是 0 就说明肯定不存在，全是 1 则大概率存在（有误判率）。
- 2) 在 Redis 里搞这个，最省事的是加载 RedisBloom 模块。它提供了 `BF.ADD`、`BF.EXISTS` 这种命令，直接就能用。比如：

```
BF.ADD user:exists "user123"
BF.EXISTS user:exists "user456"
```

背后 RedisBloom 自己管理位数组和哈希函数，你不用操心细节。

- 3) 如果不允许装模块，就得自己用 setbit 和 getbit 模拟。比如用 `key + offset` 表示某个哈希位置，自己写逻辑算 n 个哈希值。但要注意控制 key 的粒度，别让单个 key 太大，不然搞不定。

典型场景是缓存穿透防护。像商品详情页查 DB 前，先过一遍布隆过滤器，99% 的无效 ID 被挡掉，数据库压力直接降下来。



Redis 性能瓶颈时如何处理？

Redis 出现性能瓶颈，一般先看是 CPU、内存、网络还是磁盘成了短板。大多数情况瓶颈在单线程模型处理大量请求时的吞吐上限。

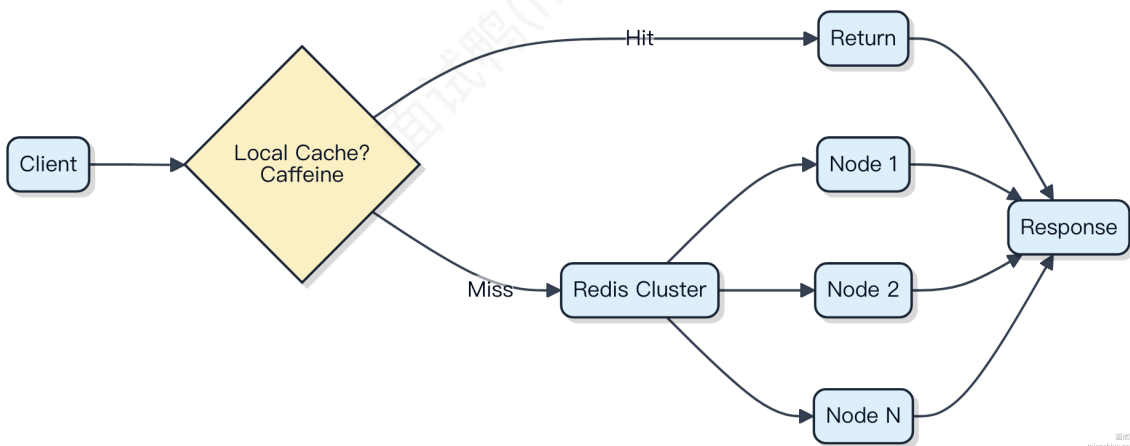
1) 先用 `INFO stats` 和 `redis-cli --latency` 观察指标。如果 QPS 上不去但 CPU 没到顶，可能是大 key 或慢查询拖累。像 `KEYS` 这种阻塞命令得禁用，改用 `SCAN`。

2) 大 key 拆分是常见优化手段。比如一个包含 10 万元素的 hash，读写都慢。可以按业务维度拆成多个小 hash，或者用 Redis Hash 的 slot 机制分散。

3) 数据结构选型也很关键。比如用 `ZSET` 做时间排序队列没问题，但如果频繁修改分数， $O(\log N)$ 的复杂度在百万级数据下就明显了。这时候可以考虑降级为 `LIST` + 外部索引，或者引入 **Redis Streams**，它更适合高吞吐的消息场景。

4) 真扛不住就横向扩展。Redis Cluster 能自动分片，数据按 key 分布到 16384 个 slot，跨节点访问会重定向。不过 multi-key 操作必须在同一个 slot，可以用 hash tag 强制路由，比如 `user:{1000}:profile` 和 `user:{1000}:orders`。

5) 极端场景下，本地缓存加一层 **Caffeine** 或 **Guava Cache**，把最热的数据挡在 Redis 前面。比如用户登录信息，99% 请求集中在 10% 的 key 上，本地缓存能减少 80% 以上的远程调用。

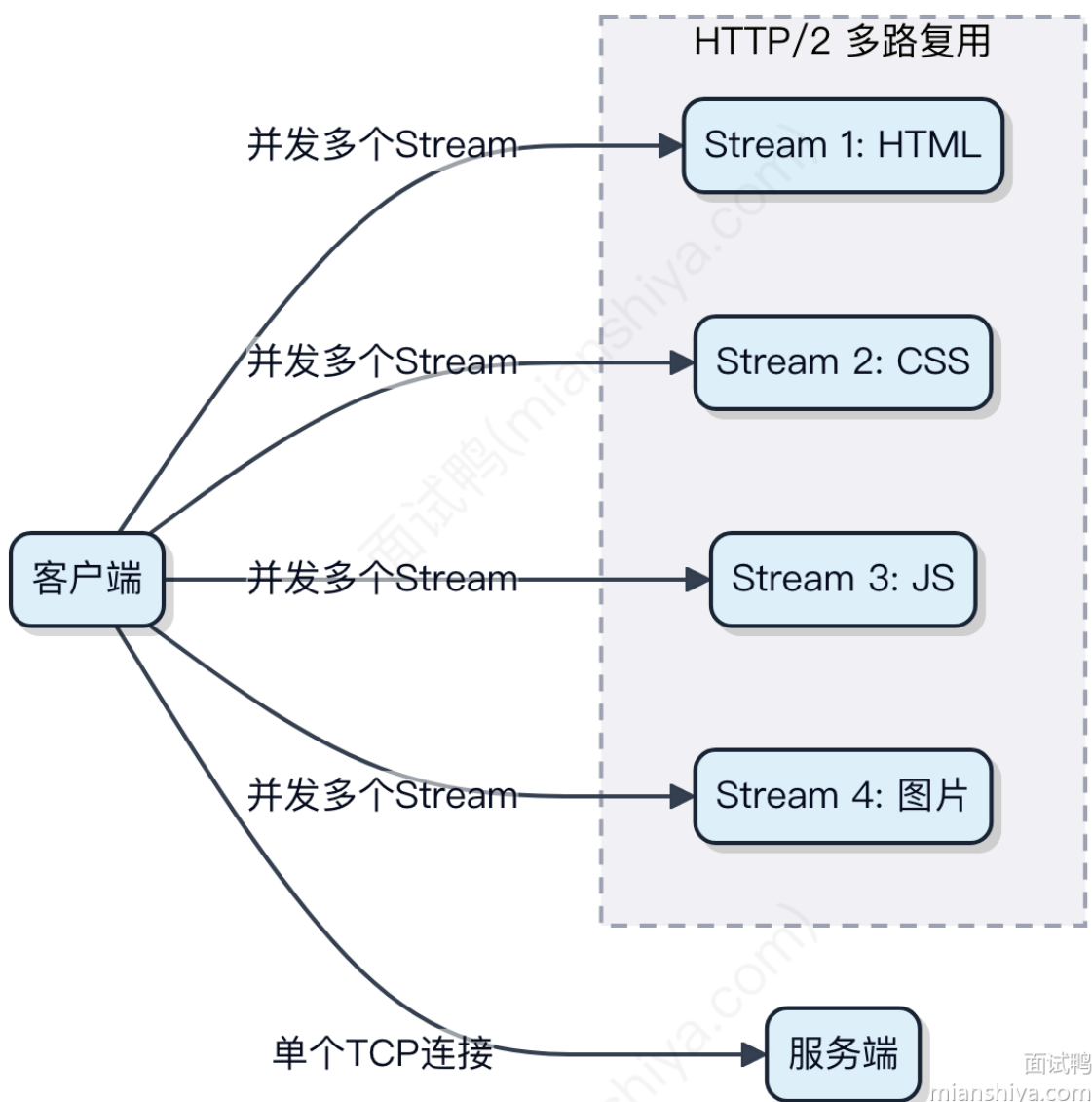


HTTP 1.0 和 2.0 有什么区别？

HTTP/2 最大的变化是引入了**二进制分帧层**，这直接改掉了 HTTP/1.1 的“一请求一响应”、串行传输的老毛病。以前每个请求都要单独建 TCP 连接，或者走管道化但容易阻塞，现在一个连接就能并发处理多个请求，效率提升非常明显。

- 1) 头部压缩。HTTP/1.1 每次都传完整头，重复的 Cookie、User-Agent 这些字段特别浪费。HTTP/2 用 HPACK 算法压缩头部，相同字段只传差量，能省下 30% 到 50% 的头部流量。
- 2) 多路复用。这是最核心的体验提升。HTTP/1.1 队头阻塞的问题在 HTTP/2 里被解决了，多个请求和响应可以同时在一个 TCP 连接上通过独立的数据帧交错传输，谁也不等谁。
- 3) 服务器推送。这个功能允许服务端提前把客户端可能需要的资源推过去，比如页面要加载 main.css，服务端可以直接推，不用等浏览器解析 HTML 再发请求。不过实际用得不多，很多场景反而会增加负担。
- 4) 优先级机制。客户端可以给不同资源设置优先级，比如 JS、CSS 比图片更重要，服务端就可以按序发送，优化页面渲染体验。

虽然有这些优势，但 HTTP/2 对网络环境要求更高。丢包严重时，因为所有数据都在一个 TCP 流里，一旦出错整个连接都会卡住。这也是后来 HTTP/3 改用 QUIC 协议的原因之一。



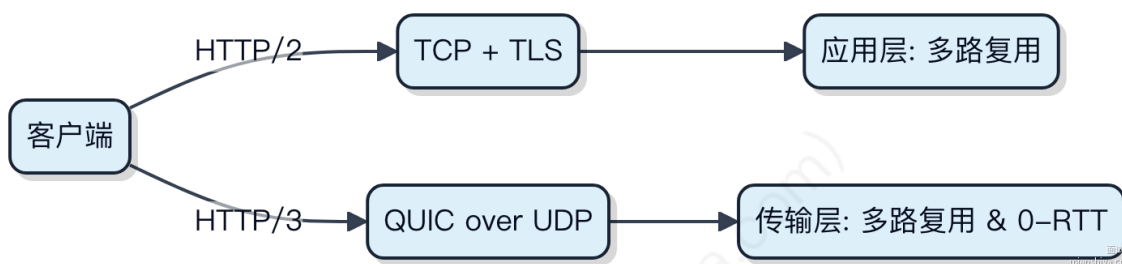
HTTP 2.0 和 3.0 有什么区别？

HTTP/2 的核心是多路复用，它把多个请求塞进同一个 TCP 连接里，并发传输。每个请求拆成帧，通过流（Stream）来区分，这样就避免了 HTTP/1.1 的队头阻塞问题。但底层还是 TCP，万一丢包，整个连接都得等重传，影响所有流。

HTTP/3 直接改了传输层，用 **QUIC** 协议替换了 TCP。QUIC 跑在 UDP 上，自己实现可靠传输和拥塞控制。最关键的是，它把连接绑定到客户端的连接 ID，而不是 IP + 端口。切换网络时，比如从 Wi-Fi 切 4G，连接不会断，直接续传。而且 QUIC 的多路复用是在传输层做的，丢一个流不影响其他流，彻底解决了队头阻塞。

实际应用中，Google 早就用 QUIC 撑起大部分流量，Cloudflare、Fastly 这些 CDN 厂商也全面支持 HTTP/3。国内阿里云、腾讯云也在逐步铺开。

代码层面你看不到太多变化，因为 HTTP/3 是浏览器和服务器自动协商的。但服务端配置要注意，比如 Nginx 需要编译 BoringSSL 或 quiche 支持，监听 UDP 443 端口。



简单说，HTTP/2 解决了应用层的并发问题，HTTP/3 连协议切换、移动网络切换这些场景都扛住了，更适应现代网络环境。

HTTP 和 HTTPS 有什么区别？

HTTP 和 HTTPS 的本质区别在于**安全性**，HTTPS 不是新的协议，而是 HTTP 跑在 SSL/TLS 之上。

1) 通信内容加密

HTTP 明文传输，中间人能直接看到请求和响应内容。HTTPS 在 TCP 握手后、HTTP 通信前，先通过 SSL/TLS 协议协商密钥，后续数据都用对称加密传输。比如浏览器访问支付宝，账号密码要是走 HTTP，网络里的路由器都能看见。

2) 身份认证

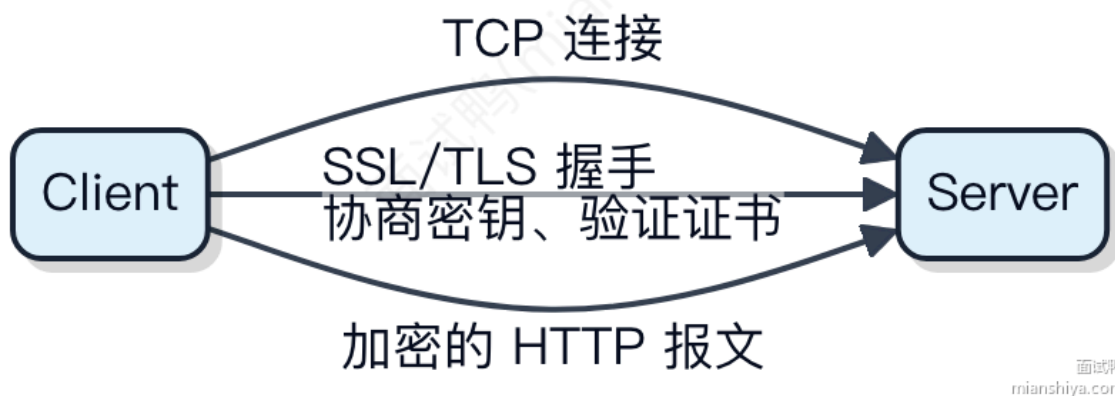
HTTPS 需要证书来证明“你真的是你”。服务端把数字证书发给客户端，客户端通过 CA（比如 Let's Encrypt、DigiCert）验证证书合法性。没有这一步，钓鱼网站随便仿个淘宝页面就搞定了。

3) 防篡改

TLS 会对数据做完整性校验，用 MAC（消息认证码）机制。哪怕中间有人改了一个字节，接收方解密时就会发现校验失败，直接断开。

性能上，HTTPS 多了握手过程，首次建立连接要多花 1-2 个 RTT。但现代 TLS 1.3 已优化到 1-RTT 甚至 0-RTT，加上 CDN 普遍支持，实际差距很小。主流站点像 Google、淘宝、微信支付全默认 HTTPS。

代码层面，Java 里用 `HttpsURLConnection` 替代 `HttpURLConnection`，关键是要信任的证书库。Netty 或 Spring WebFlux 中配置 `SSLContext` 即可启用。



现在没有理由再用纯 HTTP，搜索引擎也优先收录 HTTPS 站点。小项目可以用 Nginx + 免费证书快速上 HTTPS。

TCP 的粘包和拆包能说说吗？

TCP 本身没有“包”的概念，它传输的是字节流。所谓粘包和拆包，其实是应用层读取数据时，没按预期边界处理导致的。

发送方连续发两条消息，接收方可能一次性收到两条（粘包），也可能一条被分成两次读到（拆包）。根本原因是 TCP 不保证收发消息的次数对等，只保证字节顺序正确。

常见场景比如用 Netty 做通信框架，如果没加 **编解码器** 处理消息边界，就会出问题。解决方案得在协议层面定好规则：

- 1) 固定消息长度，不够补零
- 2) 用特殊分隔符，比如换行符或特殊字符
- 3) 头+体格式，头部存实际数据长度

最常用的是第三种，像 Dubbo、RocketMQ 内部都用 LengthFieldBasedFrameDecoder 来解。

举个例子，你发了两个 100 字节的消息，TCP 可能合并成一个 200 字节的包发过去，也可能因为 MTU 拆成 150 + 50。接收端必须靠协议解析才能还原原始消息边界。

不处理粘包拆包，服务间通信基本搞不定。尤其做 RPC 或网关，编解码这步压根不经过业务代码，全靠框架提前配好。

为什么 TCP 挥手需要有 TIME_WAIT 状态？

TCP 四次挥手中，主动关闭连接的一方会进入 TIME_WAIT 状态，这个状态的存在不是设计缺陷，而是为了解决两个关键问题。

- 1) 确保最后一个 ACK 能被对方收到。如果这个 ACK 丢了，被动关闭方会重传 FIN，此时如果主动方已经彻底关闭，就无法响应，导致对方不能正常释放连接。留一个 TIME_WAIT 时间（一般是 2MSL），就是为了能重发这个 ACK。
- 2) 防止旧连接的残留数据包干扰新连接。TCP 协议用四元组（源IP、源端口、目的IP、目的端口）标识一个连接。如果连接快速复用端口，网络中延迟的老数据包可能会被误认为是新连接的数据。等待 2MSL 能让所有老数据在网络中自然消失。

MSL 是报文最大生存时间，一般取 30 秒到 2 分钟不等，所以 TIME_WAIT 状态最长可能持续 4 分钟。虽然它保障了可靠性，但在高并发短连接场景下，比如 Web 服务器大量请求，容易出现端口耗尽的问题。这时候可以通过开启 `SO_REUSEADDR` 选项，允许处于 TIME_WAIT 的 socket 被复用，缓解这个问题。

对于客户端来说，频繁建立短连接要特别注意 TIME_WAIT 的累积；服务端通常不会成为主动关闭方，受影响较小。Nginx、Tomcat 这类服务在调优时都会关注这个状态的数量。

TCP 滑动窗口的作用是什么？

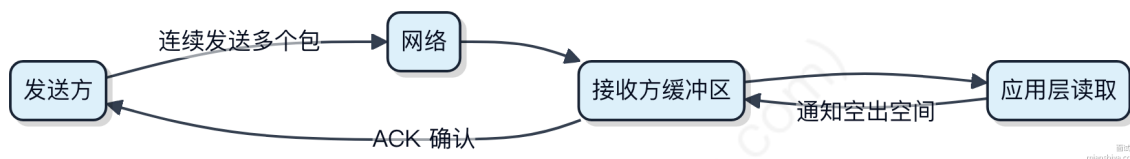
滑动窗口的核心是解决可靠传输下的效率问题。TCP 要保证不丢包、不乱序，但如果每发一个包就等 ACK 再发下一个，延迟会把吞吐压垮。滑动窗口就是让发送方不用等确认，可以连续把多个数据包扔出去，相当于“批量发货”。

接收方有个缓冲区，能存还没被应用读走的数据。这个缓冲区大小就是接收窗口（rwnd），它告诉发送方自己还能收多少。发送方根据这个值维护自己的发送窗口，已发未确认的包不能删，但可以继续发新包，直到窗口填满。

- 1) 窗口左边界是最早发出去还没确认的包
- 2) 右边界是还能发的最大序号
- 3) 收到 ACK 后左边界右移，窗口“滑动”

如果接收方处理慢，缓冲区满了，就会把窗口设为 0，发送方暂停。等应用读走数据空出空间，接收方再发个“窗口更新”报文恢复传输。

网络拥塞时，光看接收方窗口不够，还得考虑链路承载能力。所以有拥塞窗口（cwnd），和 rwnd 取小值作为实际可用窗口。慢启动、拥塞避免这些机制就是在动态调整 cwnd。

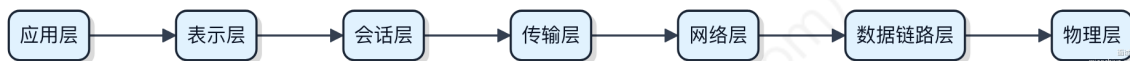


OSI 七层模型是什么？

OSI 七层模型是网络通信的参考框架，把数据从一台设备传到另一台的过程拆成了七个层次，每一层各司其职，下层为上层服务。

- 1) 物理层负责最底层的信号传输，比如网线、光纤这些介质上传输的高低电平，就是0和1的物理表示。
- 2) 数据链路层管的是相邻设备之间的数据帧传输，像以太网（Ethernet）就在这层，交换机也是工作在这里，通过MAC地址寻址。
- 3) 网络层负责跨网络的路径选择，IP协议就在这一层，路由器根据IP地址转发数据包。
- 4) 传输层保证端到端的数据可靠或高效传输，TCP和UDP是典型代表。TCP提供连接管理、重传、流量控制，而UDP不搞这些，直接发。
- 5) 会话层控制会话的建立、管理和终止，比如RPC调用或者数据库连接的会话保持。
- 6) 表示层处理数据格式转换，比如加密解密、压缩解压、字符编码转换，让不同系统的数据能互相理解。
- 7) 应用层直接面向用户程序，HTTP、FTP、DNS这些协议都在这层，你写的Web服务基本都跑在这一层之上。

实际互联网大多用的是TCP/IP四层模型，把前四层合并成“网络接口层”，会话层和表示层也常被并入应用层，但OSI模型在教学和标准化中还是很有指导意义。



Cookie、Session、Token 之间有什么区别？

这三个东西说白了都是为了解决 HTTP 无状态的问题，但思路完全不同。

Cookie 是浏览器端的一小块文本，服务器通过 Set-Cookie 响应头下发，浏览器自动带上 Cookie 请求头回传。它就存在客户端，能被 JS 读写（除非 HttpOnly），所以有 XSS 风险。一般用来存 session id 或用户偏好。

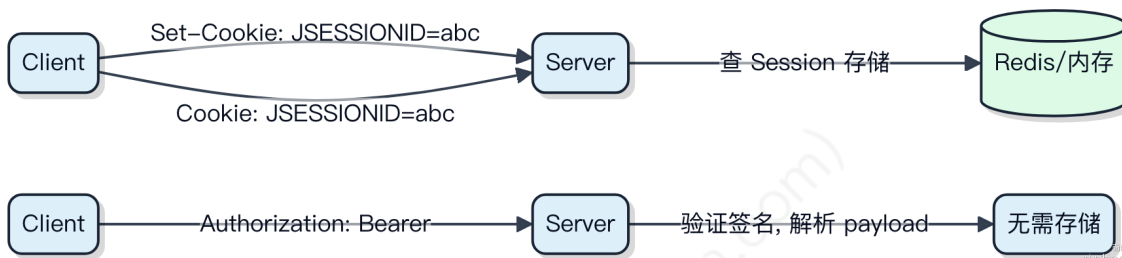
Session 是服务器端的存储机制。用户登录后，服务端生成一个 session 对象，用唯一 sessionId 当 key 存在内存或 Redis 里。sessionId 通常通过 Cookie 传给浏览器。下次请求带着这个 id，服务端就能查到状态。它的状态是集中管理的，登出可以直接销毁，但扩容麻烦，得搞 session 共享。

Token，尤其是 JWT，是无状态的方案。用户登录成功，服务器签发一个 Token，里面包含了用户信息和签名。客户端自己存，每次请求放 Header 里。服务器不存任何状态，每次靠验签解析数据。扩展性好，适合分布式，但 Token 一旦签发没法主动作废，除非加黑名单或者控制有效期很短。

- 1) Cookie 存在客户端，可以持久化
- 2) Session 存在服务端，依赖 Cookie 传 id
- 3) Token 完全自包含，服务端无状态

JWT 长这样： `header.payload.signature`，用点分割三段 Base64

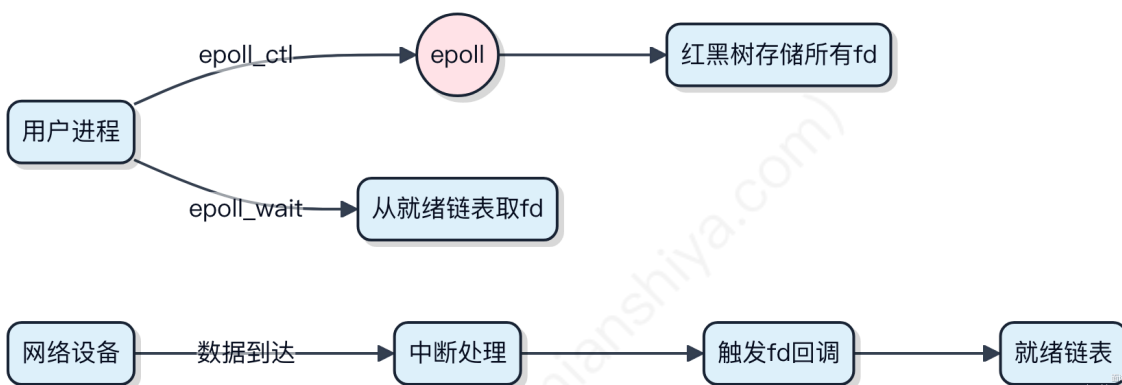
要不要共享 Session，决定了你选哪种。单体系统用 Session 最省事，微服务基本都上 Token。



Select、Poll、Epoll 之间有什么区别？

select、poll 和 epoll 都是 Linux 下处理 I/O 多路复用的机制，用来让一个线程能同时监控多个文件描述符的读写状态。它们的核心目标是一样的，但实现方式和性能差异很大。

- 1) select 使用固定大小的位图来表示 fd 集合，通常最大支持 1024 个 fd。每次调用都要把整个 fd 集合从用户态拷贝到内核态，返回后还要遍历所有 fd 判断是否就绪。这种方式在连接数多时开销大，效率低。
- 2) poll 其实跟 select 差不多，只是换成了链表结构传递 fd，突破了 1024 的限制。但它依然需要遍历所有 fd，而且每次都要传全量数据，**没有解决根本的性能问题**。
- 3) epoll 完全换了个思路，它在内核里用红黑树管理 fd，用就绪队列回调通知。调用 epoll_ctl 添加 fd 时注册回调函数，一旦有事件触发，内核直接把 fd 放进就绪队列。epoll_wait 调用时只返回就绪的 fd，不用遍历全部。这种机制让单个线程轻松扛住几万并发，Nginx、Netty 都靠它撑着。



简单说，select 和 poll 是“你有没有事？我去挨个问一遍”，而 epoll 是“有事我主动告诉你”。连接少的时候差别不大，上几千并发时，epoll 的优势就压倒性了。

什么是物理地址，什么是逻辑地址？

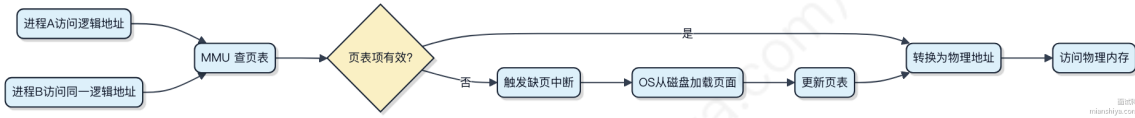
物理地址是内存芯片级别的真实地址，CPU 通过它直接访问内存硬件。你程序里看到的变量地址都不是这个，而是逻辑地址。

逻辑地址是操作系统给每个进程“画”的独立内存空间，比如 32 位系统就是从 0 到 4GB 的虚拟范围。每个进程都觉得自己独占内存，其实这是被系统管理过的假象。

真正干活的是 MMU（内存管理单元），它靠页表把逻辑地址翻译成物理地址。每次内存访问都要查页表，命中就转成物理地址去取数据，没命中就触发缺页中断，由操作系统从磁盘加载数据并更新页表。

这样设计的好处很明显：进程隔离、内存保护、支持比物理内存更大的寻址空间。像 Java 程序里的对象引用，全都是在逻辑地址空间里跑，压根不经过物理地址。

举个例子，两个进程都访问地址 0x1000，MMU 会根据各自的页表映射到不同的物理位置，互不影响。



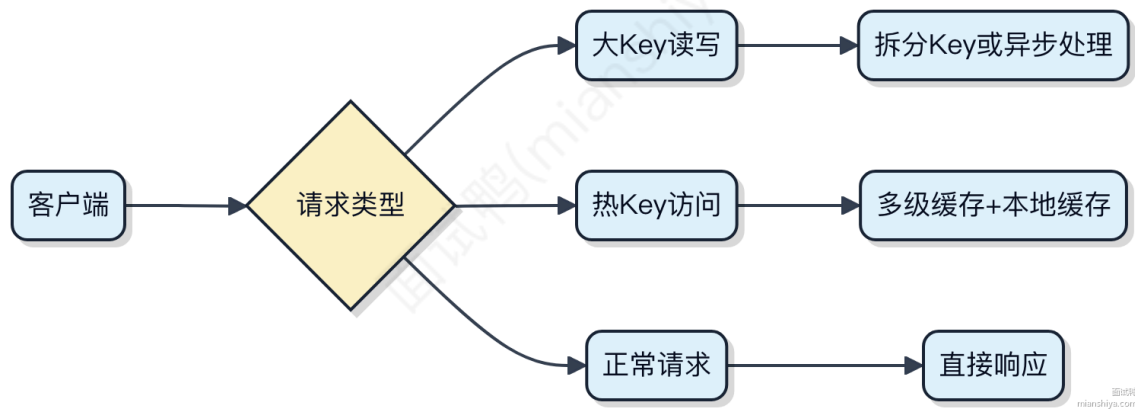
页大小通常是 4KB，一个进程的逻辑页可以映射到任意物理页框，实现非连续内存分配，解决外部碎片问题。

线上发现 Redis 机器爆了，如何优化？

Redis 机器扛不住了，先别急着加机器，得搞清楚到底是哪块出了问题。一般来说就是内存、CPU、网络或者连接数撑满了。

- 1) 先看是不是大 Key 或热 Key 搞的鬼。比如一个 Key 存了几百万条数据，查一次就卡住，删除也慢。用 `redis-cli --bigkeys` 扫一下，或者用 Redis 自带的监控工具看采样。要是发现某个 Key 特别大，考虑拆分，比如把一个大 Hash 拆成多个小 Hash，或者用本地缓存扛一部分。
- 2) 热点数据集中访问，容易压垮单个实例。像秒杀场景，一个商品信息被疯狂查。可以考虑多级缓存，比如加一层 Caffeine 做本地缓存，但要注意缓存一致性。或者用 Redis 分片，把热点打散到不同节点。
- 3) 持久化策略也可能背锅。RDB 快照如果数据量大，fork 子进程会卡主线程；AOF 日志写得频繁，磁盘 IO 上去也会拖慢响应。一般建议 RDB 和 AOF 结合，AOF 配置为每秒刷盘（`appendfsync everysec`），平衡性能和安全。
- 4) 连接数太多？可能是客户端没用连接池，或者连接泄漏。Spring Boot 里用 Lettuce 就比 Jedis 更省连接，因为它是基于 Netty 的长连接模型，压根不经过每次建连的开销。

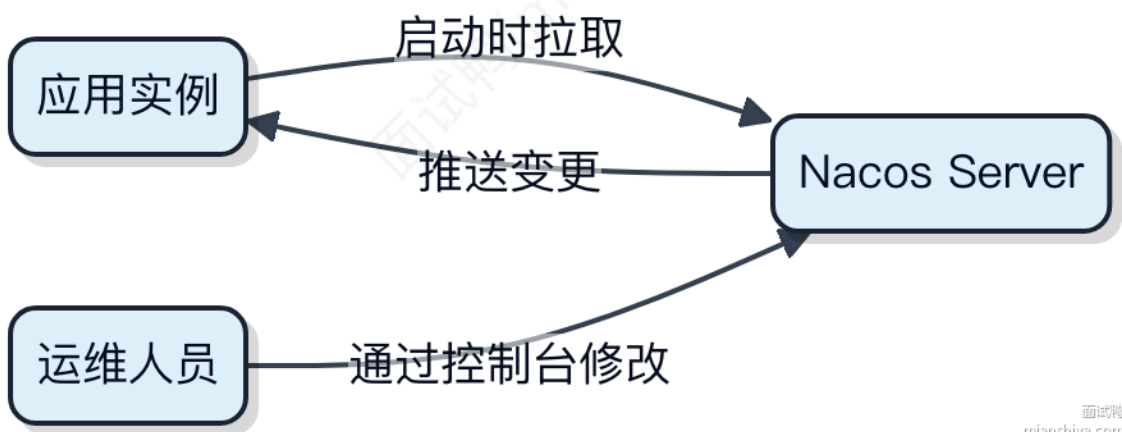
最后才是横向扩容。Redis Cluster 能自动分片，但运维复杂度上去了。也可以考虑云厂商的 Tair 或者阿里云 Redis 版，自带热点探测和大 Key 分析功能，省点心。



什么是配置中心？有哪些常见的配置中心？

配置中心本质是把应用的配置从代码里剥离出来，集中管理。服务启动时动态拉取配置，改配置不用重新发包，运维效率提升特别明显。

- 1) 最核心的是**实时生效**。比如用 Nacos 修改一个开关，下游服务能立刻感知并调整行为，不像以前改个参数得重启几十个实例。
- 2) 配置隔离靠命名空间和分组。开发、测试、生产环境各用各的配置，不会串。一个 Nacos 实例能撑起上千应用的配置管理，扛住日常变更完全没问题。
- 3) 常见方案里，Spring Cloud Config 适合纯 Spring Boot 技术栈，但功能相对基础。Apollo 是携程开源的，配置界面友好，灰度发布支持好，大中型项目用得更多。Nacos 来自阿里，除了配置还支持服务发现，微服务体系里能一箭双雕，现在新项目基本首选它。
- 4) 别小看这层能力。电商大促时调个限流阈值，如果靠改代码发布，半小时都搞不定。配置中心能让这个操作压根不经过研发，运营点两下就生效。



ZooKeeper 其实也能做配置中心，但它是通用协调服务，不是专为配置设计的，操作不够顺手，现在新项目很少这么用。

什么是 Spring Boot?

Spring Boot 本质是 Spring 框架的脚手架，目的就是要把开发从繁琐的配置里解放出来。你不用再手动写一堆 XML 去配数据源、MVC、事务，它通过**自动装配**机制帮你搞定。

启动一个 Web 项目，传统 Spring 要配 `DispatcherServlet`、组件扫描、视图解析器，可能得上百行配置。Spring Boot 只要加个 `@SpringBootApplication`，`main` 方法跑 `SpringApplication.run()` 就行了。比如用 `spring-boot-starter-web`，一引入，Tomcat 和 Spring MVC 自动就位。

它采用“约定大于配置”的思路。比如默认静态资源放在 `/static`，内嵌 Tomcat 监听 8080 端口，这些都不用额外说明。如果想改，再通过 `application.properties` 或 `application.yml` 覆盖就行。

依赖管理也是省心的地方。你用 `spring-boot-starter-parent` 作为父 POM，里面已经定好了常用库的版本号，避免自己引入时版本冲突。像整合 MyBatis、Redis、Kafka，基本都是引入对应的 starter，写点配置，直接开干。

这种快速搭建能力，让微服务架构落地变得简单。比如用 Spring Cloud + Spring Boot，几十个服务也能快速统一技术栈。不过也得注意，自动装配虽然方便，但出了问题如果不熟悉底层原理，调试起来会有点懵。

所以它不是替代 Spring，而是让 Spring 更容易上手和维护。小到单体应用，大到云原生服务，都能扛住。

Spring Boot 的核心特性有哪些？

Spring Boot 的核心特性其实围绕一个目标：让 Spring 应用能快速落地，不用在配置上折腾半天。

- 1) **自动装配**是它最硬的底气。你加个 `spring-boot-starter-web`，它就帮你把 Tomcat、Spring MVC 都配好了，不需要写一堆 XML 或 `@Configuration`。背后的逻辑是 `@ConditionalOnClass` 这些条件注解，只要类路径有某个类，就自动生效对应配置。
- 2) 起步依赖（Starter）设计得很聪明。比如用 Kafka 就引入 `spring-kafka-starter`，它把常用的依赖版本都锁定了，不会出现版本冲突。常见的像 `spring-boot-starter-data-jpa`、`spring-boot-starter-actuator` 都是这种模式。
- 3) 内嵌容器让部署变简单。以前要打 WAR 包扔到外部 Tomcat，现在直接 `java -jar` 启动，因为内嵌了 Tomcat、Jetty 或 Undertow。请求进来时，Servlet 容器直接加载应用，压根不经过外部部署。
- 4) Actuator 提供生产级监控。比如 `/actuator/health` 看健康状态，`/actuator/metrics` 查 JVM 指标，结合 Prometheus 和 Grafana 能搞监控大盘。
- 5) 外部化配置支持多环境。`application.yml` 里写 `spring.profiles.active=prod`，就能切换不同配置，本地和线上互不影响。

代码示例就是最简单的启动类：

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

这个注解本身是个组合注解，包含了 `@Configuration`、`@EnableAutoConfiguration` 和 `@ComponentScan`。

Spring Boot 是如何通过 main 方法启动 web 项目的？

Spring Boot 启动 Web 项目不是靠 main 方法本身多高级，而是靠 `SpringApplication.run()` 这个静态方法把一堆脏活累活给干了。

main 方法里通常就两行代码，核心是调用 `SpringApplication.run(Application.class, args)`。它会做几件事：推断应用类型，比如有没引入 Tomcat、Jetty 这些依赖；加载所有 `META-INF/spring.factories` 里配置的初始化和监听器；最后创建对应的上下文容器。

如果是 Web 应用，它会创建 **`AnnotationConfigServletWebServerApplicationContext`**，这个上下文知道怎么启动内嵌 Web 服务器。在刷新阶段，会触发 `ServletWebServerFactoryAutoConfiguration` 自动装配，根据 classpath 选一个可用的 Web 服务器，比如默认是 Tomcat。

- 1) 先找有没有 `ServletWebServerFactory` 类型的 Bean
- 2) 通过工厂获取 `WebServer` 实例，比如 `TomcatServletWebServerFactory`
- 3) 调用 `webServer.start()` 把内嵌容器跑起来

```
@SpringBootApplication
public class App {
```

```

public static void main(String[] args) {
    SpringApplication.run(App.class, args);
}
}

```

整个过程最关键的是自动装配 + 内嵌容器 + 特殊上下文联动。你不需要手动 new Tomcat 或 configure connector，全靠条件化配置搞定。只要加了 `spring-boot-starter-web`，启动时就会拉起一个预配置好的 Web 服务，监听 8080。



说说 Spring 启动过程？

Spring 启动过程本质是 **IoC 容器的初始化和刷新**，核心入口在 `refresh()` 方法，整个流程大概有 12 个关键步骤。

准备上下文环境时，先设置好环境变量、系统属性，校验必要的配置是否齐全。接着创建 `BeanFactory`，这一步会读取所有 `<bean>` 定义或 `@Component` 注解类，把元信息注册进去，但此时 bean 并没有被实例化。

`BeanFactory` 创建完后，开始加载各种扩展点，比如 `BeanPostProcessor` 和 `ApplicationListener`。这些扩展允许你在 bean 实例化前后插入逻辑，比如 AOP 就是在这里织入代理。

真正的单例 bean 实例化发生在 `finishBeanFactoryInitialization` 这一步。容器会遍历所有非懒加载的单例 bean，触发它们的构造、依赖注入、初始化方法（如 `@PostConstruct`）。循环依赖的解决就发生在这个阶段，Spring 用三级缓存提前暴露刚实例化但未初始化完的对象。

最后一步是发布 `ContextRefreshedEvent`，通知所有监听器上下文已就绪。像 Tomcat 嵌入式容器也是这时候启动，对外提供服务。

代码入口基本都集中在 `AbstractApplicationContext` 的 `refresh()` 方法里：

```

public void refresh() {
    prepareRefresh();
    ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();
    prepareBeanFactory(beanFactory);
    postProcessBeanFactory(beanFactory);
    invokeBeanFactoryPostProcessors(beanFactory);
    registerBeanPostProcessors(beanFactory);
    initMessageSource();
    initApplicationEventMulticaster();
    onRefresh(); // 如 Web 容器启动
    registerListeners();
    finishBeanFactoryInitialization(beanFactory);
    finishRefresh();
}

```

整个过程最关键的是 **BeanFactory 准备、扩展注册、bean 实例化、事件发布** 四个阶段。如果某个 bean 初始化失败，比如数据库连不上，容器启动就会直接卡住，应用也就搞不定了。

什么情况下需要使用分布式事务，有哪些方案？

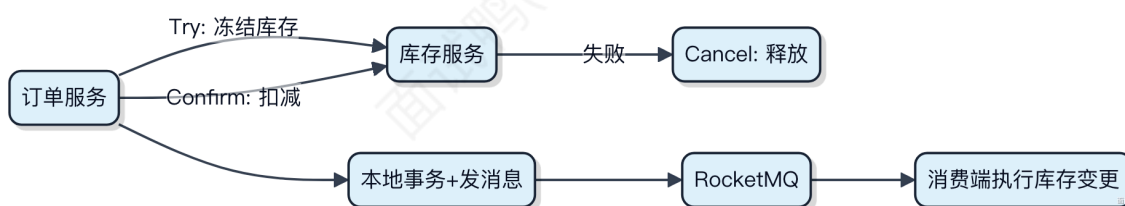
单机事务搞不定跨服务的数据一致性时就得上分布式事务。比如订单服务调库存服务，一个扣库存一个生成订单，两个数据库各管一块，本地事务压根不经过对方的数据库，这时候靠不上 ACID。

1) 最简单的办法是 **TCC**，Try-Confirm-Cancel 三步走。先预占资源 (Try)，比如冻结库存；确认 (Confirm) 真正提交；不行就回滚 (Cancel) 释放资源。逻辑复杂点，但性能好，适合高并发场景，阿里早期的金融交易就这么干。

2) **基于消息队列的最终一致性**也常用。比如用 RocketMQ 的事务消息，先把订单写进 DB，发个半消息，等库存扣成功再提交消息，消费者去减库存。哪怕中间崩了，也能靠回查机制补上。关键是状态要可查、操作要幂等。

3) **Seata 这类全局事务框架**适合想少改代码的场景。AT 模式下你写个 `@GlobalTransactional` 注解就行，它背后靠全局锁和 `undo_log` 做回滚。不过锁重了容易拖慢系统，大并发时得掂量。

4) 还有 **XA 协议**，两阶段提交的老祖宗。协调者问所有参与者“准备好了吗”，都 OK 才提交。问题是同步阻塞，协调者挂了整个事务就卡住，现在用得少了。



如何理解 Spring Boot 中的 starter?

Spring Boot 的 starter 本质是一组依赖描述符，帮你快速集成某个功能模块，不用自己一个个找依赖版本。

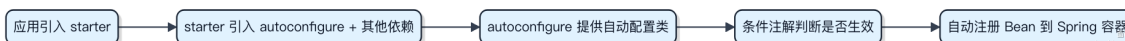
比如你要用 Redis，直接引入 `spring-boot-starter-data-redis`，它背后已经帮你组合好了 Lettuce 或 Jedis、Spring Data Redis、连接池、序列化等依赖，版本也都对齐了。你压根不需要关心具体引入了哪些库，也不用操心版本冲突。

1) 所有 starter 都遵循统一命名规范：`spring-boot-starter-xxx`，第三方框架接入也按这规则，比如 `mybatis-spring-boot-starter`。

2) starter 里不写业务逻辑，主要是 `pom.xml` 的依赖管理。真正的自动配置由对应的 `autoconfigure` 模块实现，比如 `spring-boot-autoconfigure` 里的 `RedisAutoConfiguration` 会根据类路径上的组件自动创建 `RedisTemplate`、`LettuceConnectionFactory`。

3) 自动装配基于条件注解，像 `@ConditionalOnClass`、`@ConditionalOnMissingBean`，只有满足条件才生效。比如发现 classpath 有 `Jedis.class` 才去配置 Jedis 连接工厂。

4) 你可以自定义 starter，拆成两部分：`xxx-spring-boot-starter`（空壳，引 `autoconfigure`），和 `xxx-spring-boot-autoconfigure`（核心配置，含 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 文件声明配置类）。



常见的像 `spring-boot-starter-web` 带了 Tomcat、Spring MVC、Jackson；`spring-boot-starter-jdbc` 自动配好 `DataSource` 和 `JdbcTemplate`。省去了大量基础配置的脏活累活，项目一启动就能用。

什么是 Seata?

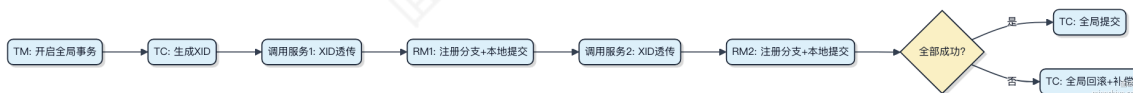
Seata 是阿里开源的分布式事务解决方案，目标是让分布式事务变得像本地事务一样简单。它不依赖数据库的 XA 协议，而是自己实现了一套高性能的事务协调机制。

它的核心是 **TC (Transaction Coordinator)**、**TM (Transaction Manager)**、**RM (Resource Manager)** 三部分。TC 是独立部署的服务，负责维护全局事务状态；TM 开启或终止全局事务；RM 负责分支事务的注册和状态汇报。

典型流程是：TM 向 TC 申请开启全局事务，拿到全局事务 ID；服务调用上下游时透传这个 ID；各个 RM 在执行本地操作时，先写 undo_log 记录反向 SQL，再提交本地事务；如果全部成功，TC 提交全局事务，否则触发回滚，通过 undo_log 做补偿。

适合微服务架构下跨数据库、跨服务的场景，比如订单、库存、账户三个系统协同完成一个业务。Spring Cloud Alibaba 集成后，加个 @GlobalTransactional 注解就能用。

常见模式有 AT 模式（自动补偿）、TCC 模式（手动定义两阶段逻辑）、Saga 模式（长事务编排）。AT 模式最常用，对业务侵入最小。



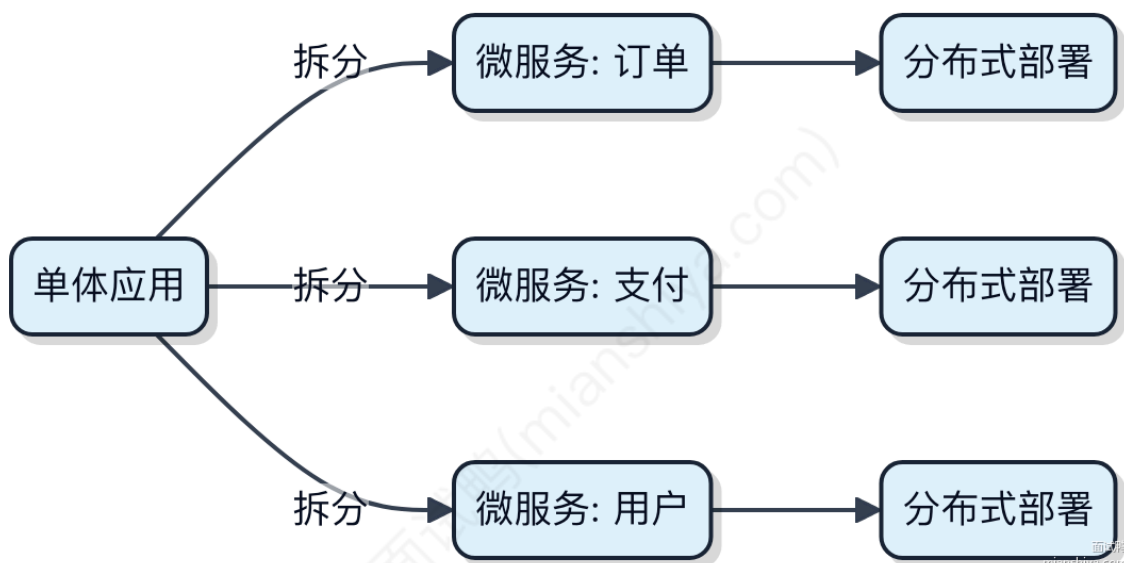
分布式和微服务有什么区别?

分布式和微服务听起来像一对双胞胎，其实它们关注的层面不一样。

- 1) 分布式系统本质是把一个大应用拆到多台机器上跑，核心目标是**解耦硬件依赖**、提升可用性和性能。比如数据库主从分离、Redis 集群，都是典型的分布式场景。它解决的是“怎么让多个节点一起干活”的问题，不关心业务长啥样。
- 2) 微服务则是业务架构的演进，把单体应用按业务域拆成一堆小服务，每个服务独立开发、部署、伸缩。像订单、支付、库存各是一个服务，用 Spring Cloud 或 Dubbo 做通信。重点在“高内聚、低耦合”的业务组织方式。

你可以理解为：微服务**天生就是分布式的**，但分布式不一定是微服务。比如一个老系统做了集群部署，但它还是单体，就不能叫微服务。

举个例子，淘宝早期是单体 Java 应用，后来扛不住流量，就拆成了用户、商品、交易等上百个微服务，每个服务还能自己做分布式部署。这时候，微服务是拆法，分布式是实现手段。



简单说，微服务决定“怎么拆业务”，分布式解决“怎么跑在多台机器上”。两者配合，才能撑住高并发、大流量的现代互联网应用。

负载均衡算法有哪些？

轮询是最常见的负载均衡算法，Nginx 默认就用这个。服务器按顺序轮流接收请求，实现简单，适合后端机器性能相近的场景。如果某台机器扛不住，得配合健康检查动态摘除。

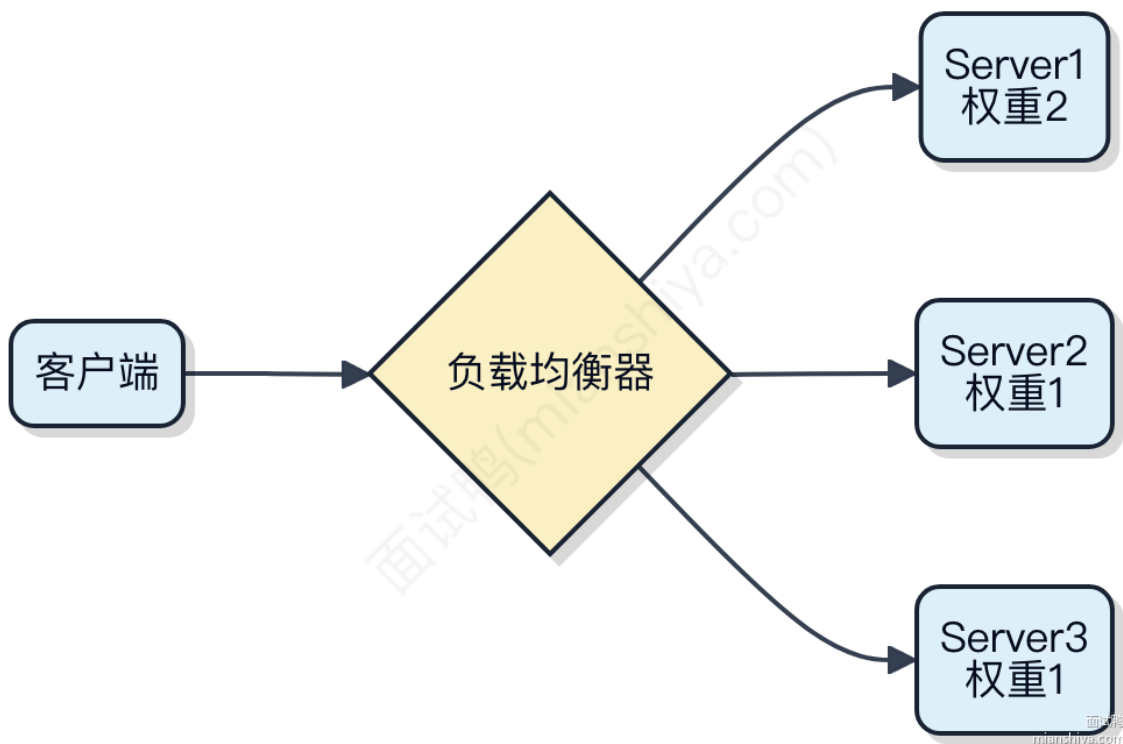
1) 加权轮询给不同机器分配权重，性能强的机器处理更多请求。比如三台机器权重 2:1:1，第一台每轮处理两次，后面各一次。Kubernetes 的 kube-proxy 在某些模式下会用到类似策略。

2) 随机算法直接选一台，看似不靠谱，但客户端足够多时分布很均匀。gRPC 的默认负载均衡就是随机，实现轻量，压根不经过中心调度。

3) 最少连接数关注实时负载，把新请求发给当前连接数最少的机器。适合长连接场景，像 WebSocket 或数据库连接池，HAProxy 支持这种策略。

4) 源地址哈希根据客户端 IP 做 hash，相同 IP 始终路由到同一台。会话保持的场景有用，比如没有共享 session 的老系统。缺点是机器扩容时大量映射失效。

一致性哈希解决普通哈希在节点变动时全量重映射的问题。把节点和请求都映射到一个环上，只影响邻近节点。分布式缓存如 Redis Cluster、Cassandra 都用了变种。

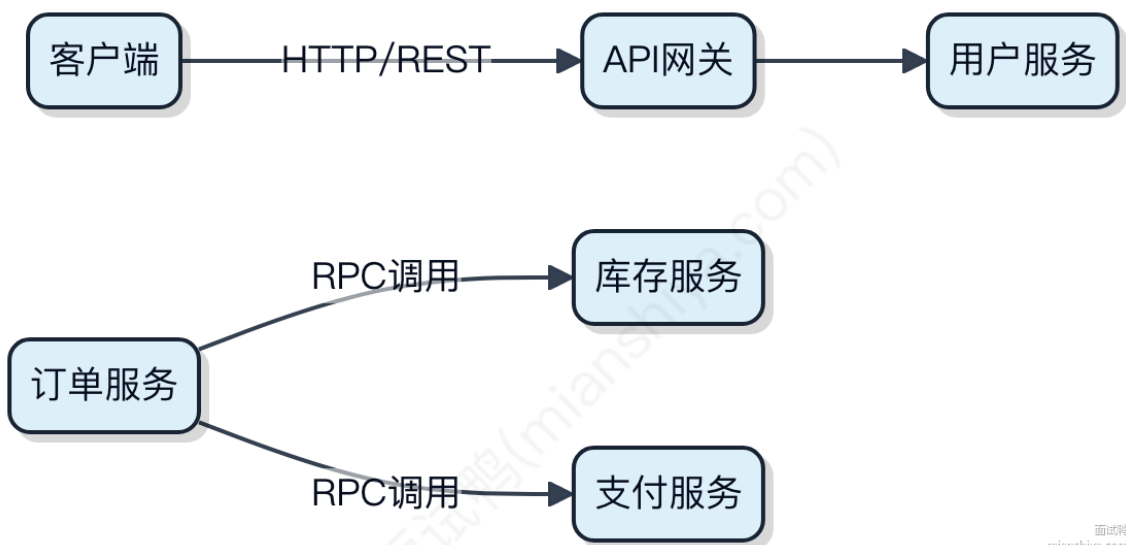


实际选型看场景。短连接无状态服务用轮询或随机就够了，有状态或连接密集型考虑最少连接或一致性哈希。

HTTP 与 RPC 之间的区别？

HTTP 是应用层协议，本质是通信规范，而 RPC 是一种编程模型，目标是让远程服务调用像本地方法调用一样简单。

- 1) 通信协议上，HTTP 通常基于文本或二进制格式（如 HTTP/1.1、HTTP/2），走的是标准端口，天然支持跨平台和浏览器交互。RPC 框架可以自定义协议，比如 Dubbo 用 Netty + 自定义私有协议，gRPC 基于 HTTP/2 但封装了严格的接口契约。
- 2) 调用方式差异明显。HTTP 一般通过 RESTful 风格的 URL + 方法（GET/POST）交互，数据多用 JSON 传输，人眼可读。RPC 更关注接口抽象，客户端直接调 `userService.getUser(1001)`，底层自动序列化、网络传输、服务寻址，开发者几乎感知不到网络细节。
- 3) 性能方面，RPC 协议更轻量，压根不经过浏览器那一套解析逻辑，序列化也常用 Protobuf 这类高效格式，延迟通常比 HTTP 低 30% 以上。像 gRPC 在内部微服务之间调用就很常见，扛住高并发没问题。
- 4) 典型场景：对外暴露 API 给前端或第三方，基本都用 HTTP + REST + JSON。内部服务间调用，尤其是对延迟敏感的场景，比如订单系统调库存，就更适合用 **RPC** 框架，比如 Dubbo 或 gRPC。



图中左侧是外部 HTTP 访问路径，右侧是内部服务通过 RPC 直接交互，职责清晰。

什么是服务降级？

服务降级不是等系统崩了才去做的事，而是提前想好“如果某个依赖出问题，我该怎么办”的兜底策略。本质是在资源有限或依赖不稳定时，牺牲非核心功能，保证核心链路可用。

比如你调用一个商品详情接口，里面要查库存、查价格、查推荐。这时候推荐服务挂了，难道整个页面都打不开？显然不行。你可以选择不展示推荐列表，但库存和价格必须返回，这就是典型的降级操作。

- 1) 降级通常发生在客户端或者网关层，比如用 Hystrix、Sentinel 这类框架配置规则。当检测到超时、异常或线程堆积时，自动走 fallback 逻辑。
- 2) 常见的降级方式有直接返回默认值、缓存数据、静态资源、简化流程。像淘宝大促时可能关闭评论加载，京东秒杀期间不查物流信息，都是为了把流量省下来给下单服务。
- 3) 和熔断的区别在于，熔断是“断开连接避免雪崩”，一般是临时性的；而降级是“我知道它有问题，但我有备选方案”。两者经常配合使用。

代码上基本就是加个注解或者配置规则：

```
@SentinelResource(value = "getRecommend", fallback = "getDefaultRecommend")
public List<Item> getRecommend() {
    return remoteCallToRecommendService();
}

public List<Item> getDefaultRecommend() {
    return Collections.emptyList(); // 降级返回空列表
}
```

要不要降级、降哪一块，得从业务角度权衡。核心原则就一条：保最重要的那个动作能跑通。

什么是服务熔断？

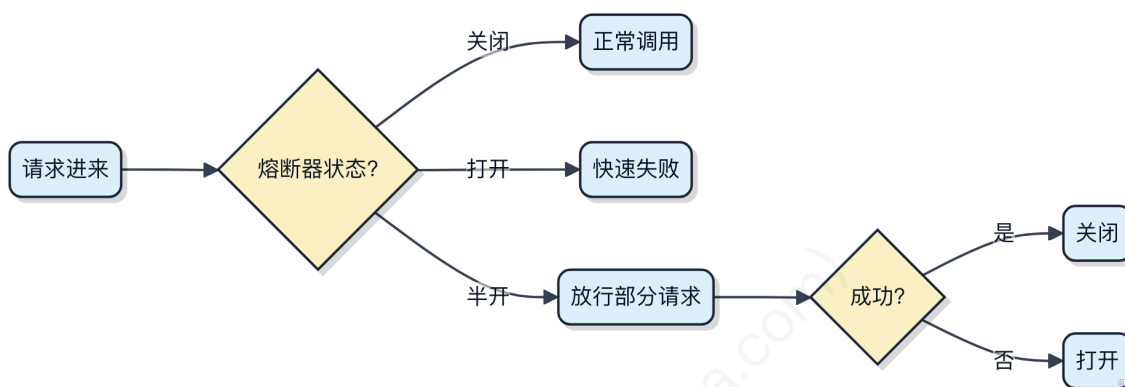
服务熔断其实和家里电闸跳掉是一个道理。系统压力太大或者下游故障，就先切断请求，避免雪崩。

- 1) 核心是**保护机制**。当某个服务调用失败率超过阈值，比如连续 5 次超时或异常，熔断器立刻切换到打开状态，后续请求直接失败，不往下游发。
- 2) 给系统一个喘息机会。默认等待 5 秒进入半开状态，放少量请求过去试探。如果成功就关闭熔断，恢复正常；失败则继续打开。
- 3) 典型实现像 Hystrix，虽然现在用得少了，但思想被广泛继承。Sentinel、Resilience4j 都有类似能力。比如 Resilience4j 可以这样配置：

```
CircuitBreakerConfig config = CircuitBreakerConfig.custom()
    .failureRateThreshold(50) // 失败率超50%触发
    .waitDurationInOpenState(Duration.ofMillis(5000)) // 等5秒尝试恢复
    .build();
```

- 4) 和限流不一样。限流是控制流量大小，熔断是应对依赖不稳定。经常配合使用，比如网关层用 Sentinel 同时做限流和熔断。

搞不定这玩意儿，线上容易出现连锁故障。一个接口卡死拖垮整个应用，尤其是微服务调用链长的场景。

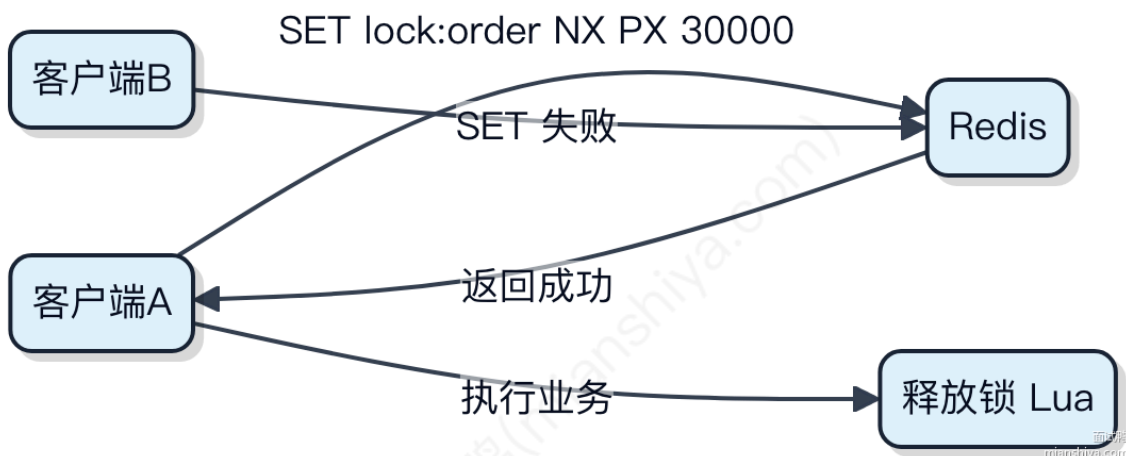


分布式锁一般都怎样实现？

分布式锁的实现，关键是要在多个节点之间协调对共享资源的访问，保证同一时刻只有一个客户端能拿到锁。常见的实现方式有三种。

- 1) 基于数据库实现，比如用 MySQL 的唯一索引或者乐观锁 version 字段。简单场景可以用，但性能差，一般扛不住高并发。
- 2) 基于 Redis，这是最常用的方案。利用 SET key value NX PX milliseconds 这个原子操作，value 一般是随机字符串或线程 ID，防止误删。释放锁时要用 Lua 脚本，确保“读取 value - 判断 - 删除”是原子的。Redisson 就是这么干的，还支持可重入、看门狗自动续期。
- 3) 基于 ZooKeeper，利用临时顺序节点和 Watch 机制。谁创建的节点序号最小，谁就获得锁。ZooKeeper 本身一致性更强，但系统复杂度也高，像 Curator 框架封装了分布式锁逻辑。

Redis 方案胜在性能好、易落地，大多数互联网公司用它。但要注意锁过期时间不好定，太短可能业务没执行完就被释放，太长又影响响应。所以**可重入**和**自动续期**很重要，不然容易出问题。



别自己从零写，直接上 Redisson 或 Curator，省事还靠谱。

MyBatis 与 Hibernate 有哪些不同？

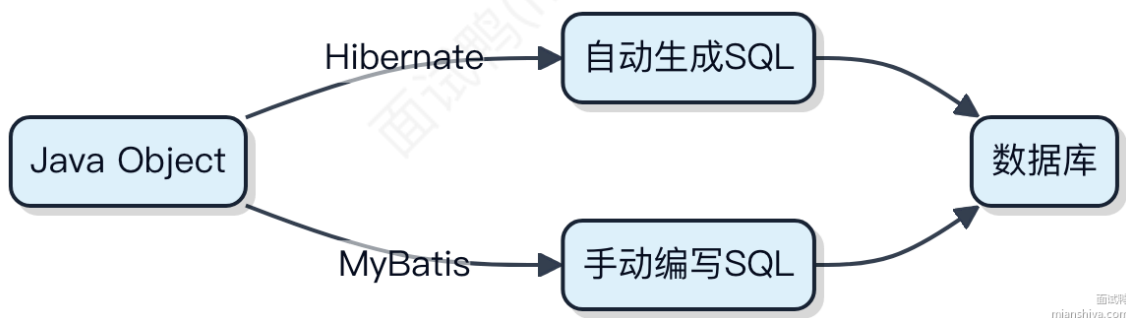
MyBatis 和 Hibernate 都是 Java 里处理数据库操作的 ORM 框架，但设计哲学差得挺远。

Hibernate 是全自动化的，你定义好实体类和映射关系，增删改查基本不用写 SQL，它自动生成并执行。比如用 `Session.save(entity)` 就能把对象存进数据库，连事务都帮你管好了。这种适合业务规则稳定、表结构规范的项目，像传统企业管理系统用 Hibernate 能省不少事。

MyBatis 则是半自动的，SQL 得自己写，通过 XML 或注解把参数和结果集跟 Java 对象做映射。比如一个带多表关联的查询，你可以精确控制 SQL 的每一部分，性能调优更方便。互联网公司搞高并发、复杂查询时，普遍选 MyBatis，毕竟对 SQL 的掌控力更强。

- 1) Hibernate 抹平了数据库差异，HQL 可以跨数据库运行，迁移成本低
- 2) MyBatis 直接暴露 SQL，灵活性高，但换数据库时可能要重写语句
- 3) Hibernate 有完整的缓存体系（一级、二级缓存），MyBatis 缓存功能相对简单

举个例子，做报表查询这种字段多、条件动态拼接的场景，Hibernate 很难搞定，SQL 不透明，拼起来费劲。MyBatis 直接写原生 SQL，配合动态标签 `<if>`、`<choose>` 处理条件，干净利落。



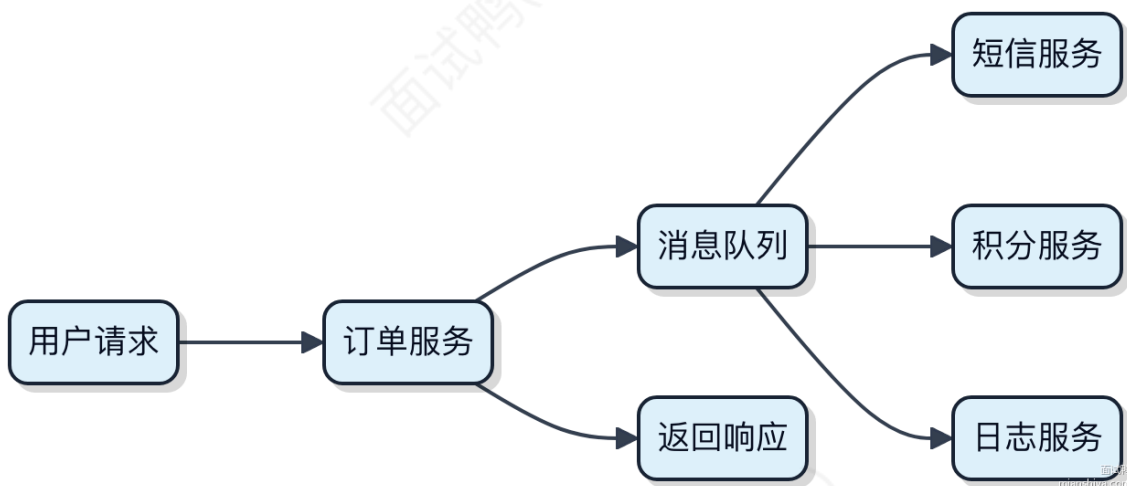
总的来说，追求开发效率、少写 SQL 用 Hibernate；需要精细控制 SQL、应对复杂场景就上 MyBatis。

为什么需要消息队列？

消息队列解决的是系统间**异步通信**和**流量削峰**的问题。你想象一下，用户下单后要发短信、更新库存、记日志，如果全都同步执行，一个环节卡住，整个下单就卡住了。更别说大促时瞬时几万订单涌进来，数据库直接被压垮。

- 1) 把这些非核心链路拆出来，丢到消息队列里异步处理。主流程只管扣减库存、生成订单，成功就完事，其他事慢慢做。这样响应快，用户体验好，系统也更容易扛住高并发。
- 2) 消息队列还能当缓冲层用。比如每秒来5000个请求，但你的订单系统只能扛2000，多出来的3000先存队列里，系统按自己节奏消费。相当于把尖锐的流量高峰给“削平”了，避免雪崩。
- 3) 系统之间不想紧耦合。A系统改了个字段，B系统要是直接调用就得跟着改。通过消息队列，A发个消息就不管了，B自己去取，谁都不依赖谁，扩展性更强。

典型场景就是电商下单后触发积分、优惠券发放，用 Kafka 或 RocketMQ 异步通知。还有日志收集，所有服务把日志扔到消息队列，再统一写入 Elasticsearch。



记住，不是所有场景都适合。如果业务要求强一致性，比如转账必须同时成功或失败，那事务消息或者直接数据库操作更靠谱。普通异步解耦，消息队列是脏活累活的好帮手。

说一下消息队列的模型有哪些？

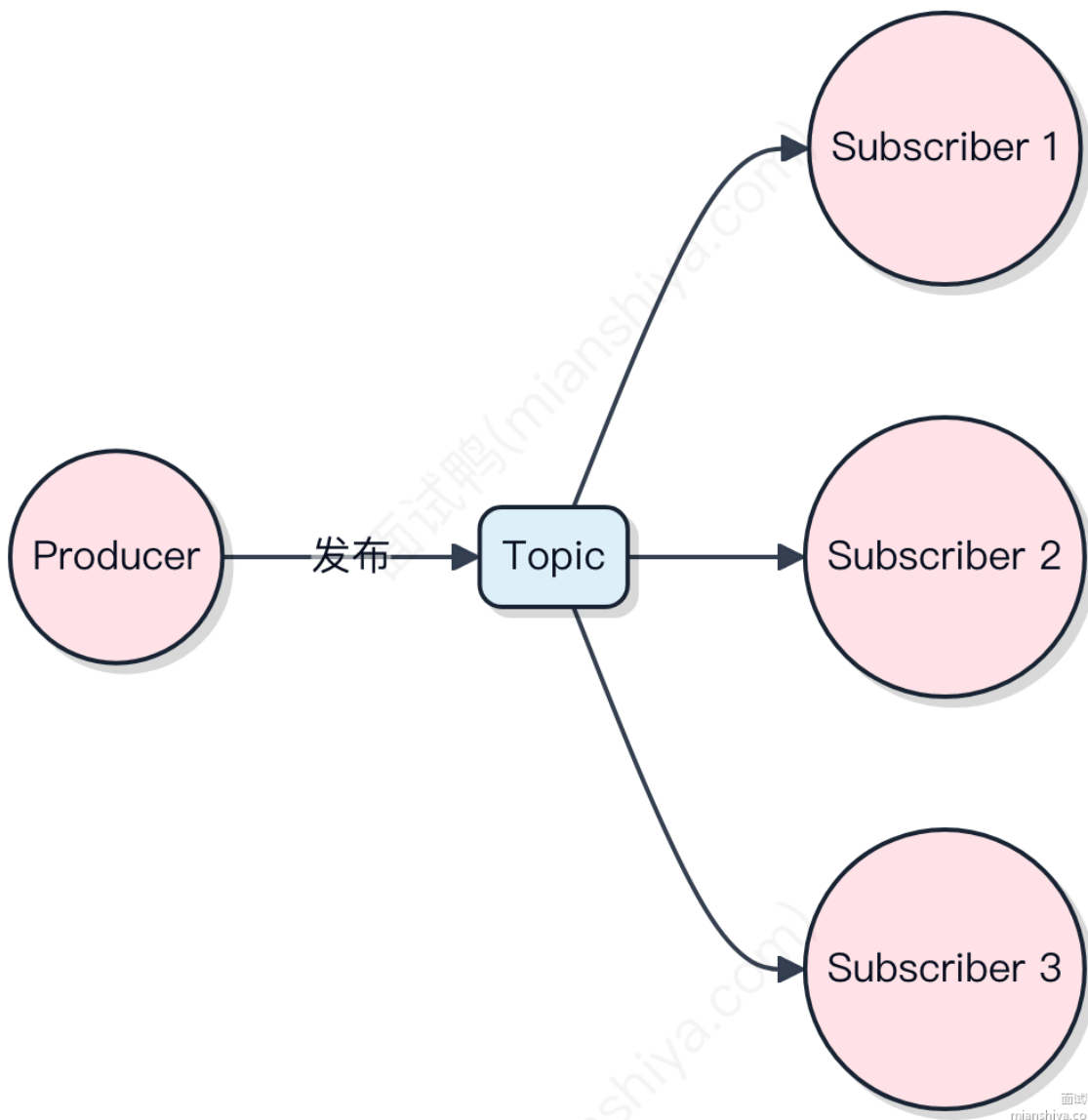
消息队列的模型其实主要就两种：**点对点模型**和**发布订阅模型**，大部分中间件都是基于这两个变种或组合。

1) 点对点模型

一个消息只能被一个消费者消费，消息发送到队列里，多个消费者监听同一个队列时，消息会被其中一个消费者取走。比如你用 RabbitMQ 发个订单处理任务，肯定不希望被三个服务都执行一遍。这种模型适合任务分发场景，ActiveMQ、RocketMQ 都支持。

2) 发布订阅模型

生产者把消息发到一个主题（Topic），所有订阅这个主题的消费者都能收到消息。典型的就广播通知，比如配置中心推送变更，Kafka 和 Redis Pub/Sub 就是这路子。注意 Kafka 的消费者组机制其实是“发布订阅 + 点对点”的混合体——组内是点对点，组间是广播。



像 RocketMQ 把这两种模型统一成 Topic + Consumer Group 的方式来管理，灵活性更高。选择哪种模型关键看业务是否需要广播语义。如果只是解耦异步处理，点对点够用；要是做事件驱动架构，发布订阅更合适。

单例模式有哪几种实现？如何保证线程安全？

单例模式的关键是确保一个类只有一个实例，并提供全局访问点。常见的实现方式有好几种，每种都有它的适用场景和坑。

1) 饿汉式在类加载时就创建实例，写法最简单，天然线程安全，但可能浪费资源。

```
public class Singleton {  
    private static final Singleton INSTANCE = new Singleton();  
    private Singleton() {}  
    public static Singleton getInstance() {  
        return INSTANCE;  
    }  
}
```

```
}  
}
```

- 2) 懒汉式延迟初始化，但直接加 synchronized 方法性能差，毕竟锁的是整个方法。
- 3) 双重检查锁定（DCL）是优化方案，只在实例未创建时才同步，注意必须用 volatile 修饰实例变量，防止指令重排序导致返回未初始化完成的对象。
- 4) 静态内部类方式利用了类加载机制保证线程安全，又实现了懒加载。外部类加载时不触发内部类初始化，直到第一次调用 getInstance 才加载 Holder 类并创建实例。

```
public class Singleton {  
    private Singleton() {}  
    private static class Holder {  
        static final Singleton INSTANCE = new Singleton();  
    }  
    public static Singleton getInstance() {  
        return Holder.INSTANCE;  
    }  
}
```

- 5) 枚举单例最简洁安全，不仅能防反射攻击，还能防止序列化破坏单例，Effective Java 推荐的方式。

```
public enum Singleton {  
    INSTANCE;  
}
```

一般项目里用静态内部类或枚举就够了。DCL 虽高效但容易写错，比如漏掉 volatile。如果涉及到反序列化或反射的场景，优先选枚举。

工厂模式和抽象工厂模式有什么区别？

工厂模式和抽象工厂模式都用来解耦对象创建，但解决的问题层次不一样。

工厂模式针对的是**单个产品等级结构**。比如你要生产不同类型的数据库连接，MySQLConnection、PostgreSQLConnection 都实现 Connection 接口。这时候用一个 DatabaseConnectionFactory，根据传入的类型返回具体实例。核心是把 new 操作统一收口，后续加新数据库类型也不用改业务代码。

```
public class ConnectionFactory {  
    public Connection getConnection(String type) {  
        if ("mysql".equals(type)) return new MySQLConnection();  
        if ("pg".equals(type)) return new PostgreSQLConnection();  
        throw new IllegalArgumentException();  
    }  
}
```

抽象工厂模式处理的是**多个产品族的创建**，强调产品之间的协调性。比如你有 MySQL + Redis 组合、PostgreSQL + Memcached 组合，每个组合是一套数据访问层方案。抽象工厂定义一套创建接口，每种实现负责创建一整套配套的对象。

```

interface DaoFactory {
    Connection createConnection();
    Cache createCache();
}

class MysqlRedisFactory implements DaoFactory {
    public Connection createConnection() { return new MySQLConnection(); }
    public Cache createCache() { return new RedisCache(); }
}

```

- 1) 工厂模式关注单一类型对象的不同实现
- 2) 抽象工厂关注一组相关或依赖对象的批量创建
- 3) 抽象工厂通常会用到多个内部工厂或工厂方法来构建完整产品族

像 Spring 的 `BeanFactory` 是典型的工厂模式，而 Java EE 中的数据源与事务管理器协同场景就适合抽象工厂。一般项目里工厂模式更常见，抽象工厂容易过度设计，除非你真有多套独立且完整的组件组合要维护。

消息队列如何保证消息不丢失？

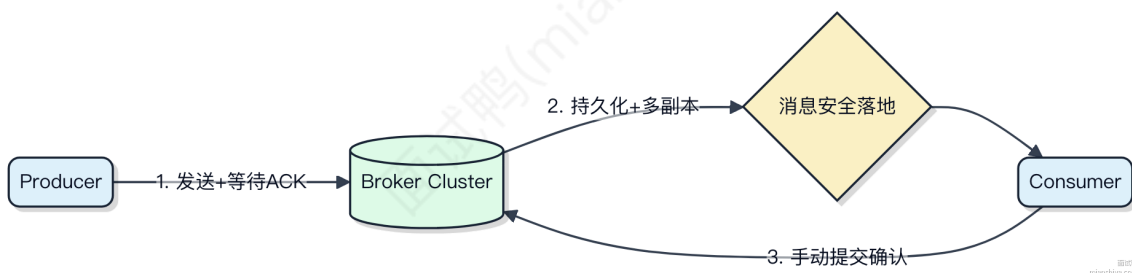
消息不丢这事，关键得把生产、存储、消费三个环节的可靠性都拉满。

生产端发消息，网络抖动或者 Broker 挂了就可能丢。解决方案是发完等 **broker** 的确认回执。比如 RabbitMQ 要开 `publisher confirm` 模式，Kafka 要把 `acks` 设成 `all`，这样 leader 和所有 ISR 副本都写入才算成功。配合上失败重试，基本就能稳住。

中间 broker 存储这块，核心是别把消息只放内存。RabbitMQ 默认持久化要开两步：交换机设 `durable`，队列也得 `durable`，发消息时 `deliveryMode=2`。Kafka 天然靠多副本，只要不是全副本宕机，数据就不会丢。但注意如果 producer 端没做 `ack` 控制，哪怕副本再强也白搭。

消费端最容易出事，比如刚拿到消息还没处理，消费者进程崩了，broker 却已经标记为“已消费”。所以必须关掉自动提交 `offset`，手动在业务逻辑执行成功后再提交。像 Kafka 要设 `enable.auto.commit=false`，处理完再 `sync` 或 `async` 提交。

整个链路就像接力赛，每一棒都得亲手交到位。



消息队列如何处理重复消息（保证消息的幂等性）？

消息重复几乎是没法彻底避免的，Kafka、RabbitMQ、RocketMQ 都不承诺消息只投递一次，所以消费端必须自己扛住幂等问题。

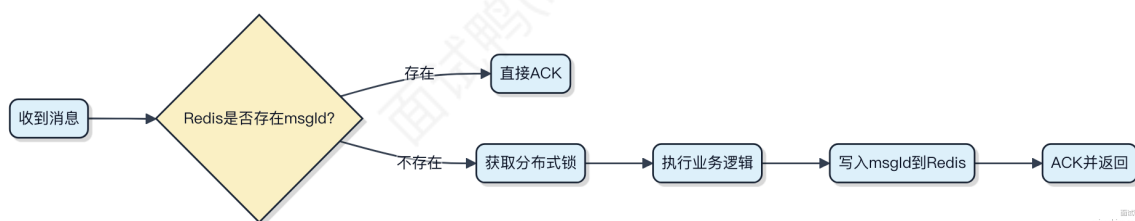
一般最简单的做法是**利用唯一标识 + 分布式锁 + 数据库或Redis记录**。比如每条消息带一个业务ID，消费者拿到后先查Redis是否处理过，没处理过才执行业务逻辑，执行完再标记为已处理。这个业务ID通常由生产者生成，比如订单场景可以用“orderNo:123”作为key。

1) 先尝试SETNX Redis，key可以是messageId，value设个过期时间 2) 如果成功，说明是第一次消费，继续走业务流程 3) 如果失败，说明已有其他实例在处理，当前直接丢弃或返回成功

更重一点的做法是在数据库建一张消费记录表，每次消费前先插入一条记录，利用主键冲突防止重复处理。这种方式一致性更好，但对DB有额外压力。

还有一个思路是把幂等性下沉到业务层。比如转账操作，不要用“加100元”，而是用“将余额更新为600元”这种状态覆盖方式。或者像RocketMQ的事务消息，通过两阶段提交+反向查询来保证最终一致。

关键点在于：消息队列本身解决的是可靠传递，不是去重。真正的幂等得靠业务逻辑兜底。



消息队列如何保证消息的有序性（顺序性）？

消息的有序性在分布式系统中是个高频问题，关键得看“顺序”的范围和级别。很多时候我们说的顺序其实是局部有序，比如同一个业务流水下的消息要按先后处理。

1) 全局有序很难搞，代价太高。像 RocketMQ 的普通集群模式，不同生产者发的消息落到不同队列，天然就乱序了。想强行全局有序就得把所有消息压到一个队列里，但这就等于单线程消费，吞吐直接被锁死，一般扛不住高并发场景。

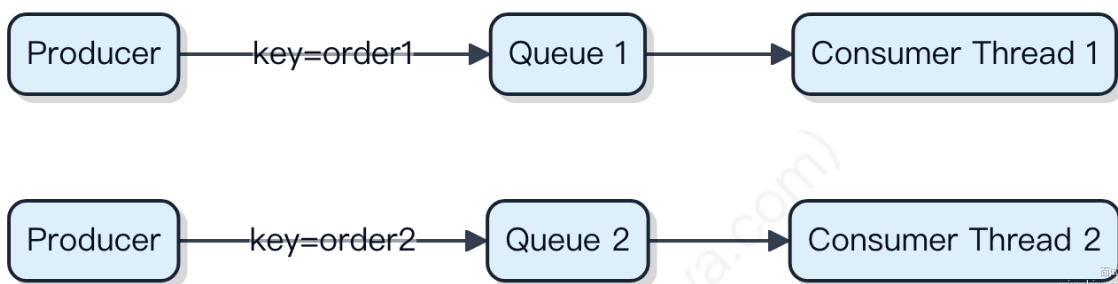
2) 更实际的做法是**分组有序**，也就是分区有序。把有顺序要求的消息通过相同的 key 路由到同一个队列。比如订单ID作为 key，所有关于这个订单的操作都进同一个 queue。Kafka 和 RocketMQ 都支持这种机制。消费者拉取时，一个队列只被一个消费者线程处理，自然保证了顺序。

3) 代码上也很简单，发送时指定 key 就行：

```
// RocketMQ 示例
Message msg = new Message("TOPIC", "TAG", "ORDER_ID_001", body);
producer.send(msg);
```

这里 `ORDER_ID_001` 作为路由 key，确保同一订单的消息落在同个队列。

4) 注意点：消费者必须单线程处理该队列，如果开了多线程，还得自己加同步控制，否则顺序一样会乱。而且一旦某个消息卡住，后面的就全堵着，这就是顺序性带来的性能 trade-off。

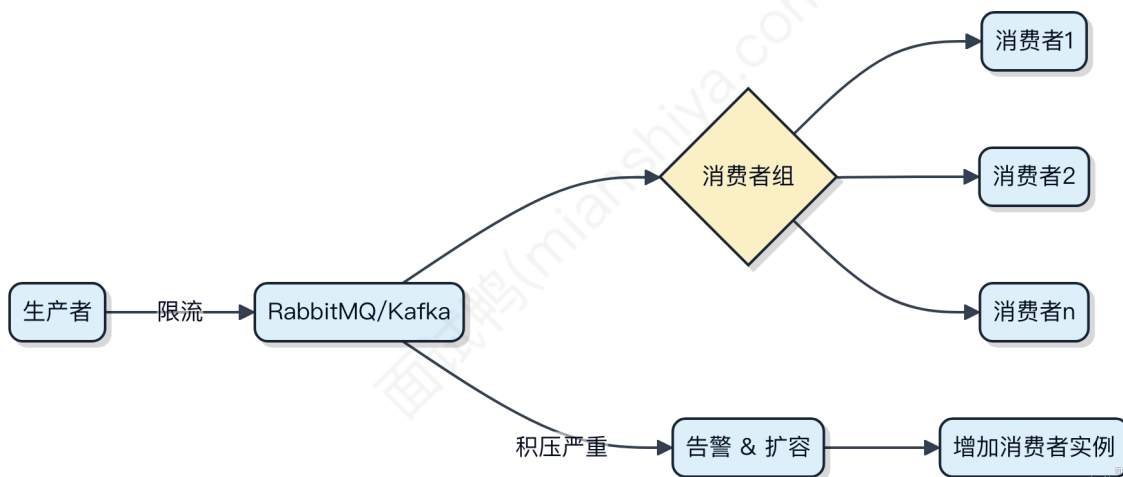


所以总结，顺序性不是免费的，靠的是 **消息分片 + 单线程消费** 来实现，适用场景如订单状态流转、数据库 binlog 同步等，对延迟敏感的链路要谨慎使用。

消息队列如何处理消息堆积？

消息堆积是生产速度远超消费能力时的典型问题，关键在于“控源头、提消费、保核心”。

- 1) 流量削峰是第一道防线。上游系统突然打来 10w 消息，消费者扛不住就得崩。一般用 **限流** 把入口流量控制住，比如通过令牌桶让每秒只放行 2k 消息进队列，剩下的等系统平稳再处理。
- 2) 提升消费能力最直接。可以横向扩容消费者实例，比如从 3 个加到 10 个，靠数量摊平压力。注意消费模型得是点对点的，像 Kafka 的 consumer group 才能并行拉取不同分区的消息。
- 3) 分级处理保障关键业务。非核心消息比如日志类，允许延迟几小时；订单支付这种高优消息，可以用独立 topic 或死信队列单独处理，避免被垃圾消息拖垮。
- 4) 最后兜底手段是丢弃或归档。实在处理不完，就把老旧消息批量归档到 HDFS 或直接丢弃，保证队列不会无限膨胀。



代码上，Kafka 消费者可以通过 `poll(500)` 控制每次拉取条数，配合 `enable.auto.commit=false` 手动提交偏移量，避免重复消费加剧压力。

消息队列设计成推消息还是拉消息？推拉模式的优缺点？

消息队列到底是推还是拉，其实没有标准答案，关键看场景和设计目标。

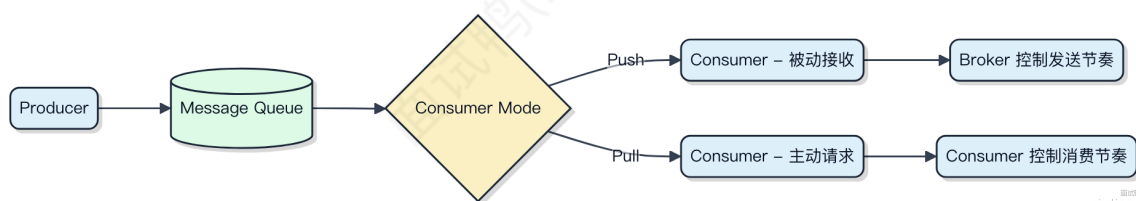
1) 推模式 (Push) 是服务端主动把消息发给消费者。好处是实时性好, 消息一到就能立刻推下去, 像 Kafka 的 consumer pull 其实底层用了长轮询模拟 push。但问题也很明显, 如果消费者处理能力跟不上, 容易被压垮, 系统搞不定。

2) 拉模式 (Pull) 是消费者自己来取消息。灵活性高, 消费者可以按自己的节奏消费, 比如 RocketMQ 就是典型的 pull 模式。但缺点是可能有延迟, 得靠轮询, 太频繁浪费资源, 间隔太久又不及时。

3) 所以很多系统走混合路线。Kafka 看起来是 pull, 但通过参数控制每次拉取数量和等待时间, 做到“准实时”。而 RabbitMQ 在 direct 等交换器下更像是 push, 配合 QoS 限流, 避免消费者过载。

真正的核心在于**流量控制**和**负载均衡**。push 容易造成雪崩, pull 可能增加 broker 查询压力。选择时得看吞吐要求、消费稳定性、网络开销这些维度。

举个例子, 日志收集这种高吞吐场景用 pull 更稳; 而订单状态通知这类强实时需求, 适合可控的 push + 限流组合。



最终选哪种, 不是技术决定, 而是业务说了算。

RocketMQ 的事务消息有什么缺点? 你还了解过别的事务消息实现吗?

事务消息听着很美, 实际用起来有不少约束和代价。RocketMQ 的事务消息本质是“半消息 + 定时回查”机制, 生产者发完半消息后执行本地事务, 再提交或回滚。这套流程搞不定强一致性, 只能保证最终一致。

1) 依赖本地事务状态持久化。你的本地事务结果得自己存好, 比如写数据库加个事务日志表, 不然宕机后 Broker 回查你都答不上来, 系统就卡住了。

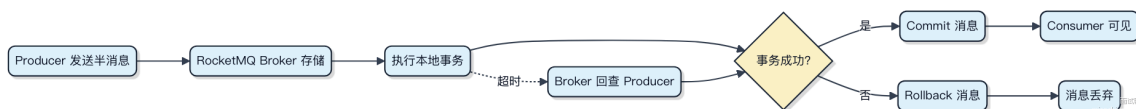
2) 回查机制有延迟。Broker 默认每隔 30 秒扫描一次未决事务, 最多查 15 次。极端情况下, 一条消息可能要等上 7 分钟才被处理, 实时性压根不满足高要求场景。

3) 生产者必须实现回查接口。这属于脏活累活, 每个业务都要写 check 逻辑, 代码侵入性强, 稍不注意还会写出幂等问题。

4) 不支持事务回滚时的反向补偿。比如下单成功但扣库存失败, 事务消息能阻止消息投递, 但已经写入的订单数据得你自己去清理, 分布式下的状态修复特别麻烦。

别的实现方案里, Apache Kafka 的事务是靠幂等写 + 两阶段提交 (2PC) 实现的, 配合 Kafka Streams 能做到端到端的恰好一次语义。不过它要求整个链路都走 Kafka, 灵活性不如 RocketMQ。

另一个是 Seata 的 Saga 模式, 把大事务拆成多个可补偿的子事务, 通过事件驱动推进或回退。适合长周期业务, 比如旅游订票这种跨多个服务的流程, 但开发成本高, 得为每个操作写补偿接口。

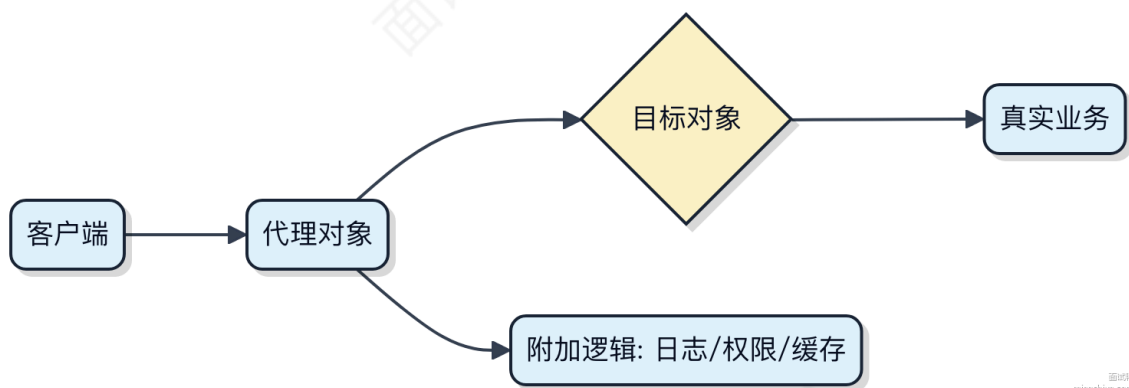


什么是代理模式？一般用在什么场景？

代理模式本质是为对象提供一个**代理**，用来控制对原对象的访问。你调用的是代理类，它帮你把请求转发给目标对象，中间可以加逻辑。

- 1) 最常见的就是 Spring 的 AOP，比如你加个 `@Transactional` 注解，Spring 就会通过代理在方法前后开启和提交事务。实际调用的不是你的原始 bean，而是生成的代理对象。
- 2) 远程调用也能用代理，像 Dubbo 里的接口注入，看着是本地调用，其实是代理帮你做了网络通信、序列化这些脏活累活。
- 3) 还有懒加载，比如 Hibernate 关联对象没查出来前是个代理，真正 get 时才去数据库捞数据，避免一次性加载太多。

代码上分静态代理和动态代理。静态代理手写一个类实现相同接口，动态代理可以用 JDK 原生的 `Proxy` + `InvocationHandler`，或者 CGLIB 字节码增强。



注意别滥用。如果只是简单转发没意义，反而增加复杂度。重点是“在不修改目标的前提下增强行为”，这是它的价值所在。像日志记录、性能监控、限流降级，都是典型场景。

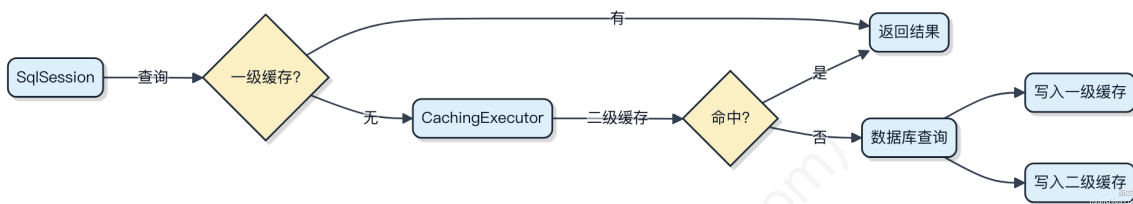
说说 MyBatis 的缓存机制？

MyBatis 的缓存分两级，一级缓存默认开启，作用域是 `SqlSession` 级别。只要你用同一个 `SqlSession` 执行相同的 SQL，参数也一样，第二次查询直接从内存取结果，不用走数据库。

- 1) 一级缓存底层其实就是一个 `HashMap`，数据结构是 key (`MappedStatement ID` + 查询参数) 到缓存对象的映射。一旦执行 `commit`、`update` 或 `close`，缓存就清空。所以事务提交后，下个会话查的数据是新的。
- 2) 二级缓存作用范围更广，跨 `SqlSession`，通常集成 `Redis`、`Ehcache` 来实现。需要手动开启，对应的 `Mapper` 类得实现 `Serializable`，配置文件里加上 `<cache/>`。多个会话查相同 `Mapper` 和参数时，先看二级缓存有没有。

但要注意，二级缓存如果没设计好，可能出脏数据。比如 A 会话更新了数据还没提交，B 会话就从缓存读了旧值。一般在读多写少场景用，像商品信息这种，配合 `TTL` 控制失效时间。

代码上，关键点是 `MappedStatement` 的 `cache` 属性是否绑定缓存实例。每次执行查询，`CachingExecutor` 先问 `cache` 有没有，没有再委托给 `BaseExecutor` 去查数据库。



简单说，一级缓存防一次会话重复查，二级缓存扛住多个会话的重复请求。真并发高并发场景，还是建议用 Redis 自定义缓存策略，比 MyBatis 默认的更可控。

简述 MyBatis 的插件运行原理，以及如何编写一个插件？

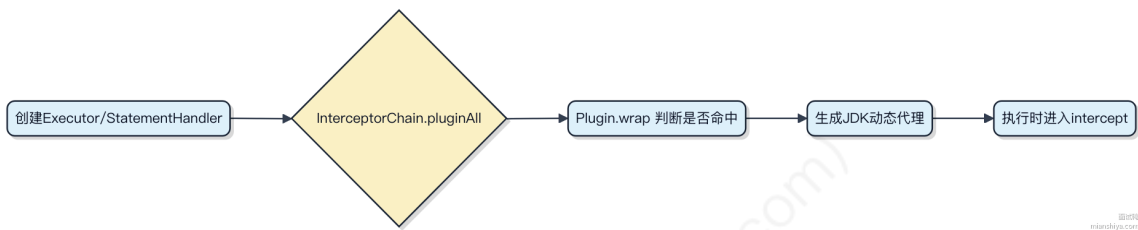
MyBatis 插件本质上是通过 **拦截器** 机制，对四大核心对象的方法调用进行增强。这四个对象分别是 Executor、StatementHandler、ParameterHandler 和 ResultSetHandler。插件不会侵入原有逻辑，而是利用动态代理，在关键方法执行时回调你定义的 intercept 方法。

- 1) 要写一个插件，先实现 Interceptor 接口，重写 intercept 方法，里面写你的增强逻辑，比如打印 SQL 执行时间。
- 2) 用 @Signature 注解声明你要拦截哪个类的哪个方法，目标明确，不能乱来。
- 3) 最后在 mybatis-config.xml 里注册这个插件，或者用编程式配置加进去。

```
@Intercepts({
    @Signature(type = StatementHandler.class, method = "prepare", args =
    {Connection.class, Integer.class})
})
public class SqlCostPlugin implements Interceptor {
    @Override
    public Object intercept(Invocation invocation) throws Throwable {
        long start = System.currentTimeMillis();
        try {
            return invocation.proceed();
        } finally {
            long cost = System.currentTimeMillis() - start;
            System.out.println("SQL 执行耗时: " + cost + "ms");
        }
    }
}
```

注意，多个插件会形成责任链，按注册顺序依次执行。代理逻辑由 Plugin.wrap 方法触发，它会判断当前对象是否匹配 @Signature 声明的目标，是的话就包一层代理。

整个过程其实就是在创建四大对象时，调用 interceptorChain.pluginAll() 把所有插件都套上去。比如 new Executor 时，会被各个插件的 plugin 方法处理，最终返回被代理后的实例。



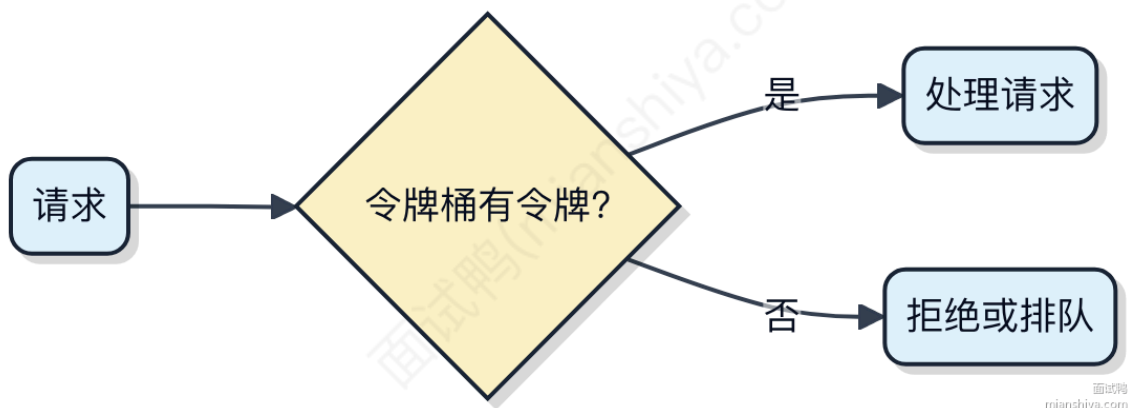
什么是限流？限流算法有哪些？怎么实现的？

限流本质是控制系统的输入或输出流量，防止突发高并发压垮服务。比如秒杀场景，系统只能扛住 2000 QPS，那就要把超过的请求挡掉或排队。

常见的限流算法就三种：计数器、滑动窗口、漏桶和令牌桶。前两个其实是一类思路，后两个更精细。

- 1) 计数器最简单，比如限制 1 秒最多 100 次请求，就用一个变量累加这 1 秒内的请求数，超了就拒绝。但它在时间边界会有突刺问题，比如第 1 秒最后 10ms 放行 100 个，第 2 秒开头又放行 100 个，瞬间 200 个请求打进来。
- 2) 滑动窗口解决了这个问题，把 1 秒拆成 10 个 100ms 的小格子，每次只看最近连续的 10 个格子的总请求数。这样流量更平滑，Redis + Lua 可以实现，Sentinel 里就有这玩意。
- 3) 漏桶模型强调恒定速率处理请求，不管进来的多快，都按固定速度“漏水”处理。超出桶容量的请求直接丢弃。它能削峰填谷，但无法应对短时间的合理 burst 流量。
- 4) 令牌桶反过来，允许一定程度的突发。比如每秒往桶里放 100 个令牌，请求来要先拿到令牌才能执行。桶里最多存 200 个令牌，意味着最多能承受 200 的瞬时并发，之后没令牌就等着或者被拒。Guava RateLimiter 用的就是这个。

代码上，用 Redis 实现分布式限流比较常见，比如用 INCR 维护窗口计数，配合 EXPIRE 设置过期时间。集群环境下也可以用 Sentinel 这种中间件统一控制。



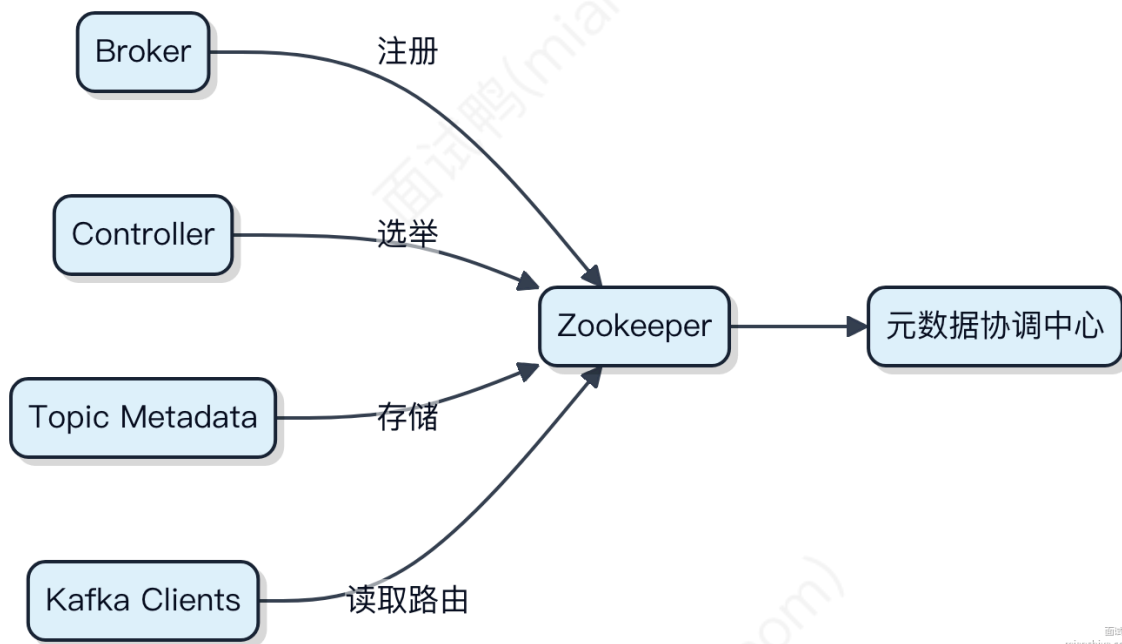
Kafka 中 Zookeeper 的作用？

Kafka 早期版本依赖 Zookeeper 主要是做分布式协调，比如管理集群元数据、控制器选举、Topic 配置这些事。你可以把 Zookeeper 当成一个高可用的分布式文件系统，Kafka 拿它来存一些关键的共享状态。

- 1) Broker 注册：每个 Broker 启动时会在 Zookeeper 的 `/brokers/ids` 下创建临时节点，其他组件就能知道有哪些 Broker 在线。一旦 Broker 宕机，节点消失，集群立刻感知。

- 2) Controller 选举: Kafka 集群里只有一个 Controller 负责分区和副本的状态管理。它是通过在 Zookeeper 创建 `/controller` 临时节点实现的, 第一个写入成功的 Broker 成为 Controller, 挂了之后其他节点重新争抢。
- 3) Topic 和分区元数据: Topic 的分区分布 (哪些副本在哪个 Broker)、配置参数都存在 Zookeeper 的 `/brokers/topics` 路径下, 所有 Broker 共享这份信息。
- 4) ACL 和配额管理: 安全相关的访问控制列表、客户端读写限流配置也放在 Zookeeper。

不过从 Kafka 2.8 开始推 **KIP-500**, 用内部的 KRaft 协议替代 Zookeeper, 目标就是干掉这个外部依赖。新版本里 Controller 自己用 Raft 算法选主, 元数据存在本地磁盘, 启动时也不再连 Zookeeper。现在 KRaft 已经 GA, 官方推荐新部署直接用 KRaft 模式。



所以面试答这题要分两段说: 老架构里 Zookeeper 是“大脑”, 但新趋势是去 Zookeeper 化, KRaft 才是未来。

Kafka为什么要抛弃 Zookeeper?

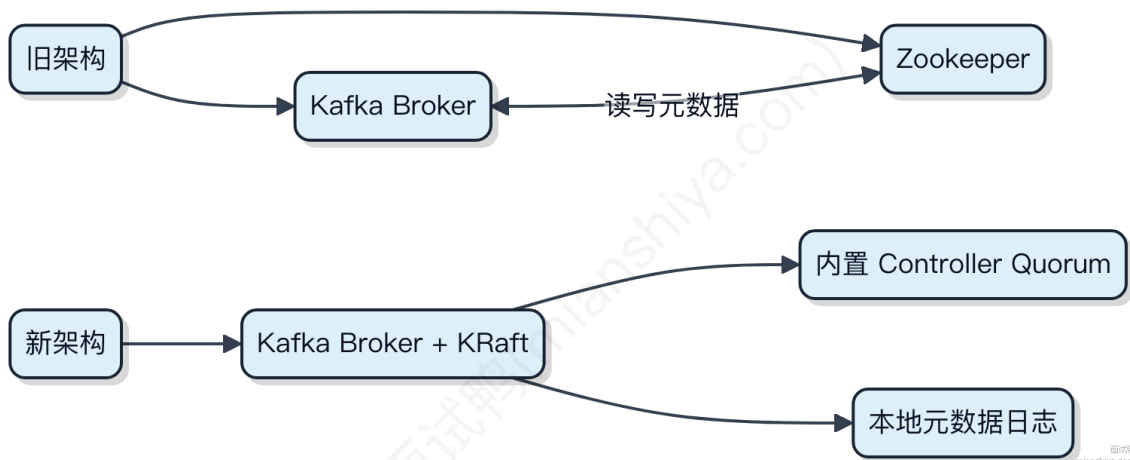
Kafka 早期依赖 Zookeeper 做元数据管理和节点协调, 比如 broker 注册、controller 选举、ACL 权限控制等。但这种架构在大规模场景下暴露了不少问题。

- 1) 运维复杂。部署 Kafka 必须搭一套 Zookeeper, 两个系统之间的交互出问题, 排查特别麻烦。比如网络抖动导致 session timeout, 容易误触发 controller 切主。
- 2) 性能瓶颈。所有元数据变更都要走 Zookeeper 的 ZAB 协议, 写入延迟高, 扩展性差。当 topic 数量达到几万时, ZK 的 watch 通知机制会成为瓶颈, 整个集群响应变慢。
- 3) 功能受限。Zookeeper 是通用协调服务, 没法针对 Kafka 的语义做优化。比如分区重分配、副本状态机这些高频操作, 都得绕一圈发到 ZK, **元数据一致性的实现路径**太长。

从 2.8 版本开始, Kafka 推出了 KRaft 模式——用内置的共识协议替代 Zookeeper。Controller 自己通过投票选主, 元数据直接由专门的日志 (MetadataLog) 持久化在本地磁盘, 不再外连。

代码层面, broker 配置改用 `process.roles` 和 `quorum.connect`, 完全脱离 `zookeeper.connect`。社区明确表示未来会彻底移除 ZK 依赖。

迁移后，集群启动更快，元数据操作延迟下降 70% 以上，单集群支持百万级分区也更现实。像 Confluent 和 AWS MSK 已全面转向 KRaft 架构。



RabbitMQ 中无法路由的消息会去到哪里？

消息发到 RabbitMQ 里头，如果 Exchange 根本找不到对应的 Queue，这条消息去哪儿了？

这得看发消息的时候有没有开启 mandatory 标志。

1) mandatory 没开，也就是默认情况，消息压根不经过任何处理就直接被 broker 丢掉了。客户端啥也不知道，就像石沉大海。

2) mandatory 开了，broker 发现这条路走不通，就会把消息扔回给生产者，这个动作叫 **Return**。Spring AMQP 里头你得注册 `RabbitTemplate.ReturnCallback` 才能收到这类消息。

所以关键就是这个 Return 机制。你可以拿它来做失败日志、监控报警，或者把消息落库做补偿。

举个例子，你往一个 direct exchange 发消息，routing key 是 order.create，但没任何一个 queue 绑定这个 key，那这条消息必然无法路由。

代码上长这样：

```
rabbitTemplate.setMandatory(true);
rabbitTemplate.setReturnCallback((message, replyCode, replyText, exchange,
routingKey) -> {
    // 处理退回的消息，比如发告警或存 DB
});
```

整个过程其实就两条路：静默丢弃，或者退回来让你自己兜底。要不要开启，取决于你对消息可靠性的要求。比如订单创建这种核心链路，肯定得开，不然搞不定问题出在哪。

RabbitMQ 中消息什么时候会进入死信交换机？

消息进不了队列，或者处理不了，总不能一直卡着吧。RabbitMQ 用死信交换机（DLX）来兜底这些“问题消息”。

消息变成死信一般就三种情况：

- 1) 消费者拒收消息，并且明确告诉 RabbitMQ 别再发了（requeue=false）。比如业务校验失败，这消息就是有问题，重试也没用。
- 2) 消息在队列里等太久，超时了（TTL 过期）。可以给单条消息设 TTL，也可以给整个队列设统一过期时间。一过期，直接扔 DLX。
- 3) 队列满了，新消息挤不进去，最老的那条就会被丢进死信交换机。这种情况常见于积压严重、消费者挂了的场景。

要启用 DLX，得在声明队列的时候绑定一个交换机，同时指定 `x-dead-letter-exchange` 参数。还可以加个路由键 `x-dead-letter-routing-key` 控制怎么转发。

```
Map<String, Object> args = new HashMap<>();
args.put("x-dead-letter-exchange", "dlx.exchange");
args.put("x-dead-letter-routing-key", "dead");
channel.queueDeclare("normal.queue", true, false, false, args);
```

这样，一旦消息符合上面三种情况之一，就会自动发到 `dlx.exchange` 上，你再绑个死信队列专门处理就行。

实际项目里，RocketMQ 的延迟消息 + 死信队列组合经常用来做订单超时关单，思路是一样的——把搞不定的先挪走，别耽误正常流程。

谈谈你了解的最常见的几种设计模式，说说他们的应用场景

工厂模式解决对象创建的复杂性问题。当系统需要根据条件生成不同类型的对象时，比如解析不同格式文件（XML、JSON），用工厂就能把创建逻辑集中管理，调用方不用关心具体类名。

- 1) 简单工厂：写一个静态方法，传个参数返回对应实例，适合产品种类少的情况
- 2) 工厂方法：每个子类对应一个工厂类，符合开闭原则，新增类型不影响原有代码
- 3) 抽象工厂：生产一整套相关的产品族，比如 MySQL 和 Oracle 的 DAO 实现分别由不同的工厂批量产出

单例模式保证一个类只有一个实例。常见的做法是私有构造器 + 静态变量 + 公共静态访问方法。注意线程安全问题，推荐使用**枚举**实现，既防反射攻击又能防止序列化破坏单例。

```
public enum Singleton {
    INSTANCE;
    public void doSomething() { /*...*/ }
}
```

观察者模式定义了一对多的依赖关系，主题变化时通知所有观察者自动更新。典型场景是事件驱动系统，比如 Spring 的 `ApplicationEvent` 机制，监听 `ContextRefreshedEvent` 做初始化动作。

代理模式在不改变原始类接口的前提下增强功能。静态代理手动写，动态代理可以用 JDK 原生（基于接口）或 CGLIB（基于继承）。Spring AOP 默认用 JDK 动态代理，除非目标类没有接口才退化到 CGLIB。

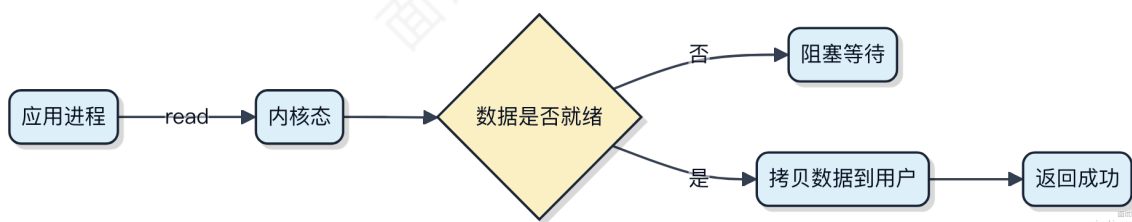
模板方法模式把算法骨架固定在父类，把可变步骤延迟到子类实现。像 Java 中的 `AbstractExecutorService.submit()` 就是模板，真正执行交给具体线程池策略。

这几个模式在框架里到处都是，MyBatis 用工厂造 `SqlSession`，Spring 大量使用代理和观察者，理解它们等于看懂了框架的筋骨。

说说你知道的几种 I/O 模型

Unix 系统里处理网络 I/O，绕不开五种经典模型，它们本质是描述数据在内核态和用户态之间搬移的过程差异。

- 1) 阻塞 I/O 最常见，比如 Java 传统 Socket 编程。调用 read 时，线程卡在那里，直到数据从网卡到达并拷贝到用户缓冲区。期间 CPU 干不了别的，**同步阻塞**，简单但并发上不去。
- 2) 非阻塞 I/O 会把 socket 设成 NONBLOCK。每次 read 直接返回，有数据就读，没数据就报 EAGAIN。需要轮询，CPU 空转严重，实际很少直接用。
- 3) I/O 多路复用才是重点。select、poll、epoll 都属于这一类。一个线程能监听多个 socket 的事件。比如 epoll_ctl 注册关注的 fd，epoll_wait 阻塞等待，哪个连接来了数据或可写了，就通知你去处理。Nginx、Redis、Netty 都靠它扛住几万并发。
- 4) 信号驱动 I/O 用 SIGIO 信号触发，内核收到数据后发信号，应用再读。异步通知，但信号处理复杂，用的人少。
- 5) 真正的异步 I/O，POSIX 的 aio_read，Linux 的 io_uring。发起读请求后立刻返回，数据由内核完整准备好并拷贝到用户空间后，再回调你的函数。整个过程不阻塞，**完全异步**，但编程模型复杂，glibc 的 aio 实现还不成熟，Java 的 AIO 实际落地不多。



主流高性能服务基本都选 epoll 这条路，代码稍微复杂点，但资源利用率最高。

说一下 Netty 的应用场景？

Netty 本质是个高性能的网络编程框架，把 NIO 那套复杂的 API 给封装得特别顺手。它真正发力的地方是需要处理大量并发连接、低延迟通信的场景。

- 1) 微服务内部通信首选。像 Dubbo、gRPC 这些 RPC 框架底层都用 Netty 做传输层，因为能轻松扛住几万甚至十几万的并发连接，吞吐量也稳。
- 2) 即时通讯和消息推送绕不开它。比如做聊天室、直播弹幕、APP 推送服务，客户端长连接一多，传统 BIO 直接崩，Netty 的事件驱动模型压根不经过线程阻塞，一条连接一个 Channel，资源占用少得多。
- 3) 网关类中间件也常见。Spring Cloud Gateway 虽然默认用 Reactor Netty，但底层还是 Netty 的能力，用来做请求路由、协议转换、限流熔断，性能损耗小。
- 4) 游戏服务器后端经常拿它打底。帧同步、状态广播这些高频操作，要求毫秒级响应，Netty 配合自定义协议编解码，能把延迟压到很低。

写个最简示例，启动一个 TCP 服务端也就十来行代码：

```
EventLoopGroup boss = new NioEventLoopGroup(1);
EventLoopGroup worker = new NioEventLoopGroup();
ServerBootstrap b = new ServerBootstrap();
b.group(boss, worker)
  .channel(NioServerSocketChannel.class)
  .childHandler(new ChannelInitializer<SocketChannel>() {
    public void initChannel(SocketChannel ch) {
```

```

        ch.pipeline().addLast(new SimpleChannelInboundHandler<ByteBuf>() {
            protected void channelRead0(ChannelHandlerContext ctx, ByteBuf msg) {
                // 处理读取数据
            }
        });
    }
});
b.bind(8080).sync();

```

这种简洁性和可控性，让 Netty 在高并发网络通信领域几乎成了事实标准。

介绍一下 Reactor 线程模型？

Reactor 模型本质是事件驱动的设计，用少量线程处理大量并发连接，避免为每个连接创建线程带来的资源开销。Java NIO 的 Selector 就是典型实现。

1) 单 Reactor 单线程

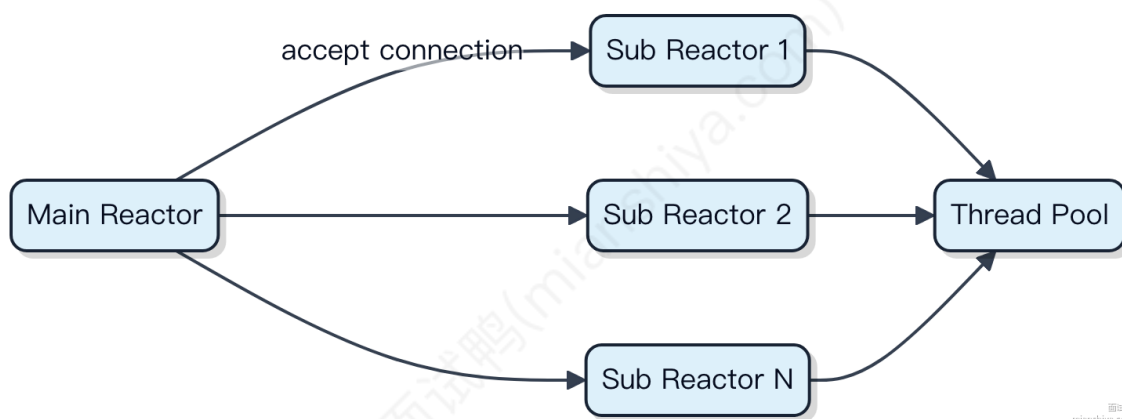
一个线程负责监听事件、分发请求、处理 I/O 和业务逻辑。适合低并发场景，比如 Redis 主线程就是这种模式。代码结构简单，但 CPU 密集任务会阻塞整个流程。

2) 单 Reactor 多线程

Reactor 线程只做事件监听和分发，具体业务交给线程池处理。这样 I/O 和计算解耦，Netty 默认就是这种模型。注意 Reactor 仍只有一个，注册、事件分发都在同一线程。

3) 主从 Reactor 多线程

多个 Reactor 分工协作，通常一个 Main Reactor 负责 Accept 新连接，把建立好的连接注册到多个 Sub Reactor 上，由它们处理读写事件。Nginx 和 Netty 都支持这种模式，能更好利用多核。



实际选型看场景：轻量通信用单线程，高并发网络服务上主从多线程。Netty 通过 `EventLoopGroup` 配置就能切换模式，灵活性很强。

简单说说 Netty 的零拷贝机制？

Netty 的零拷贝不是操作系统层面的 `zero-copy`，而是应用层通过 `CompositeByteBuf` 和 `FileRegion` 等设计，减少数据在 JVM 堆内不必要的复制和内存占用。

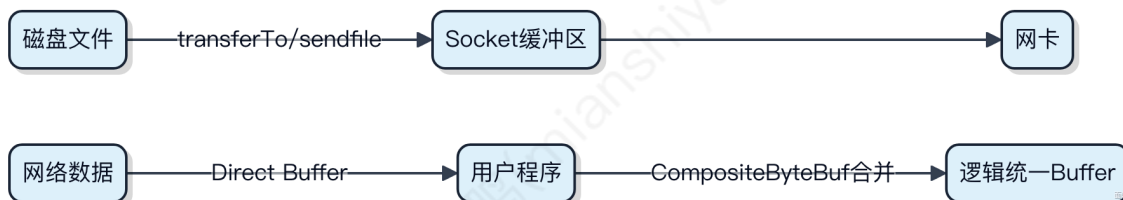
1) 比如接收多个 Buffer 拼接成完整消息时，传统做法是分配新内存，把各个片段拷贝过去。Netty 用

`CompositeByteBuf` 把多个 `ByteBuf` 聚合成一个逻辑视图，**不发生实际的数据搬移**，读取时像操作单个 Buffer 一样。

2) 发送文件时, Netty 调用 JDK 的 `FileChannel.transferTo()`, 直接从文件系统缓存写到网卡, 压根不经过用户态内存。这一步才是真正利用了 OS 的零拷贝能力, 底层对应 `sendfile` 系统调用。

3) 还有直接使用堆外内存 (Direct Buffer), 避免 JVM 堆内 GC 带来的复制开销。数据从网卡读取直接落进堆外内存, 处理完也能直接发出去, 中间不再拷贝到堆内 `byte[]`。

典型场景就是大文件传输或者协议拼包, 像 Kafka、RocketMQ 都用 Netty 做传输层, 靠的就是这种高效内存管理扛住高吞吐。



注意, 很多面试官容易把“不用拷贝”理解成完全无复制, 其实只是链路整体减少了冗余拷贝, 关键路径上能省则省。

在什么情况下, 不推荐为数据库建立索引?

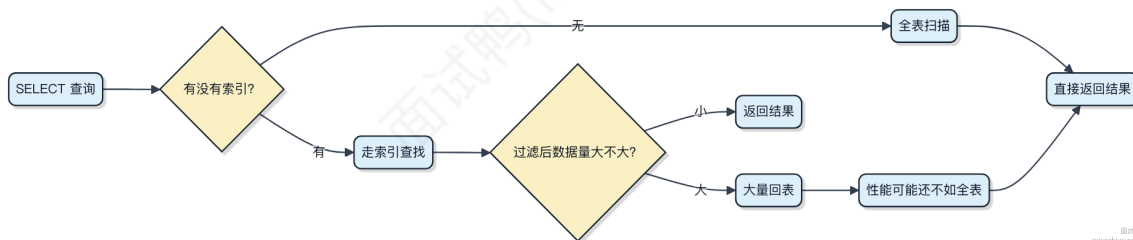
数据量小的时候, 比如表里就几百条记录, 查询全表扫描可能比走索引还快。因为索引本身要维护 B+ 树结构, 读写都有额外开销, 数据少的情况下这层跳转反而拖慢速度。

频繁更新的字段也不适合建索引。像用户余额、订单状态这种改动特别勤的列, 每次变更都要同步更新索引树, **B+ 树**为了维持平衡会触发分裂或合并, 写放大问题严重, 容易成为性能瓶颈。

区分度低的字段建了也白建。比如性别只有男/女, 或者状态码就几个固定值, 查索引后还是要回表扫一大片, 优化器干脆就走全表了。MySQL 一般预估扫描行数超过 20% 就不会用索引。

大文本字段别直接建普通索引。VARCHAR(1000) 这种往索引里一放, 一页存不了几条, 树层级拉得老高, IO 效率暴跌。真要搜可以用前缀索引, 但得权衡截取长度和冲突率。

- 1) 联合索引注意最左匹配原则, 中间断了就用不上
- 2) NULL 值多的字段效果差, 索引对空值处理比较麻烦
- 3) OR 条件如果非主键字段没覆盖到, 索引也可能失效



为什么不选择使用原生的 NIO 而选择使用 Netty 呢?

直接上 NIO 其实不是搞不定, 但开发效率和维护成本会高到离谱。NIO 的 API 本身太底层了, 像 `ByteBuffer` 的手动管理、粘包拆包处理、各种 Channel 的状态维护, 全是脏活累活。

Netty 的核心优势是把网络编程里的通用问题都封装好了。比如它用 **ByteBuf** 重新设计了缓冲区，支持池化、自动扩容，还带引用计数，内存管理比原生的 Buffer 好用太多。

粘包问题 Netty 提供了现成的解码器，像 `LengthFieldBasedFrameDecoder`，几行配置就能搞定，而原生 NIO 得自己写一大堆逻辑判断边界。

线程模型上，Netty 封装了 EventLoop，把 Reactor 模型玩得很透。一个 Channel 绑定一个 EventLoop，线程安全直接保障，不用操心并发冲突。原生 NIO 虽然也能实现多路复用，但 Reactor 的分发逻辑全得自己写，稍微复杂点就容易出错。

代码示例简单看看服务端启动：

```
new ServerBootstrap()
    .group(bossGroup, workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<SocketChannel>() {
        protected void initChannel(SocketChannel ch) {
            ch.pipeline().addLast(new MyBusinessHandler());
        }
    })
    .bind(8080);
```

这几十行代码背后，Netty 已经帮你处理了连接建立、事件轮询、异常传播、资源释放等全流程。

- 1) NIO 适合学习原理，但生产开发效率太低
- 2) Netty 提供了开箱即用的编解码、心跳、流量整形能力
- 3) 社区生态强，主流中间件如 Dubbo、RocketMQ 都基于它做通信层

Netty 性能为什么这么高？

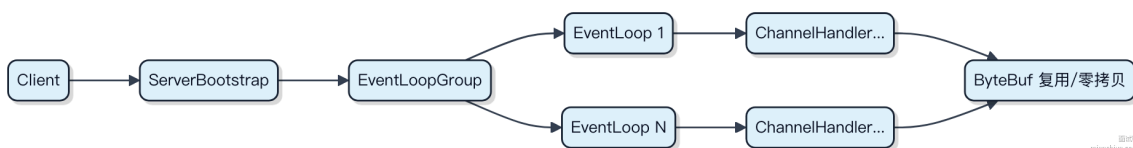
Netty 的高性能不是靠单一技术堆出来的，而是从线程模型、内存管理到 API 设计一整套协同优化的结果。

Reactor 模式是它底层的主心骨。服务端用少量线程就能同时处理成千上万的连接，不像传统 BIO 每个连接起一个线程，根本扛不住高并发。Netty 的 EventLoopGroup 把线程和事件循环包得干净利落，连接的读写事件全由固定的线程负责，避免上下文切换开销。

内存层面，对象池化玩得很深。ByteBuf 可以复用，减少 GC 压力。一次请求的数据缓冲如果每次都 new，GC 频繁时系统直接卡住。Netty 提供 PooledByteBufAllocator，默认情况下能复用内存块，吞吐量提升明显。

零拷贝不只是“不拷贝”，而是尽量让数据传输路径最短。比如 CompositeByteBuf 可以把多个 buffer 聚合成一个逻辑整体，发送时不用合并复制。文件传输用 FileRegion，直接走 JVM 之外的通道，压根不经过用户内存。

代码写起来也省心，责任链式的 ChannelPipeline 让业务逻辑拆解清晰，每个 Handler 各司其职。但别小看这设计，它保证了扩展性的同时，性能损耗几乎可以忽略。



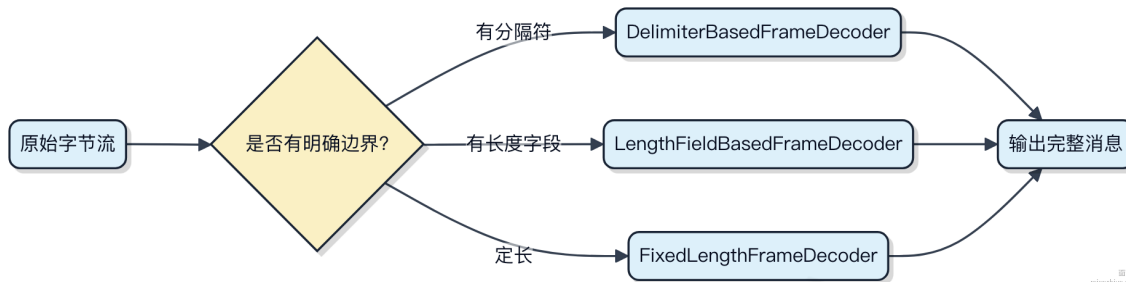
对比原生 NIO，Netty 封装了繁琐的 register、select、dispatch 流程，API 简洁不说，还避免了很多生产环境的坑，比如 epoll 空轮询导致 CPU 100%。这种细节处理，才是它能在 Dubbo、RocketMQ、gRPC 里站稳脚跟的原因。

Netty 是如何解决粘包和拆包问题的？

TCP 本身是面向流的协议，不关心应用层消息边界，这就导致接收方可能读到半条消息（拆包）或者多条消息粘在一起（粘包）。Netty 不是“解决”这个问题，而是提供了一组**解码器工具类**来帮助我们定义消息边界。

- 1) 最常见的办法是加长度字段。比如用 `LengthFieldBasedFrameDecoder`，它会先读取一个表示 body 长度的字段，再按这个长度截取完整报文。像 Dubbo、RocketMQ 内部都这么干，头部有个固定的 length 字段。
- 2) 如果协议简单，可以用分隔符。比如每条消息结尾加 `\n` 或自定义分隔符，对应 Netty 的 `DelimiterBasedFrameDecoder`。适合文本协议，比如 Redis 的 RESP 协议早期版本就靠回车换行切分。
- 3) 定长消息直接用 `FixedLengthFrameDecoder`，每个包固定大小。适用于心跳包这种场景，比如每 30 秒发一个 20 字节的心跳。

其实关键就是：**你得自己定义协议格式，让接收方能识别一条完整消息从哪开始、到哪结束**。Netty 提供这些开箱即用的工具，避免我们手动去缓冲、拼接字节流。



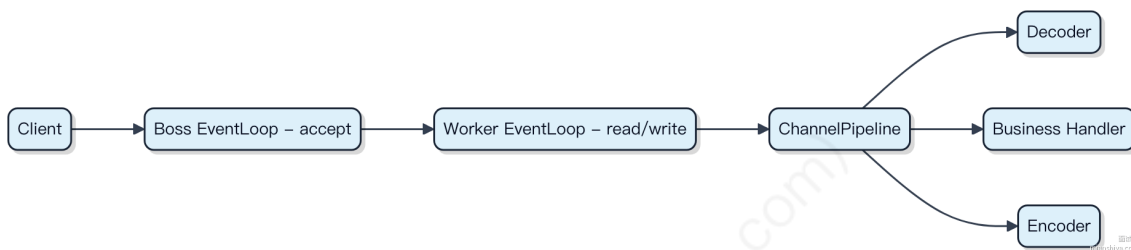
Netty 采用了哪些设计模式？

Netty 的设计里，Reactor 模式是主心骨。它用事件循环（EventLoop）来统一调度 I/O 读写和任务执行，把网络通信的复杂性屏蔽掉，让业务逻辑可以专注处理数据。服务端启动时绑定一个或多个 EventLoopGroup，每个连接由固定的 EventLoop 处理，避免锁竞争。

- 1) 单线程 Reactor：所有操作都在一个线程完成，适合低并发测试。
- 2) 多线程 Reactor：I/O 线程只负责读写，Handler 逻辑扔给业务线程池，提升吞吐。
- 3) 主从 Reactor：Boss 线程监听 accept，Worker 线程池处理 read/write，生产环境常用。

责任链模式体现在 ChannelPipeline 上。每个 ChannelHandler 是一环，进站出站事件沿链传递。你可以动态添加日志、编解码、限流等处理器，彼此解耦。比如加个 `StringDecoder` 和 `LoggingHandler` 完全不用改原有逻辑。

工厂模式藏在 ByteBuf 分配里。通过 `ByteBufAllocator` 统一创建堆内/堆外内存缓冲区，上层代码不关心具体实现。`PooledUnsafeDirectByteBuf` 和 `UnpooledHeapByteBuf` 各司其职，复用内存池减少 GC 压力。



别小看这些模式的组合，它们让 Netty 既灵活又扛住高并发。像 RocketMQ、Dubbo 底层都靠这套机制撑着。

Netty 如何解决 JDK NIO 中的空轮询 Bug?

Netty 并没有从底层修复 JDK NIO 的 epoll 空轮询 Bug，而是通过**上层机制规避**了它带来的影响。这个 Bug 表现在某些 Linux 版本的 JDK 中，Selector 会在没有实际 I/O 事件的情况下返回，导致 CPU 占用飙到 100%。

Netty 的解决方案其实很务实：它给 Selector 设置了一个“**最大连续空轮询次数**”阈值，默认是 512 次。每发生一次空轮询，计数器加 1。一旦达到这个阈值，Netty 就会重建 Selector。

重建 Selector 是关键操作。它把所有 Channel 重新注册到新的 Selector 上，绕过旧 Selector 可能存在的内核 bug 状态。这个过程虽然有点重，但远比持续空跑耗 CPU 强。

代码层面，这个逻辑在 `NioEventLoop` 的 `select` 循环里：

```
if (selectedKeys != 0 || oldWakenUp || wakenUp.get() || hasTasks() ||
    hasScheduledTasks()) {
    // 有事件或任务，正常处理
} else {
    // 空轮询，计数++
    if (selectedCnt >= SELECTOR_AUTO_REBUILD_THRESHOLD) { // 默认512
        rebuildSelector(); // 重建
        return;
    }
}
```

这种做法不依赖 JDK 修复，兼容性强，像 Dubbo、RocketMQ 这些用 Netty 做通信层的框架，都间接受益于这个防护机制。

要不要触发重建，还得看实际负载。如果真没请求，有任务调度也不会进空轮询。只有死循环无意义唤醒才算。

为什么 RocketMQ 不使用 Zookeeper 作为注册中心呢？而选择自己实现 NameServer？

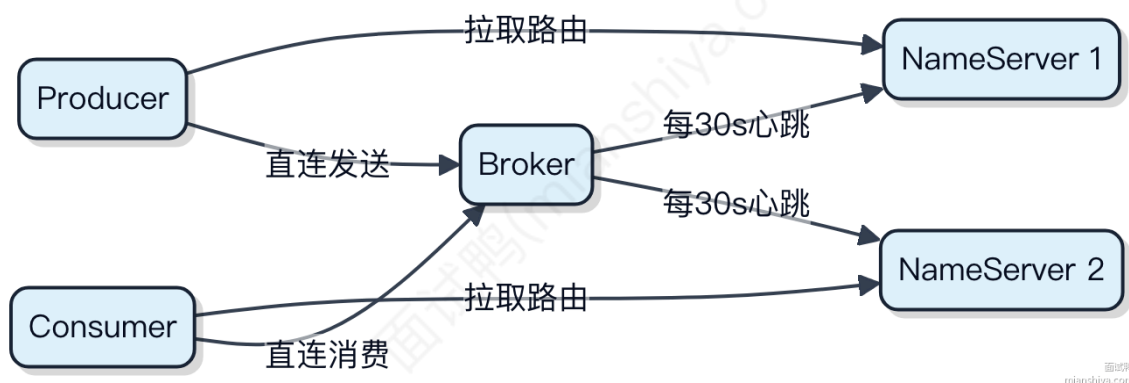
RocketMQ 没有采用 Zookeeper 作为注册中心，而是自己搞了个 NameServer，这背后其实是架构取舍的问题。

Zookeeper 虽然能做服务发现，但它设计目标是强一致性的协调服务，用的是 ZAB 协议，每次写操作都要落盘、还要多数派确认。在 Broker 频繁上下线、心跳上报的场景下，ZK 的性能压根扛不住，延迟高还容易假死。像 Kafka 就吃过这个亏，后来也在往 KRaft 转。

NameServer 的设计就轻量多了。它本质上是**个无状态的路由注册表**，Broker 每隔 30 秒主动向所有 NameServer 发送心跳，上报自己的 Topic 路由信息。NameServer 不参与消息收发，也不做持久化，宕机了 Broker 还能靠本地缓存继续工作一阵。

客户端（Producer/Consumer）启动时从 NameServer 拉取路由，之后定时更新。整个链路简单直接，没有选举开销，扩容也方便，加机器就行。

- 1) Broker 启动后连接多个 NameServer 并定时发送心跳
- 2) Producer 从任一 NameServer 获取 Topic 路由（包含 Broker 地址）
- 3) Producer 直接与 Broker 建立长连接收发消息



这套模式牺牲了一定的强一致性，换来的是高可用和低延迟。毕竟在消息系统里，**稳定和性能比绝对一致更重要**。NameServer 虽然看起来“简陋”，但恰恰是适合 RocketMQ 场景的最优解。

线上 CPU 飙高如何排查？

CPU 飙高排查本质是找到占用 CPU 的元凶线程，并定位到代码堆栈。整个过程分三步走：先看系统，再看进程，最后看线程和堆栈。

- 1) 用 `top` 命令看整体 CPU 使用情况，如果 `%Cpu(s)` 的 `us`（用户态）或 `sy`（内核态）持续高于 80%，说明系统压力大。
- 2) 继续在 `top` 中按 `P` 按 CPU 排序，找到 Java 进程的 PID。
- 3) 执行 `top -H -p <PID>`，查看该进程下所有线程的 CPU 占用，找出 top 几个高消耗线程，记下其线程 ID（TID）。

接着把 TID 转成 16 进制（比如 `printf "%x\n" <TID>`），然后用 `jstack` 抓堆栈：`jstack <PID> | grep -A 50 <hex_TID>`，就能看到对应线程的调用栈，通常是 **RUNNABLE** 状态的业务线程在死循环、正则回溯、频繁 GC 或序列化卡住。

常见场景比如：

- 正则表达式存在灾难性回溯，像 `(.+) +` 匹配长字符串，CPU 直接拉满。
- 日志里频繁输出大对象 JSON，Jackson/Gson 在序列化时卡住。
- 外部依赖超时导致线程池积压，大量线程处于 **RUNNABLE** 等待网络返回。

代码上可以用 `ThreadMXBean` 主动监控，但线上第一反应还是 `top + jstack` 组合拳最稳。如果是容器环境，记得进宿主机操作或用 `kubectl exec`。

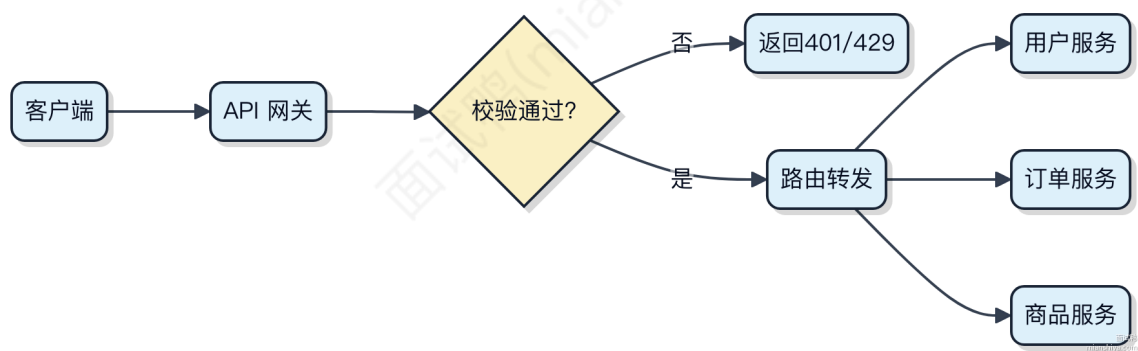
说说什么是 API 网关？它有什么作用？

API 网关本质是所有外部请求进入系统的统一入口，有点像小区的门卫。所有流量都得先经过它，再决定往哪走。

它的核心价值不是转发，而是把一堆通用的、和业务无关的脏活累活给拦下来。比如你用 Spring Cloud Gateway 或者 Kong，其实都是在配置这些能力。

1) 鉴权校验。接口不是谁都能调的，网关层可以检查 JWT token，没带或者过期了直接 401 拒掉，压根不经过后端服务。2) 限流熔断。防止突发流量把系统打崩，按 IP 或用户维度限制 QPS，超过阈值就返回 503，保护下游。3) 日志监控。统一记录请求响应时间、状态码，上报到 ELK 或 Prometheus，排查问题不用挨个服务翻日志。4) 协议转换。前端调 REST，但内部微服务可能走 gRPC，网关做一层翻译，让前端少操心。5) 路由转发。根据 path 路由到不同服务，比如 /user/** 转发到 user-service，路径规则集中管理，后端改了也不用前端跟着变。

拿电商系统举例，下单请求进来，网关先看是不是登录用户，再判断当前流量有没有超限，然后记一笔 traceId 方便链路追踪，最后才放行到订单服务。要是没有网关，每个服务都得自己实现这一套，重复代码多，策略还不一致。



RabbitMQ 如何实现延迟队列?

RabbitMQ 本身没有原生的延迟队列功能，但可以通过 **TTL (Time-To-Live)** 和 **死信队列 (DLX)** 组合实现。

- 1) 给消息或队列设置 TTL，消息在队列中存活时间超过后会自动变成“死信”。
- 2) 为队列绑定死信交换机 (DLX)，当消息过期后，RabbitMQ 会自动将其转发到指定的 DLX，再路由到对应的死信队列。
- 3) 消费者监听这个死信队列，就相当于拿到了“延迟到期”的消息。

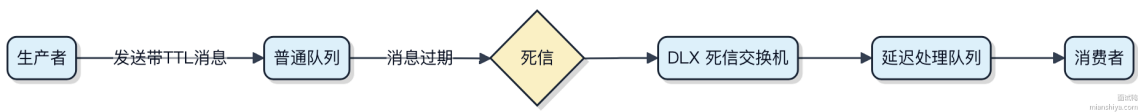
比如你用 Spring Boot 配合 RabbitMQ，可以这样定义：

```
// 队列设置过期，并绑定死信交换机
args.put("x-message-ttl", 5000);
args.put("x-dead-letter-exchange", "delay.process.exchange");
```

这种方式能支持秒级精度的延迟，适合订单超时关闭、优惠券定时失效这类场景。像美团、滴滴早期也有类似实践，基于 RabbitMQ 做任务延时调度。

不过要注意，如果大量消息同时过期，死信队列可能瞬间积压，消费端得能扛住压力。另外，RabbitMQ 是内存+磁盘存储，长时间堆积会影响性能。

真正高并发、高精度的延迟需求，一般会换用专业系统，比如 RocketMQ 的延迟消息，或者用 Redis ZSet 跑定时轮询。



说说 Redisson 分布式锁的原理？

Redisson 的分布式锁核心是基于 Redis 的 Lua 脚本和 Redis 单线程特性来保证原子性，同时支持可重入、锁续期等高级功能。

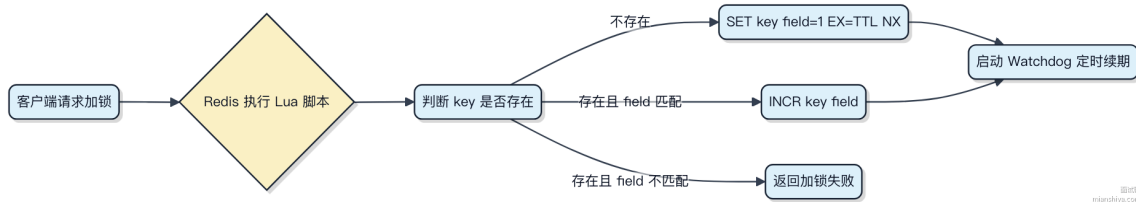
客户端加锁时，会通过一段 Lua 脚本向 Redis 写入一个 hash 结构，key 是锁名，field 是当前线程的唯一标识（比如 UUID + threadId），value 是重入次数。如果 key 不存在，就设置过期时间防止死锁；如果已存在且 field 匹配，就将 value 加一，实现可重入。

解锁同样是 Lua 脚本操作，先校验 threadId 是否匹配，不匹配直接返回失败；匹配则将重入数减一，减到 0 就删除 key 并释放锁。

最关键是锁续期机制，也就是看门狗（Watchdog）。当客户端拿到锁后，会启动一个后台任务，每过 1/3 过期时间就检查锁是否还被持有，如果还在用就自动延长 TTL，避免业务还没执行完锁就过期了。

这种设计让客户端不需要手动管理超时，只要线程活着就能保住锁。像 `RLock rlock = redisson.getLock("order"); rlock.lock();` 这样简单调用背后其实已经包含了网络通信、脚本执行和定时任务调度这些脏活累活。

使用场景上，适合订单扣减、库存超卖这类强一致性要求的场景，但要注意网络分区下可能丢失锁的问题，极端场景建议结合 RedLock 模式，不过一般单 Redis 实例配合合理超时也能扛住大部分情况。



说说 Java 中 HashMap 的原理？

HashMap 本质是个数组加链表（或红黑树）的结构，存数据靠 key 的 hash 值定位桶位置。数组初始长度是 16，负载因子默认 0.75，到 12 个元素就扩容，避免冲突太多影响性能。

key 的 hash 值不是直接用，而是再做一次扰动运算，把高位也参与映射，减少碰撞概率。定位桶用 $(n - 1) \& \text{hash}$ 而不是取模，位运算更快。

- 1) 数组里每个桶刚开始是 Node 节点，链表长度超过 8 且数组长度大于等于 64，链表转成红黑树，防止极端情况下查询退化成 $O(n)$
- 2) 如果长度小于 6，即使 hash 冲突严重也先不转树，因为数组太小扩容更划算
- 3) 删除节点时树长度小于 6 又会转回链表

put 操作流程：算 hash → 找桶 → 空则新建，否则遍历链/树插入 → 判断是否扩容。get 就反过来查。

注意多线程下 put 可能导致环形链表，所以并发场景得用 `ConcurrentHashMap`。JDK 8 虽然优化了扩容时的头插改尾插，但依然不能保证线程安全。

SpringBoot 是如何实现自动配置的？

SpringBoot 的自动配置不是魔法，靠的是约定大于配置的思想 and 几个关键机制组合起来的。

启动时会扫描 classpath 下所有 `META-INF`

`/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 文件，里面列了

一堆 **自动配置类**。这些类不会无脑加载，每个都带着条件注解，比如 `@ConditionalOnClass`、`@ConditionalOnMissingBean`，只有满足条件才生效。

举个例子，你引入了 `spring-boot-starter-data-redis`，classpath 里有了 `LettuceConnectionFactory`，自动配置发现这个类存在，而且容器里还没人创建过 `RedisTemplate`，它就帮你把连接池、模板对象都配好。

1) 自动配置类通常放在 `spring.factories` 或 `org.springframework.boot.autoconfigure.AutoConfiguration.imports` 中 2) 条件装配确保不冲突，比如只在某个类存在且某个 Bean 缺失时才创建 3) 配置优先级可通过 `@AutoConfigureAfter` 控制

别小看 `@EnableAutoConfiguration` 这个注解，它就是触发整个链路的开关。实际项目里像 Nacos、MyBatis-Plus 的 starter 都是照这个套路来，把一堆初始化逻辑封装掉，你写个依赖就能用。

说说 Springboot 的启动流程？

Spring Boot 启动看起来就一行 `run()` 调用，背后其实做了不少事。整个过程从创建上下文到刷新容器，每一步都封装好了，但得知道它干了啥。

1) 启动时先初始化一个 `SpringApplication` 实例，这里会做些前置判断，比如当前项目类型是 Servlet 还是 Reactive，然后设置默认的 `ApplicationContextInitializer` 和 `ApplicationListener`。

2) 真正 `run()` 的时候，第一步是准备环境，不管是 Web 环境还是非 Web 环境，都会读取配置文件 (`application.yml` 或 `properties`)，也支持命令行参数覆盖。

3) 接着创建合适的应用上下文 `ApplicationContext`。如果是 Web 项目，就是 `AnnotationConfigServletWebServerApplicationContext`，这个上下文能启动内嵌容器。

4) 上下文准备好后，开始加载 Bean。Spring Boot 会扫描主类所在包下的所有组件，靠的是 `@SpringBootApplication` 里的 `@ComponentScan`。

5) 最关键的一步是**刷新上下文**，这一步触发内嵌服务器启动。比如 Tomcat 就是在 `onRefresh()` 时被拉起来的，端口绑定、连接器配置都在这阶段完成。

6) 最后发布 `ApplicationReadyEvent`，表示服务已经可以接收请求了。

代码上其实就是：

```
public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
}
```

但你知道它背后把自动配置、条件装配、内嵌容器这些脏活累活全包了，才让你能一键启动。

MySQL 事务的二阶段提交是什么？

MySQL 的二阶段提交 (2PC) 是 InnoDB 在崩溃恢复时保证事务持久性和数据一致性的关键机制，主要解决 redo log 和 binlog 的一致性问题。

redo log 记录物理日志，确保事务的原子性和持久性；binlog 是逻辑日志，用于主从复制和 PITR（时间点恢复）。如果两者提交不同步，崩溃后可能不一致。比如事务提交了但 binlog 没写，备库就会丢数据。

为了解决这个问题，InnoDB 采用二阶段提交：

- 1) prepare 阶段：事务产生的 redo log 写入磁盘，并将事务状态标记为 PREPARED。此时 binlog 还没落盘。
- 2) commit 阶段：把 binlog 写入并刷盘，然后 InnoDB 再写一条 commit 标记到 redo log。只有这一步完成，事务才算真正提交。

崩溃恢复时，MySQL 会扫描 undo log 中的 PREPARED 事务，拿着事务的 XID 去 binlog 里查是否存在。如果 binlog 里有这个 XID，说明它已经完成了 prepare，就应该被提交；否则回滚。

这个机制让主从之间的数据同步更可靠，也是 MGR、XtraDB Cluster 等高可用方案的基础。但注意，二阶段提交只在开启 binlog 且使用支持事务的引擎时才生效。

MySQL 三层 B+ 树能存多少数据？

MySQL 的 InnoDB 引擎用 B+ 树组织数据，叶子节点存的是完整行记录，非叶子节点存的是索引键和指针。我们通常说的“三层 B+ 树”指的是根节点、中间层、叶子层。

先算单页容量。InnoDB 默认页大小 16KB。假设主键是 bigint，占 8 字节，指针一般按 6 字节算，那一个页能存的索引项数大约是：

每项占用 $8 + 6 = 14$ 字节

一页可存约 $16 * 1024 / 14 \approx 1170$ 个索引项

也就是说，一层非叶子节点最多指向 1170 个下级页。

再看叶子节点。假设一行数据平均 1KB，那一个 16KB 的页能存 16 行数据。

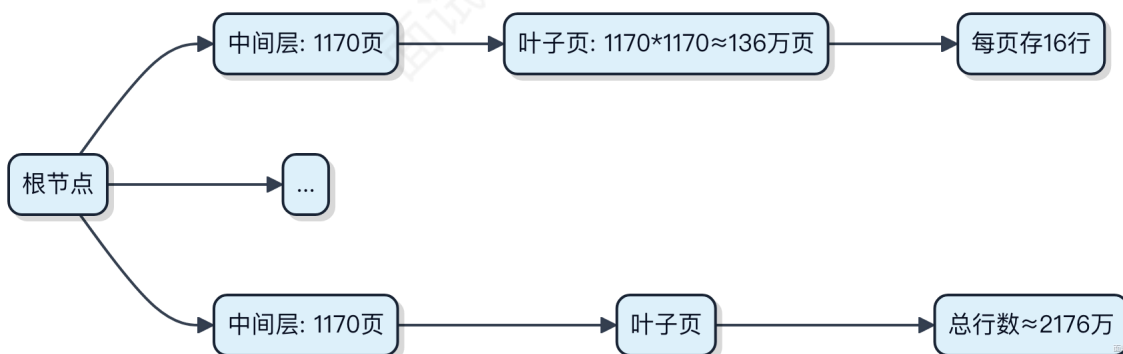
那么三层结构的数据容量就是：

根节点 → 1170 个中间页 → 每个中间页再指向 1170 个叶子页 → 总共约 $1170 * 1170 \approx 136$ 万个叶子页

每个叶子页存 16 行 → 总容量约 $136 \text{ 万} * 16 \approx 2176$ 万行

所以，一个典型的三层 B+ 树能存 **两千万级别**的数据行。

如果数据量超过这个规模，B+ 树会变成四层，查询就需要多一次磁盘 IO。从内存访问到落到磁盘，延迟可能从几十纳秒跳到几毫秒，性能差距巨大。



JVM 由哪些部分组成？

JVM 主要由类加载子系统、运行时数据区、执行引擎和本地库接口组成。

- 1) 类加载子系统负责把 .class 文件加载到内存。它包含加载、验证、准备、解析和初始化五个阶段。比如 Spring Boot 应用启动时，就是靠它把成千上万个类读进 JVM。
- 2) 运行时数据区是核心内存布局，分为方法区、堆、虚拟机栈、本地方法栈和程序计数器。其中堆是最大的一块，存放对象实例和数组，GC 主要就在这里干活。方法区存常量、静态变量、类信息，JDK 8 后用元空间实现，直接扔到了本地内存里。
- 3) 执行引擎负责执行字节码。解释执行逐条翻译，JIT 编译则把热点代码编译成机器码缓存起来，像 HotSpot 就靠这个提升性能。GC 线程也属于执行引擎的一部分，专门处理垃圾回收的脏活累活。
- 4) 本地库接口（JNI）让 Java 调用 C/C++ 代码。比如你用 NIO 操作文件，底层其实是通过 JNI 调用了操作系统 API。



整个流程串起来：类加载器把字节码载入内存，执行引擎读取并运行，需要本地功能时通过 JNI 跳出去。堆和栈的划分决定了对象生命周期和线程隔离方式，这块搞不定，线上 OOM 问题基本扛不住。

JVM 有那几种情况会产生 OOM（内存溢出）？

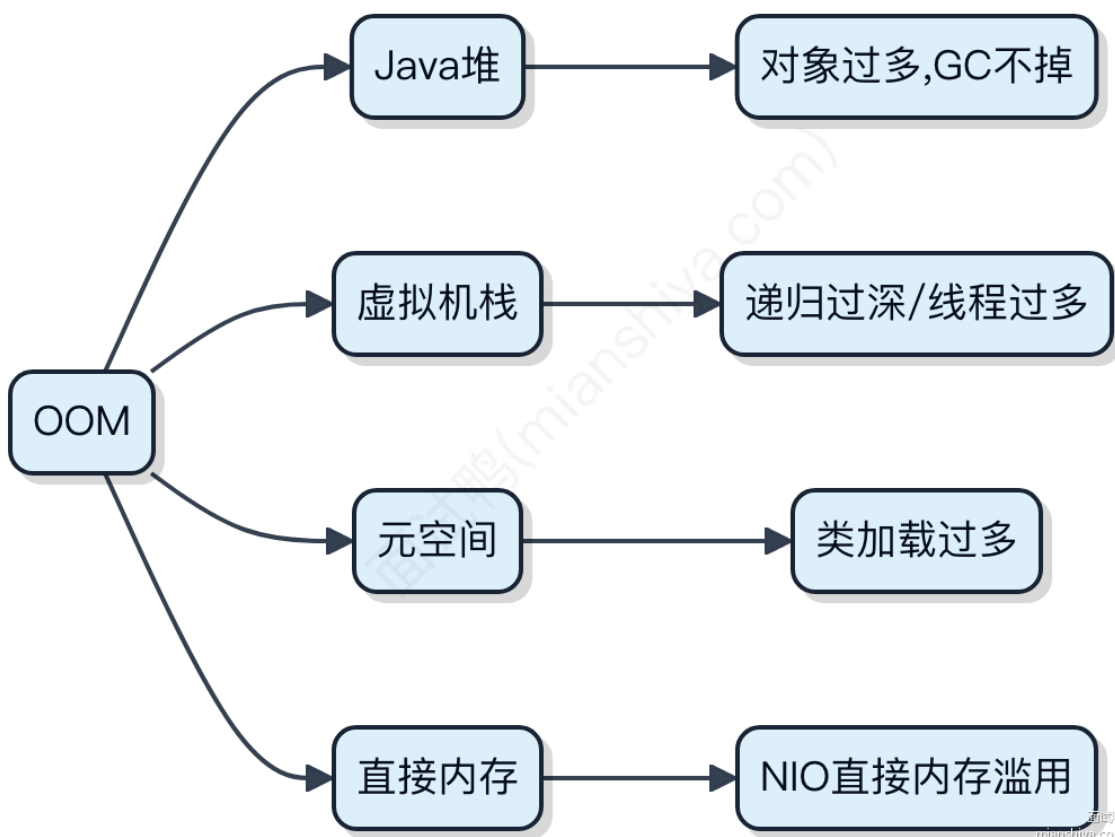
JVM 产生 OOM 的情况主要集中在几个内存区域，每个区域的溢出都有特定场景和表现。

- 1) 堆内存溢出是最常见的，对象持续创建但无法回收，最终超过 `-Xmx` 设置的上限。比如缓存没加过期策略，或者一次加载几百万条数据到 `List`，很容易把堆撑爆。
- 2) 虚拟机栈溢出一般由线程请求深度超过限制引起，典型就是递归太深或死循环。每个线程默认栈大小是 1MB 左右（取决于系统），线程数太多也会耗尽内存，这种情况其实是**栈内存不足**而不是堆问题。
- 3) 方法区（元空间）溢出在 Java 8 后主要是永久代被移除，改用本地内存存储类信息。如果动态生成大量类，比如 CGLIB 做字节码增强时没控制，元空间不断扩张又不卸载，就会 OOM。
- 4) 直接内存溢出发生在使用 `ByteBuffer.allocateDirect()` 时，这部分内存不受 GC 控制，也不算在堆内，靠 `-XX:MaxDirectMemorySize` 限制。NIO 使用不当，频繁分配直接内存且迟迟不释放，就可能触发。

代码示例：

```
// 直接内存溢出模拟
for (;;) {
    ByteBuffer.allocateDirect(1024 * 1024); // 每次分配 1MB
}
```

这类问题线上排查得看 `OutOfMemoryError` 后面的提示，比如 "Java heap space"、"Metaspace"、"Direct buffer memory" 都能直接定位区域。



Java 的 synchronized 是怎么实现的?

synchronized 的底层其实依赖 JVM 对 **monitor** 的支持，每个对象都有一个与之关联的 monitor。线程要进入 synchronized 代码块，必须先获取这个 monitor 的所有权。

monitor 在 HotSpot 虚拟机里是用 C++ 实现的 ObjectMonitor 类，它的竞争机制不是一上来就重量级加锁。而是从无锁状态开始，随着竞争情况升级：

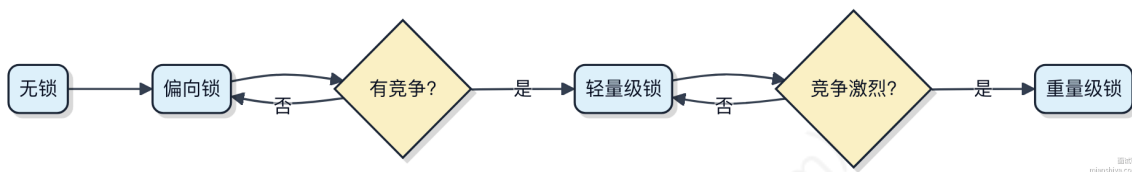
- 1) 没有竞争时，使用 **偏向锁**，记录下线程 ID，下次同一个线程进来直接通过，连 CAS 都省了
- 2) 出现竞争但不激烈，升级为 **轻量级锁**，通过 CAS 尝试把对象头的 Mark Word 替换为指向线程栈中锁记录的指针
- 3) 竞争激烈时，膨胀为 **重量级锁**，这时候线程会进入阻塞态，由操作系统来调度，性能开销大

锁的状态信息都存在对象头的 Mark Word 里，64 位虚拟机下占 8 字节，里面存了哈希码、分代年龄、锁标志位等。

```
synchronized (obj) {  
    // 字节码层面会生成 monitorenter 和 monitorexit 指令  
    // 对应到 ObjectMonitor 的 enter() 和 exit()  
}
```

解锁时要释放 monitor，并唤醒等待的线程。如果多个线程同时竞争，可能触发锁膨胀甚至全局停顿。

整个过程是 JVM 自动管理的，开发者不用干预，这也是为什么叫“内置锁”。

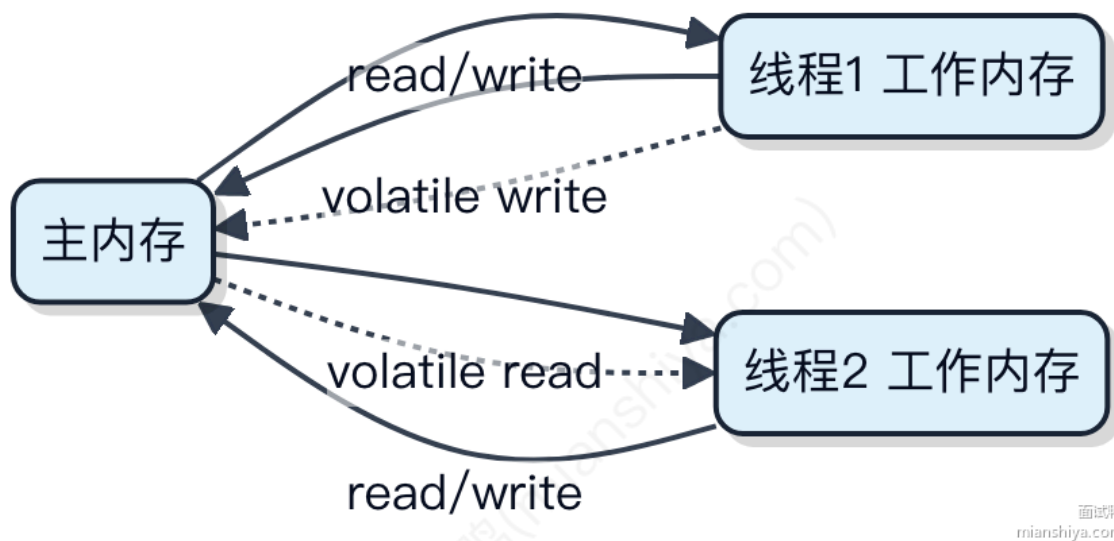


什么是 Java 内存模型（JMM）？

Java 内存模型（JMM）不是描述内存分布，而是定义多线程环境下变量的可见性、原子性和有序性规则。它屏蔽了硬件和操作系统的差异，确保程序在不同平台下对内存的访问行为一致。

每个线程有自己的工作内存，里面保存了主内存中共享变量的副本。线程对变量的操作都在工作内存里进行，修改后要写回主内存才能被其他线程看到。这就可能引发可见性问题，比如一个线程改了变量，另一个线程压根不经过主内存，读到的就是旧值。

- 1) 原子性：基本读写是原子的，但像 `i++` 这种复合操作就不行，得靠 `synchronized` 或 `volatile` 配合 CAS 来保证。
- 2) 可见性：一个线程修改了变量，其他线程能立即知道。`volatile` 变量写会强制刷回主内存，读则强制从主内存加载。
- 3) 有序性：编译器和处理器可能会重排序指令来优化性能，但 `happens-before` 规则限制了哪些重排是合法的。比如 `volatile` 写一定发生在后续的 `volatile` 读之前。



典型场景如双重检查单例，`instance` 不加 `volatile`，就可能因为重排序导致返回未初始化完的对象。用 `synchronized` 能解决，但 `volatile` 更轻量。

什么是 Java 中的原子性、可见性和有序性？

多线程环境下，这三个特性是理解并发问题的根基。

原子性指的是一个操作要么全部执行成功，要么完全不执行，不会被线程调度打断。比如 `i++` 实际上是读、改、写三步，不具备原子性。用 `AtomicInteger` 的 `incrementAndGet()` 就能保证这一步是原子的，底层靠的是 CPU 的 CAS 指令。

可见性是指当一个线程修改了共享变量的值，其他线程能立即知道这个变化。没有同步手段时，线程可能一直用着自己工作内存里的旧值。 `volatile` 关键字能解决这个问题，它会强制线程从主内存读写变量。`synchronized` 和 `final` 也能保证可见性。

有序性涉及到指令重排的问题。编译器和处理器为了优化性能，可能会改变代码执行顺序。但在多线程下，这种重排可能导致意料之外的结果。 `volatile` 能禁止特定类型的重排序，`synchronized` 则通过同一时刻只允许一个线程进入同步块来保证有序。



比如在双重检查单例里，`instance` 字段必须加 `volatile`，否则可能发生返回了一个未完全初始化的对象。重排序可能导致对象还没构造完，引用就已经指向了那块内存。

什么是 Java 中的 ABA 问题？

ABA 问题出现在使用 CAS（Compare-And-Swap）实现的无锁并发结构中，比如 `AtomicInteger` 或 `ConcurrentLinkedQueue`。它不是说值变了又变回来这么简单，而是 **CAS 操作无法感知中间是否发生过变化**。

举个例子：线程1读取某个变量值为 A，然后去执行 CAS，预期还是 A 就改成 B。但在这期间，其他线程可能把 A 改成 B 又改回 A。对线程1来说，值没变，于是顺利提交。可实际上系统状态已经不同了，比如资源被释放又分配，导致悬挂指针或重复释放等问题。

这个问题在操作系统或内存管理里尤其危险，但在 Java 里一般通过 **版本号机制** 解决。典型方案是用 `AtomicStampedReference`，它不仅存对象引用，还带一个整型的 stamp，每次修改递增。这样即使值从 A→B→A，stamp 也从 0→1→2，线程能发现版本不对，拒绝更新。

所以，单纯靠值相等来判断“没变”是不够的，得加上时间维度或者版本号。像 Redis 的乐观锁、数据库的 MVCC 其实也面临类似问题，解决思路一脉相承。

为什么 Java 8 移除了永久代（PermGen）并引入了元空间（Metaspace）？

永久代在 Java 7 及之前负责存储类的元数据，比如类名、方法、字段信息等。它有个致命问题：大小固定，默认最大就 80 多 MB，加载大量类时很容易撑满，导致 **OutOfMemoryError: PermGen space**。尤其在运行时生成类的场景，像使用 CGLIB 做动态代理的 Spring AOP，或者部署多个应用的 Tomcat 环境，这个问题特别常见。

元空间就是为了解决这个痛点来的。最关键它是 **不再使用 JVM 堆内存**，而是直接用操作系统的本地内存（native memory）。这意味着只要系统还有内存，元空间就能扩展，基本不会因为类多就 OOM。默认情况下，元空间会一直扩容，直到系统内存耗尽。

另一个好处是 GC 配合更顺了。以前永久代里的类信息和其他堆对象混在一起，GC 清理时得全堆扫描，效率低。元空间独立出来后，类的元数据由专门的机制管理，配合 **Full GC 才触发回收**，减少了干扰。

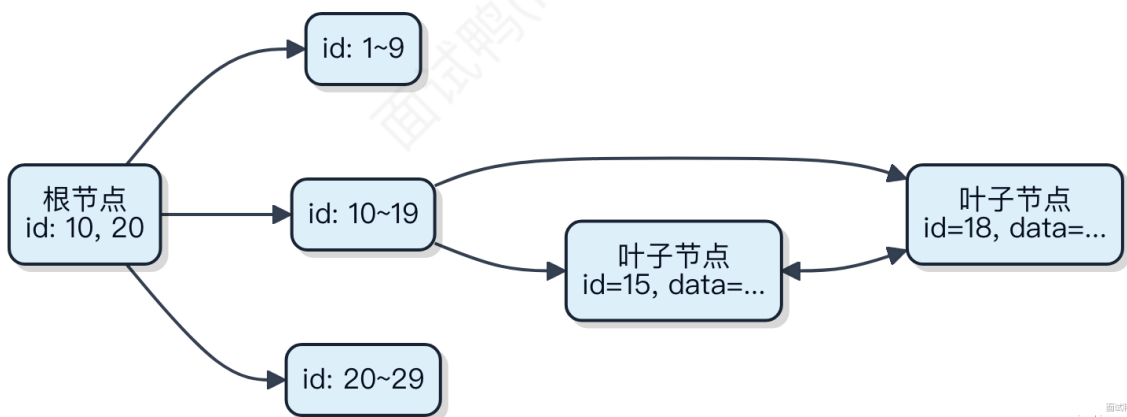
当然，元空间也不是无限的。可以通过 `-XX:MaxMetaspaceSize` 限制上限，否则可能把系统内存耗光。如果不设，它会自动调整。

简单说，从永久代到元空间，本质是把类的元数据从受限的 JVM 内存移到了更灵活的本地内存，解决了容量瓶颈和 GC 效率问题，更适合现代动态生成类的应用场景。

请详细描述 MySQL 的 B+ 树中查询数据的全过程

MySQL 的 InnoDB 引擎用 B+ 树组织主键索引，数据查询就是从根节点一路往下找叶子节点的过程。非叶子节点存的是索引键和指向子节点的指针，只有叶子节点才真正存行数据，并且叶子之间用双向链表串起来，方便范围查询。

- 1) 从根节点开始，根据查询条件比如 `WHERE id = 15`，在当前节点的键值中做二分查找，找到目标值所属的区间，决定下一步走哪个子节点。
- 2) 逐层下探，每次 IO 读一个数据页（默认 16KB），直到抵达叶子节点。这个过程一般不超过 3~4 层，也就是说多数查询 3 次磁盘 IO 就能定位到数据。
- 3) 在对应的叶子节点页内，再通过二分加遍历的方式找到具体的记录。如果查的是普通索引，可能还要回表一次主键索引拿完整数据。



注意，B+ 树所有查询都必须走到叶子节点才算结束，不像 B 树可能中途就命中。这也是它更适合数据库的原因——查询路径稳定，而且叶子连成链表后，像 `BETWEEN 15 AND 25` 这种范围扫描效率很高。聚簇索引下，主键有序插入能减少页分裂，随机写入则容易产生碎片。

MySQL 是如何实现事务的？

MySQL 的事务能力由存储引擎层实现，最典型的是 InnoDB 引擎。它通过 **redo log**、**undo log** 和 **锁机制** 三驾马车协同工作来保证 ACID。

- 1) redo log 确保持久性。当数据更新时，InnoDB 先写日志再改内存中的数据页，后续刷盘异步进行。即使宕机重启，也能靠 redo log 把未落盘的变更重放一遍。这个过程叫 crash recovery。
- 2) undo log 支持原子性和一致性。每个修改操作都会生成反向操作的日志。如果事务回滚，就用 undo log 恢复到修改前的状态。同时 MVCC（多版本并发控制）也依赖 undo log 构建历史版本，让不同事务看到一致的数据快照。
- 3) 锁机制保障隔离性。InnoDB 在行级别加锁，比如读用共享锁，写用排他锁。配合 next-key lock（记录锁 + 间隙锁），能防止幻读。不同的事务隔离级别会调整锁策略和可见性规则。

实际应用中，像支付宝转账这种场景就必须依赖这套机制，确保要么全部成功，要么全部回退。如果只靠内存操作或者普通文件写入，根本扛不住并发和故障。

MySQL 中的 MVCC 是什么？

MVCC，全称多版本并发控制，是 MySQL InnoDB 存储引擎实现高并发读的核心机制。它让读操作不用加锁也能看到一致的数据视图，读写之间不互相阻塞。

InnoDB 通过三样东西配合实现 MVCC：隐藏的 `DB_TRX_ID`（事务 ID）、`DB_ROLL_PTR`（回滚指针），再加上 **undo log**。每次修改行记录时，旧版本数据会被写进 undo log，新行指向它，形成一个版本链。

事务启动时，InnoDB 会拍一个活跃事务 ID 快照，构建 Read View。这个视图决定了你能看到哪个版本的数据。比如当前事务看不到还没提交的修改，也看不到在你之后启动的事务做的变更。

像可重复读（RR）隔离级别下，Repeatable Read 靠的就是事务开始时生成的那个 Read View 贯穿始终，中间其他事务改了数据并提交，你也看不见，保证了多次读结果一致。

但注意，MVCC 只作用于普通 SELECT 这种快照读。如果是 `SELECT ... FOR UPDATE` 或 `LOCK IN SHARE MODE`，那就是当前读，直接查最新已提交数据，得加锁。

更新操作也走当前读，所以 RR 级别能防幻读，靠的不是 MVCC，是 next-key lock。简单说，MVCC 是用来“扛住”大量并发读的脏活累活，让数据库在高并发下依然能快速响应查询。

Redis 中跳表的实现原理是什么？

跳表（Skip List）在 Redis 里用来实现有序集合（ZSet）的底层结构，特别是当元素数量多或成员字符串较长时。它本质上是带多级索引的链表，通过空间换时间，让查找、插入、删除操作的平均时间复杂度降到 $O(\log n)$ 。

Redis 的跳表每一层都是一个稀疏链表，最底层包含所有元素，每往上一层，节点数大概减少一半。每个节点有个随机生成的“层高”，最高不超过 32 层。查找时从顶层开始横向移动，遇到更大的就往下走，像坐电梯一样快速逼近目标。

```
// 简化版 zskiplistNode 结构
typedef struct zskiplistNode {
    sds ele;           // 成员值
    double score;       // 分值，排序依据
    struct zskiplistLevel {
        struct zskiplistNode *forward; // 指向同层下一个节点
        unsigned int span;             // 跨度，用于计算排名
    } level[];
} zskiplistNode;
```

插入新节点时，先通过随机函数决定它的层数，比如用 `rand() % 2` 不断判断是否升级，最多到 32 层。这样能保证索引分布大致均匀，避免退化成单链表。

- 1) 查找快：平均只要遍历 $\log n$ 个节点就能定位
- 2) 实现比红黑树简单，没那么多旋转逻辑
- 3) 范围查询友好，比如 `ZRANGE`、`ZRANGEBYSCORE` 直接链表遍历就行

这种结构在 Redis 里扛住了大量实时排行榜场景，比如游戏积分榜、热门帖子排行，跳表的优点就在于简单高效。

让你设计一个 RPC 框架，怎么设计？

先想清楚 RPC 的本质，就是让远程调用像本地调用一样简单。关键在于把“调哪儿”、“传什么”、“怎么传”、“结果怎么拿”这四件事搞定。

1) 服务暴露与发现

服务启动时得把自己的接口、IP、端口注册到注册中心，比如 ZooKeeper 或 Nacos。消费者去上面拉取可用的提供者列表，缓存下来，避免每次调用都查。

2) 动态代理拦截调用

你在代码里写 `userService.getUser(1)`，其实是个接口，没有实现。框架通过 JDK 动态代理或 ByteBuddy 生成一个代理对象，一调用就进到 `InvocationHandler`，把方法名、参数抓出来。

3) 网络通信选型

一般用 Netty 做传输层，性能好，异步非阻塞。序列化协议可以选 Protobuf、Hessian 或 JSON，Protobuf 体积小、速度快，跨语言也方便。

4) 请求唯一标识与异步等待

每个请求发出去前打上唯一 ID（比如原子递增），放进 `ConcurrentMap`，线程用 `CountDownLatch` 挂着等结果。响应回来时根据 ID 找到对应的等待线程，唤醒并返回结果。

5) 容错与负载均衡

如果节点挂了，得有重试机制，配合负载均衡策略（随机、轮询、一致性 Hash）选下一个节点。超时时间必须设，不然线程全卡死。

别忘了埋点监控，比如用 SkyWalking 记录调用耗时。一开始不用搞得太重，先把“能通”做到位，再加路由、限流、熔断这些脏活累活。

让你设计一个短链系统，怎么设计？

短链系统本质是把一个长 URL 映射到一个短字符串，核心挑战是怎么生成这个短字符串，并且能反向查回去。

1) 生成策略上，一般不用自增 ID 直接转成 62 进制（a-z, A-Z, 0-9），因为容易被遍历猜出。更稳妥的是用 **分布式唯一 ID** 生成，比如 Snowflake 算法生成 64 位 long，再转成 62 进制字符串，长度通常 6~8 位，够用还不重复。

2) 存储结构就是个 key-value，短码做主键，存原始 URL 和过期时间。读请求远多于写，所以得靠缓存扛压。Redis 是首选，先查缓存，没命中再去 MySQL 或 TiDB 查。写的时候记得双写一致，删缓存而不是更新，避免脏数据。

3) 跳转接口要快，302 重定向就行，响应头 Location 填原链接。注意短码大小写敏感问题，建议统一转小写处理。

4) 防刷和限流不能少。同一个 IP 短时间内频繁请求不同短链，可能是爬虫，可以用 Redis 记录访问频次，结合滑动窗口限流。

5) 扩展功能比如统计点击量，别直接改主表，会锁行。用 Kafka 把点击事件异步发出去，后端消费落 HBase 或 ClickHouse 做分析。

整个系统最脏活累活在 ID 生成和缓存一致性，其他都是常规操作。美团、微博的短链都这么搞。

让你设计一个分布式 ID 发号器，怎么设计？

分布式 ID 发号器的关键是生成全局唯一、趋势递增、高可用且低延迟的 ID。一般来说，会结合多种策略来满足不同场景。

1) 最简单的方案是用数据库自增主键，但单点瓶颈明显，扛不住高并发。可以改用分段预分配，比如每个应用实例从数据库一次性拿 1000 个 ID，本地自增，用完再取，减少 DB 压力。像美团的 Leaf 就是这么干的。

2) 纯内存生成常见的是 **雪花算法** (Snowflake)，64 位 Long 型，包含时间戳、机器 ID、序列号。时间戳保证趋势递增，机器 ID 区分节点，序列号解决毫秒内并发。压根不经过数据库，性能极好，TPS 能到几十万。

```
// 简化版 Snowflake 核心逻辑
long id = ((timestamp << 22) | (workerId << 12) | sequence);
```

但要注意时钟回拨问题，如果系统时间往回调，可能产生重复 ID。一般做法是等待或抛异常。

3) 如果对 ID 无序性有要求，可以用 UUID，但字符串太长，存储和索引效率差，不适合做数据库主键。

4) 更高阶的做法是混合型，比如 Leaf 同时支持号段模式和 Snowflake 模式，通过 ZooKeeper 或 Etcd 做 worker ID 分配和高可用协调。

选哪种方案，得看你的 QPS、是否允许间隙、对单调性的要求。大部分业务系统用号段 + 分布式协调服务就能搞定。

说一下 RocketMQ 中关于事务消息的实现？

事务消息解决的是本地数据库操作和发消息这两件事的原子性问题。比如扣库存成功，但发订单消息失败了，下游就收不到通知，数据就不一致。

RocketMQ 的方案是“半消息 + 定时回查”。生产者先发一个**半消息**，这个消息对消费者不可见，Broker 会存下来。发完半消息后，生产者去执行本地事务，比如改数据库。最后根据事务结果，再提交或回滚消息。

关键在于，如果生产者发完半消息就宕机，Broker 不知道结果，怎么办？它会定时发起 **事务状态回查**，主动问生产者：“你那个事务到底成没成？”生产者得实现一个检查接口，告诉 Broker 最终状态。

整个流程依赖几个角色：Producer、Broker、以及你的业务服务。正常流程是三步：发半消息 -> 执行本地事务 -> 提交/回滚。异常时，Broker 发起回查，最多查 15 次，默认间隔 30s。

代码上，你要用 `TransactionMQProducer`，然后加一个 `TransactionListener`，里面写 `executeLocalTransaction` 和 `checkLocalTransaction` 方法。

```
producer.sendMessageInTransaction(msg, null);
```

回查机制是保证最终一致的核心。但要注意幂等，因为同一个消息可能被多次消费。另外，事务消息不支持批量发送，延迟等级也有限制。

场景上，适合订单创建、支付回调这类强一致性要求的链路。不适合高频小事务，回查会有压力。

说一下 Kafka 中关于事务消息的实现？

Kafka 的事务消息主要是为了解决分布式环境下，**精确一次语义** (Exactly-Once Semantics, EOS) 的问题。你可能在用 Flink 做实时计算时见过这个功能，它背后靠的就是 Kafka 事务支持。

生产者开启事务后，可以保证一组消息要么全部提交，要么全部不提交。这在跨分区写入时特别关键，比如你要往多个 topic-partition 写数据，必须保持原子性。

实现上，Kafka 引入了事务协调器（Transaction Coordinator）和事务日志（__transaction_state）。每个生产者通过 transactional.id 绑定到一个协调器，由它来管理事务状态。整个流程分两阶段提交：

1) 生产者先向协调器发起 begin 2) 发消息时，broker 先把这些记录标记为“待提交”，存到 __consumer_offsets 和目标 topic 中 3) 提交阶段，协调器把事务结果写入事务日志，最终标记为 COMMITTED 或 ABORTED

消费者这边，默认是读不到未提交消息的，避免脏读。只有设置了 isolation.level=read_committed，才会跳过未提交记录。

代码上很简单，只要开启配置：

```
props.put("enable.idempotence", true);
props.put("transactional.id", "tx-01");
```

然后调 initTransactions()、beginTransaction()、send()、commitTransaction() 就行。

注意，事务是有代价的。每条消息会多几个字段，延迟也会上升，毕竟要走协调流程。如果你只是做单条消息的幂等写入，开 enable.idempotence 就够了，不用整个事务。

如何设计一个秒杀功能？

秒杀本质是用技术手段把瞬时超大并发转化成系统能处理的量级，关键在于“削峰”和“前置”。

1) 流量层直接拦截无效请求。Nginx 用 Lua 做本地限流，同一用户每秒只放行 1 次请求，再多直接 503。Redis 集群预热商品库存，key 设成 seckill:stock:{itemId}，值为原子整数，前端按钮灰化后仍可能被绕过，所以每扣减都必须走 DECR 并判断非负。

2) 异步化减少数据库冲击。用户中签后写 Kafka，消息消费端才真正落订单到 MySQL。这样数据库只扛住最终一致性写入，峰值写入从 10w+/s 降到几千/s，MySQL 配合分库分表就能扛住。

3) 缓存热点数据。商品详情、活动规则全放 Redis，页面静态化，CDN 托管前端资源。下单接口不做任何远程调用，所有校验在本地完成。

4) 避免超卖的核心是库存扣减原子性。Redis 扣库存 + 消息队列异步下单，两个动作必须绑定成一个逻辑单元。可以用 Lua 脚本保证原子性，脚本返回成功才发 MQ。

压测要覆盖真实场景，比如用 JMeter 模拟 5w 并发抢 100 件商品。上线前做全链路压测，重点看 Redis 和 Kafka 的负载。一般系统按 3 倍冗余设计容量，但秒杀这种极端场景，前端拦截掉 90% 以上请求才是正解。

Dubbo 和 Spring Cloud Gateway 有什么区别？

Dubbo 和 Spring Cloud Gateway 完全是不同维度的工具，一个搞服务通信，一个管流量入口。

Dubbo 是 RPC 框架，专注服务之间的远程调用。你写个接口，Dubbo 帮你把调用发到另一台机器上执行，底层用 Netty 传输，默认走 TCP 协议。它解决的是“服务 A 怎么高效、透明地调 B”的问题，像阿里内部早期就靠这玩意撑

起几千个服务协同。

Spring Cloud Gateway 是 API 网关，跑在最外层，所有外部请求先经过它。它的活是路由转发、限流、鉴权、熔断这些，基于 WebFlux 构建，走 HTTP，处理的是“用户请求进来该给谁，能不能让进”这类事。比如你访问 `api/order/1`，网关判断后转给 order-service 处理。

所以它们根本不冲突。典型微服务架构里，Gateway 在前头挡着，后面一堆 Dubbo 服务互相调用。你可以用 Nginx 或 Spring Cloud Gateway 做统一入口，内部服务之间再用 Dubbo 调，性能比 HTTP 高不少。

代码上看，Dubbo 接口像这样：

```
@DubboReference
private OrderService orderService;
// 调用就像本地方法
Order o = orderService.findById(1);
```

而 Gateway 配置路由：

```
spring:
  cloud:
    gateway:
      routes:
        - id: order_route
          uri: lb://order-service
          predicates:
            - Path=/api/order/**
```

1) Dubbo 核心是 **RPC 通信** 2) Gateway 核心是 **流量治理** 3) 一个对内，一个对外，经常一起用

怎么分析 JVM 当前的内存占用情况？OOM 后怎么分析？

分析 JVM 内存占用，第一步是看整体分布。用 `jstat -gc <pid>` 能实时看到年轻代、老年代、元空间和 GC 执行次数、耗时，一般观察 30 秒以上，判断是否有频繁 GC 或内存缓慢增长。

接着用 `jmap -heap <pid>` 查看堆内存的详细配置和使用情况，比如初始堆、最大堆、各区实际使用量。如果发现老年代使用率长期超过 70%，就得警惕了，可能快 OOM 了。

怀疑有对象泄漏时，`jmap -dump:format=b,file=heap.hprof <pid>` 抓取堆转储文件，然后用 Eclipse MAT 或 JVisualVM 分析。MAT 可以快速找出“主导集”（Dominator Tree），定位谁占了最多内存，比如某个缓存 Map 持有上百万对象。

OOM 后分析也靠 dump 文件，但得先确保加了启动参数：

```
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/path/to/dumps
```

这样 OOM 时会自动生成 dump，不然是事后诸葛也搞不定。

代码里也可以主动触发：`jcmd <pid> GC.run_finalization` 或生成 dump 辅助排查。

线上服务建议加上 `-XX:+PrintGCDetails -Xlog:gc*:file=gc.log`，GC 日志能帮你回溯 OOM 前的征兆，比如连续几次 Full GC 后内存还是下不来，基本就是泄漏了。

如果发现 Redis 内存溢出了？你会怎么做？请给出排查思路和解决方案

发现 Redis 内存溢出，别慌，先搞清楚是物理内存撑爆了，还是达到了 `maxmemory` 配置上限。这两个情况表现不一样，处理方式也不同。

1) 先看现象

如果 Redis 开始频繁写 swap，响应变慢甚至卡死，基本是物理内存耗尽。如果是开始大量返回 `OOM command not allowed`，那就是触达了 `maxmemory` 限制，**内存淘汰策略**开始干活了。

2) 查当前用了多少

上机器跑 `info memory`，重点关注 `used_memory_rss` 和 `used_memory`。前者是系统实际占用，后者是 Redis 内部统计。差太多可能有内存碎片，比如 `mem_fragmentation_ratio > 1.5` 就得注意了。

3) 谁占的内存？

用 `redis-cli --bigkeys` 快速扫一遍，看看有没有超大 hash、zset 或 list。也可能某个 key 的过期时间没设，一直堆积，比如日志类数据往里塞。监控平台像 **Prometheus + Grafana** 搭配 Redis Exporter 能帮你盯住内存增长趋势。

4) 解决方案分三层

- 紧急：调大 `maxmemory`（临时顶住），或者切到新实例做数据迁移。
- 治本：开启 **LFU/LRU 淘汰策略**，比如 `allkeys-lfu`，让冷数据自动滚蛋。
- 长期：数据结构优化，大 key 拆分，加二级缓存（比如本地 Caffeine），或者把部分数据往 **Codis** 或 **Tendis** 这类分布式方案挪。

从网络角度来看，用户从输入网址到网页显示，期间发生了什么？

用户敲下回车后，浏览器要先拿到服务器 IP 地址。本地查 hosts 文件，没找到就发 DNS 查询请求。这个请求通常先到本地 DNS 服务器，它会递归或迭代查询根域名、顶级域、权威域名服务器，最终拿到 A 记录。

有了 IP，浏览器发起 TCP 连接，三次握手建立连接。如果走 HTTPS，接着进行 TLS 握手，协商加密套件、交换密钥，确保后续通信安全。

TCP 连接建立后，浏览器开始发 HTTP 请求。比如 `GET /index.html`，携带 Host、User-Agent 等头信息。请求通过操作系统网络栈，经过路由器层层转发，最终到达目标服务器。

服务器收到请求，应用层如 Nginx 或 Tomcat 处理，可能访问数据库、缓存，生成响应内容。响应包含状态码、响应头和 HTML 内容，通过已建立的 TCP 连接返回。

浏览器收到响应，解析 HTML，遇到 CSS、JS、图片等资源，再发起多个并行请求。这些请求可能复用现有 TCP 连接（HTTP Keep-Alive），减少握手开销。

页面渲染同时进行，构建 DOM 树、CSSOM，生成渲染树，布局绘制。JS 执行可能阻塞解析，所以一般建议异步加载。

整个过程涉及多层协议协作。DNS 查询可能耗时几百毫秒，CDN 和 DNS 缓存能显著降低延迟。现代浏览器对同域名并发连接数限制在 6 个左右，超出的请求排队。

整个链路中任意环节超时或失败都会导致页面加载异常，比如 DNS 污染、TCP 重传、TLS 协商失败、后端服务无响应等。

Java 线程池核心线程数在运行过程中能修改吗？如何修改？

线程池的核心线程数在运行时是可以修改的，Java 提供了直接的方法来动态调整。关键在于 `ThreadPoolExecutor` 的 `setCorePoolSize` 方法。

- 1) 调用 `setCorePoolSize(int corePoolSize)` 可以实时修改核心线程数。如果新值大于当前值，线程池会尝试创建新线程来达到目标，前提是队列中有等待任务。如果小于当前值，超出部分的空闲线程会在下次空闲时被回收。
- 2) 这个机制让线程池具备弹性伸缩能力，适合负载波动大的场景。比如在电商大促期间，可以通过监控 QPS 动态上调核心线程数，扛住突发流量；高峰过去再降下来，避免资源浪费。
- 3) 代码上很简单：

```
executor.setCorePoolSize(20);
```

一行代码就能生效，不需要重启或重建线程池。

这种动态调整能力，是 `ThreadPoolExecutor` 区别于固定大小线程池的关键优势。很多中间件比如 **Dubbo**、**Tomcat** 的工作线程池都基于此实现运行时调参。

不过要注意，频繁修改可能引发线程震荡，建议结合监控系统做平滑调整。

MySQL 中如果我 select * from 一个有 1000 万行的表，内存会飙升么？

这个问题得看你怎么执行这条 SQL。很多人以为 `SELECT * FROM 表` 会把 1000 万行数据全捞到内存里，其实不一定。

MySQL 的查询结果默认是**流式返回**的。Server 端一边查存储引擎，一边往客户端发数据，不会一次性把所有结果塞进内存。所以服务端的内存压力其实是可控的，关键在于客户端怎么处理。

但问题出在客户端。比如你用的是 MySQL Workbench、Navicat 这类图形工具，默认会把整个结果集全拉到本地内存里渲染表格。1000 万行宽表可能直接把你的机器干趴，内存飙升的就是你本地这头。

程序里也一样。Java 用 JDBC 的 `Statement.executeQuery()`，默认行为是把结果全 fetch 回来存到 `ResultSet` 里。除非你设置了 `fetchSize(Integer.MIN_VALUE)`（MySQL 驱动特殊支持），让它用游标模式流式读取。

所以真正要关注的是：**结果集是否缓冲**。

- 1) 如果你只是想数一数，写成 `SELECT COUNT(*) FROM huge_table`，这个当然没问题，聚合后就一行。
- 2) 如果是导出数据、做迁移，建议加主键范围分页，或者用 `WHERE id > ? LIMIT 10000` 一批批拿。
- 3) 别在生产连上直接敲 `SELECT * FROM 超大表`，尤其是图形化工具，压根不经过脑子。

代码示例：

```
stmt.setFetchSize(Integer.MIN_VALUE); // MySQL 游标式读取
ResultSet rs = stmt.executeQuery("SELECT * FROM big_table");
while (rs.next()) {
```

```
// 流式处理，内存不爆炸  
}
```

本资源来自面试鸭：<https://www.mianshiya.com>

推荐更多免费学编程资源：

1. 编程导航学习网站：[学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库：[开始刷题，面试遇原题!](#)
3. 精选简历模板大全：[1 分钟搞定简历!](#)
4. AI 资源导航网站：[获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试：[随时随地提升面试能力](#)